

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

**НАВЧАЛЬНО-МЕТОДИЧНИЙ КОМПЛЕКС
НАВЧАЛЬНОЇ ДИСЦИПЛІНИ**

ПРОГРАМУВАННЯ

назва навчальної дисципліни

спеціальність 121 Інженерія програмного забезпечення

назва спеціальності

освітньо-професійна програма розробка програмного забезпечення

назва ОПП

відділення інженерії програмного забезпечення

відділення

розробник Владислав ОЛІЙНИК

ім'я та прізвище викладача

Затверджений на засіданні циклової комісії
дисциплін інженерії програмного забезпечення

назва циклової комісії

Протокол від «____» _____ 20____ року №____

Голова циклової комісії _____ Олександр БАБИЧ

підпис, ім'я та прізвище

АНОТАЦІЯ ЗМІСТУ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

Програмування

назва навчальної дисципліни

нормативна

нормативна/вибіркова

цикл професійної та практичної підготовки

цикл дисциплін за навчальним планом

Програмування є теоретичною та практичною основою професійних знань та вмінь, що формують профіль фахівця в предметній області «Інженерія програмного забезпечення». Тож НМК навчальної дисципліни посідає одне з чільних місць в підготовці фахівців з спеціальності.

Предмет навчальної дисципліни

Предметом дисципліни є методологія створення комп'ютерних програм — від формалізації ідеї до реалізації працюючого продукту за допомогою спеціалізованих мов та інструментів.

Мета навчальної дисципліни

Метою викладання навчальної дисципліни «Програмування» є: вивчення методів алгоритмізації, основ програмування на алгоритмічних мовах високого рівня і використання отриманих навиків при вирішенні задач різних типів.

Основні завдання

Основними завданнями вивчення дисципліни «ПРОГРАМУВАННЯ» є:

- формування базових знань по алгоритмізації та програмуванню про стиль написання програм, про раціональні методи їх розробки і оптимізації, про стратегії відладки і тестування програм;
- отримання базового рівня по програмуванню на мовах C, C++ з використанням простих типів даних, базових типів даних і масивів;
- вивчення структур даних в пам'яті і в файлах і алгоритмів роботи з ними з використанням мов C; C++;
- знайомство з основними принципами організації зберігання та пошуку даних, алгоритмами сортування та пошуку;
- придбання навиків використання базового набору фрагментів і алгоритмів у процесі розробки програм, навиків аналізу і “читання” програм;
- вивчення основ технології програмування та методів рішення обчислювальних задач і задач обробки символьних даних;
- формування рівня знань мови, який дозволяє вільно оперувати типами даних і змінними довільної складності, а також модульними алгоритмами їх обробки.

Навчально-методичний комплекс складається з теоретичної частини, комплексу практичних завдань, інструкцій з виконання практичних робіт, методичних рекомендацій щодо виконання курсового проекту, збірки завдань для самостійного виконання і містить всі необхідні матеріали для проведення аудиторних занять та для самостійного опрацювання.

АНОТАЦІЯ

Навчально-методичний комплекс дисципліни «ПРОГРАМУВАННЯ» складений відповідно до начального плану підготовки фахового молодшого бакалавра спеціалізації: Розробка програмного забезпечення.

Використання навчально-методичного комплексу дисципліни «ПРОГРАМУВАННЯ» передбачає формування у здобувачів освіти цілісної системи професійних компетентностей, що базуються на глибокому розумінні принципів функціонування обчислювальних систем та сучасних методологій розробки програмного забезпечення. У процесі навчання у студентів закладається фундамент алгоритмічного мислення, що дозволяє не просто писати код, а ефективно декомпонувати складні прикладні завдання на послідовність логічних операцій. Завдяки комплексному підходу майбутні фахівці опановують синтаксис та семантику мов програмування високого рівня, набувають навичок роботи з динамічними структурами даних та вчаться обирати оптимальні інструменти для реалізації конкретних проектів. Важливою складовою формування професійного профілю є оволодіння об'єктно-орієнтованою парадигмою, яка дозволяє створювати масштабовані та гнучкі програмні продукти, що відповідають сучасним вимогам ІТ-індустрії.

Крім суто технічних навичок, навчально-методичний комплекс сприяє розвитку критичного підходу до аналізу власного коду, формує вміння здійснювати тестування, налагодження та оптимізацію програм, а також готує здобувачів до роботи в командному середовищі з використанням систем контролю версій. У результаті здобувачі освіти отримують здатність самостійно проходити повний життєвий цикл розробки програмного забезпечення — від формального аналізу вимог до розгортання та супроводу готового продукту, що забезпечує їхню високу конкурентоспроможність на ринку праці.

ANNOTATION

The educational and methodological complex of the discipline "PROGRAMMING" is compiled in accordance with the initial plan for the preparation of a professional junior bachelor in the specialization: Software Development.

The use of the educational and methodological complex of the discipline "PROGRAMMING" involves the formation of a holistic system of professional competencies among students, based on a deep understanding of the principles of functioning of computer systems and modern software development methodologies. During the learning process, students are laid the foundation of algorithmic thinking, which allows them not only to write code, but also to effectively decompose complex applied tasks into a sequence of logical operations. Thanks to a comprehensive approach, future specialists master the syntax and semantics of high-level programming languages, acquire skills in working with dynamic data structures, and learn to choose the optimal tools for implementing specific projects. An important component of forming a professional profile is mastering the object-oriented paradigm, which allows you to create scalable and flexible software products that meet the modern requirements of the IT industry.

In addition to purely technical skills, the educational and methodological complex promotes the development of a critical approach to analyzing one's own code, forms the ability to test, debug, and optimize programs, and also prepares applicants to work in a team environment using version control systems. As a result, students gain the ability to independently go through the full software development life cycle — from formal requirements analysis to deployment and support of the finished product, which ensures their high competitiveness in the labor market.

ЗМІСТ

АНОТАЦІЯ ЗМІСТУ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	2
ПРОГРАМА НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	7
ВСТУП.....	9
1. МЕТА ТА ЗАВДАННЯ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	10
2. ІНФОРМАЦІЙНИЙ ОБСЯГ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	12
3. РОЗПОДІЛ ГОДИН ЗА ВИДАМИ ЗАНЯТЬ ВІДПОВІДНО ДО НАВЧАЛЬНОГО ПЛАНУ.....	13
4. РЕКОМЕНДОВАНА ЛІТЕРАТУРА.....	15
РОБОЧА ПРОГРАМА НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	16
1. ОПИС НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	18
2. МЕТА ТА ЗАВДАННЯ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	19
3. КРИТЕРІЇ ОЦІНЮВАННЯ.....	21
4. ІНФОРМАЦІЙНИЙ ОБСЯГ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ.....	22
5. РОЗПОДІЛ ГОДИН ЗА ВИДАМИ ЗАНЯТЬ ВІДПОВІДНО ДО НАВЧАЛЬНОГО ПЛАНУ.....	23
6. МЕТОДИ НАВЧАННЯ.....	25
7. ЗАСОБИ ОЦІНЮВАННЯ (МЕТОДИ КОНТРОЛЮ).....	26
8. МЕТОДИЧНЕ ЗАБЕЗПЕЧЕННЯ.....	28
9. РЕКОМЕНДОВАНА ЛІТЕРАТУРА.....	29
КОРЕКЦІЯ.....	30
ПЛАНИ-КОНСПЕКТИ ЛЕКЦІЙНИХ ЗАНЯТЬ.....	31
Тема 1.1 Мова програмування, інтегроване середовище розробки. Історія і класифікація мов програмування.....	31
Тема 2.1. Вступ в програмування. Найпростіші програми.....	45
Тема 2.2. Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).....	49
Тема 3.1. Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).....	54
Тема 3.2. Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.).....	60
Тема 4.1. Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).....	67
Тема 4.2. Структура програм. Складові частини програми. Глобальні та локальні змінні.....	74
Тема 5.1. Алгоритми і величини. Лінійні обчислювальні алгоритми.....	80
Тема 6.1. Одновимірні масиви.....	84
Тема 6.2. Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.....	89
Тема 6.3. Багатовимірні масиви.....	96
Тема 7.1. Структури.....	103
Тема 7.2. Масиви структур.....	107
Тема 8.1. Оголошення і ініціалізація змінної покажчика.....	115
Тема 8.2. Масиви покажчиків. Покажчики на функцію.....	121

Тема 9.1. Робота з текстовими файлами.....	127
Тема 10.1. Списки. Зв'язаний список.....	132
Тема 10.2. Стеки, черги, деки.....	153
Тема 10.3. Деревя.....	156
ІНСТРУКТИВНО-МЕТОДИЧНІ МАТЕРІАЛИ ДО ВИКОНАННЯ ЛАБОРАТОРНИХ ТА ПРАКТИЧНИХ ЗАНЯТЬ	160
Тема роботи 1.2 : Компілятор, інтерпретатор, компонувальник. Структура і способи опису мов програмування високого рівня.....	160
Тема роботи 2.1 : Вступ в програмування. Найпростіші програми.....	163
Тема роботи 2.2 : Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).....	165
Тема роботи 2.3 : Арифметичні вирази. Формати для виводу даних.....	169
Тема роботи 3.1 : Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).....	174
Тема роботи 3.2 : Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.).....	180
Тема роботи 4.1 : Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).....	186
Тема роботи 4.2 : Структура програм. Складові частини програми. Глобальні та локальні змінні.....	192
Тема роботи 5.1 : Алгоритми і величини. Лінійні обчислювальні алгоритми.....	198
Тема роботи 5.2 : Розгалуження і цикли в обчислювальних алгоритмах. Допоміжні алгоритми і процедури.....	204
Тема роботи 6.1 : Одновимірні масиви.....	210
Тема роботи 6.2 : Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.....	216
Тема роботи 6.3 : Багатовимірні масиви.....	222
Тема роботи 6.4: Масиви символьних строк. Оголошення і ініціалізація. Ввід-вивід. Сортування.....	228
Тема роботи 7.1: Структури.....	234
Тема роботи 7.2: Масиви структур.....	240
Тема роботи 8.1: Оголошення і ініціалізація змінної покажчика.....	247
Тема роботи 8.2: Масиви покажчиків. Покажчики на функцію.....	253
Тема роботи 9.1: Робота з текстовими файлами.....	260
Тема роботи 9.2: Створення файлу послідовного доступу.....	266
Тема роботи 9.3: Створення файлу довільного доступу.....	272
Тема роботи 9.4: Робота з двійковими файлами.....	278
Тема роботи 10.1: Списки. Зв'язаний список.....	284
Тема роботи 10.2: Стеки, черги, деки.....	290
Тема роботи 10.3: Деревя.....	296
Тема роботи 10.4: Графи.....	302
МЕТОДИЧНІ МАТЕРІАЛИ, ЩО ЗАБЕЗПЕЧУЮТЬ САМОСТІЙНУ РОБОТУ СТУДЕНТІВ	308
НАВЧАЛЬНО-МЕТОДИЧНІ МАТЕРІАЛИ ДЛЯ ПРОМІЖНОГО КОНТРОЛЮ ТА ПЕРЕЛІК ПИТАНЬ ПІДСУМКОВОГО КОНТРОЛЮ	309
Приклад тестових завдань для проміжного контролю:.....	309

Перелік екзаменаційних білетів.....	321
МЕТОДИЧНІ ВКАЗІВКИ ДО ВИКОНАННЯ КУРСОВИХ РОБІТ З ДИСЦИПЛІНИ.....	337
1. ЗАГАЛЬНІ ПОЛОЖЕННЯ.....	340
2. ПОРЯДОК ВИКОНАННЯ КУРСОВОЇ РОБОТИ.....	342
3. ЗМІСТ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ.....	344
4. ПРАВИЛА ОФОРМЛЕННЯ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ.....	349
5. ПОРЯДОК ВИКОНАННЯ ТА ЗАХИСТУ КУРСОВОЇ РОБОТИ.....	355
6. ТЕМИ ІНДИВІДУАЛЬНИХ ЗАВДАНЬ НА КУРСОВУ РОБОТУ.....	357
СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ.....	358
ДОДАТОК А.....	359
ДОДАТОК Б.....	360
ОПИС МЕТОДИЧНОГО ТА ТЕХНІЧНОГО ЗАБЕЗПЕЧЕННЯ.....	365
РЕКОМЕНДОВАНА ЛІТЕРАТУРА ТА ІНТЕРНЕТ-РЕСУРСИ.....	366

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

ЗАТВЕРДЖУЮ

Директор коледжу

_____ О.С. Пітяков

«01» червня 2023р.

ПРОГРАМУВАННЯ

(назва навчальної дисципліни)

ПРОГРАМА НАВЧАЛЬНОЇ ДИСЦИПЛІНИ ПІДГОТОВКИ ФАХОВОГО МОЛОДШОГО БАКАЛАВРА

галузь знань: 12 Інформаційні технології

спеціальність: 121 Інженерія програмного забезпечення

спеціалізація: Розробка програмного забезпечення

РОЗРОБНИК(-И) ПРОГРАМИ: викладач другої кваліфікаційної категорії
Олійник Владислав Васильович

Програма розглянута та схвалена на засіданні циклової комісії дисциплін програмної інженерії

Протокол від « 26 » травня 2023 року № 9

Голова циклової комісії_____О.В. Бабич

ВСТУП

Програма вивчення нормативної навчальної дисципліни «**Програмування**» складена відповідно до освітньо-професійної програми підготовки молодшого фахового бакалавра спеціалізації «Розробка програмного забезпечення».

Предметом вивчення навчальної дисципліни є: необхідність ознайомлення студентів з основами алгоритмізації, типовими структурами алгоритмів, мовами програмування та засобами створення програм із застосуванням ЕОМ..

Міждисциплінарні зв'язки:

- Теорія алгоритмів та структури даних
- Організація баз даних
- Об'єктно-орієнтоване програмування
- Архітектура комп'ютерів

Програма навчальної дисципліни складається з таких змістових модулів: Змістовий

модуль 1. Основні поняття та означення

Змістовий модуль 2. Елементи мови програмування

Змістовий модуль 3. Керування порядком обчислень

Змістовий модуль 4. Процедурно-орієнтоване програмування . Функції користувача

Змістовий модуль 5. Основи алгоритмізації

Змістовий модуль 6. Масиви

Змістовий модуль 7. Структури

Змістовий модуль 8. Показчики

Змістовий модуль 9. Робота з файлами

Змістовий модуль 10. Динамічні структури даних

1. МЕТА ТА ЗАВДАННЯ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

Метою викладання навчальної дисципліни «Програмування» є: вивчення методів алгоритмізації, основ програмування на алгоритмічних мовах високого рівня і використання отриманих навиків при вирішенні задач різних типів.

Основними завданнями вивчення дисципліни «Програмування» є:

- формування базових знань по алгоритмізації та програмуванню про стиль написання програм, про раціональні методи їх розробки и оптимізації, про стратегії відладки і тестування програм;
- отримання базового рівня по програмуванню на мовах C, C++ з використанням простих типів даних, базових типів даних і масивів;
- вивчення структур даних в пам'яті і в файлах і алгоритмів роботи з ними з використанням мов C; C++;
- знайомство с основними принципами організації зберігання та пошуку даних, алгоритмами сортування та пошуку;
- придбання навиків використання базового набору фрагментів і алгоритмів у процесі розробки програм, навиків аналізу і “читання” програм;
- вивчення основ технології програмування та методів рішення обчислювальних задач і задач обробки символьних даних;
- формування рівня знань мови, який дозволяє вільно оперувати типами даних і змінними довільної складності, а також модульними алгоритмами їх обробки.

Згідно з вимогами освітньо-професійної програми здобувачі освіти повинні:

знати:

- сучасні методи і засоби розробки алгоритмів і програм на мовах C, C++;
- синтаксис і семантику основних конструкцій мов C, C++;
- способи організації складних структур даних (масиви, структури, списки, дерева), основні методи представлення і алгоритми обробки цих даних;
- особливості роботи з файлами у мові C;
- особливості технології розробки програм складної структури на мові C;

вміти:

- РН04. Використовувати знання математичних методів на рівні, необхідному для розв'язання типових задач програмної інженерії;
- РН05. Розробляти та супроводжувати програмне забезпечення;
- РН10. Обирати та застосовувати ефективні методи оптимізації алгоритмів;
- РН17. Вміння демонструвати процеси та результати професійної діяльності, шляхом публічних презентацій та розробляючи пакети проектної документації, презентації, звіти.

Сформовані компетенції

Загальні компетентності:

- ЗК04. Здатність спілкуватися іноземною мовою;
- ЗК05. Знання та розуміння предметної області та розуміння професійної діяльності;
- ЗК07. Здатність застосовувати знання у практичних ситуаціях;

Спеціальні (фахові) компетентності спеціальності:

- СК01. Здатність алгоритмічно та логічно мислити.
- СК02. Здатність вдосконалювати знання і навички в галузі інформаційних технологій та усвідомлення важливості навчання протягом усього життя.
- СК03. Здатність застосовувати теоретичні та емпіричні знання для розроблення, тестування, впровадження та супроводу програмного забезпечення.

- СК07. Здатність розробляти модулі і компоненти програмного забезпечення за допомогою типових алгоритмів та інструментів.
- СК08. Здатність забезпечувати інформаційну та функціональну безпеку програмного забезпечення.
- СК-9. Уміння готувати та презентувати документацію та методичні матеріали щодо програмного забезпечення.
- СК-16. Вміння з оформлення пакету проєктної документації, підготовки презентаційних матеріалів, публічного захисту проєктів та самопрезентації.

Форма підсумкового контролю успішності навчання:

- ☐ залік
- ☒ екзамен

На вивчення навчальної дисципліни відводиться 210 годин 7 кредити ECTS.

2. ІНФОРМАЦІЙНИЙ ОБСЯГ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

Змістовий модуль 1. Основні поняття та означення

Тема 1.1. Мова програмування, інтегроване середовище розробки. Історія і класифікація мов програмування .

Тема 1.2. Компілятор, інтерпретатор, компонувальник. Структура і способи опису мов програмування високого рівня.

Змістовий модуль 2. Елементи мови програмування

Тема 2.1. Вступ в програмування. Найпростіші програми.

Тема 2.2. Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).

Тема 2.3. Арифметичні вирази. Формати для виводу даних.

Змістовий модуль 3. Керування порядком обчислень

Тема 3.1. Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).

Тема 3.2. Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.)

Змістовий модуль 4. Процедурно-орієнтоване програмування . Функції користувача

Тема 4.1. Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).

Тема 4.2. Структура програм. Складові частини програми. Глобальні та локальні змінні.

Змістовий модуль 5. Основи алгоритмізації

Тема 5.1. Алгоритми і величини. Лінійні обчислювальні алгоритми

Тема 5.2. Розгалуження і цикли в обчислювальних алгоритмах. Допоміжні алгоритми і процедури.

Змістовий модуль 6. Масиви

Тема 6.1. Одновимірні масиви

Тема 6.2. Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.

Тема 6.3. Багатовимірні масиви

Тема 6.4. Масиви символьних строк. Оголошення і ініціалізація. Ввід-вивід. Сортунанн

Змістовий модуль 7. Структури

Тема 7.1. Структури.

Тема 7.2. Масиви структур.

Змістовий модуль 8. Показчики

Тема 8.1. Оголошення і ініціалізація змінної показчика

Тема 8.2. Масиви показчиків. Показчики на функцію.

Змістовий модуль 9. Робота з файлами

Тема 9.1. Робота з текстовими файлами.

Тема 9.2. Створення файлу послідовного доступу.

Тема 9.3. Створення файлу довільного доступу

Тема 9.4. Робота з двійковими файлами.

Змістовий модуль 10. Динамічні структури даних

Тема 10.1. Списки. Зв'язаний список.

Тема 10.2. Стеки, черги, деки.

Тема 10.3. Деревя

Тема 10.4. Граф

3. РОЗПОДІЛ ГОДИН ЗА ВИДАМИ ЗАНЯТЬ ВІДПОВІДНО ДО НАВЧАЛЬНОГО ПЛАНУ

№ заняття	Назва теми	Кількість годин				
		Лекції	Семінарські та практичні заняття	Лабораторні роботи	Самостійне вивчення	Всього
Змістовий модуль 1. Основні поняття та означення						
1	Тема 1.1. Мова програмування, інтегроване середовище розробки. Історія і класифікація мов програмування .	2			6	8
2	Тема 1.2. Компілятор, інтерпретатор, компонувальник. Структура і способи опису мов програмування високого рівня.		2		6	8
Разом за змістовим модулем 1		2	2		12	16
Змістовий модуль 2. Елементи мови програмування						
3/4	Тема 2.1. Вступ в програмування. Найпростіші програми.	2	2		6	10
5/6/7	Тема 2.2. Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).	2	4		6	12
8/9	Тема 2.3. Арифметичні вирази. Формати для виводу даних.		4		6	10
Разом за змістовим модулем 2		4	10		18	32
Змістовий модуль 3. Керування порядком обчислень						
10/11/12	Тема 3.1. Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).	2	4		6	12
13/14/15/ 16	Тема 3.2. Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.)	2	6		6	14
17	Підведення підсумків за перший семестр	2				2
Разом за змістовим модулем 3		6	10		12	28
Разом за перший семестр		12	22		42	76
Змістовий модуль 4. Процедурно-орієнтоване програмування . Функції користувача						
18/19	Тема 4.1. Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).	2	2		4	8
20/21	Тема 4.2. Структура програм. Складові частини програми. Глобальні та локальні змінні.	2	2		4	8
Разом за змістовим модулем 4		4	4		8	16
Змістовий модуль 5. Основи алгоритмізації						
22/23	Тема 5.1. Алгоритми і величини. Лінійні обчислювальні алгоритми	2	2		4	8
24/25	Тема 5.2. Розгалуження і цикли в обчислювальних		4		4	8

№ заняття	Назва теми	Кількість годин				
		Лекції	Семінарські та практичні заняття	Лабораторні роботи	Самостійне вивчення	Всього
	алгоритмах. Допоміжні алгоритми і процедури.					
Разом за змістовим модулем 5		2	6		8	16
Змістовий модуль 6. Масиви						
26/27	Тема 6.1. Одновимірні масиви	2	2		4	8
28/29	Тема 6.2. Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.	2	2		4	8
30/31	Тема 6.3. Багатовимірні масиви	2	2		4	8
32	Тема 6.4. Масиви символьних строк. Оголошення і ініціалізація. Ввід-вивід. Сортюванн		2		4	6
Разом за змістовим модулем 6		6	8		16	30
Змістовий модуль 7. Структури						
33/34	Тема 7.1. Структури.	2	2		4	8
35/36/37	Тема 7.2. Масиви структур.	2	4		4	10
Разом за змістовим модулем 7		4	6		8	18
Змістовий модуль 8. Показчики						
38/39	Тема 8.1. Оголошення і ініціалізація змінної показчика	2	2		4	8
40/41	Тема 8.2. Масиви показчиків. Показчики на функцію.	2	2		2	6
Разом за змістовим модулем 8		4	4		6	14
Змістовий модуль 9. Робота з файлами						
42/43	Тема 9.1. Робота з текстовими файлами.	2	2		2	6
44	Тема 9.2. Створення файлу послідовного доступу.		2		2	4
45	Тема 9.3. Створення файлу довільного доступу		2		2	4
46	Тема 9.4. Робота з двійковими файлами.		2		2	4
Разом за змістовим модулем 9		2	8		8	18
Змістовий модуль 10. Динамічні структури даних						
47/48	Тема 10.1. Списки. Зв'язаний список.	2	2		2	6
49/50	Тема 10.2. Стеки, черги, деки.	2	2		2	6
51/52	Тема 10.3. Дерева	2	2		2	6
53	Тема 10.4. Графи		2		2	4
Разом за змістовим модулем 10		6	8		8	22
Разом за другим семестр		28	44		62	134
ВСЬОГО		40	66		104	210

4. РЕКОМЕНДОВАНА ЛІТЕРАТУРА

1. Белов Ю.А. Вступ до програмування мовою С++. / Ю.А. Белов, Т.О. Карнаух, Ю.В. Коваль, А.Б. Ставровський. – К.: Видавничо-поліграфічний центр «Київський університет», 2012. – 175 с.
2. Вступ до програмування мовою С++. Організація даних / Т. О. Карнаух, Ю. В. Коваль, М. В. Потієнко, А. Б. Ставровський. – К.: ВПЦ "Київський університет", 2015.
3. Пекарський Б. Г. Основи програмування [Текст]: навчальний посібник К. : Кондор, 2008.
4. Ковалюк Т.В. Основи програмування. – К.: Видавнича група ВНУ, 2005.

Додаткова

5. Вступ до програмування мовою С++. Організація даних / Т. О. Карнаух, Ю. В. Коваль, М. В. Потієнко, А. Б. Ставровський. . К.: ВПЦ "Київський університет", 2015.
6. Вступ до програмування мовою С++. Структури даних / Р. А. Веклич, Т. О. Карнаух, А. Б. Ставровський. . К.: ВПЦ "Київський університет", 2018.

Інформаційні ресурси

7. Програмування: С/С++/С#. [Електронний ресурс]. – Режим доступу: <http://citforum.ru/programming/c.shtml>
8. International Standard ISO/IEC 14882:2014(E) – Programming Language C++. [Електронний ресурс]. – Режим доступу: <https://isocpp.org/std/the-standard>
9. Підручник для початківців що вивчають Microsoft Visual C++. [Електронний ресурс]. – Режим доступу: http://chesworth.com/pv/languages/c/visual_cpptutorial.htm

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Циклова комісія «Дисциплін програмної інженерії»

ЗАТВЕРДЖУЮ

Заступник директора

з навчальної роботи

_____ **Олена БАБИЧ**

« 31 » серпня 2023 року

РОБОЧА ПРОГРАМА НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

ПРОГРАМУВАННЯ

(назва навчальної дисципліни)

галузь знань: 12 Інформаційні технології

спеціальність: 121 Інженерія програмного забезпечення

спеціалізація: Розробка програмного забезпечення

відділення: Комп'ютерних технологій

Робоча програма дисципліни «ПРОГРАМУВАННЯ» для здобувачів фахової передвищої освіти 2 курсу

спеціальності: 121 Інженерія програмного забезпечення

спеціалізації: Розробка програмного забезпечення

« 25 » серпня 2023 року

Розробник (-и): викладач спеціаліст другої кваліфікаційної категорії Владислав ОЛІЙНИК

Робоча програма розглянута та схвалена на засіданні циклової комісії дисциплін програмної інженерії

Протокол від «28» серпня 2023 року № 1

Голова циклової комісії _____ Олександр БАБИЧ

1. ОПИС НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

Найменування показників					Галузь знань, спеціальність, спеціалізація, освітньо-професійний ступінь	Характеристика навчальної дисципліни
Кількість кредитів ECTS – 7					галузь знань: 12 Інформаційні технології	Форма навчання: ☒ Денна ☐ Заочна
Модулів – 1						
Змістових модулів –10					спеціальність: 121 Інженерія програмного забезпечення	Вид дисципліни ☒ Нормативна ☐ Вибіркова
Загальна кількість – 210 год. аудиторних – 106 год. самостійної роботи – 44						Вид контролю: ☒ Іспит ☐ Диференційований залік
Се ме ст р	Ле кц ії	П Р/ Се мі на рс ьк і	Л Р	Са мо сті йн а ро бо та	спеціалізація: Розробка програмного забезпечення	
I						
II						
III	12	22		42		
IV	28	44		62		
V						
VI						
VII						
VIII						
					Освітньо-професійний ступінь – фаховий молодший бакалавр	

2. МЕТА ТА ЗАВДАННЯ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

Метою викладання навчальної дисципліни «Програмування» є: вивчення методів алгоритмізації, основ програмування на алгоритмічних мовах високого рівня і використання отриманих навиків при вирішенні задач різних типів.

Основними завданнями вивчення дисципліни «Програмування» є:

- формування базових знань по алгоритмізації та програмуванню про стиль написання програм, про раціональні методи їх розробки и оптимізації, про стратегії відладки і тестування програм;
- отримання базового рівня по програмуванню на мовах C, C++ з використанням простих типів даних, базових типів даних і масивів;
- вивчення структур даних в пам'яті і в файлах і алгоритмів роботи з ними з використанням мов C; C++;
- знайомство с основними принципами організації зберігання та пошуку даних, алгоритмами сортування та пошуку;
- придбання навиків використання базового набору фрагментів і алгоритмів у процесі розробки програм, навиків аналізу і “читання” програм;
- вивчення основ технології програмування та методів рішення обчислювальних задач і задач обробки символьних даних;
- формування рівня знань мови, який дозволяє вільно оперувати типами даних і змінними довільної складності, а також модульними алгоритмами їх обробки.

Згідно з вимогами освітньо-професійної програми здобувачі освіти повинні:

знати:

- сучасні методи і засоби розробки алгоритмів і програм на мовах C, C++;
- синтаксис і семантику основних конструкцій мов C, C++;
- способи організації складних структур даних (масиви, структури, списки, дерева), основні методи представлення і алгоритми обробки цих даних;
- особливості роботи з файлами у мові C;
- особливості технології розробки програм складної структури на мові C;

вміти:

- РН04. Використовувати знання математичних методів на рівні, необхідному для розв'язання типових задач програмної інженерії;
- РН05. Розробляти та супроводжувати програмне забезпечення;
- РН10. Обирати та застосовувати ефективні методи оптимізації алгоритмів;
- РН17. Вміння демонструвати процеси та результати професійної діяльності, шляхом публічних презентацій та розробляючи пакети проектної документації, презентації, звіти.

Сформовані компетенції

Загальні компетентності:

- ЗК04. Здатність спілкуватися іноземною мовою;
- ЗК05. Знання та розуміння предметної області та розуміння професійної діяльності;
- ЗК07. Здатність застосовувати знання у практичних ситуаціях;

Спеціальні (фахові) компетентності спеціальності:

- СК01. Здатність алгоритмічно та логічно мислити.
- СК02. Здатність вдосконалювати знання і навички в галузі інформаційних технологій та усвідомлення важливості навчання протягом усього життя.
- СК03. Здатність застосовувати теоретичні та емпіричні знання для розроблення, тестування, впровадження та супроводу програмного забезпечення.
- СК07. Здатність розробляти модулі і компоненти програмного забезпечення за допомогою типових алгоритмів та інструментів.
- СК08. Здатність забезпечувати інформаційну та функціональну безпеку програмного забезпечення.
- СК-9. Уміння готувати та презентувати документацію та методичні матеріали щодо програмного забезпечення.
- СК-16. Вміння з оформлення пакету проектної документації, підготовки презентаційних матеріалів, публічного захисту проєктів та самопрезентації.

3. КРИТЕРІЇ ОЦІНЮВАННЯ

Оцінювання знань, вмінь та навичок здобувачів освіти здійснюється за:

- ☒ Чотирьохбальною шкалою
- ☐ Дванадцятибальною шкалою

Оцінювання знань, вмінь та навичок здобувачів освіти за дванадцятибальною шкалою здійснюється у відповідності до наказу МОН України №329 від 13.04.2011р.(Про затвердження критеріїв оцінювання навчальних досягнень учнів (вихованців) у системі загальної середньої освіти)

Оцінювання знань, вмінь та навичок здобувачів освіти за чотирьохбальною шкалою:

5 "відмінно" – здобувач освіти володіє навчальним матеріалом у повному обсязі (міцно засвоїв увесь програмний матеріал, виявив глибоке його розуміння, вичерпно відповів і обґрунтував власні висновки, прийняв обґрунтоване рішення і вміло використав на практиці, упевнено виконав завдання);

4. "добре" – здобувач освіти засвоїв навчальний матеріал на достатньо високому рівні (загалом знає весь програмний матеріал, на питання відповідає вільно, але недостатньо широко, правильно використовує свої знання на практиці тощо);

3 "задовільно" – здобувач освіти загалом засвоїв основний навчальний матеріал, оперує ним недостатньо чітко та упевнено, слабо визначає зв'язки й відносини між предметами і явищами (виявляє знання тільки основного матеріалу, передбаченого програмою, спроможний використовувати свої знання на практиці, правильно виконує прийоми і дії та ін.);

2 "незадовільно" – здобувач освіти загалом має поверхове уявлення про основний навчальний матеріал, не може ним оперувати.

4. ІНФОРМАЦІЙНИЙ ОБСЯГ НАВЧАЛЬНОЇ ДИСЦИПЛІНИ

Змістовий модуль 1. Основні поняття та означення

Тема 1.1. Мова програмування, інтегроване середовище розробки. Історія і класифікація мов програмування

Тема 1.2. Компілятор, інтерпретатор, компонувальник. Структура і способи опису мов програмування високого рівня.

Змістовий модуль 2. Елементи мови програмування

Тема 2.1. Вступ в програмування. Найпростіші програми.

Тема 2.2. Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).

Тема 2.3. Арифметичні вирази. Формати для виводу даних.

Змістовий модуль 3. Керування порядком обчислень

Тема 3.1. Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).

Тема 3.2. Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.)

Змістовий модуль 4. Процедурно-орієнтоване програмування . Функції користувача

Тема 4.1. Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).

Тема 4.2. Структура програм. Складові частини програми. Глобальні та локальні змінні.

Змістовий модуль 5. Основи алгоритмізації

Тема 5.1. Алгоритми і величини. Лінійні обчислювальні алгоритми

Тема 5.2. Розгалуження і цикли в обчислювальних алгоритмах. Допоміжні алгоритми і процедури.

Змістовий модуль 6. Масиви

Тема 6.1. Одновимірні масиви

Тема 6.2. Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.

Тема 6.3. Багатовимірні масиви

Тема 6.4. Масиви символьних строк. Оголошення і ініціалізація. Ввід-вивід. Сортування

Змістовий модуль 7. Структури

Тема 7.1. Структури.

Тема 7.2. Масиви структур.

Змістовий модуль 8. Показчики

Тема 8.1. Оголошення і ініціалізація змінної показчика

Тема 8.2. Масиви показчиків. Показчики на функцію.

Змістовий модуль 9. Робота з файлами

Тема 9.1. Робота з текстовими файлами.

Тема 9.2. Створення файлу послідовного доступу.

Тема 9.3. Створення файлу довільного доступу

Тема 9.4. Робота з двійковими файлами.

Змістовий модуль 10. Динамічні структури даних

Тема 10.1. Списки. Зв'язаний список.

Тема 10.2. Стеки, черги, деки.

Тема 10.3. Деревя

Тема 10.4. Граф

5. РОЗПОДІЛ ГОДИН ЗА ВИДАМИ ЗАНЯТЬ ВІДПОВІДНО ДО НАВЧАЛЬНОГО ПЛАНУ

№ заняття	Назва теми	Кількість годин				
		Лекції	Семінарські та практичні заняття	Лабораторні роботи	Самостійне вивчення	Всього
Змістовий модуль 1. Основні поняття та означення						
1	Тема 1.1. Мова програмування, інтегроване середовище розробки. Історія і класифікація мов програмування .	2			6	8
2	Тема 1.2. Компілятор, інтерпретатор, компонувальник. Структура і способи опису мов програмування високого рівня.		2		6	8
Разом за змістовим модулем 1		2	2		12	16
Змістовий модуль 2. Елементи мови програмування						
3/4	Тема 2.1. Вступ в програмування. Найпростіші програми.	2	2		6	10
5/6/7	Тема 2.2. Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).	2	4		6	12
8/9	Тема 2.3. Арифметичні вирази. Формати для виводу даних.		4		6	10
Разом за змістовим модулем 2		4	10		18	32
Змістовий модуль 3. Керування порядком обчислень						
10/11/12	Тема 3.1. Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).	2	4		6	12
13/14/15/ 16	Тема 3.2. Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.)	2	6		6	14
17	Підведення підсумків за перший семестр	2				2
Разом за змістовим модулем 3		6	10		12	28
Разом за перший семестр		12	22		42	76
Змістовий модуль 4. Процедурно-орієнтоване програмування . Функції користувача						
18/19	Тема 4.1. Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).	2	2		4	8
20/21	Тема 4.2. Структура програм. Складові частини програми. Глобальні та локальні змінні.	2	2		4	8
Разом за змістовим модулем 4		4	4		8	16
Змістовий модуль 5. Основи алгоритмізації						
22/23	Тема 5.1. Алгоритми і величини. Лінійні обчислювальні алгоритми	2	2		4	8
24/25	Тема 5.2. Розгалуження і цикли в обчислювальних алгоритмах. Допоміжні алгоритми і процедури.		4		4	8

№ заняття	Назва теми	Кількість годин				
		Лекції	Семінарські та практичні заняття	Лабораторні роботи	Самостійне вивчення	Всього
Разом за змістовим модулем 5		2	6		8	16
Змістовий модуль 6. Масиви						
26/27	Тема 6.1. Одновимірні масиви	2	2		4	8
28/29	Тема 6.2. Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.	2	2		4	8
30/31	Тема 6.3. Багатовимірні масиви	2	2		4	8
32	Тема 6.4. Масиви символьних строк. Оголошення і ініціалізація. Ввід-вивід. Сортунанн		2		4	6
Разом за змістовим модулем 6		6	8		16	30
Змістовий модуль 7. Структури						
33/34	Тема 7.1. Структури.	2	2		4	8
35/36/37	Тема 7.2. Масиви структур.	2	4		4	10
Разом за змістовим модулем 7		4	6		8	18
Змістовий модуль 8. Показчики						
38/39	Тема 8.1. Оголошення і ініціалізація змінної показчика	2	2		4	8
40/41	Тема 8.2. Масиви показчиків. Показчики на функцію.	2	2		2	6
Разом за змістовим модулем 8		4	4		6	14
Змістовий модуль 9. Робота з файлами						
42/43	Тема 9.1. Робота з текстовими файлами.	2	2		2	6
44	Тема 9.2. Створення файлу послідовного доступу.		2		2	4
45	Тема 9.3. Створення файлу довільного доступу		2		2	4
46	Тема 9.4. Робота з двійковими файлами.		2		2	4
Разом за змістовим модулем 9		2	8		8	18
Змістовий модуль 10. Динамічні структури даних						
47/48	Тема 10.1. Списки. Зв’язаний список.	2	2		2	6
49/50	Тема 10.2. Стеки, черги, деки.	2	2		2	6
51/52	Тема 10.3. Дерева	2	2		2	6
53	Тема 10.4. Графи		2		2	4
Разом за змістовим модулем 10		6	8		8	22
Разом за другий семестр		28	44		62	134
ВСЬОГО		40	66		104	210

6. МЕТОДИ НАВЧАННЯ

Словесні методи навчання

- ☐ Розповіді
- ☒ Пояснення
- ☐ Дискусія
- ☒ Лекції
- ☒ Лекції з використанням мультимедійних презентацій
- ☐ Бесіди
- ☐ Евристичні
- ☐ Репродуктивні

Методи навчання за джерелами знань

- ☒ Метод роботи з книгою
- ☐ Питання
- ☐ Переказ
- ☐ Виписування
- ☐ Складання плану
- ☐ Складання таблиць, схем та ін.

Наочні методи навчання

- ☒ Демонстрації
- ☒ Ілюстрації
- ☐ Спостереження

Практичні методи навчання

- ☐ Лабораторна робота
- ☒ Практична робота
- ☒ Виконання вправ
- ☒ Самостійна робота
- ☐ Проблемно-пошукові роботи
- ☐ Розгляд творчо-аналітичних завдань
- ☒ Робота в групах
- ☒ Розв'язування ситуаційних завдань

7. ЗАСОБИ ОЦІНЮВАННЯ (МЕТОДИ КОНТРОЛЮ)

Засоби оцінювання:

- ☒ Екзамен
- ☐ Диференційований залік
- ☒ Підсумковий тест
- ☒ Курсова робота
- ☐ Курсовий проект
- ☐ Командний проект
- ☐ Комплект звітів про виконання лабораторних /практичних робіт
- ☐ Реферат, есе
- ☐ Розрахункова / розрахунково-графічна робота
- ☐ Презентація результатів виконаних завдань та досліджень
- ☐ Виконання завдання на лабораторному обладнанні

Методи контролю:

Поточний:

- ☒ Усне опитування
- ☐ Фронтальне
- ☐ Групове
- ☒ Індивідуальне
- ☒ Комбіноване
- ☐ _____

Письмовий контроль:

- ☒ Самостійна робота
- ☒ Тести
- ☐ Вправи
- ☐ Диктанти
- ☐ Реферати
- ☒ Презентації
- ☐ Звіти
- ☐ _____

Періодичний контроль:

☐ Підсумкові письмові контрольні роботи за змістовими модулями

☒ Тематичні роботи

☐ _____

Підсумковий контроль:

☒ Екзамен

Диференційований залік

☐ Захист курсового проекту

☒ Захист курсової роботи

☐ Захист розрахунково-графічної роботи

☐ _____

8. МЕТОДИЧНЕ ЗАБЕЗПЕЧЕННЯ

- ☒ Конспект лекцій
- ☒ Презентації лекцій
- ☐ Роздатковий матеріал
- ☐ Таблиці, схеми, ментальні карти
- ☐ Електронний підручник з дисципліни
- ☒ Програмне забезпечення навчального призначення
- ☐ Пакет завдань для контрольної роботи
- ☒ Пакет завдань для самостійної роботи
- ☒ Комплект тестових завдань
- ☐ Комплексна контрольна робота
- ☒ Екзаменаційні білети
- ☒ Методичні вказівки до
 - ☐ Семінарських занять
 - ☒ Практичних занять
 - ☐ Лабораторних занять
 - ☒ Самостійної та індивідуальної роботи

9. РЕКОМЕНДОВАНА ЛІТЕРАТУРА

Основна

1. Белов Ю.А. Вступ до програмування мовою С++. / Ю.А. Белов, Т.О. Карнаух, Ю.В. Коваль, А.Б. Ставровський. – К.: Видавничо-поліграфічний центр «Київський університет», 2012. – 175 с.
2. Вступ до програмування мовою С++. Організація даних / Т. О. Карнаух, Ю. В. Коваль, М. В. Потієнко, А. Б. Ставровський. – К.: ВПЦ "Київський університет", 2015.
3. Пекарський Б. Г. Основи програмування [Текст]: навчальний посібник К. : Кондор, 2008.
4. Ковалюк Т.В. Основи програмування. – К.: Видавнича група BHV, 2005.

Додаткова

5. Вступ до програмування мовою С++. Організація даних / Т. О. Карнаух, Ю. В. Коваль, М. В. Потієнко, А. Б. Ставровський. . К.: ВПЦ "Київський університет", 2015.
6. Вступ до програмування мовою С++. Структури даних / Р. А. Веклич, Т. О. Карнаух, А. Б. Ставровський. . К.: ВПЦ "Київський університет", 2018.

Інформаційні ресурси

7. Програмування: С/С++/С#. [Електронний ресурс]. – Режим доступу: <http://citforum.ru/programming/c.shtml>
8. International Standard ISO/IEC 14882:2014(E) – Programming Language C++. [Електронний ресурс]. – Режим доступу: <https://isocpp.org/std/the-standard>
9. Підручник для початківців що вивчають Microsoft Visual C++. [Електронний ресурс]. – Режим доступу: http://chesworth.com/pv/languages/c/visual_cpptutorial.htm

КОРЕКЦІЯ

Навчальний рік 2021-2022

Аудиторні год. (за семестр)	Зняття	До виконання	Корекція заняття (об'єднання тем, винесення на самостійне опрацювання)	Підпис

Навчальний рік 2022-2023

Аудиторні год. (за семестр)	Зняття	До виконання	Корекція заняття (об'єднання тем, винесення на самостійне опрацювання)	Підпис

Навчальний рік 2023-2024

Аудиторні год. (за семестр)	Зняття	До виконання	Корекція заняття (об'єднання тем, винесення на самостійне опрацювання)	Підпис

ПЛАНІ-КОНСПЕКТИ ЛЕКЦІЙНИХ ЗАНЯТЬ

Змістовий модуль 1. Основні поняття та означення(3 семестр)

Тема 1.1 Мова програмування, інтегроване середовище розробки. Історія і класифікація мов програмування

Навчальна мета: Ознайомити студентів з історією виникнення та принципами класифікації мов програмування; розкрити призначення та склад компонентів інтегрованого середовища розробки (IDE); навчити класифікувати мови за рівнями та парадигмами для свідомого вибору стеку технологій у майбутній професійній діяльності.

Розвиваюча мета полягає у формуванні аналітичного та системного мислення студентів через дослідження еволюційного шляху технологій програмування. Заняття спрямоване на розвиток здатності порівнювати різні парадигми та рівні абстракції, що дозволяє майбутнім фахівцям не просто вивчати синтаксис, а розуміти внутрішню логіку побудови програмних систем.

Виховна мета спрямована на формування професійної етики та культури майбутнього розробника, де особлива увага приділяється розумінню програмування як інтелектуального надбання людства. Вона передбачає виховання поваги до інтелектуальної праці та досвіду попередніх поколінь програмістів, чиї фундаментальні ідеї стали основою сучасної цифровізації.

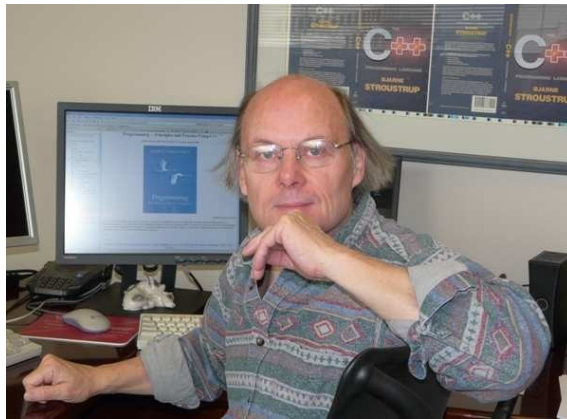
Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

ПЕРЕДМОВА

Процес оволодіння навиками програмування мовою C++ має багато спільного з науково-дослідною діяльністю. Пов'язано це з тим, що ця мова поєднує декілька технологій програмування – традиційну, тобто структурне програмування (представлене мовою C), об'єктно-орієнтоване програмування (представлене таким поняттям як клас, який підвищує потужність мови C++ порівняно з мовою C) і узагальнене програмування (програмування за допомогою шаблонів мови C++). Розроблена Б'ярном Страуструпом (англ. Bjarne Stroustrup¹) мова C++ постійно перебуває в стані розвитку, позаяк відбувається її доповнення новими функціональними можливостями, однак на даний час, з прийняттям ще у 1998 р. стандарту ISO/ANSI C++, специфікація та синтаксис мови стабілізувалися. Сучасні компілятори підтримують практично усі функції, дозволені цим стандартом, і більшість програмістів мали достатньо часу, щоб встигнути звикнути до них.



Б'ярн Страуструп

1. ІСТОРІЯ ВИНИКНЕННЯ МОВИ ПРОГРАМУВАННЯ C++

Мова програмування C++ – одна з найдосконаліших мов, яку має засвоїти потенційний програміст-початківець. Ця заява може видатися дуже серйозною, але вона – не перебільшення. Мова C++ – центр притягання, навколо якого "обертається" все сучасне програмування. Її синтаксис і принципи розроблення програм визначають суть об'єктно-орієнтованого програмування. Понад це, мова C++ втручала шлях для розроблення багатьох інших мов майбутнього. Наприклад, мови Java та C# – прямі "нащадки" мови C++. Її також можна назвати універсальною мовою програмування, оскільки вона дає змогу фаховим програмістам обмінюватися різноманітними ідеями. Сьогодні бути професійним програмістом високого класу означає бути компетентним у тонкощах мови C++, позаяк це ключ до оволодіння сучасними перспективними технологіями програмування.

Приступаючи до вивчення мови програмування C++, важливо знати, як вона вписується в історичний контекст мов програмування. Розуміючи, що привело до її створення, які принципи розроблення вона представляє і що вона успадкувала від своїх попередників, студентам буде легше оцінити суть новаторства і унікальність засобів мови програмування C++. Саме тому у цьому розділі спробуємо зробити короткий екскурс в історію створення мови програмування C++.

+, заглянути в її витоки, проаналізувати її взаємини з безпосереднім попередником, тобто мовою С, розглянути її області застосування та принципи програмування, які вона підтримує. Тут також студент дізнається, яке місце займає мова С++ серед інших мов програмування.

1.1.Витоки мови програмування С++

Історія розроблення мови програмування С++ починається з мови С, тобто її побудовано на фундаменті та синтаксисі мови С. Мова С++ насправді є надбудовою мови С, тобто усі компілятори С++-програм можна використовувати для компілювання С-програм. Мову С++ можна назвати розширеною та поліпшеною версією мови С, у якій реалізовано технологію об'єктоорієнтованого, узагальненого та процедурного програмування. Вона також містить ряд інших удосконалень мови С, наприклад, розширений набір бібліотечних функцій. Щоб до кінця зрозуміти і оцінити переваги мови програмування С++, необхідно зрозуміти все "як" і "чому" відносно мови С.

1.1.1. Причини створення мови програмування С

Поява мови С потрясла комп'ютерний світ. Її вплив не можна було недооцінити, оскільки вона докорінно змінила підхід до методології програмування і його сприйняття. Мова С стала вважатися першою сучасною "мовою програміста", оскільки до її винаходу комп'ютерні мови в основному розроблялися або як навчальні вправи, або як результат діяльності науководослідних структур. З появою мови програмування С все пішло інакше. Вона була задумана і розроблена реальними програмістами-практиками, яка відображала їх підхід до методології програмування. Її синтаксис і засоби були багато разів продумані, відточені та протестовані користувачами, які дійсно працювали з нею. Як наслідок, появилася мова, яку сприйняли багато програмістів-практиків. Вона швидко знайшла багато прихильників, які мали про неї найкращі враження, що сприяло широкому її використанню різними програмістами. Загалом мову програмування С було розроблено фаховими програмістами-практиками для програмістів-аматорів. Саме це і зумовило її шалений успіх як серед професійних програмістів, так і серед аматорів.

Мову програмування С винайшов Деніс Рітчи (Dennis Ritchie) для комп'ютера PDP-11 (розробка компанії DEC – Digital Equipment Corporation), який працював під управлінням операційної системи (ОС) UNIX. Мова С – результат тривалого робочого процесу, який спочатку був пов'язаний з іншою мовою – BCPL, створеною Мартіном Річардсом (Martin Richards). Мова BCPL індукувала появу мови, що отримала початкову назву В (її автор – Кен Томпсон (Ken Thompson)), після чого на початку 70-х років вона привела до розроблення мови С.

Впродовж багатьох років стандартом для мови програмування С де-факто слугувала мова, яку підтримувала ОС UNIX і описана в книзі Брайана Кернігана (Brian Kernighan) і Деніса Рітчи *The C Programming Language* (PrenticeHall, 1978). Проте формальна відсутність стандарту стала причиною розбіжностей між різними реалізаціями мови С. Щоб змінити ситуацію, на початку літа 1983 р. було започатковано комітет із створення ANSI-стандарту, покликаного – раз і назавжди визначити мову програмування С. Кінцеву версію цього стандарту було прийнято у грудні 1989 р., а його перше видання стало доступним для програмістів на початку 1990 р. Ця версія мови програмування С отримала назву C89, тому саме вона стала фундаментом для розроблення мови С++.

Мову програмування С часто називають комп'ютерною мовою середнього рівня. Однак це визначення не має негативного відтінку, оскільки воно зовсім не означає, що мова С є менш потужною та розвиненою (порівняно з іншими мовами "високого рівня") або її складно використовувати (подібно до мови Assembler¹). Мова С належить до середнього рівня тільки тому, що вона поєднує елементи мов високого рівня, таких як Pascal, Modula-2 або Visual Basic

з функціональними можливостями мови Assembler.

З погляду теорії програмування, у мову високого рівня закладено прагнення створити програмісту максимум зручностей у вигляді вбудованих засобів. Мова низького рівня не забезпечує програміста нічим, окрім доступу до реальних машинних команд. Мова середнього рівня надає програмісту певний (невеликий) набір інструментів, які дають змогу йому самому розробляти конструкції програм вищого рівня. Іншими словами, мова середнього рівня пропонує програмісту вбудовані можливості в поєднанні з її гнучкістю.

Будучи мовою середнього рівня, мова С дає змогу маніпулювати бітами, байтами і адресами, тобто базовими елементами, з якими працює комп'ютер. Таким чином, у мові С не передбачено спроби відокремити апаратні засоби комп'ютера від програмних. Наприклад, розмір цілочисельного значення у мові С безпосередньо пов'язаний з розміром слова центрального процесора. У більшості мов високого рівня існують вбудовані настанови, призначені для зчитування і запису дискових файлів. У мові С всі ці процедури виконуються за допомогою виклику бібліотечних функцій, а не за допомогою ключових слів, визначених у самій мові. Такий підхід підвищує функціональну гнучкість мови програмування С.

Мова С дає змогу програмісту (правильніше сказати, стимулює його) визначати підпрограми для виконання операцій високого рівня. Ці підпрограми називаються *функціями*, які є "будівельними" блоками мови С. Програміст може без особливих зусиль створити бібліотеку функцій, призначених для виконання різних задач, які має використовувати його програма. У цьому сенсі програміст може персоніфікувати мову С відповідно до своїх потреб.

Необхідно згадати ще про один аспект мови С, дуже важливий і для мови програмування С++: С – *структурована мова*, найхарактернішою особливістю якої є використання блоків. Блок – набір настанов, які логічно пов'язані між собою. Наприклад, уявіть собі настанову **if**, яка, при успішній перевірці свого виразу, має виконати п'ять окремих настанов. А якщо ці настанови спеціально згрупувати і звертатися до них як до єдиного цілого, то така група утворює блок.

Структурована мова підтримує концепцію функцій і локальних змінних. *Локальна змінна* – звичайна змінна, яка відома тільки функції, у якій вона визначена. Структурована мова також підтримує ряд таких циклічних конструкцій, як **while**, **do-while** і **for**. Проте використання настанови **goto** або забороняється, або не рекомендується, а також не несе того змістового навантаження щодо передачі керування, яке властиве їй у таких мовах програмування, як Basic або Fortran. Структурована мова дає змогу формалізувати код програми, тобто робити відступи під час написання настанов, і не вимагає строгої прив'язки до полів, як це реалізовано в ранніх версіях мови Fortran.

Нарешті (і це, можливо, найважливіше), мова С не несе відповідальності за некоректні дії програміста. Основний принцип використання мови С полягає у тому, щоб дати змогу програмісту робити все те, що він захоче, але за наслідки роботи програми (нехай навіть дуже незвичайні або зовсім підозрілі) відповідають не засоби мови, а програміст. Мова С надає програмісту практично повну владу над розробленою ним програмою та комп'ютером, який її виконує, але за наслідки такого володарювання він несе персональну відповідальність.

1.1.2 Передумови виникнення мови програмування С++

Наведена вище характеристика мови С може викликати справедливий подив: навіщо ж тоді, мовляв, було винайдено мову С++? Якщо мова програмування С – така хороша і корисна, то чому виникла потреба в чомусь ще новому? Виявляється, вся справа в складності написання досконалих кодів програм. Впродовж всієї історії розвитку програмування ускладнення кодів програм спонукало програмістів шукати шляхи, які б дали змогу справитися з тими труднощами, що виникали. Закладені можливості у мову програмування С++ можна вважати

одним із способів їх подолання. Спробуємо краще розкрити цей взаємозв'язок.

Ставлення до процесу програмування різко змінилося з моменту винаходу персонального комп'ютера. Основна причина – прагнення "приборкати" все зростаючу складність написання кодів програм. Наприклад, програмування для перших обчислювальних машин полягало в перемиканні тумблерів на передній панелі так, щоб їх положення відповідали двійковим кодам машинних команд. Доки величини програм не перевищували декількох сотень команд, такий метод ще мав право на існування. Але у міру їх подальшого зростання було винайдено мову низького рівня *Assembler*, яка дала змогу програмістам використовувати символічне представлення машинних команд. Оскільки розміри програм продовжували зростати, то бажання справитися з вищим рівнем їх складності викликало згодом появу мов програмування високого рівня, розроблення та використання яких дало змогу програмістам значно більше інструментів – нових і різних.

Першою широко поширеною мовою програмування була, звичайно ж, мова *Fortran*. Незважаючи на те, що це був дуже значний крок на шляху прогресу у області програмування, *Fortran* все ж таки важко назвати тією мовою, яка сприяла написанню яскравих і простих для розуміння програм. Шістдесяті роки XX ст. вважаються періодом появи технології структурного програмування. Саме її і було реалізовано у мові *C* та багатьох інших мовах. За допомогою структурованих мов можна було розробляти програми середньої складності, до того ж без особливих зусиль з боку програміста. Але, якщо програмний проект досягав певного розміру, то навіть з використанням згаданих структурних методів його складність різко зростала і виявлялася непосильною для можливостей навіть професійних програмістів. Коли (до кінця 1970-х) до "критичної" точки підійшло достатньо багато проектів, почали появлятися нові технології програмування, одна з яких отримала назву *об'єктно-орієнтованого програмування*¹ (ООП). Опанувавши методи ООП.

1.1.3 Поява мови програмування C++

Отже, мова програмування C++ з'явилася як відповідь на потребу подолати всезростаючу складність написання кодів програм складної структури. Вона була створена Б'ярном Страуструпом (Bjarne Stroustrup) в 1979 р. в компанії AT&T Bell Laboratories (м. Муррей-Хилл, шт. Нью-Джерси). Спочатку вона мала ім'я "C з класами" (C with Classes), але в 1983 р. її було перейменовано під назвою C++. У 1998 р. вона була стандартизована Міжнародною організацією стандартизації (МОС) під номером 14882:1998 – мова програмування C++. Згодом робоча група МОС почала працювати над новою версією стандарту під кодовою назвою C++09 (раніше відомою як C++0X), який мав вийти в 2009 р. під загальною назвою "ISO/IEC 14883(2009): Programming Language C++".

Стандарт мови програмування C++ станом на 1998 рік складався з двох основних частин: ядра мови і стандартної бібліотеки. Стандартна бібліотека C++ увірала в себе бібліотеку шаблонів STL (Standard Template Library), що розроблялася одночасно із стандартом. Зараз назва STL офіційно не вживається, проте в колах програмістів, які програмують мовою C++, ця назва використовується для позначення частини стандартної бібліотеки, що містить визначення шаблонів, контейнерів, ітераторів¹, алгоритмів і функторів².

Стандарт мови програмування C++ містить нормативне посилання на стандарт мови *C* від 1990 р. і не визначає самостійно ті функції стандартної бібліотеки, які було запозичено із стандартної бібліотеки мови *C*. Поза цим, існує величезна кількість бібліотек мови C++, котрі не входять в її стандарт, тобто можна успішно використовувати різні бібліотеки і з мови *C*.

Стандартизація визначила специфікацію та синтаксис мови програмування C++, проте за цією назвою можуть ховатися також неповні, обмежені до-стандартні варіанти мови. Спочатку мова розвивалася поза формальними рамками, спонтанно, у міру надходження тих завдань, що

ставилися перед нею. Розвиток мови супроводив розвиток крос-компілятора Cfront. Нововведення в мові відбивалися в зміні номера версії крос-компілятора, які розповсюджувалися і на саму мову. Проте але, стосовно теперішнього часу, то нумерація версії мови програмування C++ не ведеться.

Мова програмування C++ багато в чому є надбудовою мови C. Мова C++ містить такі нові можливості як: оголошення у вигляді виразів, перетворення типів у вигляді функцій, динамічні оператори **new** і **delete**, тип **bool**, посилання, розширене поняття константності та змінності, вбудовані функції, аргументи за замовчуванням, перевизначення функцій, простори імен, класи (включаючи і всі пов'язані з класами можливості, такі як успадкування, функції-члени (методи), віртуальні функції, абстрактні класи і конструктори), перевизначення операторів, шаблони, оператор '::', оброблення винятків, динамічну ідентифікацію і багато що інше. Мова програмування C++ є також мовою строгого типування даних і накладає більше вимог щодо дотримання типів даних, порівняно з мовою C.

У мові програмування C++ з'явилися коментарі у вигляді подвійної косої риски ("//"), які були в попереднику мови C – мові BCPL. Деякі особливості мови програмування C++ пізніше були перенесені в мову C, наприклад ключові слова **const** та **inline**, оголошення в циклах **for** і коментарі в стилі C++ ("//"). У пізніших реалізаціях мови програмування C також були представлені можливості, яких немає в мові C++, наприклад макроси `vararg` і покращена робота з масивами-параметрами.

Отже, мова програмування C++ повністю містить всі особливості мови C, всі її засоби і атрибути, володіє всіма її перевагами. Для неї також залишається у силі принцип функціонування мови C, згідно з яким програміст, а не мова, несе власну відповідальність за результати роботи своєї програми. Саме цей момент дає змогу зрозуміти, що процес розроблення мови C++ не був спробою створити нову мову програмування. Це було мабуть удосконалення вже наявної (і при цьому дуже успішної) мови.

Більшість нововведень, якими Б'ярн Страуструп збагатив мову C, було використано для підтримки технології ООП. По суті мова C++ стала об'єктно-орієнтованою версією мови C. Взявши мову C за основу, Б'ярн Страуструп підготував плавний перехід до ООП. Тепер, замість того, щоби вивчати абсолютно нову мову, C-програмісту достатньо було засвоїти тільки ряд нових засобів, і він міг пожинати плоди використання технології ООП. Проте в основу мови програмування C++ лягла не тільки мова C. Б'ярн Страуструп стверджує, що деякі об'єктно-орієнтовані засоби були інспіровані іншою об'єктно-орієнтованою мовою, а саме Simula67. Таким чином, мова C++ є симбіозом двох могутніх технологій програмування.

1.1.4 Етапи вдосконалення мови програмування C++

Історія розвитку мови програмування C++ містить такі ключові події:

- квітень 1979 – початок роботи над мовою C з класами (C with Classes);
- жовтень 1979 – робоча версія мови C з класами (Cpre);
- серпень 1983 – назва мови програмування C++ вперше використовується в компанії AT&T Bell Laboratories;
- 1984 – затверджена офіційна назва мови програмування C++;
- лютий 1985 – перший зовнішній випуск мови програмування C++ – Cfront Release E (Educational – випуск для навчальних закладів);
- жовтень 1985 – перший комерційний випуск – Cfront 1.0;
- лютий 1987 – другий комерційний випуск – Cfront 1.2;
- грудень 1987 – перший випуск GNU C++ (1.13);
- 1988 – перші випуски Oregon Software C++ і Zortech C++;
- червень 1989 – комерційний випуск Cfront 2.0;

- 1989 – вийшла у світ книга "The Annotated C++ Reference Manual" (ARM); засновано комітет ANSI C++;
- 1990 – перша технічна зустріч комітету ANSI C++; прийнято шаблони (**templates**), виняткові ситуації (**exceptions**); перший випуск Borland C++;
- 1991 – перша зустріч ISO; комерційний випуск Cfront 3.0 (з шаблонами); книга "The C++ Programming Language" (2-га редакція);
- 1992 – перші випуски IBM, DEC, Microsoft C++;
- 1993 – RTTI (Run-time type identification – визначення типу даних під час виконання) прийнято; простори назв (**namespaces**) і **string** (шаблонний за символьним типом) прийнято;
- 1994 – прийнято STL – Стандартна Бібліотека Шаблонів;
- 1996 – прийнято **export**;
- 1997 – остаточне голосування комітету за завершений стандарт C++;
- 1998 – ратифіковано стандарт ISO C++;
- 2003 – технічні поправки до стандарту; початок роботи над C++0x;
- 2005 – перше голосування за можливості стандарту C++0x; auto, static_assert, rvalue references прийняті в загальному використанні;
- 2006 – перше офіційне голосування за стандарт C++0x.

З моменту винаходу мова програмування C++ зазнала три значних вдосконалень, тобто щоразу вона як доповнювалася новими засобами, так і в чомусь змінювалася. Першій ревізії мова C++ була піддана в 1985 р., а другої вона зазнала в 1990 р. Третя ревізія відбулася в процесі її стандартизації, яка активізувалася на початку 1990-х. Спеціально для цього був сформований об'єднаний ANSI/ISO-комітет, який 25 січня 1994 р. прийняв перший проект запропонованого на розгляд стандарту. У цей проект були внесені всі засоби, вперше визначені Б'ярном Страуструпом, і додано нові. Але в цілому він відображав стан мови програмування C++ на той момент часу.

Незабаром, після завершення роботи над першим проектом стандарту мови програмування C++, відбулася подія, яка примусила значно розширити наявний стандарт. Йдеться про створення Стандартної Бібліотеки Шаблонів (Standard Template Library – бібліотека STL). Як буде сказано пізніше, бібліотека STL – набір узагальнених функцій, які можна використовувати для оброблення даних. Вона достатньо велика за розміром. Комітет ANSI/ISO проголосував за внесення бібліотеки STL у специфікацію мови програмування C++. Додавання бібліотеки STL розширило сферу застосування засобів мови C++ далеко за межами початкового її визначення.

Необхідно зазначити, що стандартна бібліотека мови програмування C++ містить стандартну бібліотеку мови C з невеликими змінами, які роблять її більш відповідною мові C++. Інша велика частина бібліотеки C++ базується на бібліотеці STL. Вона надає такі важливі інструменти, як контейнери (наприклад, вектори і списки) і ітератори (узагальнені покажчики), що надають доступ до цих контейнерів як до масивів. Окрім цього, бібліотека STL дає змогу аналогічно працювати і з іншими типами контейнерів, наприклад, асоціативними списками, стеками, чергами. Використовуючи шаблони, можна писати узагальнені алгоритми, здатні працювати з будь-якими контейнерами або послідовностями, доступ до членів яких забезпечують ітератори. Так само, як і в мові програмування C, можливості бібліотек активізуються використанням директиви #include для включення стандартних файлів. Всього в стандарті мови програмування C++ визначено 50 таких файлів.

Бібліотека STL до внесення її в стандарт мови програмування C++ була сторонньою розробкою, на початку – фірми HP, а потім SGI. Стандарт мови не називає її "STL", оскільки ця бібліотека стала невід'ємною частиною мови, проте багато програмістів до цих пір використовують цю назву, щоб відрізнити її від решти частини стандартної бібліотеки (потоків

введення/виведення (Iostream), підрозділ мови C тощо). Проект під назвою STLport, який базується на SGI STL, здійснює постійне оновлення STL, Iostream і рядкових класів. Деякі інші проекти також займаються розробленням приватних додатків стандартної бібліотеки для різних конструкторських завдань. Кожен виробник компіляторів мови програмування C++ обов'язково поставляє якусь реалізацію цієї бібліотеки, оскільки вона є дуже важливою частиною стандарту і широко використовується програмування.

Причиною успіху бібліотеки STL, зокрема її вхід до стандартної бібліотеки мови програмування C++, була направленість на широке коло виконуваних завдань і узагальнена структура. В цьому сенсі, близькою за духом бібліотеки STL, на сьогодні є бібліотека Boost, яка теж є бібліотекою загального застосування і теж впливає на формування стандартної бібліотеки мови C++.

Проте внесення бібліотеки STL, окрім іншого, уповільнило процес стандартизації мови програмування C++, причому достатньо істотно. Окрім бібліотеки STL, в саму мову програмування було додано декілька нових засобів і внесено багато дрібних змін. Тому версія мови C++ після розгляду комітетом із стандартизації стала набагато більша і складніша порівняно з початковим варіантом Б'ярна Страуструпа. Кінцевий результат роботи комітету датується 14 листопада 1997 р., а реально ANSI/ISO-стандарт мови C++ побачив світ у 1998 р. Саме ця специфікація мови програмування C++ зазвичай називається *Standard C++*, саме вона описана у цьому посібнику. Ця версія мови програмування C++ підтримується всіма основними C++-компіляторами, в т.ч. Visual C++ (Microsoft) і C++ Builder (Borland). Тому коди програм, наведені у цьому посібнику, повністю придатні для застосування в усіх сучасних C++-середовищах.

Мова програмування C++ продовжує розвиватися, щоб відповідати сучасним вимогам. Одна з науково-дослідних груп, що займається розвитком мови C++ в її сучасному вигляді і яка направляє комітету зі стандартизації поради щодо її поліпшення, – група Boost. Наприклад, один з напрямів діяльності цієї групи – вдосконалення можливостей мови шляхом залучення до неї особливостей метапрограмування.

1.2 Введення в алгоритми, їх класифікація та подання

Перше визначення. Алгоритмом називається система формальних правил, яка чітко та однозначно визначає процес вирішення поставленого завдання у вигляді кінцевої послідовності дій або операцій.

Друге визначення. Алгоритм – це є завершений набір чітких інструкцій по перетворенню інформації (команд), виконання яких призводить до досягнення поставленої мети. Кожна інструкція алгоритму містить точний опис деякої елементарної дії по перетворенню інформації, а також (в явному або неявному вигляді) вказівку на інструкцію, яку необхідно виконувати наступної.

Перша згадка терміну “алгоритм” зустрічається в працях середньоазіатського вченого **Мухаммеда бен Муси аль-Хорезмі** в IX столітті. Термін використовувався спочатку для визначення правил обчислень в десятковій позиційній системі числення.

Алгоритми повинні мати наступні властивості:

1. Дискретність – процес вирішення розбивається на кроки. Кожен крок – це одна дія або виклик підлеглому алгоритму.
2. Результативність – припускає доступність результату рішення задачі для користувача (друк результату на екран або у файл).
3. Визначеність або детермінованість – запис алгоритму повинен бути чітко визначений. Не допускається неоднозначність тлумачення запису алгоритму.
4. Кінцевість (фінітність) – алгоритм повинен призводити до вирішення завдання

обов'язково за кінцевий час.

5. Масовість – вірний результат за алгоритмом повинен бути отриманий на різній безлічі вхідних даних, які припустимі в конкретному завданні.

6. Ефективність – дозволяє вирішити задачу за прийнятний для розробника час при найменших обчислювальних витратах.

Залежно від мети, початкових умов завдання, шляхів його вирішення, визначення дій розробника, алгоритми поділяються на:

1. Керуючі (механічні) – задають певні дії, позначаючи їх в єдиній послідовності, що забезпечує однозначний результат. Наприклад, алгоритми роботи машин, верстатів, двигунів і т.п. Як правило дані для таких алгоритмів надходять від зовнішніх процесів, якими вони керують.

2. Обчислювальні – обробляють порівняно прості види даних, наприклад, числа, вектора, матриці.

3. Інформаційні алгоритми – це колекція простих процедур, що обробляють великі обсяги інформації. Наприклад, процедури виконують пошук необхідної числової або символічної інформації, що відповідає визначеним ознакам. Причому ефективність роботи таких алгоритмів залежить від організації даних.

Як правило серед обчислювальних та інформаційних алгоритмів можна виділити також ймовірнісні (стохастичні) та евристичні алгоритми.

Ймовірнісні – пропонують програму рішення задачі декількома шляхами або способами, що приводять до досягнення результату.

Евристичні – досягнення кінцевого результату однозначно не визначено, тому що не визначена вся послідовність дій. У таких алгоритмах використовуються універсальні логічні процедури та способи прийняття рішень, засновані на аналогії, асоціаціях і минулому досвіді вирішення схожих завдань. При реалізації евристичних алгоритмів велику роль відіграє інтуїція розробника.

У програмуванні алгоритми поділяються на три типи:

1. Лінійні – набір команд (вказівок), які виконуються послідовно один за одним.

2. Розгалужені (рос.: разветвленные) – містять хоча б одну перевірку умови, в результаті якої забезпечується перехід на один з можливих варіантів вирішення.

3. Циклічні – передбачають багаторазове повторення однієї і тієї ж дії або операцій над новими вхідними даними.

Способи подання алгоритмів:

1. Словесний – зміст етапів обчислень задається на природній мові в довільній формі з необхідною деталізацією.

2. Формульно-словесний – завдання інструкцій з використанням математичних символів виразів в поєднанні зі словесними поясненнями.

3. Блок-схемний – кожен етап процесу обробки даних представляється у вигляді геометричних фігур (блоків), які мають певну конфігурацію в залежності від характеру виконуваних операцій.

4. Псевдокод – сукупність операторів мови програмування та природної мови.

5. Структурні діаграми – використовуються в якості структурних блок-

схем для показу міжмодульних зв'язків, а також для відображення структур та систем обробки даних.

6. Мови програмування – фіксовані системи позначень для опису структур даних та алгоритмів, призначені для виконання обчислювальними машинами.

Приклад словесного способу. Завдання 1. Потрібно написати алгоритм обміну значеннями змінних А та В, в разі, якщо $A > B$.

Крок 1. Отримати значення А.

Крок 2. Отримати значення В.

Крок 3. Порівнюємо: якщо $A > B$, тоді виконуємо крок 4, інакше – переходимо до кроку

Крок 4. Зберігаємо значення В в тимчасовій змінній С.

Крок 5. Надаємо змінній В значення змінної А.

Крок 6. Надаємо змінній А значення змінної С.

Крок 7. Виводимо значення змінних А та В на екран. Крок 8.

Кінець алгоритму.

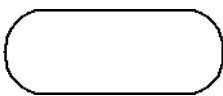
Приклад формульно-словесного способу. Завдання 2. Обчислити площу трикутника за трьома сторонами а, b, с.

Крок 1. Обчислити напівпериметр трикутника $p = (a+b+c)/2$. Крок 2.

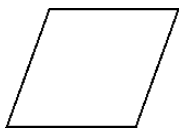
Обчислити площу трикутника $S = \sqrt{p(p-a)(p-b)(p-c)}$. Крок 3. Вивести на екран результат S та припинити обчислення.

При описі алгоритму блок-схемами використовують зображення безлічі стандартних блоків, детально описаних в міждержавному стандарті ГОСТ 19.701-90 «ЕСПД. Схеми алгоритмів, програм даних і систем. Умовні позначення і правила виконання». (ISO 5807: 1985 «Обработка информации. Символы, що застосовуються в документації, і позначення для блок-схем даних програм та систем, схем програмних мереж системних ресурсів» – «Information processing – Documentation symbols and conventions for data, program and system flowcharts, program network charts and system resources charts»).

Основними блоками вважаються наступні:



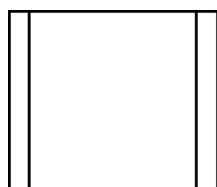
Термінатор – відображає вихід в зовнішнє середовище та вхід із зовнішнього середовища (початок або кінець схеми програми, зовнішнє використання та джерело або пункт призначення даних).



Дані – відображає дані, носій даних не визначено.

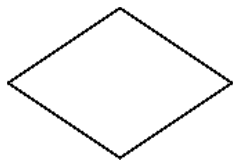


Процес – відображає функцію обробки даних будьякого виду (виконання певної операції або групи операцій, що приводить до зміни значення, форми або розміщення інформації або до визначення, за яким з декількох напрямків потоку слід рухатися).

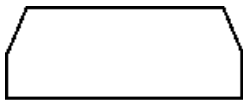


Попередньо визначений процес – процес, що складається з однієї або декількох операцій або кроків програми, визначених

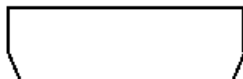
у іншому місці (в підпрограмі, модулі).



Рішення – відображає рішення або функцію перемикача типу, що має один вхід та ряд альтернативних виходів, один і тільки один з яких може бути активізований після обчислення умов, визначених всередині цього символу. Відповідні результати обчислення можуть бути записані по сусідству з лініями, що відображають ці шляхи.



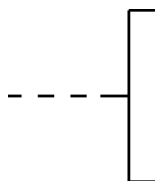
Межа циклу – складається з двох частин, що відображають початок і кінець циклу. Обидві частини символу мають один і той же ідентифікатор. Умови для ініціалізації, збільшення, завершення і т.д. поміщаються всередині символу на початку або в кінці в залежності від розташування операції, перевіряє умову.



Підготовка – відображає модифікацію команди або групи команд з метою впливу на деяку подальшу функцію (установка перемикача, модифікація індексного регістру або ініціалізація програми).



З'єднувач – відображає вихід в частину схеми та вхід з іншої частини цієї схеми й використовується для обриву лінії та продовження її в іншому місці. Відповідні символи-з'єднувачі повинні містити одне і теж унікальне позначення.



Коментар – використовують для додавання описових коментарів або пояснювальних записів з метою пояснення або приміток. Пунктирні лінії в символі пов'язані з відповідним символом або можуть обводити групу символів. Текст коментарів або приміток повинен бути поміщений біля обмежуючої фігури.

Введення в програмування та мови

Мова програмування – це певна знакова система для планування поведінки комп'ютера.

Комп'ютерна програма являє собою опис тієї послідовності дій, яку повинен виконати комп'ютер. Цей опис, з точки зору машини, виглядає як послідовність машинних команд, реалізація яких закладена в електронні схеми процесору комп'ютера.

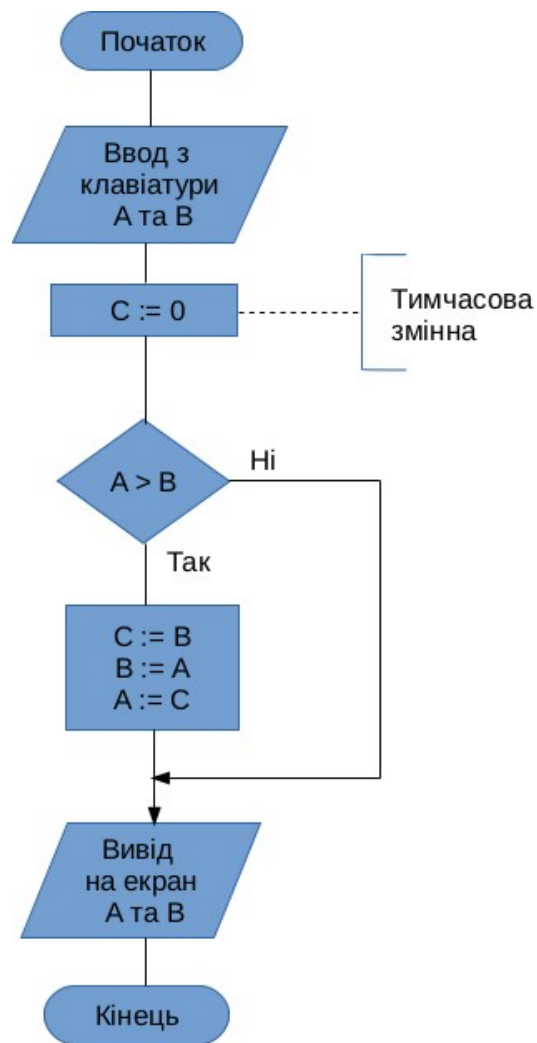


Рис. 1.3. Приклад подання алгоритму за допомогою блок-схем

Мови програмування високого рівня призначені насамперед для зручності та швидкості створення й розвитку програм програмістами. Вони не призначені для прямого виконання на процесорах комп'ютера. Для виконання на процесорах різних архітектур їх необхідно переводити до форми подання машинними командами. Кажуть, що перед виконанням такі програми повинні бути піддані трансляції.

Транслятором називають програму, яка в якості вхідних даних сприймає програми деякою вхідною мовою, а на виході формує еквівалентні за своєю функціональністю програми, але вже на іншій, так званій об'єктній мові (не має відношення до так званих об'єктних мов програмування високого рівня). Наприклад, Асемблером називають транслятор, у якого об'єктною мовою є деякий різновид машинної мови будь-якого апаратного комп'ютера, а вихідною мовою – символічне уявлення машинної мови. Вхідну мову зазвичай називають мовою асемблера. Найчастіше кожна команда мовою оригіналу перекладається в одну команду на об'єктній мові.

Відомі дві основні різновиди трансляторів: компілятори та інтерпретатори.

Компілятори спочатку повністю переводять весь текст програми з мови високого рівня в мову машинних команд для подальшого запуску отриманої програми. При цьому в подальшому користувачеві не потрібен вхідний код і він може багаторазово запускати отриману програму на комп'ютері.

До найбільш відомих мов програмування з компіляторами відносять, наприклад, C, C++, Objective-C, Pascal, Object Pascal, Fortran.

Першою високорівневою мовою програмування з компілятором, яка отримала широке застосування, був Fortran (IBM, 1957 рік). Мова була створена в період з 1954 по 1957 рік групою програмістів під керівництвом **Джона Бекуса** в корпорації IBM. Вона призначалася для наукових та технічних розрахунків.

Мова C була розроблена в 1972 році **Денісом Рітчі** та **Кеном Томпсоном** (AT&T Bell Telephone Laboratories).

Інтерпретатори обробляють текст коду програми на мові високого рівня і виконують його в міру обробки (читання). Переклад програми на машинну мову не запам'ятовується. Тому для багаторазового повторення виконання програми доводиться знову запускати її код через інтерпретатор.

До найбільш відомих мов програмування з інтерпретатором відносять, наприклад, Python, Ruby, Matlab.

Є мови програмування, для яких були розроблені як інтерпретатори, так і компілятори. Наприклад, мова програмування BASIC – свого часу був

розроблений інтерпретатор Altair BASIC (1975) для комп'ютера Altair 8800, а також серед багатьох реалізацій можна пригадати компілятор FreeBASIC (2004).

Зауважимо, що є мови, назви яких є акронімами, або скороченнями, а деякі отримали свою назву на честь певних світових вчених, або за назвою певної географічної місцевості. Прикладом мови програмування з назвою, що є акронімом, є все той же BASIC – це акронім від слів Beginner's All-purpose Symbolic Instruction Code (універсальний код символічних інструкцій для початківців). Назву цієї мови спрощено також називають як початкова, елементарна. Ще прикладом є мови Fortran – це скорочення від FORmula TRANslator (перекладач формул), а також ALGOL (ALGOrithmic Language).

Найвідоміші приклади мов програмування з назвами на честь вчених або географічних міст: Pascal (на честь Блеза Паскаля), Ada (на честь Августи Ади Кінг), Karel (на честь Карела Чапека, який придумав слово “робот”), Haskell (на честь американського математика Гаскелла Карпі), Kotlin (назва походить від Kotlin Island неподалік від Санкт-Петербургу).

Існує ряд сучасних мов програмування, які використовують компілятори для перекладу тексту коду мови високого рівня в проміжний, але не в машинний, а так званий проміжний байт-код, який в подальшому буде інтерпретований певною програмою, яку часто називають віртуальною машиною. Такі віртуальні машини для виконання проміжних байт-кодів збирають під певні апаратні платформи. Найбільш відомими представниками

цього сімейства мов, що використовують свої власні віртуальні машини виконання проміжних байт-кодів, є мови Java (використовує Java Virtual Machine) та C# (використовує Common Language Runtime).

Оскільки багато мов програмування розвиваються, то у багатьох існують версії, а також діалекти. наприклад:

- мова C – ANSI C, C99, C11 та ін.;
- мова C++ – C++ 98, C++03, C++11 та ін.;
- мова Fortran – FORTRAN 66, FORTRAN 77, Fortran 2003, Fortran 2008;
- мова Python – Python 2, Python 3. Та т.ін.

Окрему категорію займають сценарні (скриптові) мови програмування, які фактично належать до інтерпретаторів, але ставлять перед собою за мету дати можливість створювати програми, що використовують більш високорівневі готові програмні компоненти та конструкції для вирішення завдань в контекстах певних середовищ. Наприклад, в середовищі Web-браузера або в операційній системі (ОС). Прикладами можуть служити ECMAScript (приклади діалектів: JavaScript, ActionScript), VBScript, PHP, Perl, Bash, PowerShell. Багато фахівців також схильні відносити до цієї категорії мови Python та Ruby.

Запитання для контролю знань:

1. Хто вважається автором першої в історії програми для обчислювальної машини?
2. До якого покоління мов програмування належить мова C++?
3. Чим відрізняються мови низького рівня від мов високого рівня?
4. Для чого використовується дебаггер (debugger) в середовищі розробки?
5. Яку мову програмування найчастіше використовують для системного програмування (написання ядер ОС, драйверів) завдяки її близькості до апаратного забезпечення?

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).

Змістовий модуль 2. Елементи мови програмування

Тема 2.1. Вступ в програмування. Найпростіші програми.

Навчальна мета: є ознайомлення учнів із базовими принципами структурного програмування на мові C++ та принципами роботи компілятора для перетворення вихідного коду у виконуваний файл. Під час уроку передбачається опанування структури найпростішої програми, включаючи підключення бібліотек через препроцесор, роботу з головною функцією `int main()` та використання оператора виведення даних `std::cout`. У результаті навчання учні мають навчитися самостійно створювати та запускати базові консольні додатки у середовищі розробки, дотримуючись правил синтаксису мови, а також розвинути навички логічного проектування алгоритмів та культуру оформлення програмного коду.

Розвиваюча мета: Заняття має на меті активно розвивати логічне та аналітичне мислення учнів через виявлення причинно-наслідкових зв'язків у структурі коду C++, а також стимулювати формування алгоритмічного підходу до розв'язання задач, де будь-яка проблема розкладається на послідовність чітких інструкцій. Особлива увага приділяється розвитку когнітивної гнучкості під час пошуку та виправлення синтаксичних помилок, тренуванню концентрації та уважності до деталей (пунктуації та специфічних символів мови), а також формуванню навичок абстрактного мислення шляхом проектування віртуальних моделей реальних процесів у програмному середовищі.

Виховна мета: Процес написання коду має на меті виховувати у здобувачів освіти наполегливість, терплячість та дисциплінованість під час налагодження програм, а також формувати відповідальне ставлення до точності та грамотності виконання завдань. Заняття сприяє вихованню культури програмування через дотримання загальноприйнятих стандартів оформлення коду та поваги до інтелектуальної власності.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

1. Розроблення найпростішої C++-програми

Перш ніж заглибитись детально в теорію програмування, розглянемо спочатку роботу найпростішої C++-програми. Почнемо з введення коду програми, а потім перейдемо до її компілювання та виконання.

Код програми Демонстрація механізму роботи першої програми

```
// Програма №1 – перша C++-програма.  
#include <iostream> // Потокowe введення-виведення  
using namespace std; // Використання стандартного простору імен  
int main()           // Початок виконання програми.  
{
```

```
cout << "Це моя перша C++-програма.";
getch(); return 0;
} // Завершення роботи програми.
```

Отже, програміст повинен виконати такі дії: ввести код програми; скопіювати її; запустити на виконання.

Перш ніж приступати до виконання цих дій, необхідно визначити два терміни: початковий код і об'єктний код. *Початковий код* – версія програми, яку може читати людина. Наведений вище код програми – приклад початкового коду програми. Виконувана версія програми називається *об'єктним*, або *виконуваним кодом*.

Особливості введення коду програми:

Коди програм, які представлено у цьому навчальному посібнику, у разі потреби необхідно ввести вручну. У цьому випадку бажано використовувати будь-який текстовий редактор (наприклад, WordPad), а не текстовий процесор (Word processor). Йдеться про те, що під час введення коду програми мають бути створені виключно текстові файли, а не файли, у яких разом з текстом зберігається інформація про його форматування. Необхідно також пам'ятати, що зайва інформація про форматування тексту значно перешкоджає роботі C++-компілятора, часто видає помилку компілювання.

Ім'я файлу, який міститиме початковий код програми, формально може бути будь-яким. Але C++-програми зазвичай зберігаються у файлах з розширенням *.cpp. Тому називайте свої C++-програми будь-якими іменами, але як розширення використовуйте *.cpp. Наприклад, назвіть свою першу програму Savchuk_Prog.cpp (це ім'я вживатиметься в подальших настановах), а для інших програм (якщо не буде спеціальних вказівок) вибирайте імена на свій власний розсуд.

Компілювання програми:

Спосіб компілювання програми Savchuk_Prog.cpp залежить від використовуваного компілятора і вибраних опцій. Понад це, багато компіляторів, наприклад Visual C++ (Microsoft) і C++ Builder (Borland), надають два різні способи компілювання програм: за допомогою компілятора командного рядка та інтегрованого середовища розробки (Integrated Development Environment – IDE). Тому для компілювання C++-програм неможливо дати універсальні настанови, які підійдуть для всіх компіляторів. Це означає, що програміст повинен дотримуватися настанов, наведених у супровідній документації, що додається до Вашого компілятора.

Але, як було зазначено вище, найпопулярнішими компіляторами є Visual C++ і C++ Builder, тому для зручності роботи читачів, які їх використовують, ми наведемо тут настанови з компілювання програм, які відповідають цим компіляторам. Найпростіше в обох випадках компілювати і виконувати програми, наведені у цьому посібнику, з використанням компіляторів командного рядка. Так ми і вчинимо.

Щоб скопіювати програму Savchuk_Prog.cpp, використовуючи Visual C++, введіть такий командний рядок:

```
C:\...>cl -GX Savchuk_Prog.cpp
```

Опція -GX призначена для підвищення якості компілювання. Щоб використовувати компілятор командного рядка Visual C++, необхідно виконати пакетний файл VCVARS32.bat, який входить до складу Visual C++.

Щоб скопіювати програму Savchuk_Prog.cpp, використовуючи C++ Builder, введіть такий командний рядок:

```
C:\...>bcc32 Savchuk_Prog.cpp
```

Внаслідок роботи C++-компілятора утворюється об'єктний код, який може виконати комп'ютер після завершення роботи компілятора. Для Windows-середовища виконуваний файл матиме те саме ім'я, що і початковий, але з іншим розширенням, а саме розширенням *.exe. Отже, виконувана версія програми Savchuk_Prog.cpp зберігатиметься в окремому файлі під назвою Savchuk_Prog.exe.

Виконання програми

Скомпільована програма готова до виконання. Оскільки результатом роботи C++-компілятора є виконуваний об'єктний код, то для запуску програми як команду достатньо ввести її ім'я. Наприклад, щоб виконати програму Savchuk_Prog.exe, потрібно ввести такий запис у командному рядку:

```
C:\...>Savchuk_Prog.cpp
```

Результатом виконання цієї програми є такий:

Це моя перш C++-програма.

Якщо Ви використовуєте інтегроване середовище розробки, то виконати програму можна шляхом вибору з меню команди Run (Виконати). Безумовно, точніші настанови наведено в супровідній документації, що додається до Вашого компілятора. Але, як згадувалося вище, найпростіше компілювати і виконувати наведені у цьому посібнику програми за допомогою програмного середовища C++ Builder.

Необхідно відзначити, що всі ці програми є консольними додатками, а не додатками, які базуються на застосуванні вікон, тобто вони виконуються в сеансі запрошення на введення команди. При цьому Вам, мабуть, відомо, що мова програмування C++ не просто підходить для Windows-програмування, вона – основна мова, що використовують для розроблення Windows-додатків. Проте жодна з програм, представлених у цьому посібнику, не використовує графічного інтерфейсу користувача (GUI – graphics user interface). Йдеться про те, що Windows – достатньо складне середовище для написання програм, що містить багато другорядних тем, не пов'язаних безпосередньо з мовою програмування C++. Водночас консольні додатки є набагато коротшими від графічних і краще надається для вивчення процесу програмування. Засвоївши мову програмування C++, Ви зможете без перешкод застосувати свої знання у сфері створення Windows-додатків.

Оброблення синтаксичних помилок

Кожному програмісту відомо, наскільки легко під час введення коду програми в комп'ютер вносяться випадкові помилки (друкарські огріхи). На щастя, під час компілювання такої програми компілятор "сигналізує" повідомленням про наявність *синтаксичних помилок*. Більшість C++-компіляторів спробують "побачити" сенс в початковому коді програми, незалежно від того, що Ви ввели. Тому повідомлення про помилку не завжди відображає дійсну причину проблеми. Наприклад, якщо в попередній програмі випадково опустити відкриту фігурну дужку після імені функції **main()**, компілятор вкаже як джерело помилки настанову **cout**. Тому під час отримання повідомлення про помилку прогляньте два-три рядки коду програми, які безпосередньо передують рядку з "виявленою" помилкою. Адже іноді трапляється так, що компілятор починає виявляти збої програми тільки через декілька рядків після реальної помилки.

Багато C++-компіляторів видають як результати своєї роботи не тільки повідомлення про помилки, але і попередження (warning). У мову програмування C++ "від народження" закладено доброзичливе ставлення до програміста, тобто вона дає змогу програмісту чинити практично все, що коректно з погляду синтаксису. Проте навіть "всепрощаючим" C++-компіляторам деякі синтаксично правильні речі можуть видатися підозрілими. У таких ситуаціях і видається попередження. Тоді програміст сам повинен оцінити, наскільки справедливі підозри компілятора. Деякі компілятори дуже пильні, тобто інколи виводять попередження з приводу набору абсолютно коректних настанов. Окрім цього, компілятори дають змогу використовувати різні опції, які можуть інформувати про деякі речі, що можуть цікавити Вас. Іноді така інформація має форму застережливого повідомлення навіть незважаючи на відсутність "складу" попередження.

Запитання для контролю знань:

1. Яке основне призначення препроцесорної директиви `#include` у програмі на C++?
2. Яка функція є обов'язковою точкою входу в будь-яку консольну програму на мові C++?
3. Яку роль відіграє компілятор у процесі створення програми?
4. Який результат виконання команди `std::cout << "Hello" << " " << "World";` ?

5. Що станеться, якщо ви забудете закрити фігурну дужку } у кінці функції main()?

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).

Тема 2.2. Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).

Навчальна мета: формування в учнів фундаментального розуміння того, як комп'ютер оперує інформацією через механізм змінних та класифікацію даних за їхньою природою. Під час уроку ми детально розберемо процес оголошення комірок пам'яті, приділяючи особливу увагу відмінностям між цілими та дробовими числами, що критично важливо для коректних обчислень.

Розвиваюча мета: спрямована на стимулювання логічного та алгоритмічного мислення учнів через виявлення причинно-наслідкових зв'язків між обраним типом даних та точністю обчислювального процесу. Ми працюватимемо над розвитком здатності до абстракції, що дозволяє учневі сприймати змінну не просто як символ, а як іменовану ділянку пам'яті з певними характеристиками.

Виховна мета: полягає у формуванні відповідального та критичного ставлення до результатів інтелектуальної праці, що проявляється через ретельну перевірку коректності введених даних та отриманих результатів. Ми прагнемо виховувати в учнях культуру цифрової взаємодії, де особлива увага приділяється повазі до кінцевого користувача програми — це реалізується через створення зрозумілих підказок при введенні та охайне оформлення виводу інформації.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Присвоєння значень змінним

Можливо, найважливішою конструкцією в будь-якій мові програмування є присвоєння змінній деякого значення. *Змінна* – іменована область пам'яті, у якій можуть зберігатися різні значення. При цьому значення змінної у процесі виконання програми можна змінити один або кілька разів. Іншими словами, вміст змінної є змінним, а не фіксованим.

У наведеному нижче коді програми створюється цілочисельна змінна з іменем *x*, якій присвоюється значення 1023, а потім на екрані монітора з'являється таке повідомлення:

Ця програма виводить значення змінної *x*: 1023.

Код програми 2.2. Демонстрація механізму розроблення другої програми

```
// Програма №2 – використання змінної.  
#include <iostream> // Потокowe введення-виведення  
using namespace std; // Використання стандартного простору імен  
  
int main()  
{  
    int x; // Тут визначається тип змінної  
    x = 1023; // Тут змінній x присвоюється число 1023.  
    cout << "Ця програма виводить значення змінної x: ";
```

```

    cout << x;    // Відображення числа 1023.
    getch();
    return 0;
}

```

Що ж нового у цьому коді програми? По-перше, настанова:

```
int x;    // Тут визначається тип змінної.
```

оголошує змінну з іменем `x` цілого типу. У мові програмування C++ всі змінні мають бути оголошені до їх використання. У визначенні змінної, окрім її імені, треба вказати, значення якого типу вона може зберігати. Тим самим оголошується *тип* змінної. У цьому випадку змінна `x` може зберігати цілочисельні значення, тобто цілі числа, що знаходяться в діапазоні -32 768 +32 767. У мові програмування C++ для оголошення змінної цілого типу достатньо поставити перед її іменем ключове слово `int`. Таким чином, настанова `int x;` оголошує змінну `x` типу `int`. Нижче дізнаємося, що мова програмування C++ підтримує широкий діапазон вбудованих типів змінних¹.

По-друге, у процесі виконання такої настанови змінній присвоюється конкретне значення:

```
x = 1023;    // Тут змінній x присвоюється число 1023.
```

У мові програмування C++ *оператор присвоєння* позначають одиночним знаком рівності (=). Його дія полягає в копіюванні значення, розташованого праворуч від оператора, у змінну, вказану зліва від нього. Після виконання цієї настанови присвоєння змінна `x` міститиме число 1023.

Результати, що були видані цією програмою, відображаються на екрані за допомогою двох настанов `cout`. Зверніть увагу на використання такої настанови для виведення значення змінної `x`:

```
cout << x;    // Відображення числа 1023.
```

У загальному випадку для відображення значення змінної достатньо в настанові `cout` помістити її ім'я праворуч від оператора "<<":

```
cout << "Ця програма виводить значення змінної x: " << x;
```

Оскільки у цьому конкретному випадку змінна `x` містить число 1023, то його і буде відображено на екрані.

Перед останнім оператором `return` в секції `main()` знаходиться функція `getch()`, за допомогою якої Ви зможете побачити результати роботи програми на екрані.

Перш ніж переходити до наступного розділу, спробуйте присвоїти змінній `x` інші значення (у початковому коді програми) і подивіться на результати виконання цієї програми після внесення відповідних змін.

2. Типи даних

Тип даних це множина допустимих значень, які може приймати той чи інший об'єкт.

Тип залежить від внутрішнього представлення інформації, що визначає як саме зберігаються й обробляються дані даного типу.

Тип даних визначає:

- внутрішнє представлення даних в пам'яті комп'ютера;
- обсяг пам'яті, що виділяється під дані;
- діапазон значень, які можуть приймати величини цього типу;
- операції та функції, які можна застосовувати до даних цього типу.

Стандарт C89 визначає п'ять фундаментальних типів, на основі яких формують інші типи. Це:

char, int, float, double, void.

Залежно від компіляторів і процесорів діапазон значень цих типів може бути різним, але: **char** – завжди байт, **int** – завжди слово (16 бітів на 16розрядному і 32 – на 32-

розрядному процесорах, але також не завжди). В C++ до цих типів додаються **wchar_t**, **bool**, у C99 **_Bool**, **_Complex**.

Стандарт визначає тільки мінімальний діапазон значень кожного типу, але не розмір у байтах.

Тип **void** має порожню множину значень, його використовують для оголошення функції, яка не повертає значення, для створення порожнього списку аргументів, для створення універсального вказівника, в операціях приведення типу.

Також для створення типу можуть бути використані модифікатори (специфікатори, описувачі) типу, які передують опису базового типу в оголошеннях. Це:

signed, **unsigned** – визначають наявність знаку;

long, **short** – визначають розмір.

Для **char** використовують тільки два перші, для решти типів – усі, при цьому можуть бути використані сполучення одного з перших з одним із останніх.

Повна інформація про типи C89(C99) представлена у додатку А.

Цілі типи. Базовими цілими типами в C є типи **int** та **char**. До цих типів можуть бути застосовані усі перелічені модифікатори.

Приклад оголошення, де оголошення подані як переліки і деякі зі змінних одразу ініціалізовані.

```
int a, b=23, c;  
long int len=-40000, big; unsigned  
u=45;
```

Якщо в оголошенні заданий модифікатор без типу, то за умовчанням це **int**. До даних цілого типу можуть бути застосовані наступні *арифметичні операції* :

+	додавання
-	віднімання, також унарний мінус
*	множення
/	цілочисельне ділення: $13/3 = 4$
%	остача від цілочисельного ділення: $13\%3 = 1$
=	оператор присвоювання
++x;	префіксний інкремент $x=x+1$
x++;	постфіксний інкремент $x=x+1$
--x;	префіксний декремент $x=x-1$
x--;	постфіксний декремент $x=x-1$

Порозрядні операції застосовують тільки до даних цілого типу і вони визначають дію безпосередньо над бітами чисел:

&	бінарна операція , побітове І
	бінарна операція , побітове АБО

$\wedge$	бінарна операція , виключне АБО
$\sim$	унарна операція <i>НЕ</i> – заперечення, доповнення до 1
$\gg$	бінарна операція ,зсув вправо, в напрямку від старшого біта
$\ll$	бінарна операція ,зсув вліво, в напрямку від молодшого біта

Дійсні типи. Для зберігання дійсних чисел застосовуються типи даних **float** (з одинарною точністю) і **double** (з подвійною точністю). Формат зберігання – з плаваючою крапкою.

Приклади оголошення

```
double result, some=4.5, eps=.1e-8; long double p,  
p1=-35.67891234005;
```

До даних дійсного типу можуть бути застосовані наступні арифметичні операції :

$+$	додавання
$-$	віднімання, також унарний мінус
$*$	множення
$/$	ділення: $15/5 = 7.5$

Для застосування математичних функцій до даних числових типів, потрібно підключити заголовковий файл з математичними функціями **#include <math.h>**. Деякі функції, оголошені у цьому файлі, потрібні для виконання першої роботи комп'ютерного практикуму, представлені у додатку Б.

Дані з плаваючою крапкою можна привести до цілого типу округленням дійсного аргументу *x* до найближчого цілого:

```
long int lround(double x);
```

Символьні типи. У стандарті C немає типу даних, який можна було б вважати дійсно символьним. Для представлення символьної інформації використовують тип **char**. Змінна такого типу призначена для зберігання одного символу. Приклад оголошення з ініціалізацією.

```
char c='A', a,b='5';
```

Оскільки в пам'яті комп'ютера символи зберігаються у вигляді їх ASCIIкодів – цілих чисел, то тип **char** насправді є підмножиною типу **int**. Під величину символьного типу відводиться 1 байт.

Для зберігання інформації у вигляді рядка, використовують масиви із елементів типу **char**.

Логічний тип. У стандарті C89 немає окремо визначеного логічного типу.

Тому у разі потреби його заміняє тип **int** зі значеннями **0** і **1**. У пізніших стандартах C і C++ з'явився спеціальний булевий тип **bool** зі значеннями **true(1)** та **false(0)**.

Запитання для контролю знань:

1. Який тип даних найкраще обрати для змінної, що зберігатиме результат додавання двох цілих чисел?
2. Що станеться, якщо спробувати додати ціле число (наприклад, 5) до дробового (наприклад, 2.5) у більшості сучасних мов програмування?
3. Яка основна мета операції 'вводу' (input) у програмі для знаходження суми двох чисел?
4. Чому важливо ініціалізувати (надавати початкове значення) змінні перед використанням?
5. Який оператор зазвичай використовується для присвоєння значення суми змінній (наприклад, `result = a + b`)?

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).

Змістовий модуль 3. Керування порядком обчислень

Тема 3.1. Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).

Навчальна мета: полягає у формуванні в студентів цілісної системи знань та практичних навичок застосування керуючих конструкцій розгалуження в алгоритмах та програмуванні. Студент має глибоко засвоїти логіку функціонування умовного оператора if-else, навчитися правильно формулювати прості та складені логічні вирази, а також опанувати методику побудови вкладених умов для вирішення багаторівневих завдань. Особлива увага приділяється вивченню оператора множинного вибору switch, де метою є розуміння його синтаксичних особливостей, принципів оптимізації коду при роботі з великою кількістю дискретних значень та коректного використання переривань.

Розвиваюча мета: спрямована на активізацію аналітичного мислення студентів через опанування механізмів прийняття рішень у програмуванні. Вона передбачає розвиток здатності до логічного прогнозування результатів виконання коду при різних вхідних даних та формування вміння декомпонувати складні умови на послідовність елементарних кроків. Під час лекції акцент зміщується на розвиток когнітивної гнучкості, що дозволяє студенту порівнювати різні підходи до реалізації алгоритму та обирати найбільш оптимальний з точки зору швидкодії та читабельності коду. Також метою є вдосконалення навичок абстрактного мислення, що проявляється у вмінні переводити вербальні описи життєвих ситуацій у формалізовані логічні структури if-else або switch.

Виховна мета: зосереджена на формуванні професійної відповідальності та культури програмування як важливої складової етики розробника. Вона передбачає виховання прискіпливого ставлення до точності та однозначності формулювань, адже кожна помилка в логічному розгалуженні безпосередньо впливає на надійність системи. Через вивчення структур if-else та switch у студентів виховується прагнення до лаконічності та інтелектуальної чесності — вміння створювати прозорий і зрозумілий для інших код, що свідчить про повагу до колег, які працюватимуть із ним у майбутньому.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Операції порівняння та логічні операції

Операції порівняння – це бінарні операції, в яких значення двох змінних порівнюються один з одним.

>	більше ніж
>=	більше або дорівнює
<	менше ніж
<=	менше або дорівнює
==	дорівнює
!=	не дорівнює

Використання логічних операцій дає можливість реалізувати операції формальної логіки засобами мови програмування C.

! унарна операція логічне НІ – заперечення, таблиця істинності:

x	!x
0	1
1	0

Бінарні операції

&& логічне І – кон'юнкція (^)

|| логічне АБО – диз'юнкція (v)

Таблиця істинності виглядає так:

x	y	x&& y	x y
0	0	0	0
0	1	0	0
1	0	0	0
1	1	1	1

2. Умовний оператор if

Оператор умовного переходу **if** використовують для розгалуження процесу обчислень на два напрямки. Синтаксис оператора:

if (вираз) оператор_1; else
оператор_2;

де **вираз** — це вираз, який має логічне значення (**true** — «істинний» або **false** — «хибний»). Виконання оператору **if** реалізоване так: спочатку компілятор обчислить **вираз** і, якщо його значення не дорівнює нулю («істина»), виконується **оператор_1**, у протилежному випадку — **оператор_2**, після чого управління буде передано наступному за умовним оператору,

наприклад:

```
if ( i < j ) i++; else{
    j = i-3; //оператор_2 складений i++;
};
printf(" i=%d, j=%d\n", i, j);
```

У загальному випадку **вираз** може не використовувати операцій порівняння,

наприклад:

```
if(n++)m++;else k++;
```

Оператор **if** може мати скорочену форму запису, тоді у записі оператора друга частина, тобто **else**, відсутня. Синтаксис такого оператора наступний:

```
if (вираз) оператор_1;
```

Для такого оператора, якщо **вираз** приймає значення «неправда», що відповідає значенню 0, управління одразу буде передане на наступний після **if** оператор програми, наприклад :

```
n=0;
if (n) n++; //оператор_2 відсутній
printf(" n=%d\n", n); // n++ не виконався, n==0
```

Якщо у будь-якій гілці розгалуження необхідно виконати декілька операторів, їх слід розташовувати у блоці, як у першому прикладі.

Синтаксис C припускає, що у випадку застосування вкладених умовних операторів, при цьому кожне **else** відповідає найближчому до нього попередньому **if**. Наприклад, розглянуті фрагменти тотожні:

<pre>if (a > b){ if (a > c) max = a; else max = c; }</pre>	<pre>if (a > b) if (a > c) max = a; else max = c;</pre>
--	---

У деяких випадках замість оператора **if** зручніше використати тернарний оператор "?". Тобто оператор

```
if (вираз) {оператор_1} [else {оператор_2}];
```

можна записати так:

```
вираз ? оператор_1 : оператор_2;
```

Приклад застосування тернарного оператора замість оператора **if**

<pre>x=10; if (x<9) y=100; else y=200;</pre>	<pre>x=10; y = (x<9) ? 100 : 200;</pre>
---	--

У наведеному прикладі результати роботи обох фрагментів будуть тотожними:

Приклад . Розглянемо як приклад розв'язок добре відомого рівняння

$$ax^2 + bx + c = 0$$

Математична модель:

1) Якщо $a=0$, $b=0$ і $c=0$ – безліч розв'язків.

2) Якщо $a=0$, $b=0$, $c \neq 0$ – немає розв'язку.

3) Якщо $a=0$, $b \neq 0$ – лінійне рівняння. Якщо $a \neq 0$ – квадратне рівняння і якщо детермінант $D = b^2 - 4ac \geq 0$

Алгоритм розв'язку такої задачі можна побудувати двома шляхами – з використанням так званої квазілінійної моделі, яка практично відповідає побудованій математичній моделі, або з використанням вкладених умов.

Перший алгоритм передбачає використання у програмі складених логічних виразів, наприклад

```
printf("enter a,b,c "); scanf("%lf%lf%lf", &a,
&b, &c); if((a==0)&&(b==0)&&(c==0))
    printf("set of solutions");
if((a==0)&&(b==0)&&(c!=0))
    printf("no solutions");
if((a==0)&&(b!=0))
    printf("linear equation x=%f\n", -c/b); if(a!=0) {
    d=b*b+4*a*c;
    if(d<0)
        printf("no valid solutions"); else {
        x1=(-b+sqrt(d))/(2*a);
        x2=(-b-sqrt(d))/(2*a);
        printf("roots x1=%f, x2=%f\n", x1, x2);
    }
}
```

Якщо значення дійсного числа не є щойно уведеним з клавіатури, а отримане в результаті обчислень, які завжди накопичують похибку, варто перевірку організувати з врахуванням точності. Для цього або уводять з клавіатури, або задають як константу (частіше – макроконстанту) деяке число ϵ .

Наприклад

```
#define EPSILON 0.0001
...
if(fabs(x-1)>epsilon)
    y=25.5/(x-1);
else
    y=1.0
```

3. Оператор switch

Довгі ланцюжки операторів **if-else** важкі для сприйняття, тому, якщо варіант вибору шляху подальшого обчислення залежить від цілих значень, використовують оператор вибору (*перемикач*). Синтаксис:

switch (вираз_цілого_типу)

```
{  
    case константний вираз 1: [оператор1; ] [break;] case константний  
    вираз 2: [оператор2; ] [break;]  
    ...  
    case константний вираз n : [оператор n; ] [break;] [default: оператор  
    n+1] // аналог else в if  
}
```

Семантика оператора: після обчислення **виразу_цілого_типу** його значення по чергові порівнюється з кожним з **константних виразів** *i*, коли відбудеться співпадіння, наприклад, з **виразом i**, управління передається на **оператор i**. Якщо після оператора стоїть оператор **break**, то після виконання **оператора i** управління передається на наступний за **switch** оператор.

Якщо жодного співпадіння не відбулося, управління передається на мітку **default** за її наявності або просто жодна з гілочок оператора виконана не буде і виконання програми продовжиться з наступного за **switch** оператора.

Оператор **break**; відноситься до операторів передачі управління і приводить до переривання виконання оператора, у якому він використовується. Управління передається наступному оператору. Використовується в **switch** і в циклах.

Приклад. Виведення назв перших трьох чисельників англійською мовою

```
switch (number)  
{  
    case 1:      printf(" % d one \n", number);break; case 2: printf(" % d  
    two \n", number);break; case 3:  printf(" % d three\n", number);break;  
    default: printf ("good bye \n");  
}
```

Як правило, оператор вибору використовують саме у такому форматі, з **break**. Але оскільки з **case** завжди пов'язане тільки одне значення, а інколи потрібно виконати однакові дії для двох або більше міток, тоді **break** не використовують. Якщо оператор **break** відсутній, то управління передається на наступний **case** і так до закінчення всього оператора **switch** включно з **default**, або до появи першого оператора **break**. Наприклад, програма переведення оцінки із 12-бальної у 5-бальну.

```
int main(){  
    printf("enter your mark\n"); scanf("%d",  
    &n);  
    switch (n){  
        case 0: case 1: case 3: printf("bad\n");           break;  
        case 4:  
        case 5: case 6: printf("satisfactory\n"); break;  
    }
```

```

        case 7:
        case 8: case 9: printf("good\n"); break;
        case 10:
        case 11: case 12: printf("excellent\n");           break;
        default: printf ("incorrect mark\n");
    }
    return 0;
}

```

Дві різні мітки не можуть мати однакові значення. Вираз умови в інструкції **switch** може бути як завгодно складним, у тому числі включати виклики функцій. Оператори вибору можуть бути вкладеними один у другий і в інші конструкції мови.

Запитання для контролю знань:

1. Якими операторами може бути реалізоване розгалуження у програмі?
2. Опишіть оператор вибору switch.
3. Навіщо потрібен оператор break в конструкції switch?
4. Чи можна об'єднувати розділи case?
5. Чи можна вкладати умовні оператори один в другий?

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).

Тема 3.2. Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.)

Навчальна мета: полягає у формуванні в студентів цілісної системи знань про принципи побудови циклічних алгоритмів та їх практичну реалізацію мовами програмування. Опрацювання теми передбачає ґрунтовне засвоєння синтаксису та семантики основних операторів повторення, зокрема циклу з параметром для ітерації у відомому діапазоні, а також ітераційних структур із передумовою та післяумовою для опрацювання процесів, тривалість яких залежить від динамічних логічних виразів.

Розвиваюча мета: спрямована на активне стимулювання аналітичного мислення та здатності до абстрагування шляхом виявлення повторюваних процесів у складних обчислювальних задачах. Вона передбачає розвиток уміння порівнювати різні типи циклічних структур і критично оцінювати їхню ефективність для конкретних сценаріїв, що сприяє формуванню гнучкості інтелекту та логічної послідовності дій. Крім того, процес навчання має на меті вдосконалення навичок прогнозування результатів роботи алгоритму, виховання уваги до деталей при відстеженні змін станів змінних, а також розвиток алгоритмічної інтуїції, яка дозволяє знаходити оптимальні шляхи оптимізації програмного коду та самостійно виправляти логічні помилки.

Виховна мета: зосереджена на формуванні професійної відповідальності та культури програмування, що виявляється у прагненні створювати чистий, зрозумілий та ефективний код. Навчальний процес сприяє вихованню наполегливості та терплячості під час відлагодження циклічних процесів, де навіть незначна помилка в умові може призвести до зациклення або некоректних обчислень. Робота над задачами з розрахунку послідовностей стимулює розвиток уважності, акуратності та критичного ставлення до власних результатів, прищеплюючи студентам усвідомлення того, що інтелектуальна праця потребує дисципліни мислення та системного підходу до вирішення будь-яких прикладних проблем.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

Оператори циклу використовують для здійснення багаторазового повторення деякої послідовності дій.

Кожен цикл складається з умови та тіла циклу, що виконується декілька разів. Один прохід циклу називається ітерацією. У мові C існують три циклічні конструкції: `while`, `do while`, `for`.

1. Цикл з передумовою `while`

Відповідно до теорії будь-яка структурна програма може бути реалізована за допомогою лінійних блоків, умовних блоків і циклів з передумовою. Тобто цикл з передумовою – основна циклічна конструкція, яка може замінити будь-яку. Але для зручності ввели також і інші.

Цикл називається циклом з передумовою тому, що умова в ньому перевіряється до початку наступної ітерації циклу

Синтаксис циклу з передумовою:

`while` (вираз) оператор;

Тіло циклу представлено **оператором**, який може бути простим або складеним.

Складеним називається послідовність операторів, розділених знаком «;», які сформовані у **блок**, обмежений фігурними дужками.

Порядок виконання оператора наступний.

Спочатку буде перевірена умова входу у цикл. Якщо умова істинна, тобто значення **виразу** не дорівнює нулю, буде виконаний **оператор**, після чого управління знову буде передане на початок циклу – перевірку умови входу в цикл.

Так цикл буде повторюватись поки умова входу не стане хибна, тобто значення **виразу** стане рівним нулю, тоді цикл закінчиться і управління одразу буде передане на наступний за ним оператор.

Приклад використання:

```
int i=0, sum=0; while (i
< n)
    sum += ++i * i;//простий оператор
```

Оператор **`while`** зручно застосовувати у випадках, коли кількість ітерацій заздалегідь невідома. Наприклад, потрібно обчислити суму рядучерговий член ряду не стане меншим за деяке наперед задане число

```
#define SQ1(x) 1./((x)*(x)) int main()
{
    double s=0, eps; int k=1;
    printf("enter epsilon\n"); scanf("%lf",
```

```

&eps); while(SQ1(k)>eps)
{
    s=s+SQ1(k); k+
    +;
}
printf("result=%f \n", s); return 0;
}

```

Якщо значення **виразу** одразу дорівнюватиме нулю, цикл з передумовою не виконається жодного разу, управління буде передане до наступного за цим циклом оператора. Наприклад, якщо у розглянутому вище прикладі помилково увести.

2. Цикл з постумовою do while

Циклічну конструкцію з постумовою застосовують у випадках, коли тіло циклу потрібно виконати хоча б один раз. Тіло циклу представлено простим або складеним **оператором**, умова входу в цикл перевіряється **після** його виконання. Синтаксис:

do оператор while (вираз);

Так само, як і в циклі з передумовою, умова виконання циклу – ненульове значення **виразу**.

Синтаксисом циклу **do while** не передбачені обов'язкові фігурні дужки для випадку з одним оператором, але їх використання є бажаним для забезпечення більшої читабельності програми.

Типові випадки використання циклів з постумовою – організація меню, перевірка коректності уведених даних, наближені обчислення за ітераційною формулою з заданою точністю, тощо.

Приклад використання для забезпечення коректності введення інформації:

```

do{
    printf("enter the number of students \n");
    scanf("%d",&numb);
}while(numb<0);
printf("ok, number of students=%d\n", numb);

```

Як і для циклу з передумовою, робота циклу з постумовою буде закінчена по нульовому значенню виразу.

Приклад використання для обчислення суми ряду з заданою точністю.
Дано дійсні числа x, ε ($x \neq 0, 0 < \varepsilon < 1$). Обчислити з точністю до ε та

дослідити збігання послідовності для обраного значення x з виведенням кожного p -го доданку та відповідного значення суми:

$$\sum_{k=0}^{\infty} \frac{(-1)^{k+1} (x/2)^{2k}}{2(k+1)!}.$$

Для розв'язку даної задачі варто спочатку побудувати математичну модель.

$$\frac{a_{k+1}}{a_k} = \frac{(-1) \left(\frac{x}{2}\right)^2}{k+2}, \text{ тоді } a_{k+1} = \frac{(-1) \left(\frac{x}{2}\right)^2}{k+2} a_k \text{ і обчислення проводитимуться поки}$$

```
int main(){
    double s=0, a=1., x, eps; int k=0, p;
    printf("enter x and p\n"); scanf("%lf%d", &x,
    &p);
        do
        {
            printf("enter 0<epsilon<1\n"); scanf("%lf", &eps);
        } while(fabs(eps)>1); do
        {//формування наступного члена ряду через попередній a=-a*x*x/(4*(k+2));
            s=s+a;
            if(k%p); //чи це p-тий член ряду?
                else printf("a(%d)=%f, s=%f\n", k, ak, S);
            k++;
        } while(fabs(a)>eps); printf("s=%f\n", s);
        return 0;
    }
```

3. Цикл з відомою кількістю кроків for

Синтаксис циклу **for** наступний:

for([ініціалізація];[вираз];[модифікація])[оператор];

У секції **ініціалізації** виконують оголошення параметрів циклу у вигляді списку, з використанням операції «кома». Цих параметрів може бути декілька, обмежень на типи немає. В C++ та пізніших специфікаціях C вони можуть бути тут же і ініціалізовані, якщо їх використання у програмі ніде крім даного циклу не заплановане.

Вираз – це умова виконання циклу. У цій секції може бути лише один вираз. Це може бути складеною умовою, але операція кома тут не допустима.

Секція **модифікацій** виконується після кожної ітерації циклу і слугує для зміни значень параметрів. Виразів модифікації може бути декілька, тобто можна використовувати операцію

кома.

Оператор визначає тіло циклу. Він може бути як простий, так і складений, тоді стають обов'язковими фігурні дужки.

Приклад циклу:

```
int n=10, y, k;  
for (k = 1; k <= n; k++) printf("k=%d, k^2=%d", k,  
    k*k);
```

У наведеному прикладі на екран будуть виведені цілі від 1 до 10 і їх квадрати.

Спочатку буде виконаний етап ініціалізації циклу, вираз за виразом, зліва направо, якщо секція ініціалізації не порожня. Ця секція виконується один раз, на самому початку роботи циклу. Приклад циклу зі списком у секції ініціалізації:

```
int n, y, k;  
for (k = 0, n = 20; k <= n; k++, n--) y = k * n;
```

Потім буде обчислено значення **виразу** – виразу умови продовження циклу.

Якщо значення цього виразу відмінно від нуля (тобто істинно), буде виконаний **оператор** циклу. Це етап виконання тіла циклу. Якщо значення виразу умови продовження циклу хибне, виконання циклу переривається і управління передається на наступний після **for** оператор.

Після цього будуть обчислені значення виразів, розташованих у секції модифікації. Це етап обчислення параметрів циклу для здійснення наступної ітерації циклу.

На останніх двох етапах значення раніше визначених змінних можуть бути змінені, тому наступна ітерація циклу буде розпочата з етапу визначення умови продовження циклу.

Цей цикл є циклом з передумовою, оскільки умова входу в цикл перевіряється перед початком чергової ітерації. Відповідно, вираз, що визначає умову може одразу дорівнювати нулю і тоді вхід до циклу не відбудеться взагалі, управління буде передане наступному за циклом оператору.

Кожна з секцій може бути відсутня, як і всі одночасно. Але знаки ; у дужках після **for** є обов'язковими. Наприклад

```
int n=10, y=0, k=0;  
for (;k <= n; k++, n--) y += k * n;
```

У наведеному прикладі відсутня перша секція.

Безкінечні цикли

Якщо умова входу у цикл побудована некоректно, цикл може ніколи не закінчитися, наприклад: **for (k = 0, n = 5; k <= n; k++, n++) y = k * n;**

Іноколи безкінечний цикл – це не помилка, просто потрібно організувати цикл, який

ніколи не повинен закінчитися, окрім випадку з певною нестандартною ситуацією.

Наступний цикл, що виводить синуси уведених кутів, працюватиме до тих пір, поки не буде уведений нечисловий символ.

```
double angle; for(;;)
{
    printf("enter      angle      \n");
    if(scanf("%lf", &angle))
        printf("sin(%f)=%f\n", angle, sin(angle)); else break;
}
```

Тут **scanf** поверне 0, якщо буде уведене не число.

Вкладені цикли

Цикли можуть бути вкладеними, якщо виникає необхідність багаторазового виконання певної дії, яка сама по собі також представляє циклічну конструкцію.

Глибина вкладеності мовою не регламентована. Програміст повинен сам стежити, щоб границі зовнішнього і внутрішнього циклів не перетиналися.

Вкладеними можуть бути цикли різних типів.

Наприклад, фрагмент програми з реалізацією імітації годинника з виведенням поточного значення годин, хвилин і секунд.

```
int h=0, m, s;
printf("hours, minutes, seconds\n"); while(h<24){
    for(m=0; m<60; m++) for(s=0;
        s<60; s++)
        printf("%5i, %6i, %6i\n", h, m, s);
    h++;
}
```

Запитання для контролю знань:

1. Опишіть синтаксис і призначення кожної секції оператора **for**.
2. Опишіть механізм дії оператора **for**.
3. Що таке лічильник циклу?
4. Скільки і якого типу може бути лічильників у циклі **for** в С?
5. Параметри яких типів можна використовувати у циклі **for**?
6. Чи допустимо використання оператора кома при визначенні умови виконання циклу?
7. Які обмеження на глибину вкладеності циклів ви знаєте і з чим вони пов'язані?
8. Що таке «безкінечний цикл»? У яких випадках можна використовувати безкінечні цикли?
9. Про що потрібно пам'ятати, щоб не організувати безкінечний цикл?
10. Які різновиди визначення умови входу в цикл в С ви знаєте?
11. Що таке тіло циклу? Ітерація циклу?
12. Які принципові відміни між циклами з переді постумовою?

13. У якому випадку цикл не буде виконано жодного разу?
14. У якому випадку цикл буде виконано хоч раз за будь-яких умов?

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).

Змістовий модуль 4. Процедурно-орієнтоване програмування . Функції користувача

Тема 4.1. Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).

Навчальна мета: зосереджена на формуванні у студентів цілісного розуміння концепції модульного програмування через опанування механізмів створення та застосування підпрограм. У ході вивчення теми закладається фундамент для усвідомленого розрізнення процедур і функцій як основних структурних одиниць коду, що дозволяють уникати дублювання та підвищувати читабельність програм. Особлива увага приділяється засвоєнню синтаксичних особливостей оголошення користувацьких функцій, де ключовим аспектом стає механізм повернення результату, на відміну від процедур, призначених для виконання послідовності дій. Студенти мають навчитися не лише технічно описувати підпрограми, а й стратегічно визначати доцільність їх використання для розв'язання конкретних алгоритмічних задач.

Розвиваюча мета: спрямована на стимулювання аналітичного мислення та формування навичок декомпозиції, що дозволяють студентам розкладати складні, багатошарові завдання на простіші, логічно завершені компоненти. Через вивчення підпрограм активізується здатність до абстрагування, коли учень вчиться бачити за конкретним кодом загальний алгоритмічний шаблон, придатний для багаторазового використання в різних контекстах.

Виховна мета: зосереджена на формуванні професійної відповідальності та етики розробника через усвідомлення значущості якісного й структурованого коду. Використання підпрограм прищеплює студентам повагу до праці колег, адже створення зрозумілих модулів є проявом турботи про тих, хто буде супроводжувати чи масштабувати проєкт у майбутньому. Це виховує культуру взаємодопомоги та командної взаємодії, де кожен окремий блок коду стає внеском у загальну стабільність системи.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Функції – "будівельні блоки" C++-програми

Функція – у програмуванні – один з видів підпрограми. Особливість, що відрізняє її від іншого виду підпрограм – процедури, полягає в тому, що функція повертає значення, а її виклик може використатися в програмі як вираз.

Підпрограма – частина програми, яка реалізує певний алгоритм і дає змогу звернення до неї з різних частин загальної (головної) програми. Підпрограма часто використовується для скорочення розмірів програм у тих завданнях, у процесі розв'язання яких необхідно виконати декілька разів однаковий алгоритм при різних значеннях параметрів. Оператори (команди), які реалізують відповідну підпрограму, записують один раз, а в необхідних місцях розміщують оператори передачі управління на цю підпрограму.

З погляду теорії систем, функція в програмуванні – окрема система (підсистема, підпрограма), на вхід якої надходять керуючі впливи у вигляді значень аргументів. На виході системи одержуємо результат виконання програми, що може бути як скалярною величиною, так і векторним значенням. За ходом виконання функції можуть виконуватися також деякі зміни в керованій системі, причому як оборотні, так і необоротні.

У деяких мовах програмування (наприклад, у мові Pascal) функції існують поряд із процедурами (підпрограмами, що не повертають значення), в інші, наприклад, у мові C++, є єдиним реалізованим видом підпрограми (тобто всі підпрограми є функціями й можуть повертати значення).

Побічним ефектом функції називається будь-яка зміна функцією стану програмного середовища, крім повернення результату (зміна значень глобальних змінних, виділення й звільнення пам'яті, введення-виведення і так далі). Теоретично найбільш правильним є використання функцій, що не мають побічного ефекту (тобто таких, внаслідок виклику яких повертається обчислене значення і тільки), хоча на практиці доводиться використати функції з побічним ефектом, хоча б для забезпечення введення-виведення й відображення результатів роботи програми. Існує специфічна парадигма програмування – функціональне програмування, у якій будь-яка програма є набір вкладених викликів функцій, що не викликають побічних ефектів. Найбільш відома мова програмування, що реалізує цю парадигму – Лісп¹. У ньому будь-яка операція, будь-яка конструкція мови, будь-який вираз, окрім константи, є викликами функцій.

2. Основні поняття функції

Будь-яка C++-програма складається з "будівельних блоків", що називаються *функціями*. Функція – підпрограма, яка містить одну або декілька C++настанов і здійснює одну або декілька задач. Хороший стиль програмування мовою C++ передбачає, що кожна функція виконує тільки одну задачу, наприклад, окремо введення та виведення даних. Кожна функція має ім'я, яке використовують для її виклику. Своїм функціям програміст може давати будь-які імена за винятком імені **main()**.

У мові програмування C++ жодна функція не може бути вбудована в іншу. На відміну від таких мов програмування, як Pascal, Modula-2 і деяких інших, які дають змогу використовувати вкладені функції, у мові програмування C++ всі функції розглядаються як окремі компоненти¹.

Під час позначення функцій у цьому посібнику використовують домовленість (зазвичай її дотримується в літературі, присвяченій мові програмування C++), згідно з якою ім'я функції завершується парою круглих дужок. Наприклад, якщо функція має ім'я `getVal`, то її згадка в тексті позначиться як `Put()`. Дотримання цієї домовленості дасть змогу легко відрізнити імена змінних від імен функцій.

У вже розглянутих прикладах програм функція **main()** була єдиною. Як було зазначено вище,

функція **main()** – перша функція, яка виконується під час запуску програми. Її повинна містити кожна C++-програма. Взагалі, функції, які Вам належить використовувати, бувають двох типів. До першого типу належать функції, написані програмістом (**main()** – приклад функції такого типу). Функції іншого типу знаходяться в *стандартній бібліотеці* C++-компілятора². Як правило, C++-програми містять як функції, написані програмістом, так і функції, які надає компілятор.

Оскільки функції утворюють фундамент мови програмування C++, займемося ними ґрунтовніше. Наведена вище програма містить дві функції: **main()** і **FunC()**. Ще до виконання цієї програми (або читання подальшого опису) уважно вивчіть її текст і спробуйте передбачити, що вона повинна відобразити на екрані.

Код програми: Демонстрація механізму реалізації структури програми, що містить дві функції: **main()** і **FunC()**

```
#include <iostream>    // Потокowe введення-виведення
using namespace std;   // Використання стандартного простору імен

void FunC();           // Попереднє оголошення прототипу функції FunC()

int main()
{
    cout << "У функції main():";
    FunC();             // Викликаємо функцію FunC().
    cout << "Знову у функції main():"
    getch(); return 0;
}

void FunC()            // Визначення функції FunC()
{
    cout << " У функції FunC(). ";
}
```

Програма працює таким чином. Спочатку викликається функція **main()** і здійснюється її перша **cout**-настанова. Потім з функції **main()** викликається функція **FunC()**. Зверніть увагу на те, як цей виклик реалізується у програмі: вказується ім'я функції **FunC**, за яким стоїть пара круглих дужок і крапка з комою. Виклик будь-якої функції є C++-настановою і тому повинен завершуватися крапкою з комою. Потім функція **FunC()** здійснює свою єдину **cout**-настанову і передає керування назад функції **main()**, причому тому рядку коду програми, який розташований безпосередньо за викликом функції. Нарешті, функція **main()** здійснює свою другу **cout**-настанову, яка завершує всю програму. Отже, на екрані ми повинні побачити такі результати:

У функції **main()**. У функції **FunC()**. Знову у функції **main()**.

У цьому кодї програми необхідно розглянути таку настанову:

```
void FunC();           // Попереднє оголошення прототипу функції FunC()
```

Як зазначено в коментарі, це – *прототип оголошення функції* **FunC()**. Хоча детальніше прототипи буде розглянуто нижче, все ж таки без коротких пояснень тут не обійтись. Прототип функції оголошує функцію до її визначення. Прототип дає змогу компіляторові дізнатися тип значення, що повертається цією функцією, а також кількість і тип параметрів, які вона може мати. Компіляторові потрібно знати цю інформацію до першого виклику функції. Тому прототип розташовується ще до функції **main()**. Єдиною функцією, яка не вимагає прототипу, є **main()**, оскільки вона є вбудованою у мові програмування C++.

Як бачимо, функція **FunC()** не містить настанови **return**. Ключове слово **void**, яке передує як прототипу, так і визначенню функції **FunC()**, формально заявляє про те, що функція **FunC()** не повертає ніякого значення. У мові програмування C++ функції, які не повертають значень, оголошуються з використанням ключового слова **void**.

3. Передавання аргументів функції

Функції можна передати одне або декілька значень. Значення, що передається функції, називають *аргументом*. Хоча у програмах, які ми розглядали дотепер, жодна з функцій (ні **main()**, ні **FunC()**) не отримувала ніяких значень, функції у мові програмування C++ можуть приймати один або декілька аргументів. Верхня межа кількості аргументів, що приймаються, визначається конкретним компілятором. Згідно зі стандартом мови програмування C++, він дорівнює 256.

Аргумент — значення, що передається функції під час її виклику.

Розглянемо коротку програму, яка для відображення абсолютного значення числа використовує стандартну бібліотечну (тобто вбудовану) функцію **abs()**. Ця функція приймає один аргумент, перетворює його в абсолютне значення і повертає результат.

Код програми 2.7. Демонстрація механізму використання функції **abs()** **#include**

```
<iostream>          // Потокове введення-виведення
#include <cstdlib>     // Використання бібліотечних функцій
using namespace std;  // Використання стандартного простору імен
int main()
{
    cout << abs(-10);
    getch(); return 0;
}
```

У цьому коді програми функції **abs()** як аргументу передається число -10. Функція **abs()** приймає цей аргумент під час виклику і повертає його абсолютне значення, яке у цьому випадку дорівнює числу 10. Важливо запам'ятати: якщо функція приймає аргумент, то його вказують усередині круглих дужок, розташованих відразу після імені функції.

Значення, що повертається функцією **abs()**, використовується настановою **cout** для відображення на екрані абсолютного значення числа -10.

Зверніть також увагу на те, що наведений вище код програми містить заголовок **<cstdlib>**. Цей заголовок необхідний для забезпечення можливості виклику функції **abs()**. Кожного разу, коли Ви використовуєте бібліотечну функцію, у програмі необхідно помістити відповідний заголовок. Заголовок, окрім іншої інформації, повинен містити прототип бібліотечної функції.

Параметр — визначена функцією змінна, яка приймає аргумент, що передається функції.

Під час розроблення функції, яка приймає один або декілька аргументів, іноді необхідно оголосити змінні, які зберігатимуть значення аргументів. Ці змінні називають *параметрами* функції. Наприклад, наступна функція виводить добуток двох цілочисельних аргументів, що передаються функції під час її виклику:

```
void funZ(int x, int y)
{
    cout << x * y << " ";
}
```

Під час кожного виклику функції **funZ()** здійснюється множення значення, що передається параметру **x**, на значення, передане параметру **y**. Проте пам'ятайте, що **x** і **y** – просто змінні, які приймають значення, які передаються під час виклику функції. Розглянемо таку коротку програму, яка демонструє використання функції **funZ()**.

Код програми: Демонстрація механізму використання функції **funZ()** **#include**

```
<iostream>          // Потокове введення-виведення
#include <conio>       // Для консольного режиму роботи
using namespace std;  // Використання стандартного простору імен
```

```

void funZ(int x, int y); // Попереднє оголошення прототипу функції funZ()
int main()
{
    funZ(10, 20);
    funZ(5, 6);
    funZ(8, 9);
    getch(); return 0;
}
void funZ(int x, int y) // Визначення функції funZ()
{
    cout << x * y << " ";
}

```

Ця програма виведе на екран числа 200, 30 і 72. Під час виклику функції `funZ()` C++-компілятор копіює значення кожного аргумента у відповідний параметр. У цьому випадку під час першого виклику функції `funZ()` число 10 копіюється в змінну `x`, а число 20 – в змінну `y`. Під час другого виклику 5 копіюється в `x`, а 6 – в `y`. Під час третього виклику 8 копіюється в `x`, а 9 – в `y`.

Якщо Ви ніколи не працювали з мовою програмування, у якій дозволені функції, які параметризуються, то описаний вище процес може видатися дещо дивним. Проте хвилюватися не варто: у міру перегляду інших C++-програм Вам стане значно зрозумілішим принцип використання функцій, їх аргументів і параметрів.

4. Повернення функціям аргументів

У мові програмування C++ багато бібліотечних функцій повертають значення. Наприклад, вже знайома нам функція `abs()` повертає абсолютне цілочисельне значення свого аргументу. Функції, які написано програмістом, також можуть повертати значення. У мові програмування C++ для повернення значення використовують настанову `return`. Загальний формат цієї настанови є таким:

```
return значення;
```

Неважко здогадатися, що тут елемент *значення* є значенням, що повертається функцією. Щоб продемонструвати механізм повернення функціями значень, переробимо попередню програму так, як це показано далі. У цій версії функція `funZ()` повертає добуток своїх аргументів. Зверніть увагу на те, що розташування функції праворуч від оператора присвоєння означає присвоєння змінній (розташованій зліва) значення, що повертається цією функцією.

```

#include <iostream> // Потокowe введення-виведення
using namespace std; // Використання стандартного простору імен
int funZ(int x, int y); // Попереднє оголошення прототипу функції funZ()
int main()
{
    int result;

    result = funZ(10, 11); // Присвоєння значення, що повертається функцією.
    cout << "Відповідь дорівнює " << result;
    getch(); return 0;
}

```

```
// Ця функція повертає значення.
int funZ(int x, int y)    // Визначення функції funZ()
{
    return x * y;        // Функція повертає добуток x і y.
}
```

У наведеному прикладі функція `funZ()` повертає результат обчислення виразу `x * y` за допомогою настанови `return`. Потім значення цього результату присвоюється змінній `rezult`. Таким чином, значення, що повертається настановою `return`, стає значенням функції `funZ()` у програмі, яка її викликає.

Оскільки у цій версії програми функція `funZ()` повертає значення, то її ім'я у визначенні не передуює слово `void`. Оскільки існують різні типи змінних, існують і різні типи значень, що повертаються функціями. Тут функція `funZ()` повертає значення цілого типу. Тип значення, що повертається функцією, передуює її імені як у прототипі, так і у визначенні.

У попередніх версіях мови програмування C++ для типів значень, що повертаються функціями, існувала домовленість, що діє за замовчуванням. Якщо тип значення, що повертається функцією, не вказано, то передбачалося, що ця функція повертає цілочисельне значення. Наприклад, функція `funZ()`, згідно з тією домовленістю, могла бути записана так:

```
funZ(int x, int y)    // За замовчуванням тип значення,
                    // що повертається функцією, використовується тип int.
{
    return x * y;    // Функція повертає добуток x і y.
}
```

У цьому записі за замовчуванням передбачається цілочисельний тип значення, що повертається функцією, оскільки не задано ніякого іншого типу. Проте правило встановлення цілого типу за замовчуванням було знехтуване стандартом мови програмування C++. Незважаючи на те, що більшість компіляторів підтримують це правило заради зворотної сумісності, програміст повинен безпосередньо задавати тип значення, що повертається кожною функцією, яку він пише. Але, якщо Вам доведеться мати справу зі старими версіями C++-програм, то цю домовленість необхідно мати на увазі.

Досягши настанови `return`, функція негайно завершується, а увесь решта програмний код ігнорується. Функція може містити декілька настанов `return`. Повернення з функції можна забезпечити за допомогою настанови `return` без вказання значення, що повертається, але таку її форму допустимо застосовувати тільки для функцій, які не повертають ніяких значень і оголошені з використанням ключового слова `void`.

Запитання для контролю знань:

1. У чому полягає основна відмінність між процедурою та функцією користувача?
2. Що таке формальні та фактичні параметри?
3. Які переваги надає використання підпрограм у розробці складного ПЗ?
4. Яким чином здійснюється передача параметрів за значенням та за посиланням?
5. Чи може функція викликати іншу функцію або саму себе?
6. Які вимоги ставляться до назв підпрограм?

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).
4. Жуковський С. С. Основи програмування : Навчальний посібник / С. С. Жуковський, О. В. Вакалюк. — Житомир : Вид-во ЖДУ ім. І. Франка, 2016. — 182 с.
5. Бублик В. В. Об'єктно-орієнтоване програмування : Підручник / В. В. Бублик. — Київ : ІТ-книга, 2015. — 624 с.

Тема 4.2. Структура програм. Складові частини програми. Глобальні та локальні змінні.

Навчальна мета: полягає у формуванні в учнів цілісного розуміння архітектури програмного коду та механізмів управління даними. Під час вивчення матеріалу студент має засвоїти логічну послідовність побудови коду, починаючи від підключення необхідних бібліотек і оголошення констант до розробки головної функції та допоміжних підпрограм. Особлива увага приділяється опануванню концепції області видимості ідентифікаторів, де учень повинен навчитися чітко розрізняти змінні, що доступні в межах усієї програми, та ті, що існують лише всередині конкретного блоку чи функції.

Розвиваюча мета: спрямована на активізацію аналітичних здібностей студентів через глибоке розуміння ієрархії та взаємозв'язків усередині програмного коду. Вона передбачає стимулювання логічного мислення шляхом моделювання процесів передачі даних між різними частинами програми, що дозволяє учням бачити не просто рядки коду, а цілісну динамічну систему.

Виховна мета: зосереджена на формуванні професійної відповідальності та етики сучасного розробника через усвідомлення важливості чистоти та прозорості програмного коду. Робота над структурою програми та розмежуванням областей видимості змінних виховує в учнів акуратність, системність та увагу до деталей, оскільки будь-яка необачність у глобальному просторі імен може призвести до критичних помилок.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Локальні змінні

Змінні, які оголошуються усередині функції, називаються локальними. Їх можуть використовувати тільки настанови, що належать тілу цієї функції. Локальні змінні, оголошені в одній функції, невідомі жодним зовнішнім функціям. Розглянемо конкретний приклад.

```
#include <iostream>    // Потокowe введення-виведення
using namespace std;    // Використання стандартного простору імен
void FunC();            // Попереднє оголошення прототипу функції FunC()
int main()
{
    int x;              // Локальна змінна для функції main().
    x = 10;
    FunC(); cout << endl;
                        cout << x;    // Виводиться число 10.
    getch(); return 0;
}

void FunC()            // Визначення функції FunC()
{
    int x;              // Локальна змінна для функції FunC() x = -199;
    cout << x;          // Виводиться число -199.
}
```

У цьому коді програми цілочисельна змінна з іменем `x` оголошена двічі: спочатку у функції `main()`, а потім у функції `FunC()`. Але змінна `x` з функції `main()` не має жодного стосунку до змінної `x` з функції `FunC()`. Іншими словами, зміни, яким піддається змінна `x` з функції `FunC()`, ніяк не позначаються на змінній `x` з функції `main()`. Тому наведена вище програма виведе на екран числа - 199 і 10.

Локальна змінна відома тільки функції, у якій вона визначена.

У мові програмування C++ локальні змінні створюються під час виклику функції та руйнуються при виході з неї. Те саме можна сказати і про пам'ять, що виділяється для локальних змінних: під час виклику функції в неї записуються відповідні значення, а при виході з функції пам'ять звільняється. Це означає, що локальні змінні не підтримують своїх значень між викликами функцій.

У деяких літературних джерелах, присвячених мові програмування C++, локальна змінна називається *динамічною* або *автоматичною змінною*. Але у цьому посібнику ми дотримуватимемося поширенішого і відомого терміну – *локальна змінна*.

Формальний параметр — локальна змінна, яка набуває значення аргументу, що передається функції.

2. Глобальні змінні

Щоб наділити змінну "всепрограмною" популярністю, її необхідно зробити глобальною. На відміну від локальних, глобальні змінні зберігають свої значення протягом всього часу життя (часу існування) програми. Щоби створити глобальну змінну, її необхідно оголосити поза всіма

функціями. Доступ до глобальної змінної можна отримати з будь-якої функції.

Глобальні змінні відомі всій програмі.

У наведеному нижче коді програми змінна `pm` оголошується поза всіма функціями. Її оголошення передує функції **`main()`**. Але її з таким самим успіхом можна розмістити у іншому місці, головне, щоб вона не належала якійнебудь іншій функції. Пам'ятайте: оскільки змінну необхідно оголосити до її використання, глобальні змінні найкраще оголошувати на початку програми.

```
#include <iostream>    // Потокове введення-виведення
using namespace std;   // Використання стандартного простору імен
void Fun1();           // Попереднє оголошення прототипу функції Fun1()
void Fun2();           // Попереднє оголошення прототипу функції Fun2()
int pm;                // Це глобальна змінна.
int main()
{
    int i;              // Це локальна змінна.
    for(i=0; i<10; i++) {
        pm = i * 2;
        Fun1();
    }
    getch();
    return 0;
}

void Fun1()             // Визначення функції Fun1()
{
    cout << "pm: " << pm; // Звернення до глобальної змінної.
    cout << endl;        // Виведення символу нового рядка.
    Fun2();
}

void Fun2()             // Визначення функції Fun2()
{
    int pm;              // Це локальна змінна.
    for(pm=0; pm<3; pm++) cout << ' ';
}
```

Незважаючи на те, що змінна `pm` не оголошується ні у основній функції **`main()`**, ні у функції `Fun1()`, обидві вони можуть її використовувати. Але у функції `Fun2()` оголошується локальна змінна `pm`. Тут під час звернення до змінної `pm` здійснюється доступ до локальної, а не до глобальної змінної. Важливо пам'ятати: якщо глобальна і локальна змінні мають однакові імена, то всі посилання на суперечливе ім'я змінної усередині функції, у якій визначена локальна змінна, належать локальній, а не глобальній змінній.

3. Поняття про модифікатори типів даних

У мові програмування C++ перед такими типами даних, як **`char`**, **`int`** і **`double`**, дозволяється використовувати модифікатори. Модифікатор слугує для зміни значення базового типу, щоб він точніше відповідав конкретній ситуації. Перерахуємо можливі модифікатори типів:

Модифікатори **signed**, **unsigned**, **long** і **short** можна застосовувати до цілочисельних базових типів. Окрім цього, модифікатори **signed** і **unsigned** можна використовувати з типом **char**, а модифікатор **long** – з типом **double**. Всі допустимі комбінації базових типів і модифікаторів для 16- і 32-розрядних середовищ наведено в табл. 3.2 і 3.3. У цих таблицях також вказано типи і розміри значень в бітах і діапазони представлення для кожного типу. Безумовно, реальні діапазони, що підтримуються Вашим компілятором, необхідно уточнити у відповідній документації.

Вивчаючи ці таблиці, зверніть увагу на кількість бітів, що виділяються для зберігання коротких, довгих і звичайних цілочисельних значень. Зауважте: у більшості 16-розрядних середовищ розмір (у бітах) звичайного цілочисельного значення збігається з розміром короткого цілого. Також зверніть увагу на те, що в більшості 32-розрядних середовищ розмір (у бітах) звичайного цілочисельного значення збігається з розміром довгого цілого.

Тип	Розмір у бітах	Діапазон
char	8	-128 ÷ +127
unsigned char	8	0 ÷ +255
signed char	8	-128 ÷ +127
int або enum	16	-32 768 ÷ +32 767
unsigned int	16	0 ÷ +65 535
signed int	16	Аналогічний типу int
short int	16	Аналогічний типу int
unsigned short int	16	Аналогічний типу unsigned int
signed short int	16	Аналогічний типу short int
long int	32	-2 147 483 648 ÷ +2 147 483 647
signed long int	32	Аналогічний типу long int
unsigned long int	32	0 ÷ +4 294 967 295
float	32	3,4E-38 ÷ 3,4E+38
double	64	1,7E-308 ÷ 1,7E+308
long double	80	3,4E-4932 ÷ 1,1E+4932

"Собака заритий" в C++-визначенні базових типів. Згідно зі стандартом мови програмування C++, розмір довгого цілого повинен бути не меншим за розмір звичайного цілочисельного значення, а розмір звичайного цілочисельного значення повинен бути не меншим від розміру короткого цілого. Розмір звичайного цілочисельного значення повинен залежати від середовища виконання. Це означає, що в 16-розрядних середовищах для зберігання значень типу **int** використовується 16 біт, а в 32-розрядних – 32 біти. При цьому найменший допустимий розмір для цілочисельних значень в будь-якому середовищі повинен становити 16 біт. Оскільки стандарт мови програмування C++ визначає тільки відносні вимоги до розміру цілочисельних типів, то немає гарантії, що один тип буде більший (за кількістю бітів), ніж інший.

Незважаючи на дозвіл мови C++, використання модифікатора **signed** для цілочисельних типів є надлишковим, оскільки оголошення за замовчуванням передбачає значення зі знаком. Строго кажучи, тільки конкретна реалізація визначає, яким буде **char**-оголошення: зі знаком чи без нього. Але для більшості компіляторів під оголошенням типу **char** розуміють значення зі знаком. Таким чином, у таких середовищах використання модифікатора **signed** або **char**-оголошення також є надлишковим. У цьому посібнику вважається, що **char**-значення мають знак.

Відмінність між цілочисельними значеннями із знаком і без нього полягає в інтерпретації старшого розряду. Якщо задано цілочисельне значення із знаком, C++-компілятор згенерує код з урахуванням того, що старший розряд значення використовується як прапорець знаку. Якщо прапорець знаку дорівнює 0, то число вважається позитивним, а якщо він дорівнює 1 – негативним. Негативні числа майже завжди представляються в додатковому коді. Для отримання додаткового коду програми всі розряди числа беруться в зворотному коді, а потім отриманий результат збільшується на одиницю.

Цілочисельні значення із знаком ("+" чи "-") використовуються в багатьох алгоритмах, але максимальне число, яке можна представити із знаком, становить тільки половину від максимального числа, яке можна представити без знаку. Розглянемо, наприклад, максимально можливе 16-розрядне ціле число (32 767): 01111111 11111111

Якби старший розряд цього значення із знаком дорівнював 1, то воно б інтерпретувалося як -1 (у додатковому коді). Але, якщо оголосити його як **unsigned int**-значення, то після встановлення його старшого розряду в +1 ми отримали б число 65 535.

Щоб зрозуміти відмінність в C++-інтерпретації цілочисельних значень із знаком і без нього, виконаємо таку коротку програму.

```
#include <iostream>           // Потокowe введення-виведення
using namespace std;          // Використання стандартного простору імен

int main()
{
    short int c;               // Коротке int-значення із знаком
    short unsigned int d;      // Коротке int-значення без знаку

    d = 60000;
    c = d;
    cout << c << " " << d;

    getch(); return 0;
}
```

У мові програмування C++ передбачено скорочений спосіб оголошення **unsigned**-, **short** і **long**-значень цілого типу. Це означає, що під час оголошення **int**-значень достатньо використовувати слова **unsigned**, **short** і **long**, не вказуючи тип **int**, тобто тип **int** мається на увазі. Наприклад, такі дві настанови оголошують цілочисельні змінні без знаку:

```
unsigned x; unsigned int y;
```

Змінні типу **char** можна використовувати не тільки для зберігання ASCII-символів, але і для зберігання числових значень. Змінні типу **char** можуть містити "невеликі" цілі числа в діапазоні -128...+127 і тому їх можна використовувати замість **int**-змінних, якщо Вас влаштовує такий діапазон представлення чисел. Наприклад, у наведеному нижче коді програми **char**-змінну використовують для керування циклом, який виводить на екран алфавіт англійської мови.

```
#include <iostream>           // Потокowe введення-виведення
using namespace std;          // Використання стандартного простору імен

int main()
{
    char letter;

    for(letter = 'z'; letter >= 'A'; letter--) cout << letter;

    getch(); return 0;
}
```

Запитання для контролю знань:

1. У чому полягає суттєва різниця між оголошенням та визначенням змінної?
2. Дайте визначення глобальним змінним: де вони оголошуються та яка їхня область видимості?
3. Що таке локальні змінні та який їхній життєвий цикл у межах функції чи блоку?
4. Що станеться, якщо локальна та глобальна змінні мають однакові імена? Поясніть механізм затінення (shadowing).
5. Чому використання локальних змінних вважається більш безпечним підходом з точки зору архітектури програми?

Література:

1. Васильєв О. М. Програмування мовою С++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою С++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).
4. Жуковський С. С. Основи програмування : Навчальний посібник / С. С. Жуковський, О. В. Вакалюк. — Житомир : Вид-во ЖДУ ім. І. Франка, 2016. — 182 с.
5. Бублик В. В. Об'єктно-орієнтоване програмування : Підручник / В. В. Бублик. — Київ : ІТ-книга, 2015. — 624 с.

Змістовий модуль 5. Основи алгоритмізації

Тема 5.1. Алгоритми і величини. Лінійні обчислювальні алгоритми

Навчальна мета: полягає у формуванні в учнів фундаментального розуміння поняття алгоритму як чіткої послідовності дій та вивченні базових складників програмування. Головним завданням є опанування поняття величини, її властивостей та типів, а також засвоєння правил побудови лінійних структур, де команди виконуються послідовно одна за одною. У результаті навчання учні мають навчитися аналізувати умову задачі, визначати вхідні та вихідні дані, будувати логічні схеми розв'язання та реалізовувати найпростіші обчислювальні процеси, де результат досягається шляхом прямого виконання арифметичних операцій без розгалужень чи повторень.

Розвиваюча мета: спрямована на активне стимулювання логічного та алгоритмічного мислення учнів через вибудовування чітких причинно-наслідкових зв'язків між етапами розв'язання задачі. Вона передбачає розвиток здатності до абстрагування, що дозволяє учням сприймати реальні процеси як сукупність інформаційних об'єктів та операцій над ними. Особлива увага приділяється формуванню інтелектуальної гнучкості, вмінню аналізувати структуру лінійних процесів, критично оцінювати ефективність обраного шляху обчислень та розвивати когнітивну стійкість під час роботи з формальними мовами й математичними моделями.

Виховна мета: зосереджена на формуванні відповідального ставлення до точності та достовірності результатів власної інтелектуальної праці. Вона передбачає виховання культури запису алгоритмів, що привчає учнів до охайності, дисциплінованості та дотримання встановлених стандартів. Важливим аспектом є розвиток самостійності у прийнятті рішень під час побудови обчислювальних моделей, наполегливості у подоланні труднощів при налагодженні логічних кроків, а також плекання поваги до інтелектуальної власності та етичних норм використання інформаційних технологій у повсякденному житті.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

Оператори відношення та логічні (булеві) оператори, які часто йдуть "рука в руку", використовуються для отримання результатів у вигляді значень ІСТИНА/ФАЛЬШ. Оператори відношення оцінюють за "двобальною системою" відношення між двома значеннями, а логічні визначають різні способи поєднання дійсних і помилкових значень. Оскільки оператори відношення генерують ІСТИНА/ФАЛЬШ – результати, то вони часто виконуються з логічними операторами. Тому вони і розглядаються у одному розділі

Табл. 3.8. Оператори відношення та логічні оператори

Оператори відношення	Значення
=	дорівнює
!=	не дорівнює
>	більше
<	менше
>=	більше або дорівнює
<=	менше або дорівнює
Логічні оператори	Значення
&&	І
	АБО
!	НЕ

Оператори відношення та логічні (булеві) оператори перераховано в табл. 3.8. Зверніть увагу на те, що у мові програмування C++ як оператор відношення "не дорівнює" використовує символ " !=", а для оператора "дорівнює" – подвійний символ рівності (==). Згідно зі стандартом мови програмування C++, результат виконання операторів відношення і логічних операторів має тип **bool**, тобто у процесі виконання операцій відношення і логічних операцій виходять значення **true** або **false**. Під час використання старших компіляторів результати виконання цих операцій мали тип **int** (нуль або ненульове ціле, наприклад 1). Ця відмінність в інтерпретації значень має в основному теоретичну основу, оскільки мова програмування C++ автоматично перетворює значення **true** в 1, а значення **false** – в 0, і навпаки.

Операнди, що беруть участь в операціях "з'ясування" відношення, можуть мати практично будь-який тип, головне, щоб їх можна було порівнювати. Що стосується логічних операторів, то їх операнди повинні мати тип **bool**, і результат логічної операції завжди матиме тип **bool**. Оскільки у мові програмування C++ будь-яке ненульове число оцінюється як істинне (**true**), а нуль еквівалентний помилковому значенню (**false**), то логічні оператори можна використовувати в будь-якому виразі, який дає нульовий або ненульовий результат.

Логічні оператори використовують для підтримки базових логічних операцій І, АБО і НЕ відповідно до такої таблиці істинності. У ній 1 використовується як значення ІСТИНА, а 0 – як значення ФАЛЬШ.

p	q	p І q	p АБО q	НЕ p
0	0	0	0	1
0	1	0	1	1
1	1	1	1	0
1	0	0	1	0

Незважаючи на те, що мова програмування C++ не містить вбудованого логічного оператора, що

"виключає АБО" (XOR), його неважко "створити" на основі вбудованих. Подивіться, як наведено нижче функція використовує оператори І, АБО і НЕ для виконання операції, що "виключає АБО":

```
bool XOR (bool a, bool b)
{
    return (a || b) &&! (a && b);
}
```

Ця функція використовується у наведеному нижче коді програми. Вона відображає результати застосування операторів І, АБО і що "виключає АБО" до значень, що вводяться Вами ж¹.

```
XOR()
#include <iostream>    // Потокowe введення-виведення
using namespace std;   // Використання стандартного простору імен
bool XOR(bool a, bool b); // Попереднє оголошення логічної функції
int main()
{
    bool p, q;
    cout << "Введіть P (0 або 1): "; cin >> p; cout << "Введіть Q (0 або 1): "; cin >>
    q; cout << "P І Q: " << (p && q) << endl; cout << "P АБО Q: " << (p || q)
    << endl;
    cout << "P XOR Q: " << XOR(p, q) << endl;
    getch(); return 0;
}

bool XOR(bool a, bool b)    // Визначення логічної функції
{
    return (a || b) &&! (a && b);
}
```

Ось як виглядає можливий результат виконання цієї програми.

```
Введіть P (0 або 1): 1
Введіть Q (0 або 1): 1 P І Q: 1
P АБО Q: 1 P XOR Q: 0
```

У цьому коді програми зверніть увагу ось на що. Хоча параметри функції XOR() вказані з типом **bool**, користувач вводить цілочисельні значення (0 або 1). У цьому немає нічого дивного, оскільки мова програмування C++ автоматично перетворить число 1 в **true**, а 0 – в **false**. І навпаки, при виведенні на екран **bool**-значення, що повертається функцією XOR(), воно автоматично перетвориться в число 0 або 1 (залежно від того, яке значення "повернулося": **false** або **true**). Цікаво відзначити, що, коли типи параметрів функції XOR() і тип значення, що повертається нею, замінити типом **int**, ця функція працюватиме абсолютно так само. Причина проста: вся справа в автоматичних перетвореннях, що виконуються C++-компілятором між цілочисельними і булевими значеннями.

Як оператори відношення, так і логічні оператори мають нижчий пріоритет порівняно з арифметичними операторами. Це означає, що такий вираз, як $10 > 1+12$ буде обчислено так, як би його було записано у такому вигляді:

```
10 > (1+12)
```

Результат цього виразу, звичайно ж, дорівнює значенню ФАЛЬШ. Окрім цього, погляньте ще раз на настанови виведення результатів роботи попередньої програми на екран.

```
cout << "P І Q: " << (p && q) << endl; cout << "P АБО Q: " << (p || q) << endl;
```

Без круглих дужок, у які поміщені вирази $p \&\& q$ і $p || q$, тут обійтися не можна, оскільки оператори **&&** і **||** мають нижчий пріоритет, ніж оператор виведення даних.

За допомогою логічних операторів можна об'єднати в одному виразі будь-яку кількість операцій відношення. Наприклад, у цьому виразі об'єднано відразу три операції відношення:

```
var>15 ||!(10<pm) && 3<=item
```

У наведеній нижче таблиці наведено пріоритет операторів відношення і логічних операторів.

Табл. 3.9. Пріоритет операторів відношення та логічних операторів

Пріоритет	Оператори
Найвищий	!
	> >= < <=
	= !=
	&&
Нижчий	

Запитання для контролю знань:

1. Що таке лінійний алгоритм?
2. Дайте визначення алгоритму. Які основні властивості він повинен мати, щоб бути виконуваним?
3. Що таке величина в програмуванні? Наведіть приклад величини, використовуючи метафору "шухляди"
4. Як працює команда присвоювання (:=)? Опишіть покроково, що відбувається в пам'яті комп'ютера під час виконання операції `a := a + 5`.

Література:

1. Васильєв О. М. Програмування мовою C++: Навчальний посібник / О. М. Васильєв. — Тернопіль : Навчальна книга – Богдан, 2019. — 464 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).

Змістовий модуль 6. Масиви

Тема 6.1. Одновимірні масиви

Навчальна мета: Навчальна мета заняття спрямована на комплексне опанування поняття одновимірного масиву як упорядкованої сукупності однотипних даних, що об'єднані спільним іменем. У ході вивчення теми необхідно забезпечити глибоке розуміння механізмів оголошення масивів у пам'яті комп'ютера, принципів ініціалізації та способів звернення до окремих елементів через їхні індекси.

Розвиваюча мета: спрямована на інтенсивне формування алгоритмічного та логічного мислення через усвідомлення принципів структурування та систематизації великих обсягів даних. Процес роботи з масивами стимулює здатність до аналізу та синтезу, оскільки вимагає від студента вміння розбивати складну задачу на послідовність ітераційних кроків та впевнено оперувати абстракціями індексів.

Виховна мета: спрямована на формування академічної доброчесності та високої культури програмування, що виявляється у відповідальному ставленні до написання чистого, зрозумілого та оптимізованого коду. Процес роботи з масивами виховує у студентів наполегливість та витримку, оскільки пошук помилок в індексації та налагодження циклів обробки даних потребують особливої концентрації та терпіння.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Одновимірні масиви

Масив (array) – перелік змінних однакового типу, звернення до яких відбувається із застосуванням імені, загального для всіх його елементів. У мові програмування C++ масиви можуть бути одно, дво та багатовимірними, хоча в основному використовуються одновимірні масиви. Масиви є зручними засобами для групування взаємопов'язаних між собою змінних.

Одновимірний масив – перелік взаємопов'язаних між собою змінних. Для оголошення одновимірного масиву використовують така форма запису:

тип ім'я_масиву[розмір];

У цьому записі за допомогою елемента запису *тип* оголошується базовий тип масиву. Базовий *тип* визначає тип даних кожного елемента, з яких складається масив. Кількість елементів, які зберігатимуться в масиві, визначається елементом *розмір*. Наприклад, у процесі виконання наведеної нижче настанови оголошується **int**-масив (що складається з 10 елементів) з іменем *Array*:

```
int Array[10];
```

Індекс у прямокутних дужках після імені масиву вказує на конкретний елемент масиву. Доступ до окремого елемента масиву здійснюється за допомогою індексу, який описує позицію елемента усередині масиву.

Перший елемент масиву має нульовий індекс. Оскільки масив *Array* містить 10 елементів, то його індекси змінюються від 0 до 9. Щоб отримати доступ до елемента масиву за індексом, достатньо вказати потрібний номер елемента в квадратних дужках. Так, наприклад, першим елементом масиву *Array* є *Array[0]*, а останнім – *Array[9]*. Розглянемо наведену нижче програму, яка ініціалізує масив *Array* квадратами чисел від 1 до 10.

```
#include <iostream>           // Потокowe введення-виведення
#include <conio>                // Консольний режим роботи
using namespace std;          // Використання стандартного простору імен

int main()
{
    int Array[10]; // Ця настанова резервує область пам'яті для 10 елементів типу int. clrscr();           // Очищення екрану

    // Поміщаємо в масив значення.
    for(int t=0; t<10; ++t) Array[t] = (t+1)*(t+1);

    // Відображається масив.
    for(int t=0; t<10; ++t) cout << Array[t] << " ";

    getch(); return 0;
}
```

У мові програмування C++ всі масиви займають суміжні елементи пам'яті. Іншими словами, елементи масиву в пам'яті розташовані послідовно один за одним. Клітина з найменшою адресою належить до першого елемента масиву, а з найбільшою – до останнього. Наприклад, після виконання цього фрагмента коду програми

```
int Array[7];
for(int j=0; j<7; j++) Array[j] = j*j;
```

масив *Array* виглядатиме так:

i[0] i[1] i[2] i[3] i[4] i[5] i[6]

0	1	4	9	16	25	36
---	---	---	---	----	----	----

Для одновимірних масивів загальний розмір масиву в байтах обчислюється так:

всього байтів = розмір типу елемента в байтах кількість елементів.

Масиви часто використовують під час програмування, оскільки дають змогу легко обробляти велику кількість взаємопов'язаних між собою змінних. Наприклад, у наведеному нижче коді

програми створюється масив з десяти елементів, кожному елементу присвоюється випадкове число, а потім на екрані відображаються мінімальне і максимальне його значення.

```
#include <iostream>           // Потокowe введення-виведення
#include <cstdlib>             // Використання бібліотечних
// функцій
#include <conio>                // Консольний режим роботи
using namespace std;          // Використання стандартного простору імен

int main()
{
    int min, max, Array[10];
    clrscr();                  // Очищення екрану
    for(int i=0; i<10; i++) Array[i] = rand();

    // Знаходимо мінімальне значення. min = Array[0];
    for(int i=1; i<10; i++)
        if(min > Array[i]) min = Array[i];
    cout << "Мінімальне значення: " << min << endl;

    // Знаходимо максимальне значення. max = Array[0];
    for(int i=1; i<10; i++)
        if(max < Array[i]) max = Array[i];
    cout << "Максимальне значення: " << max << endl;

    getch(); return 0;
}
```

У мові програмування C++ не можна присвоїти один масив іншому. У наведеному нижче коді програми, наприклад, присвоєння `aMas = bMas`; неприпустимо:

```
int aMas[10], bMas[10];
//...
aMas = bMas; // Помилка!!!
```

Щоб помістити вміст одного масиву в інший, необхідно окремо виконати присвоєння кожного значення:

```
int aMas[10], bMas[10], i;
//...
for(i=1; i<10; i++) aMas[i] = bMas[i]; // Правильно!
//...
```

2. Організація контролю меж масивів

У мові програмування C++ не здійснюється ніякої перевірки порушення контролю меж масивів, тобто нічого не може перешкодити програмісту звернутися до масиву за його межами. Якщо це відбувається у процесі виконання настанови присвоєння, то можуть бути змінені значення в елементах пам'яті, виділених деяким іншим змінним або навіть Вашій програмі. Іншими словами, звернення до масиву (розміром у N елементів) за межею N-го елемента може призвести до руйнування програми за відсутності яких-небудь зауважень з боку компілятора і без видачі повідомлень про помилки під час роботи програми.

Це означає, що вся відповідальність за дотримання "кордонів" масивів покладається тільки на програмістів, які повинні гарантувати коректну роботу з масивами. Іншими словами, програміст зобов'язаний використовувати масиви достатньо великого розміру, щоб в них можна було без ускладнень поміщати дані. Однак найкраще у програмі передбачити перевірку перетину "кордонів" масивів.

Наприклад, C++-компілятор "мовчки" скомпілює і дасть змогу запустити таку програму на виконання, хоча у ній відбувається вихід за межі масиву `myArray`.

```
int main()
{
    int myArray[10];

    for(int i=0; i<100; i++) myArray[i] = i;

    return 1;
}
```

У цьому випадку цикл **for** виконає 100 ітерацій, хоча масив `myArray` призначений для зберігання тільки десяти елементів. У процесі виконання цієї програми можливий перезапис важливої інформації, що може призвести до аварійної зупинки програми.

Вас, можливо, дивує така "непередбачливість" мови програмування C++, яка виражається у відсутності вбудованих засобів динамічної перевірки на "недоторканність" меж масивів. Нагадаємо, проте, що мова програмування C++ призначена для професійних програмістів, і її завдання – надати їм можливість створювати максимально ефективний програмний код. Будь-яка перевірка коректності доступу засобами мови програмування C++ істотно уповільнює виконання програми. Тому такі дії залишено на розгляд програмістам. Як буде показано далі, у разі потреби програміст може сам визначити тип масиву і закласти у ньому перевірку непорушності меж.

3. Сортування елементів масиву

Однією з найпоширеніших операцій, що виконуються над масивами, є сортування. Існує багато різних алгоритмів сортування. Широко застосовується, наприклад, сортування перемішуванням і сортування методом Шелла. Відомий також алгоритм Quicksort (швидке сортування з розбиттям початкового набору даних на дві половини так, що будь-який елемент першої половини впорядкований щодо будь-якого елемента другої половини). Проте найпростішим вважають алгоритм сортування методом бульбашок. Незважаючи на те, що бульбашкове сортування не відрізняється високою ефективністю (і справді, його продуктивність неприйнятна для сортування великих масивів), його цілком успішно можна застосовувати для сортування масивів малого розміру.

Алгоритм сортування методом бульбашок отримав свою назву від способу, використовуваного для впорядковування елементів масиву. Тут виконуються операції порівняння, що повторюються, і у разі потреби міняються місцями суміжні елементи. При цьому елементи з меншими значеннями поступово переміщуються до одного кінця масиву, а елементи з великими значеннями – до іншого. Цей процес нагадує поведінку бульбашок повітря в резервуарі з водою. Бульбашкове сортування здійснюється шляхом декількох проходів по масиву, під час яких у разі потреби здійснюється перестановка елементів, що опинилися "не на своєму місці". Кількість проходів, що гарантують отримання відсортованих елементів масиву, дорівнює кількості елементів у масиві, зменшеному на одиницю.

У наведеному нижче коді програми реалізовано алгоритм сортування елементів масиву (цілого типу), що містить випадкові числа. Ця програма заслуговує уважного розбору.

```
#include <iostream>           // Потокове введення-виведення #include <cstdlib>       // Використання бібліотечних
функцій #include <conio>       // Консольний режим роботи
using namespace std;          // Використання стандартного простору імен

int main()
{
    int Array[10];
    int a, b, t, size = 10;    // Кількість елементів, що підлягають сортуванню.

    // Поміщаємо в масив випадкові числа.
    for(t=0; t<size; t++) Array[t] = rand();

    // Відображаємо початковий масив.
    cout << "Початковий масив: ";
    for(t=0; t<size; t++) cout << Array[t] << " ";
```

```

cout << endl;

// Реалізація методу бульбашкового сортування.
for(a=1; a<size; a++)
    for(b=size-1; b>=a; b--) {
        if(Array[b-1]> Array[b]) { // Елементи нерегульовані.
            // Міняємо елементи місцями.
            t = Array[b-1]; Array[b-1] = Array[b]; Array[b] = t;
        }
    } // Кінець бульбашкового сортування.

// Відображаємо відсортований масив.
cout << "Відсортований масив: ";

for(t=0; t<size; t++) cout << Array[t] << " ";
cout << endl;

getch(); return 0;
}

```

Хоча алгоритм бульбашкового сортування придатний для невеликих масивів, для масивів великого розміру він стає неефективним. Більш універсальним вважають алгоритм Quicksort. У стандартну бібліотеку мови програмування C++ включена функція **qsort()**, яка реалізує одну з версій цього алгоритму. Але, перш ніж використовувати її, Вам необхідно вивчити більше засобів мови програмування C++.

Запитання для контролю знань:

1. Яке з визначень найбільш точно описує одновимірний масив?
2. Якщо масив містить N елементів, який індекс буде мати останній елемент при індексації з нуля?
3. Що означає поняття 'однотипність' елементів масиву?
4. Як називається операція звернення до кожного елемента масиву по черзі?
5. Який цикл найчастіше використовується для обробки одновимірних масивів?
6. Для чого використовується змінна-прапорець (flag) при роботі з масивами?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Коноваленко І. В. Програмування мовою C++: Навчальний посібник / І. В. Коноваленко, Ф. Р. Вишневський. — Тернопіль : ТНТУ імені Івана Пулюя, 2016. — 188 с. (с. 10-25 — Історія та класифікація).
3. Бернацький П. В. Основи алгоритмізації та програмування: Підручник / П. В. Бернацький, В. В. Бісікало. — Вінниця : ВНТУ, 2017. — 134 с. (с. 88-92 — Інтегровані середовища розробки).
4. Жуковський С. С. Основи програмування : Навчальний посібник / С. С. Жуковський, О. В. Вакалюк. — Житомир : Вид-во ЖДУ ім. І. Франка, 2016. — 182 с.
5. Бублик В. В. Об'єктно-орієнтоване програмування : Підручник / В. В. Бублик. — Київ : ІТ-книга,

2015. — 624 с.

Тема 6.2. Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.

Навчальна мета: полягає у формуванні в студентів цілісного розуміння принципів роботи з текстовими даними як ключовим елементом сучасної розробки програмного забезпечення. Студенти мають опанувати техніки стандартного введення та виведення символьних рядків, глибоко засвоїти синтаксис і логіку використання спеціалізованих функцій для маніпуляції строками, а також навчитися ефективно інтегрувати текстові змінні в архітектуру програм через їхню передачу у функції та процедури.

Розвиваюча мета: орієнтована на інтенсивне вдосконалення алгоритмічного мислення та здатності до аналітичного розбору складних структур даних. Вона передбачає розвиток у студентів когнітивної гнучкості при виборі методів обробки текстової інформації, а також формування інтуїтивного розуміння того, як високорівневі операції зі строками трансформуються у конкретні маніпуляції з пам'яттю.

Виховна мета: спрямована на формування професійної відповідальності за якість створеного програмного продукту та культури написання чистого, зрозумілого коду. Вона сприяє розвитку прискіпливості до деталей і терплячості під час налагодження складних алгоритмів, що виховує в майбутніх фахівцях наполегливість у досягненні поставленої мети.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Побудова символьних рядів

Найчастіше одновимірні масиви використовуються для побудови символьних рядків. У мові програмування C++ рядок визначається як символьний масив, який завершується нульовим символом ('\0'). Під час визначення довжини символьного масиву необхідно враховувати ознаку його завершення, тобто задавати його довжину на одиницю більше довжини найбільшого рядка, які передбачають зберігати у цьому масиві.

Рядок – символьний масив, який завершується нульовим символом.

Оголошуючи масив str, призначеного для зберігання 10-символьного рядка, потрібно використовувати таку настанову:

```
char str[11];
```

Заданий тут розмір (11) дає змогу зарезервувати місце для нульового символу в кінці рядка.

Як було зазначено вище, мова програмування C++ дає змогу визначати рядкові літерали. Пригадаємо, що *рядковий літерал* – перелік символів, поміщений в подвійні лапки. Ось декілька прикладів:

```
"Привіт"  
"Мені подобається мова програмування C++" "%@#@#"  
""
```

Рядок, наведений останнім (""), називається *нульовим*. Він складається тільки з одного нульового символу (ознаки завершення рядка). Нульові рядки використовуються для представлення порожніх рядків.

Програмісту не потрібно вручну добавляти в кінець рядкових констант нульові символи. C++-компілятор робить це автоматично. Отже, рядок "Привіт" в пам'яті розміщується так, як це показано на цьому рисунку:

П	Р	И	В	І	Т	'\0'
---	---	---	---	---	---	------

2. Зчитування рядів з клавіатури

Найпростіше зчитувати рядок з клавіатури, створивши масив, який прийме цей рядок за допомогою настанови cin. Зчитування рядка, введеного користувачем з клавіатури, відображено у наведеному нижче коді програми.

Код програми 5.4. Демонстрація механізму використання cin-настанови для зчитування рядка з клавіатури

```
#include <iostream>           // Потокowe введення-виведення  
#include <conio>               // Консольний режим роботи  
using namespace std;          // Використання стандартного простору імен  
  
int main()  
{  
    char str[80];  
  
    cout << "Введіть рядок: "; cin >> str; // Зчитуємо рядок з клавіатури.  
    cout << "Ось Ваш рядок: ";  
    cout << str;  
  
    getch(); return 0;  
}
```

Хоча ця програма формально коректна, однак вона не позбавлена недоліків. Розглянемо такий результат її виконання:

Введіть рядок: Це перевірка
Ось Ваш рядок: Це

Як бачимо, під час виведення рядка, введеного з клавіатури, програма відображає тільки слово "Це", а не весь рядок. Йдеться про те, що оператор "<<" припиняє зчитування рядка, як тільки трапляється символ пропуску, табуляції або нового рядка (називатимемо ці символи

пропускними). Для вирішення цього питання можна використовувати ще одну бібліотечну функцію **gets()**. Загальний формат її виклику є таким:

```
gets(ім'я_масиву);
```

Якщо у програмі необхідно зчитувати рядок з клавіатури, то викличте функцію **gets()**, а як аргумент передайте ім'я масиву, не вказуючи індексу. Після виконання цієї функції заданий масив міститиме текст, введений з клавіатури. Функція **gets()** зчитує символи, які вводяться користувачем, доти, доки він не натисне на клавішу <Enter>. Для виклику функції **gets()** у програму необхідно включити заголовок **<cstdio>**.

У наведеній нижче версії попередньої програми продемонстровано механізм використання функції **gets()**, яка дає змогу ввести в масив рядок символів, що містить пропуски.

```
#include <iostream>           // Потокowe введення-виведення
#include <cstdio>              // Підтримка системи введення-виведення
#include <conio>                // Консольний режим роботи

using namespace std;          // Використання стандартного простору імен

int main()
{
    char str[80];

    cout << "Введіть рядок: "; gets(str); // Зчитуємо рядок з клавіатури.
    cout << "Ось Ваш рядок: ";
    cout << str;

    getch(); return 0;
}
```

Цього разу після запуску нової версії програми на виконання і введення з клавіатури тексту "Це простий тест" рядок зчитується повністю, а потім так само повністю і відображається:

```
Введіть рядок: Це простий тест Ось Ваш рядок: Це простий тест
```

У цьому коді програми зверніть увагу на таку настанову:

```
cout << str;
```

У цьому записі (замість звичного літерала) використовують ім'я рядкового масиву. І хоча причина такого використання настанови **cout** нам стане зрозумілою після вивчення ще декількох розділів цього навчального посібника, поки що стисло зазначимо, що ім'я символьного масиву, який містить рядок, можна використовувати скрізь, де допустимо застосування рядкового літерала. При цьому майте на увазі, що ні оператор "<<", ні функція **gets()** не виконують граничної перевірки (на відсутність порушення меж масиву). Тому, якщо користувач введе рядок, довжина якого перевищує розмір масиву, то можливі неприємності, про які згадувалося вище. Із сказаного виходить, що обидва описаних тут варіанти зчитування рядків з клавіатури є потенційно небезпечними.

Мова програмування C++ підтримує багато функцій для оброблення рядків.

Найпоширенішими з них є такі: **strcpy()**, **strlen()**, **strcmp()**.

Для виклику всіх цих функцій у програму необхідно включити заголовок **<cstring>**. Тепер познайомимося з кожною функцією окремо.

3. Використання функцій роботи з рядками

Загальний формат виклику функції **strcpy()** є таким:

```
strcpy(to, from);
```

Функція **strcpy()** копіює вміст рядка *from* в рядок *to*. Необхідно пам'ятати, масив, який використовують для зберігання рядка *to*, повинен бути достатньо великим, щоб в нього можна було помістити рядок з масиву *from*. Інакше масив *to* переповниться, тобто відбудеться вихід за його межі, що може призвести до руйнування програми.

Використання функції **strcpy()** продемонстровано у наведеному нижче коді програми, яка копіює рядок "Привіт" в рядок *str*:

```
#include <iostream>           // Потокowe введення-виведення
#include <cstring>            // Робота з рядковими типами даних
using namespace std;          // Використання стандартного простору імен
```

```
int main()
{
    char str[80];

    strcpy(str, "Привіт");
    cout << str << endl << "";

    getch(); return 0;
}
```

Звернення до функції **strcat()** має такий формат:

strcat(s1, s2);

Функція **strcat()** приєднує рядок *s2* до кінця рядка *s1*; при цьому рядок *s2* не змінюється. Обидва рядки повинні завершуватися нульовим символом. Результат виклику цієї функції, тобто остаточний рядок *s1* також завершуватиметься нульовим символом. Використання функції **strcat()** продемонстровано у наведеному нижче коді програми, яка має вивести на екран рядок "Привіт усім!".

```
#include <iostream>           // Потокowe введення-виведення
#include <cstring>             // Робота з рядковими типами даних
#include <conio>                // Консольний режим роботи
using namespace std;          // Використання стандартного простору імен

int main()
{
    char s1[20], s2[10]; strcat(s1, "Привіт"); strcat(s2, " усім!"); strcat(s1, s2);
    cout << s1;

    getch(); return 0;
}
```

Звернення до функції **strcmp()** має такий формат:

strcmp(s1, s2);

Функція **strcmp()** порівнює рядок *s2* з рядком *s1* і повертає значення 0, якщо вони однакові. Якщо рядок *s1* лексикографічно (тобто відповідно до алфавітного порядку) більший від рядка *s2*, то повертається позитивне число. Якщо рядок *s1* лексикографічно менший від рядка *s2*, то повертається негативне число. Використання функції **strcpy()** продемонстровано у наведеному нижче коді програми, яка слугує для перевірки правильності пароля, введеного користувачем (для введення пароля з клавіатури і його верифікації слугує функція **password()**).

```
#include <iostream>           // Потокowe введення-виведення
#include <cstring>             // Робота з рядковими типами даних
#include <cstdlib>             // Підтримка системи введення-виведення
using namespace std;          // Використання стандартного простору імен

bool password(); int main()
{
    if(password()) cout << "Вхід дозволено" << endl; else cout << "У доступі відмовлено" << endl;
    getch(); return 0;
}

// Функція повертає значення true, якщо пароль прийнятий, і значення false інакше.
bool password()
{
    char str[80];

    cout << "Введіть пароль: "; gets(str); if(strcmp(str, "пароль")) { // Рядки різні.
        cout << "Пароль недійсний" << endl; return false;
    }
}
```

```

    }
    // Порівнювані рядки збігаються
return true;
}

```

Під час застосування функції **strcmp()** важливо пам'ятати, що вона повертає число (тобто значення **false**), якщо порівнювані рядки однакові. Отже, якщо Вам необхідно виконати певні дії за умови збігу рядків, то програміст повинен використовувати оператор НЕ (!). Наприклад, у процесі виконання наведеної нижче програми запит вхідних даних продовжується доти, доки користувач не введе слово "Вихід".

```

#include <iostream>      // Потокowe введення-виведення
#include <cstdio>         // Підтримка системи введення-виведення
#include <cstring>        // Робота з рядковими типами даних
#include <conio>          // Консольний режим роботи
using namespace std;    // Використання стандартного простору імен

int main()
{
    char str[80];
    for(;;) {
        cout << "Введіть рядок: "; gets(str); if(!strcmp("Вихід", str)) break;
    }
    getch(); return 0;
}

```

Загальний формат виклику функції **strlen()** є таким:

```
strlen(s);
```

У цьому записі *s* – рядок. Функція **strlen()** повертає довжину рядка, вказаного аргументом *s*. У процесі виконання наведеної нижче програми буде показано довжину рядка, введеного з клавіатури.

Код програми 5.10. Демонстрація механізму використання функції strlen()

```

#include <iostream>      // Потокowe введення-виведення
#include <cstdio>         // Підтримка системи введення-виведення
#include <cstring>        // Робота з рядковими типами даних
using namespace std;    // Використання стандартного простору імен

int main()
{
    char str[80];

    cout << "Введіть рядок: "; gets(str);
    cout << "Довжина рядка дорівнює: " << strlen(str);

    getch(); return 0;
}

```

Якщо користувач введе рядок "Привіт усім!", програма виведе на екрані число 12. При підрахунку символів, з яких складається заданий рядок, ознака завершення рядка (нульовий символ) не враховується.

А у процесі виконання цієї програми рядок, введений з клавіатури, буде відображено на екрані в зворотному порядку. Наприклад, під час введення слова "привіт" програма відобразить слово тівірп. Необхідно пам'ятати, що рядки є символьні масиви, які дають змогу посилатися на кожен елемент (символ) окремо.

```

#include <iostream>      // Потокowe введення-виведення
#include <cstdio>         // Підтримка системи введення-виведення
#include <cstring>        // Робота з рядковими типами даних
#include <conio>          // Консольний режим роботи
using namespace std;    // Використання стандартного простору імен

int main()
{
    char str[80];

```

```

    cout << "Введіть рядок: "; gets(str);
    for(int i=strlen(str)-1; i>=0; i--) cout << str[i];

    getch(); return 0;
}

```

У наведеному нижче прикладі продемонструємо використання всіх розглянутих вище чотирьох рядкових функцій.

```

#include <iostream>           // Потокowe введення-виведення
#include <cstdio>              // Підтримка системи введення-виведення
#include <cstring>             // Робота з рядковими типами даних
#include <conio>               // Консольний режим роботи
using namespace std;         // Використання стандартного простору імен

int main()
{
    char s1[80], s2[80];
    clrscr();                // Очищення екрану

    cout << "Введіть два рядки: "; gets(s1); gets(s2); cout << "Їх довжини дорівнюють: " << strlen(s1); cout << " "
    << strlen(s2) << endl;
    if(!strcmp(s1, s2))
        cout << "Рядки однакові" << endl;
        else
        cout << "Рядки не однакові" << endl;

    strcat(s1, s2);
    cout << s1 << endl;

    strcpy(s1, s2);
    cout << s1 << " і " << s2 << " ";
    cout << "тепер однакові" << endl;

    getch(); return 0;
}

```

Якщо запустити цю програму на виконання і на пропозицію ввести рядки "привіт" і "усім", то вона відобразить на екрані такі результати:

```

Їх довжини дорівнюють: 6 4
Рядки не однакові привіт усім
всім і всім тепер однакові

```

Останнє нагадування: не забувайте, що функція **strcmp()** повертає значення **false**, якщо рядки однакові. Тому, якщо Ви перевіряєте рівність рядків, необхідно використовувати оператор **!** (НЕ), щоб реверсувати умову (тобто змінити її на зворотне), як було показано в попередній програмі.

4. Механізм використання ознак завершення рядка

Факт завершення нульовими символами всіх C++-рядків можна використовувати для спрощення різних операцій над ними. Наведений нижче приклад дає змогу переконатися у тому, наскільки простий програмний код потрібен для заміни всіх символів рядка їх прописними еквівалентами.

```

#include <iostream>           // Потокowe введення-виведення
#include <cstring>             // Робота з рядковими типами даних
#include <cctype>              // Робота з символьними аргументами
using namespace std;         // Використання стандартного простору імен

int main()
{
    char str[80];
    int i;

    strcpy(str, "test");
    for(i=0; str[i]; i++) str[i] = toupper(str[i]); cout << str;
}

```

```
    getch(); return 0;  
}
```

Ця програма у процесі виконання виведе на екран слово TEST. Тут використовується бібліотечна функція **toupper()**, яка повертає прописний еквівалент свого символьного аргументу. Для виклику функції **toupper()** необхідно приєднати до програми заголовок **<cctype>**.

Зверніть увагу на те, що як умову завершення циклу **for** використано масив **str**, індексований змінній **i**, що керує (**str[i]**). Такий спосіб керування циклом цілком прийнятний, оскільки за дійсне значення у мові програмування C++ приймається будь-яке ненульове значення. Згадаймо, що всі друкарські символи представляються значеннями, не дорівнюють нулю, і тільки символ, що завершує рядок, дорівнює нулю. Отже, цей цикл працює доти, доки індекс не вкаже на нульову ознаку кінця рядка, тобто доки значення **str[i]** не стане нульовим. Оскільки нульовий символ відзначає кінець рядка, цикл зупиниться точно там, де потрібно. При подальшій роботі з цим навчальним посібником Ви побачите багато прикладів, у яких нульова ознака кінця рядка використовують так само.

Запитання для контролю знань:

1. Який символ використовується у мові C для позначення кінця рядка (нуль-термінатор)?
2. Яка функція стандартної бібліотеки повертає довжину рядка без урахування завершального нуль-символа?
3. Який метод введення рядка є найбільш безпечним з точки зору запобігання переповненню буфера?
4. Що поверне функція **strcmp(s1, s2)**, якщо рядок **s1** лексикографічно 'менший' за рядок **s2**?
5. Який результат виведе функція **puts(s)**, якщо **s = "Hello"**?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Тема 6.3. Багатовимірні масиви

Навчальна мета: формування у студентів комплексного розуміння структури та принципів роботи з багатовимірними масивами як фундаментальним інструментом для моделювання складних даних. Навчальний процес спрямований на те, щоб кожен учасник опанував логіку організації матриць та тензорів, навчився вільно оперувати індексами для доступу до елементів на різних рівнях вкладеності та зрозумів механізми розподілу пам'яті для таких структур. Окрему увагу приділено розвитку практичних навичок проектування алгоритмів, що використовують вкладені цикли для ітерації по багатовимірних просторах, що є критично важливим для вирішення задач у сферах математичного моделювання, обробки зображень та розробки ігрових механік. В результаті заняття студент має не просто вивчити синтаксис оголошення масивів, а навчитися аналітично підходити до вибору розмірності даних залежно від поставленої прикладної задачі.

Розвиваюча мета: стимулювання логічного та абстрактного мислення студентів через усвідомлення переходу від лінійного сприйняття даних до просторового. Заняття спрямоване на розвиток когнітивної гнучкості, що дозволяє візуалізувати складні ієрархічні зв'язки та будувати ментальні моделі багатовимірних об'єктів. Працюючи з індексацією, студенти вдосконалюють свою аналітичну точність, концентрацію уваги та здатність до декомпозиції складних операцій на послідовність вкладених кроків. Окрім цього, вивчення багатовимірних масивів сприяє формуванню алгоритмічної інтуїції, необхідної для оптимізації процесів та знаходження ефективних шляхів обробки великих масивів інформації в умовах обмежених ресурсів.

Виховна мета: полягає у формуванні професійної відповідальності та культури написання коду, що базується на принципах логічного порядку та поваги до майбутніх користувачів програмного продукту. Через роботу зі складними структурами даних у студентів виховується наполегливість, терплячість та уважність до деталей, оскільки робота з багатовимірними масивами не пробає неохайності в індексації. Процес розв'язання алгоритмічних задач сприяє становленню інтелектуальної самостійності та впевненості у власних силах при зустрічі з комплексними викликами. Крім того, заняття стимулює прагнення до оптимізації та раціонального використання ресурсів, що виховує ощадливе ставлення до обчислювальних потужностей як важливу рису сучасного інженера-програміста.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Організація двовимірних масивів

У програмуванні масив (англ. array) – одна з найпростіших структур даних, сукупність елементів одного типу даних, впорядкованих за індексами, які зазвичай репрезентовані натуральними числами, що визначають положення елемента в масиві. Масив може бути двовимірним (матрицею), та багатовимірним, тобто таким, де індексом є не одне число, а кортеж (сукупність) з декількох чисел, кількість яких збігається з розмірністю масиву. У переважній більшості мов програмування масив є стандартною вбудованою структурою даних.

У мові програмування C++ можна використовувати двовимірні масиви. Двовимірний масив, по суті, є списком одновимірних масивів. Щоб оголосити двовимірний масив цілочисельних значень розміром 10 20 з іменем num, достатньо записати таке:

```
int num[10][20];
```

Зверніть особливу увагу на це оголошення. На відміну від багатьох інших мов програмування, у яких під час оголошення масиву значення розмірностей відокремлюються комами, у мові програмування C++ кожна розмірність полягає у власну пару квадратних дужок.

Щоб отримати доступ до елемента масиву num з координатами 3 5, необхідно використовувати запис num[3][5]. У наведеному нижче прикладі в двовимірний масив поміщаються послідовні числа від 1 до 12.

```
#include <iostream>           // Потокowe введення-виведення
using namespace std;          // Використання стандартного простору імен

int main()
{
    int t, i, num[3][4];
    for(t=0; t<3; ++t) {
        for(i=0; i<4; ++i) {
            num[t][i] = (t*4)+i+1;
            cout << num[t][i] << " ";
        }
        cout << endl;
    }
    getch(); return 0;
}
```

У наведеному прикладі елемент масиву num[0][0] набуде значення 1, елемент num[0][1] – значення 2, елемент num[0][2] – значення 3 і т.д. Значення елемента num[2][3] буде дорівнювати числу 12. Схематично цей масив можна представити так, як це показано на рис. 5.1.

У двовимірному масиві позиція будь-якого елемента визначається двома індексами. Якщо представити двовимірний масив у вигляді таблиці даних, то один індекс означає рядок, а другий – стовпець. З цього виходить, якщо доступ до елементів масиву надати в порядку, у якому вони реально зберігаються в пам'яті, то правий індекс змінюватиметься швидше, ніж лівий.

	0	1	2	3	Правий індекс
0	1	2	3	4	
1	5	6	7	8	
2	9	10	11	12	

Рис. 5.1. Схематичне представлення масиву num

Для визначення кількості байтів пам'яті, займаної двовимірним масивом, використовують така формула:

кі-сть байтів = кі-сть рядків кі-сть стовпців розмір типу в байтах

Отже, двовимірний цілочисельний масив розмірністю 10 5 займає в пам'яті 10 5 2, тобто 100 байтів (якщо цілочисельний тип має розмір 2 байт).

2. Організація багатовимірних масивів

У мові програмування C++, окрім двовимірних, можна визначати масиви трьох і більш вимірів. Ось як оголошується багатовимірний масив:

```
тип ім'я[розмір1][розмір2]...[розмірN];
```

Наприклад, за допомогою такого оголошення створюється тривимірний цілочисельний масив розміром 4*10*3:

```
int multidim[4][10][3];
```

Як було зазначено вище, пам'ять, виділена для зберігання всіх елементів масиву, використовується протягом всього часу наявності масиву. Масиви з числом вимірювань, що перевищує три, використовуються нечасто, хоча б тому, що для їх зберігання потрібен великий об'єм пам'яті. Наприклад, зберігання елементів чотиривимірного символьного масиву розміром 10 6 9 4 займе 2 160 байтів. А якщо кожен розмір збільшити в 10 разів, то займана масивом пам'ять зросте до 21 600 000 байтів. Як бачимо, великі багатовимірні масиви здатні "з'їсти" великий об'єм пам'яті, а програма, яка їх використовує, може дуже швидко наštтовхнутися на проблему відсутності пам'яті.

3. Ініціалізація «розмірних» масивів

Ознакою масиву при описі є наявність парних дужок []. Константа або константний вираз в квадратних дужках задає число елементів масиву. При описі масиву може бути виконана ініціалізація його елементів. Існує два методи ініціалізації елементів масивів: розмірних і безрозмірних масивів.

У мові програмування C++ передбачено можливість ініціалізації елементів масиву. Формат ініціалізації елементів масиву подібний до формату ініціалізації інших змінних:

```
тип ім'я_масиву[розмір] = {перелік_значень};
```

У цьому записі елемент *перелік_значень* є перелік значень ініціалізації елементів масиву, розділених між собою комами. Тип кожного значення ініціалізації повинен бути сумісний з базовим типом масиву (елементом *тип*). Перше значення ініціалізації буде збережено в першій позиції масиву, друге значення – в другій і т.д. Зверніть увагу на те, що крапка з комою ставиться після закритої фігурної дужки (}).

Наприклад, в такому прикладі 10-елементний цілочисельний масив ініціалізувався числами від 1 до 10:

```
int Array[10] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};
```

Після виконання цієї настанови елемент `Array[0]` набуде значення 1, а елемент `Array[9]` – значення 10.

Для символьних масивів, призначених для зберігання рядків, передбачено скорочений варіант ініціалізації, який має таку форму:

```
char ім'я_масиву[розмір] = "рядок";
```

Наприклад, такий фрагмент коду програми ініціалізує масив `str` фразою "привіт".

```
char str[7] = "привіт";
```

Це рівнозначно по-елементній ініціалізації:

```
char str[7] = {'п', 'р', 'и', 'в', 'і', 'т', '\0'};
```

Оскільки у мові програмування C++ рядки повинні завершуватися нульовим символом, то переконайтеся, що під час оголошення масиву його розмір вказано з врахуванням ознаки кінця. Саме тому в попередньому прикладі масив `str` оголошений як 7-елементний, хоча у слові "привіт" тільки шість букв. Під час використання рядкового літерала компілятор додає нульову ознаку кінця рядка автоматично.

Багатовимірні масиви ініціалізуються за аналогією з одновимірними. Наприклад, такий фрагмент коду програми масив `Array` ініціалізувався числами від 1 до 10 і квадратами цих чисел.

```
int Array[10][2] = { 1, 1,
```

```

2, 4,
3, 9,
4, 16,
5, 25,
6, 36,
7, 49,
8, 64,
9, 81,
10, 100
};

```

Тепер розглянемо, як елементи масиву Array розташовуються в пам'яті (рис. 5.2).

	0	1	2	← Правий індекс
0	1	1		
1	2	4		
2	3	9		
3	4	16		
4	5	25		
5	6	36		
6	7	49		
7	8	64		
8	9	81		
9	10	100		

↑ Лівий індекс

Рис. 5.2. Схематичне представлення ініціалізованого масиву Array

Під час ініціалізації багатовимірного масиву перелік ініціалізацій кожної розмірності (підгрупу ініціалізацій) можна помістити у фігурні дужки. Ось, наприклад, як виглядає ще один варіант запису попереднього оголошення:

```

int Array[10][2] = {
    {1, 1},
    {2, 4},
    {3, 9},
    {4, 16},
    {5, 25},
    {6, 36},
    {7, 49},
    {8, 64},
    {9, 81},
    {10, 100}
};

```

Під час використання підгруп ініціалізацій відсутні члени підгрупи ініціалізуються нульовими значеннями автоматично.

У наведеному нижче коді програми масив Array використовують для пошуку квадрата числа, введенного користувачем. Програма спочатку здійснює пошук заданого числа в масиві, а потім виводить відповідне йому значення квадрата.

```

#include <iostream>      // Потокowe введення-виведення
using namespace std;    // Використання стандартного простору імен

```

```

int Array[10][2] = {
    {1, 1},
    {2, 4},
    {3, 9},
    {4, 16},
    {5, 25},
    {6, 36},
    {7, 49},
    {8, 64},
    {9, 81},
    {10, 100}
};

int main()
{
    int i, j;

    for(;;) {
        cout << "Введіть число від 1 до 10: "; cin >> i;

        if(i>=1)    (i<=10) break;

        };

// Пошук значення i.
for(j=0; j<10; j++)
    if(Array[j][0] == i) break;

    cout << "Квадрат числа " << i << " дорівнює " << Array[j][1];

    getch(); return 0;

}

```

Глобальні масиви ініціалізувалися на початку виконання програми, а локальні – під час кожного виклику функції, у якій вони містяться. Розглянемо такий приклад:

```

#include <iostream>           // Потокowe введення-виведення
#include <cstring>            // Робота з рядковими типами даних
using namespace std;        // Використання стандартного простору імен

void Fun1();

int main()
{
    Fun1();
    Fun1();

    getch(); return 0;
}

void Fun1() {
    char str[80] ="Це просто тест\n";

    cout << str;
    strcpy(str, "Змінено\n");    // Змінюємо значення рядка str.
    cout << str;
}

```

Внаслідок виконання ця програма відображає на екрані такі результати:

```

Це просто тест Змінено
Це просто тест Змінено

```

У цьому коді програми масив str ініціалізувався під час кожного виклику функції Fun1(). Той факт, що під час її виконання масив str змінюється, ніяк не впливає на його повторну ініціалізацію при подальших викликах функції Fun1(). Тому під час кожного входження в неї на екрані монітора з'являється такий текст:

```

Це просто тест

```

4. Ініціалізація «безрозмірних» масивів

Припустимо, що ми використовуємо такий варіант ініціалізації елементів масиву для побудови таблиці повідомлень про помилки:

```
char e1[14] = "Ділення на 0\n";  
char e2[23] = "Кінець файлу\n";  
char e3[21] = "У доступі відмовлено\n";
```

Неважко припустити, що вручну незручно підраховувати символи у кожному повідомленні, щоб визначити коректний розмір масиву. На щастя, у мові програмування C++ передбачено можливість автоматичного визначення довжини масивів шляхом використання їх "безрозмірного" формату. Якщо в настанові ініціалізації масиву не вказано його розмір, то мова програмування C++ автоматично створить масив, розмір якого буде достатнім для зберігання всіх значень ініціалізацій. При такому підході попередній варіант ініціалізації елементів масиву для побудови таблиці повідомлень про помилки можна переписати так:

```
char e1[] = "Ділення на 0\n";  
char e2[] = "Кінець файлу\n";  
char e3[] = "У доступі відмовлено\n";
```

Крім зручності в первинному визначенні масивів, метод "безрозмірної" ініціалізації дає змогу змінити будь-яке повідомлення без переліку його довжини. Тим самим усувається можливість внесення помилок, викликаних випадковим прорахунком.

"Безрозмірна" ініціалізація елементів масивів не обмежується одновимірними масивами. Під час ініціалізації багатовимірних масивів Вам треба вказати усі дані, за винятком крайньої зліва розмірності, щоб C++-компілятор міг належним чином індексувати масив. Використовуючи "безрозмірну" ініціалізацію масивів, можна створювати таблиці різної довжини, даючи змогу компілятору автоматично виділяти область пам'яті, достатню для їх зберігання.

У такому прикладі масив Array[][] оголошується як "безрозмірний":

```
int Array[][2] = { 1, 1,  
2, 4,  
3, 9,  
4, 16,  
5, 25,  
6, 36,  
7, 49,  
8, 64,  
9, 81,  
10, 100  
};
```

Перевага такої форми оголошення перед "габаритною" (з точним указанням усіх розмірностей) полягає у тому, що програміст може подовжувати або укорочувати таблицю значень ініціалізації, не змінюючи розмірності масиву.

Запитання для контролю знань:

1. Що таке багатовимірний масив у програмуванні?
2. Як зазвичай звертаються до елемента у двовимірному масиві (матриці)?
3. Яка структура найкраще підходить для обходу всіх елементів двовимірного масиву?
4. Якщо масив оголошено як A[3][4], скільки всього елементів він містить?
5. Яка головна діагональ квадратної матриці?
6. У тривимірному масиві (кубі) третя розмірність зазвичай інтерпретується як?
7. При обробці матриці за допомогою двох циклів (i та j), що станеться, якщо переплутати індекси та написати A[j][i] замість A[i][j]?

Література:

1. Васильєв О. М. Програмування на С++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою С++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Змістовий модуль 7. Структури

Тема 7.1. Структури.

Навчальна мета: полягає у формуванні в учнів цілісного розуміння концепції користувацьких типів даних як інструменту для моделювання об'єктів реального світу. Студент має усвідомити фундаментальну відмінність між масивом, що об'єднує однорідні елементи, та структурою, яка дозволяє групувати логічно пов'язані змінні різних типів під одним іменем.

Розвиваюча мета: спрямована на активізацію аналітичного мислення студентів через перехід від лінійного сприйняття даних до об'єктного моделювання. Вона передбачає стимулювання здатності систематизувати інформацію та виявляти логічні зв'язки між окремими характеристиками об'єкта, що сприяє формуванню системного погляду на архітектуру програмного забезпечення.

Виховна мета: зосереджена на формуванні професійної відповідальності та культури написання коду, що є фундаментальним для роботи в команді. Вона передбачає виховання почуття естетики в програмуванні через дотримання стандартів іменування полів структур та охайність у структуруванні даних, що демонструє повагу автора до колег, які працюватимуть із цим кодом у майбутньому.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Організація роботи зі структурами даних(основні положення)

У програмуванні **структури даних** – способи організації даних у комп'ютерах. Часто разом зі структурою даних пов'язується і специфічний перелік операцій, що можуть бути виконаними над даними, організованими в таку структуру. Правильний підбір структур даних є надзвичайно важливим для ефективного функціонування відповідних алгоритмів їх оброблення. Добре побудовані структури даних дають змогу оптимізувати використання машинного часу та пам'яті комп'ютера для виконання найбільш критичних операцій.

Відома формула "Програма = Алгоритми + Структури даних" дуже точно виражає потребу відповідального ставлення до такого підбору. Тому іноді навіть не обраний алгоритм для оброблення масиву даних визначає вибір тої чи іншої структури даних для їх збереження, а навпаки. Підтримка базових структури даних, які використовуються в програмуванні, включена в комплекти стандартних бібліотек мови програмування C++.

У мові програмування C++ **структура** представляє колекцію змінних, об'єднаних загальним іменем, яка забезпечує зручний засіб зберігання споріднених даних в одному місці. **Структура** – сукупність різних типів даних, оскільки вони складаються з декількох різних, але логічно взаємопов'язаних між собою змінних. З цих самих причин структури іноді називають *складеними* або *конгломератними типами даних*.

Структура — група взаємопов'язаних між собою змінних.

Перед визначенням структурних змінних, необхідно визначити формат структури. Це робиться за допомогою оголошення структури. Оголошення структури дає змогу компілятору зрозуміти, змінні якого типу вона містить. Змінні, які належать до структури, називаються її *членами*. Члени структури також називають *полями*.

Член структури — змінна, яка є частиною структури.

У загальному випадку всі члени структури мають бути логічно пов'язані одна з одною! Наприклад, структури зазвичай використовують для зберігання такої інформації, як поштові адреси, банківські реквізити, елементи книжкової бібліографії і т.ін. Безумовно, відносини між членами структури абсолютно суб'єктивні і визначаються програмістом. Компілятор "нічого про них не знає" (або "не хоче знати").

Ключове слово **struct** означає початок оголошення структури.

Загальний формат оголошення структури має такий вигляд:

```
struct ім'я_типу_структури {  
    тип ім'я_члена_1; тип ім'я_члена_2; тип ім'я_члена_3;  
    .....  
    тип ім'я_члена_n;  
} структурна_змінна_1, структурна_змінна_2, ..., структурна_змінна_t;
```

Розглянемо деякі приклади оголошення структур. Визначимо структуру, яка може містити інформацію про продукцію, що зберігаються на складі приватної фірми. Один запис інвентарної відомості зазвичай складається з декількох даних, наприклад: назву продукції, вартості та наявної кількості. Тому для керування такою інформацією зручно використовувати саме таку структуру. У наведеному нижче коді програми оголошується структура, яка визначає такі елементи: назву продукції, її вартість, роздрібну ціну, наявну кількість, кількість днів до поновлення запасів. Цих даних часто цілком достатньо для керування складським господарством. Про початок оголошення структури компіляторіві повідомляє ключове слово **struct**:

```
struct invStruct {           // Попереднє оголошення типу структури  
    char nameProd[40];      // Назва продукції
```

```

double varProd; // Вартість продукції
double rozdrCina; // Роздрібна ціна
int payavKilk; // Наявна кількість
int kilkDniv; // Кількість днів до поновлення запасів
};

```

Ім'я типу структури — її специфікатор типу даних.

Зверніть увагу на те, що оголошення структури завершується крапкою з комою, тобто вона може бути настановою. Ім'ям типу структури тут є `invStruct`. Іншими словами, ім'я `invStruct` ідентифікує конкретну структуру даних і є її специфікатором типу.

У нашому оголошенні структури насправді не було створено жодної структурної змінної, а визначено тільки формат типу даних. Щоб за допомогою цієї структури визначити реальну структурну змінну (тобто фізичний об'єкт), потрібно записати таку настанову:

```
invStruct InvVidom;
```

Ось тепер визначається структурна змінна типу `invStruct` з іменем `InvVidom`.

Під час визначення структурної змінної компілятор мови програмування C++ автоматично виділить об'єм пам'яті, достатній для зберігання всіх членів структури. На рис. 10.1 показано, як змінну `InvVidom` буде розміщено в пам'яті комп'ютера (у припущенні, що **double**-значення займає 8 байтів, а **int**-значення — 4 байти).

```

nameProd 40 байт |
                  |
                  | varProd 8 байт |
                  |                 |
rozdrCina 8 байт | = InvVidom         |
                  |                 |
                  | payavKilk 4 байт |
                  |                 |
                  | kilkDniv 4 байт |||||

```

Рис. 10.1. Розміщення структурної змінної `InvVidom` у пам'яті комп'ютера

Одночасно з оголошенням імені типу структури можна визначити одну або декілька структурних змінних:

```

struct invStruct { // Попереднє оголошення типу структури
    char nameProd[40]; // Назва продукції
    double varProd; // Вартість продукції
    double rozdrCina; // Роздрібна ціна
    int payavKilk; // Наявна кількість
    int kilkDniv; // Кількість днів до поновлення запасів
} InvVidomA, InvVidomB, InvVidomC; // Визначення структурної змінної

```

Цей код програми оголошує структурний тип `invStruct` і визначає структурні змінні `InvVidomA`, `InvVidomB` і `InvVidomC` цього типу. Важливо розуміти, що кожна структурна змінна містить власні копії членів структури. Наприклад, поле `varProd` структури `InvVidomA` ізолювано від поля `varProd` структури `InvVidomB`. Отже, зміни, що вносяться в певне поле однієї структурної змінної, ніяк не впливають на вміст такого самого поля іншої структурної змінної.

Якщо для коду програми достатньо тільки однієї структурної змінної, то в оголошенні структури необов'язково міститиме ім'я структурного типу. Для розуміння сказаного розглянемо такий приклад:

```

struct { // Попереднє оголошення типу структури
    char nameProd[40]; // Назва продукції
    double varProd; // Вартість
    double rozdrCina; // Роздрібна ціна
    int payavKilk; // Наявна кількість
    int kilkDniv; // Кількість днів до поновлення запасів
}

```

```
} vidom;           // Визначення структурної змінної
```

Цей код програми визначає одну структурну змінну `vidom` відповідно до оголошення структури, яка їй передує.

2. Доступ до членів структури

До окремих членів структури доступ здійснюється за допомогою оператора "крапка". Наприклад, у процесі виконання такого коду програми значення 10.39 буде присвоєно полю `vartProd` структурної змінної `InvVidom`, яка була оголошена вище:

```
InvVidom.vartProd = 10.39;
```

Щоб звернутися до члена структури, потрібно перед його іменем поставити ім'я структурної змінної та оператор "крапка". Так здійснюється доступ до всіх членів структури. Загальний формат доступу до члена структури записується так:

ім'я_структурної_змінної.ім'я_члена_структури

Оператор "крапка" (.) дає змогу отримати доступ до будь-якого члена відповідної структури.

Щоб вивести значення поля `vartProd` структурної змінної `InvVidom` на екран, необхідно записати таку настанову:

```
cout << InvVidom.vartProd;
```

Аналогічним способом можна використовувати символьний масив

`InvVidom.nameProd` у виклику функції **gets()**:

```
gets(InvVidom.nameProd);
```

У цьому записі функції **gets()** буде передано символьний покажчик на початок області пам'яті, відведеної члену `nameProd` структури `InvVidom`.

Якщо виникає потреба отримати доступ до окремих елементів масиву структур під назвою, наприклад, `InvVidom.nameProd`, необхідно використати індексацію масиву. Наприклад, за допомогою цього коду програми можна по-символьно вивести на екран монітора вміст поля масиву структур `InvVidom.nameProd`:

```
for(int t=0; InvVidom.nameProd[t]; t++)  
cout << InvVidom.nameProd[t];  
cout << endl;
```

Запитання для контролю знань:

1. Яке основне призначення структур у мовах програмування (наприклад, C++)?
2. Яке ключове слово використовується для оголошення структури?
3. Який оператор використовується для доступу до полів конкретного примірника структури?
4. Чи можуть поля структури мати різні типи даних?
5. Який обсяг пам'яті займає структура?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Тема 7.2. Масиви структур.

Навчальна мета: полягає у формуванні в студентів системного розуміння принципів організації та обробки складних типів даних у мові програмування. Під час вивчення матеріалу здобувачі мають усвідомити логічний зв'язок між концепцією окремої структури та механізмом масивів, що дозволяє ефективно моделювати реальні об'єкти з багатьма характеристиками.

Розвиваюча мета: спрямована на стимулювання аналітичного мислення та здатності до абстрагування під час проектування архітектури програмного забезпечення. Вивчення цієї теми сприяє розвитку вміння розкладати складні об'єкти реального світу на складові атрибути та синтезувати їх у цілісні моделі даних, що є критично важливим для формування інженерного підходу до вирішення задач.

Виховна мета: зосереджена на формуванні професійної відповідальності та культури програмування як важливої складової етики майбутнього фахівця. Робота зі складними структурами даних вимагає від студента особливої акуратності, дисциплінованості та уваги до деталей, адже найменша неточність у зверненні до полів структури може призвести до критичних помилок у системі. Це сприяє вихованню наполегливості у розв'язанні складних задач та прагнення до створення чистого, зрозумілого й документованого коду, який поважає роботу інших розробників.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Поняття про масиви структур

Структури можуть бути елементами масивів, тобто, масиви структур використовуються достатньо часто. Щоб визначити масив структур, необхідно спочатку оголосити структуру, а потім визначити масив елементів цього структурного типу. Наприклад, щоб визначити 100-елементний масив структур типу `invStruct` (який було оголошено вище), достатньо записати таку настанову:

```
invStruct prodArray[100];
```

Щоб отримати доступ до конкретної структури в масиві структур, необхідно індексувати ім'я структури. Щоб відобразити на екрані вміст члена `nayavKilk` третьої структури, достатньо використати таку настанову:

```
cout << prodArray[2].nayavKilk;
```

Щоб продемонструвати застосування структур, розробимо просту програму керування складом, у якій для зберігання інформації про продукцію (інвентарну відомість) використовується масив структур типу `invStruct`. Різні функції, які визначені у цьому коді програми, взаємодіють із структурою та її членами по-різному.

Запис інвентарної відомості можна зберігати в структурі типу `invStruct`, організовані у масиві під назвою `prodArray`:

```
const int rozmir = 100;
struct invStruct {           // Попереднє оголошення типу структури
    char nameProd[40];       // Назва продукції
    double vartProd;         // Вартість продукції
    double rozdrCina;        // Роздрібна ціна
    int nayavKilk;           // Наявна кількість
    int killDniv;            // Кількість днів до поновлення запасів
} prodArray[rozmir];        // Визначення масиву структур
```

Розмір масиву вибрано довільно. При бажанні його можна легко змінити. Зверніть увагу на те, що розмірність масиву задано з використанням **const**-змінної. А оскільки розмір масиву в усій програмі використовується декілька разів, то застосування **const**-змінної для цього є виправданим. Щоб змінити розмір масиву, достатньо змінити значення константної змінної `rozmir`, а потім перекомпілювати програму. Використання **const**-змінної для визначення "магічного числа", яке часто уживається у програмі, – звичайна практика в професійному C++-коді програми.

Розроблена програма повинна забезпечити виконання таких дій:

- введення інформації про продукцію, що зберігатимуться на складі;
- відображення інвентарної відомості;
- модифікування полів заданого запису.

Передусім напишемо основну функцію `main()`, яка має мати приблизно такий вигляд:

```
int main()
{
    char vybir;
    InitList(); // Ініціалізація елементів масиву структур.
    for(;;) {
        // Отримання команди меню, вибраної користувачем. vybir = MenuSelect();
        switch(vybir) { // Введення запису в інвентарну відомість.
            case 'e': Enter();
                break;
            // Відображення на екрані записів інвентарної відомості.
            case 'd': Display();
                break;
            // Модифікування полів наявного запису відомості.
            case 'u': UpDate();
                break; case 'q': return 0;
        }
    }
}
```

Функція `main()` починається з виклику функції `InitList()`, яка ініціалізує масив структур. Потім організовується цикл, який відображає меню і обробляє команду, вибрану користувачем. Наведемо код функції `InitList()`:

```
void InitList() // Ініціалізація елементів масиву структур.
{
    // Ім'я нульової довжини означає порожній запис.
    for(int t=0; t<rozmir; t++) *prodArray[t].nameProd = '\0';
}
```

Функція `InitList()` готує масив структур для подальшого використання, поміщаючи в перший байт поля `nameProd` нульовий символ. Передбачається, якщо поле `nameProd` порожнє, то на даний момент структура, у якій воно міститься, просто не використовується.

Функція `MenuSelect()` відображає команди меню і приймає варіант, вибраний користувачем:

```
int MenuSelect() // Отримання команди меню, вибраної користувачем.
{
    char ch; cout << endl; do {
        cout << "(E)nter" << endl; // Ввести новий запис.
        cout << "(D)isplay" << endl; // Відобразити всю відомість.
        cout << "(U)pdate" << endl; // Змінити поля запису.
        cout << "(Q)uit\n" << endl; // Вийти з програми.
        cout << "Виберіть команду: "; cin >> ch;
    } while(!strchr("edup", tolower(ch))); return tolower(ch);
}
```

Користувач вибирає із запропонованого меню команду, вводячи потрібну букву. Наприклад, щоб відобразити всю інвентарну відомість, потрібно натиснути букву "D".

Функція `MenuSelect()` викликає бібліотечну функцію мови C++ `strchr()`, яка має такий прототип:

```
char *strchr(const char *str, int ch);
```

Ця функція проглядає рядок, який адресується покажчиком `str`, на предмет входження в неї символу, який зберігається в молодшому байті змінної `ch`. Якщо такий символ виявиться, то функція поверне покажчик на нього. І у цьому випадку значення, що повертається функцією, за визначенням буде ІСТИННИМ. Однак, якщо такого збігу символів не відбудеться, то функція поверне нульовий покажчик, який за визначенням є значення ФАЛЬШ. Так перевірка тут організована для того, щоби виявити, чи є значення, що вводяться користувачем, допустимими командами меню.

Функції `Enter()` передусє виклик функції `GetPut()`, яка "підказує" користувачу порядок введення даних і приймає їх. Розглянемо програмні коди обох функцій:

```
void Enter() // Введення запису в інвентарну відомість.
{
    int i;
    // Знаходимо першу вільну структуру.
    for(i=0; i<rozmir; i++)
        if(!*prodArray[i].nameProd) break;

    // Якщо масив повний, то значення "i" буде дорівнювати rozmir.
    if(i==rozmir) {
        cout << "Перелік повний" << endl; return;
    }
    GetPut(i); // Введення інформації.
}

void GetPut(int i) // Введення інформації.
{
    cout << "Назва продукції : "; cin >> prodArray[i].nameProd; cout << "Вартість продукції : ";
    cin >> prodArray[i].vartProd; cout << "Роздрібна ціна : "; cin >>
    prodArray[i].rozdrCina;
    cout << "Наявна кількість: "; cin >> prodArray[i].nayavKilk;
    cout << "Кількість днів до поновлення запасів: ";
}
```

```

        cin >> prodArray[i].kilkDniv;
    }

```

Оскільки інформація про продукцію на складі може змінюватися, програма ведення інвентарної відомості повинна давати змогу вносити зміни в її окремі записи. Це реалізується шляхом виклику функції `Update()`:

```

void Update() // Модифікування полів наявного запису відомості.
{
    int i; char name[80];

    cout << "Введіть назву продукції: "; cin >> name;

    for(i=0; i<rozmir; i++)
        if(!strcmp(name, prodArray[i].nameProd)) break;

    if(i==rozmir) {
        cout << "Продукцію не знайдено" << endl; return 0;
    }

    cout << "Введіть нову назву продукції" << endl; GetPut(i); // Введення інформації.
}

```

Ця функція пропонує користувачу ввести назву продукції, інформацію про який йому потрібно змінити. Потім вона проглядає весь перелік наявних записів і, якщо вказана користувачем продукція у ньому є, то викликається функція `GetPut()`, яка забезпечує прийняття від користувача нової інформації.

Нам залишилося розглянути функцію `Display()`. Вона виводить на екран записи інвентарної відомості в повному обсязі. Код функції `Display()` має такий вигляд:

```

void Display() // Відображення на екрані записів інвентарної відомості.
{
    for(int t=0; t<rozmir; t++)
        if(*prodArray[t].nameProd) {
            cout << "Назва продукції:" << prodArray[t].nameProd << endl;
            cout << "Вартість продукції:" << prodArray[t].vartProd << " грн" << endl;
            cout << "У роздріб:" << prodArray[t].rozdrCina << " грн" << endl;
            cout << "У наявності:" << prodArray[t].nayavKilk << " шт" << endl;
            cout << "Залишилося:" << prodArray[t].kilkDniv << " днів" << endl;
        }
}

```

Спробуйте також розширити програму, додавши функції пошуку в списку заданої продукції, видалення вже непотрібного запису або повного очищення інвентарної відомості.

```

#include <vcl>
#include <iostream> // Потокowe введення-виведення
#include <conio> // Консольний режим роботи
using namespace std; // Використання стандартного простору імен

const int rozmir = 100;

struct invStruct { // Попереднє оголошення типу структури
    char nameProd[40]; // Назва продукції
    double vartProd; // Вартість продукції
    double rozdrCina; // Роздрібна ціна
    int nayavKilk; // Наявна кількість
    int kilkDniv; // Кількість днів до поновлення запасів
} prodArray[rozmir]; // Визначення масиву структур

// Попереднє оголошення процедур і функцій
void Enter(), InitList(), Display();
void Update(), GetPut(int i);
int MenuSelect();

int main() // Початок основної програми
{

```

```

char vybir;
InitList(); // Ініціалізація елементів масиву структур.

for(;;) {
    // Отримання команди меню, вибраної користувачем. vybir = MenuSelect();
    switch(vybir) {
        // Введення запису в інвентарну відомість.
        case 'e': Enter();
                        break;
        // Відображення на екрані записів інвентарної відомості.
        case 'd': Display();
                        break;
        // Модифікування полів наявного запису відомості.
        case 'u': UpDate();
        break; case 'q': return 0;
    }
}

} // Кінець основної програми

// Визначення процедур і функцій
void InitList() // Ініціалізація елементів масиву структур.
{
    // Ім'я нульової довжини означає порожній запис.
    for(int t=0; t<rozmir; t++) *prodArray[t].nameProd = '\0';
}

int MenuSelect() // Отримання команди меню, вибраної користувачем.
{
    char ch; cout << endl; do {
        cout << "(E)nter" << endl; // Ввести новий запис.
        cout << "(D)isplay" << endl; // Відобразити всю відомість.
        cout << "(U)pdate" << endl; // Змінити поля запису.
        cout << "(Q)uit\n" << endl; // Вийти з програми.
        cout << "Виберіть команду: "; cin >> ch;
    } while(!strchr("eduoq", tolower(ch)));

    return tolower(ch);
}

void Enter() // Введення запису в інвентарну відомість.
{
    int i;
    // Знаходимо першу вільну структуру.
    for(i=0; i<rozmir; i++)
        if(!*prodArray[i].nameProd) break;

    // Якщо масив повний, то значення "i" буде дорівнювати rozmir.
    if(i==rozmir) {
        cout << "Перелік повний" << endl; return;
    }
    GetPut(i); // Введення інформації.
}

void GetPut(int i) // Введення інформації.
{
    cout << "Назва продукції : "; cin >> prodArray[i].nameProd;
    cout << "Вартість продукції : "; cin >> prodArray[i].vartProd;
    cout << "Роздрібна ціна : "; cin >> prodArray[i].rozdrCina;
    cout << "Наявна кількість : "; cin >> prodArray[i].nayavKilk;
    cout << "Кількість днів до поновлення запасів: ";
    cin >> prodArray[i].kilkDniv;
}

void UpDate() // Модифікування полів наявного запису відомості.
{
    int i; char name[80];
    cout << "Введіть назву продукції: "; cin >> name;

    for(i=0; i<rozmir; i++)
        if(!strcmp(name, prodArray[i].nameProd)) break;

```



```

        if(i==rozmir) {
cout << "Продукцію не знайдено" << endl;
return;
        }
        cout << "Введіть нову назву продукції" << endl; GetPut(i); // Введення інформації.
    }

void Display() // Відображення на екрані записів інвентарної відомості.
{
    for(int t=0; t<rozmir; t++)
        if(*prodArray[t].nameProd) {
            cout << "Назва продукції:" << prodArray[t].nameProd << endl; cout << "Вартість продукції:" << prodArray[t].vartProd << "
грн" << endl; cout << "У роздріб :" << prodArray[t].rozdrCina << " грн" << endl;
            cout << "У наявності:" << prodArray[t].nayavKilk << " шт" << endl; cout << "Залишилося :" << prodArray[t].kilkDniv << " днів"
<< endl;
        }
}

```

2. Механізм присвоєння структур

Вміст однієї структури можна присвоїти іншій, якщо обидві ці структури мають однаковий тип. Наприклад, наведений нижче код програми присвоює значення структурної змінної strVar1 змінній strVar2.

```

#include <vcl>
#include <iostream> // Потокowe введення-виведення
#include <conio> // Консольний режим роботи
using namespace std; // Використання стандартного простору імен

struct demoStruct { // Попереднє оголошення типу структури
    int a, b;
};

int main()
{
    demoStruct strVar1, strVar2; strVar1.a = strVar1.b = 10; strVar2.a = strVar2.b = 20;

    cout << "Структури до присвоєння" << endl;
    cout << "strVar1: " << strVar1.a << " " << strVar1.b << endl;
    cout << "strVar2: " << strVar2.a << " " << strVar2.b << endl << endl; strVar2 = strVar1; // Присвоєння структур
    cout << "Структури після присвоєння" << endl;
    cout << "strVar1: " << strVar1.a << " " << strVar1.b << endl;
    cout << "strVar2: " << strVar2.a << " " << strVar2.b << endl;

    getch(); return 0;
}

```

Внаслідок виконання цієї програми на моніторі буде відображено такі результати:

Структури до присвоєння. strVar1: 10 10
strVar2: 20 20

Структури після присвоєння. strVar1: 10 10
strVar2: 10 10

У мові програмування C++ кожне нове оголошення структури визначає новий тип. Отже, навіть якщо дві структури фізично однакові, але мають різні імена типів, компілятор вважатиме їх різними і не дасть змоги присвоїти значення однієї з них іншій. Розглянемо такий фрагмент коду програми. Він некоректний і тому не буде компільована:

```

struct demoStructA {
    int a, b;
};

struct demoStructB {
    int a, b;
};

```

```
};

int main()
{
    demoStructA strVar1; demoStructB strVar2;

    strVar2 = strVar1; // Помилка через невідповідності типів.
    .....
}
```

Незважаючи на те, що структури `demoStructA` і `demoStructB` фізично однакові, з погляду компілятора вони є окремими типами.

3. Передача структури функції як аргументу

При передачі структури функції як аргумент використовується механізм передачі параметрів за значенням. Це означає, що будь-які зміни, внесені у вміст структури в тілі функції, якій вона передана, не впливають на структуру, що використовується як аргумент цієї функцією. Проте необхідно мати на увазі, що передача великих структур вимагає значних витрат системних ресурсів. Як правило, чим більше даних передається функції, тим більше витрачається системних ресурсів.

Використовуючи структуру як параметр, необхідно також пам'ятати, що тип аргументу повинен відповідати типу параметра. Наприклад, у наведеному нижче коді програми спочатку оголошується структура `demoStruct`, а потім функція `FunA()` приймає параметр типу `demoStruct`.

```
#include <vc1>
#include <iostream>      // Потокowe введення-виведення
#include <conio>          // Консольний режим роботи
using namespace std;     // Використання стандартного простору імен

struct demoStruct {      // Попереднє оголошення типу структури
    int a;
    char ch;
};

void FunA(demoStruct pm); // Попереднє оголошення прототипу функції

int main()
{
    demoStruct arg;      // Визначення змінної arg типу demoStruct.

    arg.a = 1000; arg.ch = 'x';
    FunA(arg);           // Передача структури функції як аргументу
    getch(); return 0;
}

void FunA(demoStruct pm) // Визначення функції
{
    cout << "Передає функції структура: "
         << "a=" << pm.a << "; ch=" << pm.ch << endl;
}
```

Внаслідок виконання цієї програми на моніторі буде відображено такий результат:

```
Передає функції структура: a= 1000; ch= x
```

У цьому коді програми як аргумент `arg` у функції `main()`, так і параметр `pm` у функції `FunA()` мають однаковий тип. Тому аргумент `arg` можна передати функції `FunA()`. Якби типи цих структур були різні, у процесі компілювання коду програми було б видано повідомлення про помилку.

4. Повернення функцією структури як значення

Для повернення функцією структури як значення використовується механізм повернення параметрів за значенням. Це означає, що будь-які зміни, внесені у вміст структури, яка визначена в тілі функції, впливають на структуру, якій присвоюється значення, що повертається цією функцією. Проте необхідно мати на увазі, що повернення великих структур вимагає значних витрат системних ресурсів. Як правило, чим більше даних повертається функцією, тим більше витрачається системних ресурсів.

Використовуючи структуру як параметр, необхідно також пам'ятати, що тип структури повинен відповідати типу повернутого значення. Наприклад, у наведеному нижче коді програми спочатку оголошується структура `demoStruct`, а потім функція `FunA()` приймає параметр типу `demoStruct`.

```
#include <vcl>
#include <iostream>      // Потокowe введення-виведення
#include <conio>          // Консольний режим роботи
using namespace std;     // Використання стандартного простору імен

struct demoStruct {      // Попереднє оголошення типу структури
    int a;
    char ch;
};

void FunA(demoStruct pm, char zm);    // Попереднє оголошення прототипу функції
demoStruct FunB(demoStruct pm);      // Попереднє оголошення прототипу функції

int main()
{
    demoStruct arg1={1000,'x'}, arg2;    // Визначення структурних змінних

    FunA(arg1,'1');    // Передача структури функції як аргументу arg2 = FunB(arg1);    // Повернення
    функцією структури як значення FunA(arg2,'2');    // Передача структури функції як аргументу

    getch(); return 0;
}

void FunA(demoStruct pm, char zm)    // Визначення функції
{
    cout << "Передаєна функції структура arg" << zm
    << "; a= " << pm.a << "; ch= " << pm.ch << endl;
}

demoStruct FunB(demoStruct pm)    // Визначення функції
{
    demoStruct pn;

    pn.a = 3*pm.a; pn.ch = 'y';
    return pn;
}
```

Внаслідок виконання цієї програми на моніторі буде відображено такий результат:

```
Передаєна функції структура arg1: a= 1000; ch= x
Передаєна функції структура arg2: a= 3000; ch= y
```

У цьому коді програми як аргументи `arg1` і `arg2` у функції `main()`, так і параметр `pm` у функції `FunA()` мають однаковий тип. Аналогічно у функції `FunB()` визначена структурна змінна `pn` має цей самий тип. Тому аргументи `arg1` і `arg2` можна передати функції `FunA()`, а значення структурної змінної `pn` можна повернути з функції `FunB()` у функції `main()`. Якби типи цих структур були різні, у процесі компілювання коду програми було б видано повідомлення про помилку. Окрім цього, у функції `FunA()` як аргумент передається проста змінна типу `char` для ідентифікування назви структурної змінної.

Запитання для контролю знань:

1. Як правильно оголосити масив з 10 структур типу Student (де Student уже визначено)?
2. Який оператор використовується для доступу до поля структури, що є елементом масиву `arr[i]`?
3. Як звернутися до поля `age` третього елемента масиву структур `people`?
4. Чи можна ініціалізувати масив структур під час оголошення?
5. Яка основна перевага використання масиву структур замість кількох окремих масивів для кожного поля?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Змістовий модуль 8. Показчики

Тема 8.1. Оголошення і ініціалізація змінної показчика

Навчальна мета: полягає у формуванні в учнів фундаментального розуміння механізмів прямої адресації пам'яті та принципів роботи з посиланнями на дані.

Розвиваюча мета: спрямована на формування в учнів абстрактного та аналітичного мислення через усвідомлення непрямої адресації даних. Вивчення показчиків стимулює здатність візуалізувати складні архітектурні процеси, що відбуваються в оперативній пам'яті, допомагаючи перейти від сприйняття змінної як «контейнера з числом» до розуміння її як «об'єкта в адресному просторі».

Виховна мета: зосереджена на формуванні професійної відповідальності та культури програмування, де кожен рядок коду безпосередньо впливає на стабільність системи. Оскільки робота з показчиками відкриває прямий доступ до пам'яті комп'ютера, навчання має на меті виховання дисциплінованості та ретельності, привчаючи студента до обов'язкової ініціалізації змінних та уникнення небезпечних станів програми.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Основні поняття про покажчики

Покажчики, поза сумнівом, – один з найважливіших і складних аспектів мови програмування C++. Значною мірою потужність багатьох засобів мови програмування C++ визначається використанням покажчиків. Наприклад, завдяки ним забезпечується підтримка зв'язних списків і динамічного виділення області пам'яті, саме вони дають змогу функціям змінювати значення своїх аргументів. Однак ці питання будуть розглядатися в подальших розділах, а поки що (тобто у цьому розділі) розглянемо основні особливості застосування покажчиків і продемонструємо, як можна уникнути деяких потенційних проблем, пов'язаних з їх використанням.

Під час розгляду основних понять про покажчики нам доведеться використовувати таке поняття як розмір базових C++-типів даних. Для цього запам'ятаємо тільки те, що символи займають в пам'яті один байт, цілочисельні значення – чотири, значення з плинною крапкою типу **float** – чотири, а значення з плинною крапкою типу **double** – вісім (ці розміри характерні для типового 32-розрядного середовища).

Покажчики – змінні, які зберігають адреси іншої змінної. Найчастіше ці адреси позначають місцезнаходження в пам'яті інших змінних. Наприклад, якщо змінна *x* містить адресу змінної *y*, то про змінну *x* говорять, що вона "вказує" на *y*.

Змінні-покажчики (або *змінні типу покажчика*) мають бути відповідно оголошені. Формат оголошення змінної-покажчика є таким:

```
тип *ім'я_змінної;
```

У цьому записі елемент *тип* означає базовий тип покажчика, причому він повинен бути допустимим C++-типом. Елемент *ім'я_змінної* є іменем змінної-покажчика.

Для розуміння сказаного розглянемо такий приклад. Щоб оголосити змінну *p* покажчиком на **int**-значення, використаємо таку настанову:

```
int *p;
```

Для оголошення покажчика на **float**-значення використаємо таку настанову:

```
float *p;
```

У загальному випадку використання символу "зірочка" (*) перед іменем змінної в настанові оголошення перетворює цю змінну на покажчик.

Тип даних, на які посилатиметься покажчик, визначається його базовим типом. Розглянемо ще один приклад:

```
int *ip;           // Покажчик на цілочисельне значення
double *dp;        // Покажчик на значення типу double
```

Як зазначено в коментарях до цієї програми, змінна *ip* – покажчик на **int** значення, оскільки його базовим типом є тип **int**, а змінна *dp* – покажчик на **double**-значення, оскільки його базовим типом є тип **double**. Отже, в поперед-

ніх прикладах змінну *ip* можна використовувати для вказівки на **int**-значення, а змінну *dp* – на **double**-значення. Проте запам'ятайте: не існує реального засобу, який би зміг перешкодити покажчику посилатися на "казна-що". Ось через це покажчики потенційно небезпечні.

2. Оператори роботи з покажчиками

З покажчиками використовуються два оператори: "*" і "&". Оператор "&" – унарний. Він повертає адресу пам'яті, за якою розташований його операнд. Наприклад, у процесі виконання такого фрагмента коду програми

```
ptr = &balance;
```

у змінну *ptr* поміщається адреса змінної *balance*. Ця адреса відповідає області у внутрішній пам'яті комп'ютера, яка належить змінній *balance*. Виконання цієї настанови ніяк не впливає на значення змінної *balance*. Призначення оператора "&" можна "перекласти" українською мовою як "адреса змінної", перед якою він знаходиться. Отже, наведену вище настанову присвоєння

можна виразити так: "змінна `ptr` отримує адресу змінної `balance`". Щоб краще зрозуміти суть цього присвоєння, припустимо, що змінна `balance` розташована у області пам'яті з адресою 100. Отже, після виконання цієї настанови змінна `ptr` набуде значення 100.

Другий оператор роботи з покажчиками (*) слугує доповненням до першого (&). Це також унарний оператор, але він звертається до значення змінної, розташованої за адресою, заданою його операндом. Іншими словами, він посилається на значення змінної, яка адресується заданим покажчиком. Якщо (продовжуючи роботу з попередньою настановою присвоєння) змінна `ptr` містить адресу змінної `balance`, то у процесі виконання настанови

```
value = *ptr;
```

змінній **value** буде присвоєне значення змінної `balance`, на яку вказує змінна `ptr`. Наприклад, якщо змінна `balance` містить значення 3200, то після виконання останньої настанови змінна **value** міститиме значення 3200, оскільки це якраз те значення, яке зберігається за адресою 100. Призначення оператора "*" можна виразити словосполученням "за адресою". У цьому випадку попередню настанову можна прочитати так: "змінна **value** набуває значення (розташоване) за адресою `ptr`". Дію наведених вище двох настанов схематично показано на рис. 6.1.

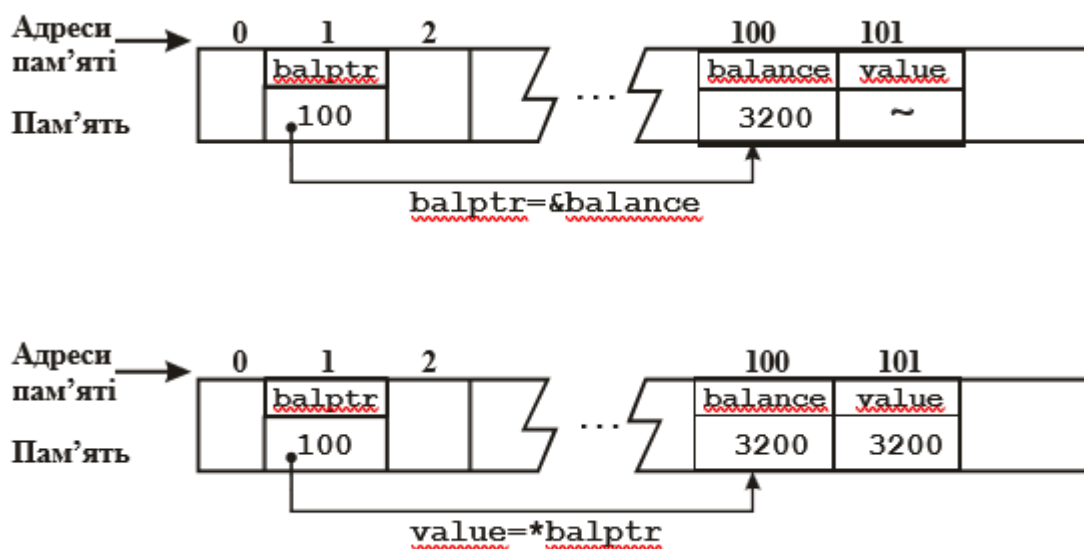


Рис. 6.1. Дія операторів "*" і "&"

Послідовність операцій, відображених на рис. 6.1, безпосередньо реалізується у наведеному нижче коді програми:

```
#include <iostream>    // Потокowe введення-виведення
#include <conio>        // Консольний режим роботи
using namespace std;   // Використання стандартного простору імен

int main()
{
    int balance;
    int *ptr;
    int value;

    balance = 3200; ptr = &balance; value = *ptr;
    cout << "Баланс дорівнює: " << value << endl;

    getch(); return 0;
}
```

Внаслідок виконання ця програма відображає на екрані такі результати:

```
Баланс дорівнює: 3200
```

Шкода, але знак множення (*) і оператор із значенням "за адресою" позначаються однаковими символами "зірочка", що іноді збиває з пантелику новачків у мові програмування C++. Ці операції

ніяк не пов'язані одна з іншою. Майте на увазі, що оператори "*" і "&" мають вищий пріоритет, ніж будь-який з арифметичних операторів, за винятком унарного мінуса, пріоритет якого такий самий, як у операторів, що вживаються для роботи з покажчиками.

Операція непрямого доступу — механізм використання покажчика для доступу до деякого об'єкта.

Операції, що виконуються за допомогою покажчиків, часто називають *операціями непрямого доступу*, оскільки ми опосередковано отримуємо доступ до змінної за допомогою деякої іншої змінної.

3. Важливість застосування базового типу покажчика

На прикладі попередньої програми було показано можливість присвоєння змінній **value** значення змінної **balance** за допомогою операції непрямого доступу, тобто з використанням покажчика. Можливо, при цьому у Вас промайнуло запитання: "Як C++-компілятор дізнається про те, скільки необхідно скопіювати байтів у змінну **value** з області пам'яті, яка адресується покажчиком **ptr**?". Сформулюємо те саме запитання в більш загальному вигляді: як C++-компілятор передає належну кількість байтів у процесі виконання операції присвоєння з використанням покажчика? Відповідь звучить так. Тип даних, який адресується покажчиком, вважається базовим типом покажчика. Оскільки **ptr** є покажчиком на цілочисельний тип, то у цьому випадку C++-компілятор скопіює в змінну **value** з області пам'яті, яка адресується покажчиком **ptr**, чотири байти інформації (що справедливо для 32-розрядного середовища). Але якби ми мали справу з **double**-покажчиком, то в аналогічній ситуації скопіювалося б вісім байтів інформації.

Змінні-покажчики повинні завжди вказувати на відповідний тип даних. Наприклад, під час оголошення покажчика типу **int** компілятор "припускає", що всі значення, на які посилається цей покажчик, мають тип **int**. C++-компілятор просто не дасть змоги виконати операцію присвоєння за участю покажчиків (з обох боків від оператора присвоєння), якщо типи цих покажчиків несумісні (по суті не однакові).

Наприклад, такий фрагмент коду програми є некоректним:

```
int *p;
double f;
//... p = &f; // Помилка!
```

Некоректність цього фрагмента полягає в неприпустимості присвоєння **double**-покажчика **int**-покажчику. Вираз **&f** генерує покажчик на **double**-значення, а **p** — покажчик на цілочисельний тип **int**. Ці два типи несумісні, тому компілятор відзначить цю настанову як помилкову і не скомпілює програму.

Як було заявлено вище, під час присвоєння два покажчики мають бути сумісні за типом, проте це серйозне обмеження можна подолати (правда, на свій страх і ризик) за допомогою операції приведення типів. Наприклад, такий фрагмент коду програми тепер формально є більш коректним:

```
int *p;
double f;
//...
p = (int *) &f; // Тепер формальне все ОК!
```

Операція приведення до типу **(int *)** викличе перетворення **double** до **int**-покажчика. Все ж таки використання операції приведення з цією метою є дещо сумнівним, оскільки саме базовий тип покажчика визначає, як компілятор поводитиметься з даними, на які він посилається. У цьому випадку, незважаючи на те, що покажчик **p** (після виконання останньої настанови) насправді вказує на значення з плинною крапкою, компілятор, як і раніше, "вважає", що він вказує на цілочисельне значення (оскільки **p** за визначенням — **int**-покажчик).

Щоб краще зрозуміти, чому використання операції приведення типів під час присвоєння одного покажчика іншому не завжди є допустимим, розглянемо таку програму.

```
#include <iostream> // Потокowe введення-виведення
using namespace std; // Використання стандартного простору імен
```



```

int main()
{
    double x, y;
    int *p;
    x = 123.23;
    p = (int *) &x;    // Використовуємо операцію приведення типів
                        // для присвоєння double-показчика int-показчику.
    y = *p;             // Що відбувається у процесі виконання цієї настанови?
    cout << y;          // Що виведе ця настанова?

    getch(); return 0;
}

```

Як бачимо, у цьому коді програми змінній **p** (точніше, показчику на цілочисельне значення) присвоюється адреса змінної **x** (яка має тип **double**). Отже, коли змінній **y** присвоюється значення, яке адресується показчиком **p**, то змінна **y** отримує тільки чотири байти даних (а не всі вісім, що є потрібними для **double**-значення), оскільки **p** – показчик на цілочисельний тип **int**. Таким чином, у процесі виконання **cout**-настанови на екран буде виведено не число 123.23, а, висловлюючись мовою програмістів, "сміття", яке насправді є молодшими 4-ма байтами мантиси.

4. Присвоєння значень за допомогою показчика

Під час присвоєння значення області пам'яті, яка адресується показчиком, його (показчик) можна використовувати з лівого боку від оператора присвоєння. Наприклад, у процесі виконання такої настанови (якщо **p** – показчик на цілочисельний тип)

```
*p = 101;
```

число 101 присвоюється області пам'яті, в **p**, яка адресується показчиком. Таким чином, цю настанову можна прочитати так: "за адресою **p** поміщаємо значення 101". Щоб інкрементувати або декрементувати значення, розташоване у області пам'яті, яка адресується показчиком, можна використовувати настанову, подібну до такої:

```
(*p)++;
```

Круглі дужки тут є обов'язковими, оскільки оператор "*" має нижчий пріоритет, ніж оператор "++".

Присвоєння значень з використанням показчиків продемонстровано у наведеному нижче коді програми.

```

#include <iostream>    // Потокowe введення-виведення
using namespace std;  // Використання стандартного простору імен

int main()
{
    int *p, n; p = &n;
    *p = 100;
    cout << n << " "; (*p)++;
    cout << n << " "; (*p)--;
    cout << n << endl;

    getch(); return 0;
}

```

Ось такі результати генерує ця програма. 100 101 100

Запитання для контролю знань:

1. Який символ використовується при оголошенні змінної для позначення того, що вона є показчиком?
2. Що саме зберігається у змінній-показчику після її коректної ініціалізації адресою іншої

змінної?

3. Який оператор використовується для отримання адреси існуючої змінної для ініціалізації покажчика?
4. Яке значення рекомендується присвоювати покажчику при оголошенні, якщо ви ще не маєте конкретної адреси для нього?
5. Якщо ми маємо `int x = 10; int *p = &x;`, що виведе команда друку значення `*p`?
6. Чому важливо вказувати тип даних при оголошенні покажчика (наприклад, `float *ptr`)?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Тема 8.2. Масиви покажчиків. Покажчики на функцію.

Навчальна мета: Метою цього заняття є формування у студентів комплексного розуміння механізмів непрямої адресації через поглиблене вивчення складних структур даних та динамічного керування логікою програми. Навчальний процес спрямований на те, щоб кожен учасник опанував техніку створення та використання масивів, елементами яких є адреси в пам'яті, що дозволяє ефективно працювати з нерегулярними структурами даних та рядками довільної довжини. Окрему увагу приділено концепції покажчиків на функції як інструменту для реалізації принципів зворотного виклику та динамічного вибору алгоритмів під час виконання коду.

Розвиваюча мета: стимулювання аналітичного мислення та здатності до абстрактної візуалізації складних процесів у пам'яті комп'ютера. Вивчення масивів покажчиків та покажчиків на функції сприяє формуванню структурного підходу до розв'язання задач, де студент вчиться бачити програму не як лінійну послідовність команд, а як динамічну систему взаємопов'язаних адрес та об'єктів.

Виховна мета: полягає у формуванні професійної відповідальності та етики програмування, що ґрунтується на розумінні критичної важливості точності при роботі з прямою адресацією пам'яті. Робота з масивами покажчиків та функціями через адреси вимагає від студента особливої дисципліни та культури написання коду, оскільки найменша необачність у таких структурах може призвести до критичних помилок у безпеці або стабільності системи.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

1. Показчики і масиви – взаємозамінні поняття

У мові програмування C++ показчики і масиви тісно пов'язані між собою, причому настільки, що часто поняття "показчик" і "масив" взаємозамінні. У цьому підрозділі ми спробуємо простежити цей зв'язок. Для початку розглянемо такий фрагмент програми:

```
char str[80]; char *p1; p1 = str;
```

У цьому записі `str` є іменем масиву, що містить 80 символів, а `p1` – показчик на тип **char**. Особливий інтерес представляє третій рядок, у процесі виконання якого змінній `p1` присвоюється адреса першого елемента масиву `str`. Іншими словами, після цього присвоєння `p1` вказуватиме на елемент `str[0]`. Йдеться про те, що у мові програмування C++ використання імені масиву без індексу генерує показчик на перший елемент цього масиву. Таким чином, у процесі виконання операції присвоєння `p1 = str` адреса `str[0]` присвоюється показчику `p1`.

Ім'я масиву без індексу утворює показчик на початок цього масиву.

Оскільки після розглянутого вище присвоєння показчик `p1` вказуватиме на початок масиву `str`, то `p1` можна використовувати для доступу до елементів цього масиву. Наприклад, якщо потрібно отримати доступ до п'ятого елемента масиву `str`, використовується один з таких виразів: `str[4]` або `*(p1+4)`.

Необхідність використання круглих дужок, у які поміщено вираз `p1+4`, пов'язана з тим, що оператор `"*"` має вищий пріоритет, ніж оператор додавання `"+"`. Без цих круглих дужок вираз звівся би до отримання значення, яке адресується показчиком `p1`, тобто значення першого елемента масиву, яке потім було б збільшено на 4.

2. Основні відмінності між індексуванням елементів масивів і арифметичними операціями над показчиками

У мові програмування C++ передбачено два способи доступу до елементів масивів: за допомогою індексування елементів масивів і арифметики показчиків. Йдеться про те, що арифметичні операції над показчиками іноді виконуються швидше, ніж індексування елементів масивів, особливо під час доступу до елементів, розташуваних яких відрізняється строгою впорядкованістю. Оскільки швидкодія часто є визначальним чинником при виборі тих або інших рішень під час програмування, то використання показчиків для доступу до елементів масиву – характерна особливість багатьох C++-програм. Окрім цього, іноді показчики дають змогу написати дещо компактніший код програми порівняно з використанням індексування елементів масивів.

Щоб краще зрозуміти відмінність між індексуванням елементів масивів і арифметичними операціями над показчиками, розглянемо дві версії однієї і тієї ж самої програми. У наведеному нижче коді програми з рядка тексту виділяють слова, розділені між собою пропусками. Наприклад, з рядка "Привіт, друг" програма повинна виділити слова "Привіт" і "друг". Програмісти зазвичай називають такі розмежовані символні послідовності *лексемами* (*tmp*). У процесі виконання програми вхідний рядок по-символьно копіюється в інший масив (з іменем *tmp*) доти, доки не трапиться пропуск. Після цього виділена лексема виводиться на екран, і процес триває доти, доки не буде досягнуто кінця рядка. Наприклад, якщо як вхідний рядок використовувати рядок "Це тільки простий тест.", то програма відобразить таке:

Це тільки простий тест.

Ось як виглядає версія програми поділу рядка на слова з використанням арифметичних показчиків

```
#include <vcl>
#include <iostream> // Потокowe введення-виведення
#include <cstdio>    // Підтримка системи введення-виведення
#include <conio>     // Консольний режим роботи
using namespace std; // Використання стандартного простору імен
```

```

int main()
{
    char str[80], tmp[80];
    char *p, *q;

    cout << "Введіть речення: ";

    gets(str); p = str;

    // Зчитуємо лексему з рядка.
    while(*p) {
        q = tmp;    // Встановлюємо q для вказівки на початок масиву tmp.

        /* Зчитуємо символи доти, доки не трапиться або пропуск, або нульовий символ (ознака завершення
        рядка). */ while(*p != ' ' && *p) {
            *q = *p; q++; p++;
        }
        if(*p) p++; // Переміщаємося за пропуск.
        *q = '\0'; // Завершуємо лексему нульовим символом.
        cout << tmp << endl;
    }

    getch(); return 0;
}

```

А ось як виглядає версія тієї ж самої програми з використанням індексування елементів масивів.

```

#include <vcl>
#include <iostream>    // Потокowe введення-виведення
#include <cstdio>       // Підтримка системи введення-виведення
#include <conio>        // Консольний режим роботи
using namespace std;   // Використання стандартного простору імен

int main()
{
    char str[80], tmp[80];

    cout << "Введіть речення: "; gets(str);

    // Зчитуємо лексему з рядка.
    for(int i=0;; i++) {
        // Зчитуємо символи доти, доки не трапиться пропуск,
        // або нульовий символ (ознака завершення рядка).
        for(int j=0; str[i] != ' ' && str[i]; j++, i++) tmp[j] = str[i];
        tmp[j] = '\0'; // Завершуємо лексему нульовим символом.
        cout << tmp << endl;
        if(!str[i]) break;
    }

    getch(); return 0;
}

```

У цих програм може бути різна швидкодія, що зумовлено особливостями генерування коду програми С++-компіляторами. Як правило, під час використання індексування елементів масивів генерується довший код (з великою кількістю машинних команд), ніж це робиться у процесі виконання арифметичних дій над покажчиками. Тому не дивно, що професійно в написаному С++-кодi програми частіше трапляються версії, які орієнтуються на оброблення покажчиків. Але, якщо Ви – програміст-початківець, то сміливо використовуйте індексування елементів масивів, доки не навчитеся вільно володіти покажчиками.

Як було показано вище, можна отримати доступ до масиву, використовуючи арифметичні дії над покажчиками. Цікаво, що у мові програмування С++ покажчик, який посилається на масив, можна індексувати так, як би це було ім'я масиву (це свідчить про тісний зв'язок між покажчиками

і масивами). Відповідний такому підходу синтаксис забезпечує альтернативу арифметичним операціям над покажчиками, оскільки він є дещо зручнішим у деяких ситуаціях. Розглянемо конкретний приклад.

```
#include <iostream>      // Потокowe введення-виведення
#include <cctype>          // Робота з символьними аргументами
#include <conio>           // Консольний режим роботи
using namespace std;     // Використання стандартного простору імен

int main()
{
    char str[20] = "Я тебе люблю";
    char *p;

    p = str;              // Індесуємо покажчик.

    for(int i=0; p[i]; i++) p[i] = toupper(p[i]); // Повертає прописні символи
    cout << p;            // Відображаємо рядок.

    getch(); return 0;
}
```

У процесі виконання програма відобразить на екрані таке:

Я тебе люблю

Ось як працює ця програма. Спочатку в масив `str` вводиться рядок " Я тебе люблю". Потім адреса початку цього рядка присвоюється покажчику `p`. Після цього кожен символ рядка `str` за допомогою функції **`toupper()`** перетвориться в його прописний еквівалент за допомогою індексування покажчика `p`. Пам'ятайте, що вираз `p[i]` за своєю дією однаковий виразу `*(p+i)`.

3. Масиви покажчиків

Покажчики, подібно до інших типів даних, можуть зберігатися в масивах. Ось, наприклад, як виглядає оголошення 10-елементного масиву покажчиків на `int`-значення.

```
int *Array[10];
```

У цьому записі кожен елемент масиву `Array` містить покажчик на цілочисельне значення. Щоб присвоїти адресу `int`-змінній з іменем `var` третьому елементу цього масиву покажчиків, записується таке:

```
Array[2] = &var;
```

Необхідно пам'ятати, що тут `Array` – масив покажчиків на цілочисельні значення. Елементи цього масиву можуть містити тільки значення, які є адресами змінних цілого типу. Ось тому змінна `var` передує оператору `"&"`.

Щоб присвоїти значення змінної `var` цілочисельній змінній `x` за допомогою масиву `Array`, використовують такий синтаксис:

```
x = *Array[2];
```

Оскільки адреса змінної `var` зберігається в елементі `Array[2]`, то застосування оператора `"*"` до цієї індексованої змінної дасть змогу набути значення змінній `var`.

Подібно до інших масивів, масиви покажчиків можна ініціалізувати. Як правило, масиви ініціалізованих покажчиків використовують для зберігання покажчиків на рядки. Наприклад, щоб створити функцію, яка виводить щасливі передбачення, можна таким чином визначити масив **`fortunes`**:

```
char *fortunes[] = {
    "Незабаром гроші потечуть до Вас рікою.\n",
    "Ваше життя осяє нове кохання.\n",
    "Ви житимете довго і щасливо.\n",
}
```

```

        "Гроші, вкладені зараз в справу, незабаром принесуть дохід.\n" ,
        "Близький друг шукатиме Вашої підтримки.\n"
    };

```

Не забувайте, що мова програмування C++ забезпечує зберігання всіх рядкових літералів у таблиці рядків, пов'язаній з конкретною програмою, тому масив потрібен тільки для зберігання покажчиків на ці рядки. Таким чином, для виведення другого повідомлення достатньо використовувати настанову, подібну до такої:

```
cout << fortunes[1];
```

Наведений нижче код програми передбачень є цілком коректною. Для отримання випадкових чисел у ній використовується функція **rand()**, а для отримання випадкових чисел в діапазоні від 0 до 4 – оператор ділення за модулем, оскільки саме такі числа можуть слугувати для доступу до елементів масиву за індексом.

```

#include <vcl>
#include <iostream>      // Потокowe введення-виведення
#include <cstdlib>        // Використання бібліотечних функцій
#include <conio>           // Консольний режим роботи
using namespace std;     // Використання стандартного простору імен
char *fortunes[] = {
    "Незабаром гроші потечуть до Вас рікою.\n" , "Ваше життя осяє нове кохання.\n" ,
    "Ви житимете довго і щасливо.\n" ,
    "Гроші, вкладені зараз в справу, незабаром принесуть дохід.\n" , "Близький друг шукатиме Вашої підтримки.\n"
};
int main()
{
    int chance;
    cout << "Щоб дізнатися про свою долю, натисніть будь-яку клавішу: ";
    // Рандомізуємо генератор випадкових чисел.
    while(!kbhit()) rand(); cout << endl;
    chance = rand(); chance = chance % 5;

    cout << fortunes[chance];

    getch();

    return 0;
}

```

Зверніть увагу на цикл **while**, який викликає функцію **rand()** доти, доки не буде натиснуто на яку-небудь клавішу. Оскільки функція **rand()** завжди генерує одну і ту саму послідовність випадкових чисел, важливо мати можливість програмно використовувати цю послідовність з певної довільної позиції¹. Ефект випадковості досягається за рахунок повторних звернень до функції **rand()**. Коли користувач натисне на клавішу, цикл зупиниться на деякій випадковій позиції послідовності чисел, що генеруються, і ця позиція визначить номер повідомлення, яке буде виведено на екран. Нагадаємо, що функція **kbhit()** є достатньо поширеним розширенням бібліотеки функцій мови програмування C++, яка забезпечується багатьма компіляторами, але не входить в стандартний пакет бібліотечних функцій мови C++.

У наведеному нижче прикладі використовується двовимірний масив покажчиків для розроблення програми, яка відображає синтаксис-пам'ятку за ключовими словами мови програмування C++. У програмі ініціалізується перелік покажчиків на рядки. Перша розмірність масиву призначена для вказівки на ключові слова мови програмування C++, а друга – на короткий їх опис. Перелік завершується двома нульовими рядками, які використовуються як ознака кінця списку. Користувач вводить ключове слово, а програма повинна вивести на екран його опис. Як бачимо, цей перелік містить всього декілька ключових слів. Тому його продовження залишається за Вами.

```

#include <iostream>      // Потокowe введення-виведення
#include <cstring>       // Робота з рядковими типами даних

```

```

#include <conio>           // Консольний режим роботи
using namespace std;      // Використання стандартного простору імен

char *keyword[][2] = {
    "for", "for(ініціалізація; умова; інкремент)", "if", "if(умова)... else ...",
    "switch", "switch(значення) { case-перелік }", "while", "while(умова)...",
    // Сюди потрібно додати решту ключових слів мови програмування C++. "", "" // Перелік повинен завершуватися
    нульовими рядками.
};

int main()
{
    char str[80];

    cout << "Введіть ключове слово: "; cin >> str;
    // Відображаємо синтаксис.
    for(int i=0; *keyword[i][0]; i++)
        if(!strcmp(keyword[i][0], str)) cout << keyword[i][1];

    getch(); return 0;
}

```

Ось приклад виконання цієї програми:

Введіть ключове слово: for for(ініціалізація; умова; інкремент)

У цьому коді програми зверніть увагу на керівний вираз, організований оператором циклу **for**. Він призводить до завершення циклу, коли елемент **keyword[i][0]** містить покажчик на нуль, який інтерпретується як значення ФАЛЬШ. Отже, цикл зупиняється тоді, коли трапляється нульовий рядок, який завершує масив покажчиків.

Запитання для контролю знань:

1. Що зберігається в елементах масиву покажчиків?
2. Як правильно оголосити покажчик на функцію, яка приймає два інпути типу `int` і повертає `void`?
3. Навіщо найчастіше використовується масив покажчиків на рядки (`char *arr[]`) замість двовимірного масиву (`char arr[][N]`)?
4. Припустимо, є `int a = 10; int *p = &a; int **pp = &p;`. Як отримати значення 'а' через 'pp'?
5. Яка основна перевага використання покажчиків на функції?
6. Що станеться, якщо викликати функцію через покажчик, який дорівнює `NULL`?
7. Як виглядає виклик функції через покажчик `void (*f_ptr)()`?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Змістовий модуль 9. Робота з файлами

Тема 9.1. Робота з текстовими файлами.

Навчальна мета: Навчальна мета заняття з теми «Робота з текстовими файлами» полягає у формуванні в учнів цілісного розуміння механізмів взаємодії програмного коду із зовнішніми носіями даних. Під час вивчення матеріалу студент має опанувати логіку відкриття та закриття файлових потоків, усвідомити різницю між режимами доступу, такими як читання, запис та додавання інформації, а також навчитися коректно опрацьовувати текстовий вміст на рівні символів, рядків або цілих блоків даних.

Розвиваюча мета: спрямована на активізацію аналітичних здібностей учнів через виявлення причинно-наслідкових зв'язків між структурою збережених даних та ефективністю їх подальшої обробки. Процес навчання стимулює розвиток логічного мислення, оскільки вимагає від студента вміння прогнозувати поведінку програми в умовах обмежених ресурсів або при виникненні виняткових ситуацій, як-от відсутність доступу до файлу чи пошкодження його вмісту. Робота з текстовими масивами сприяє вдосконаленню навичок абстрагування, дозволяючи переходити від маніпуляцій з окремими змінними до оперування великими обсягами інформації.

Виховна мета: зосереджена на формуванні відповідального ставлення до інформації як до цінного ресурсу та основи цифрової етики. Вона спрямована на виховання культури опрацювання даних, що передбачає дбайливе ставлення до конфіденційності, авторства та цілісності чужої інформації. Під час роботи з файлами в учнів закладаються основи професійної сумлінності: розуміння того, що кожна помилка в коді може призвести до незворотної втрати або викривлення важливих відомостей, що стимулює розвиток почуття обов'язку за результати власної праці.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

Незважаючи на те, що файлова система в С відрізняється від тієї, що використовується у мові програмування С++, між ними є багато загального. Система оброблення файлів складається з декількох взаємопов'язаних функцій. Найбільш популярні з них перераховані в табл. А.3.

Табл. А.3. С-функції оброблення файлів

Функція	Призначення
<u>fopen()</u>	Відкриває потік
<u>fclose()</u>	Закриває потік
<u>fputc()</u>	Записує символ у потік
<u>fgetc()</u>	Зчитує символ з потоку
<u>fwrite()</u>	Записує блок даних у потік
<u>fread()</u>	Зчитує блок даних з потоку
<u>fseek()</u>	Встановлює індикатор позиції файлу на заданий байт у потоці
<u>fprintf()</u>	Робить для потоку те, що функція <u>printf()</u> робить для консолі
<u>fscanf()</u>	Робить для потоку те, що функція <u>scanf()</u> робить для консолі ()
<u>feof()</u>	Повертає значення <u>true</u> , якщо досягнуто кінець файлу
<u>ferror()</u>	Повертає значення <u>true</u> , якщо виникла помилка
<u>rewind()</u>	Встановлює індикатор позиції файлу у початок файлу
<u>remove()</u>	Видаляє файл

Загальний потік, який "цементує" С-систему введення-виведення, є *файловий покажчик* (file pointer). Файловий покажчик це покажчик на інформацію про файл, яка містить його ім'я, статус і поточну позицію. По суті, файловий покажчик ідентифікує конкретний дисковий файл і використовується потоком, щоб повідомити всім С-функціям введення-виведення, де вони повинні виконувати операції. Файловий покажчик це змінна-покажчик типу **FILE**, який визначено у заголовку **stdio.h**.

Функція **fopen()** здійснює три завдання.

- відкриває потік;
- пов'язує файл з потоком;
- повертає покажчик типу **FILE** на цей потік.

Найчастіше під файлом маємо на увазі дисковий файл. Функція **fopen()** має такий прототип:

```
FILE *fopen(const char * filename, const char *mode);
```

У цьому записі параметр *filename* вказує на ім'я файлу, що відкривається, а параметр *mode* на рядок, що містить потрібний статус (режим) відкриття файлу. Можливі значення параметра *mode* показані у наведеній табл. А.4. Параметр *filename* повинен представляти рядок символів, які становлять ім'я файлу, яке допустимо у даній операційній системі. Цей рядок може містити специфікацію шляху, якщо діюче середовище підтримує таку можливість.

Якщо функція **fopen()** успішно відкрила заданий файл, вона повертає покажчик **FILE**. Цей покажчик ідентифікує файл і використовується більшістю інших файлових системних функцій. Він не повинен піддаватися модифікації кодом програми. Якщо файл не вдається відкрити, повертається нульовий покажчик.

Як видно з табл. А.4, файл можна відкривати або в текстовому, або у двійковому режимі. Під час відкриття в текстовому режимі виконуються перетворення деяких послідовностей символів.

Табл. А.4. Допустимі значення параметра **mode**

mode	І призначення
"r"	Відкриває текстовий файл для зчитування
"w"	Створює текстовий файл для запису
"a"	Відкриває текстовий файл для запису у кінець файлу
"rb"	Відкриває двійковий файл для зчитування
"wb"	Створює двійковий файл для запису
"ab"	Відкриває двійковий файл для запису у кінець файлу
"r+"	Відкриває текстовий файл для зчитування і запису
"w+"	Створює текстовий файл для зчитування і запису
"a+"	Відкриває текстовий файл для зчитування і запису у кінець файлу
"r+b"	Відкриває двійковий файл для зчитування і запису
"w+b"	Створює двійковий файл для зчитування і запису
"a+b"	Відкриває двійковий файл для зчитування і запису у кінець файлу

Якщо виникає потреба відкрити файл **test** для запису, використаємо таку настанову:

```
fp = fopen("test", "w");
```

У цьому записі змінна **fp** має тип **FILE ***. Проте часто для відкриття файлу використовують такий програмний код:

```
if((fp = fopen("test", "w"))==NULL) { printf("Не вдається відкрити файл."); exit(1); }
}
```

При такому методі виявляється будь-яка помилка, пов'язана з відкриттям файлу (наприклад, під час використання захищеного від запису або заповненого диска), і тільки після цього можна робити спробу запису у заданий файл. **NULL** це макроім'я, що є визначеним у заголовку **stdio.h**.

Якщо Ви використовуєте функцію **fopen()**, щоб відкрити файл виключно для виконання операцій виведення (записи), будь-який вже наявний файл із заданим іменем буде стертий, і замість нього буде створено новий. Якщо файл з таким іменем не існує, він буде створено. Якщо виникає потреба додавати дані у кінець файлу, використовується режим "a". Якщо опиниться, що вказано файл не існує, він буде створено. Щоб відкрити файл для виконання операцій зчитування, необхідна наявність цього файлу. Інакше функція поверне значення помилки. Нарешті, якщо файл відкривається для виконання операцій зчитування-запису, то у разі його існування він не буде видалений; але його немає, він буде створено.

Функція **fputc()** використовують для виведення символів у потік, заздалегідь відкрито для запису за допомогою функції **fopen()**. Її прототип має такий вигляд:

```
int fputc(int ch, FILE *fp);
```

У цьому записі параметр **fp** файловий покажчик, що повертається функцією **fopen()**, а параметр **ch** символ, що виводиться. Файловий покажчик повідомляє функції **fputc()**, в який дисковий файл необхідно записати символ. Незважаючи на те, що параметр **ch** має тип **int**, у ньому використовується тільки молодший байт.

При успішному виконанні операції виведення функція **fputc()** повертає записаний у файл символ, інакше значення **EOF**.

Функція **fgetc()** використовують для зчитування символів з потоку, відкритого в режимі зчитування за допомогою функції **fopen()**. Її прототип має такий вигляд:

```
int fgetc(FILE *fp);
```

У цьому записі параметр **fp** файловий покажчик, що повертається функцією **fopen()**. Незважаючи на те, що функція **fgetc()** повертає значення типу **int**, його старший байт дорівнює нулю. Під час виникнення помилки або досягненні кінця файлу функція **fgetc()** повертає значення **EOF**. Отже, для того, щоби рахувати весь вміст текстового файлу (до самого кінця), можна використати такий програмний код:

```

ch = fgetc(fp); while(ch != EOF) {
    ch = fgetc(fp);
}

```

Файлова система у мові С може також обробляти двійкові дані. Якщо файл відкрито у режимі введення двійкових даних, то не виключено, що може бути зчитано ціле число, дорівнює значенню **EOF**. У цьому випадку під час використання такого коду програми перевірки досягнення кінця файлу, як **ch != EOF**, буде створена ситуація, еквівалентна отриманню сигналу про досягнення кінця файлу, хоча насправді фізичний кінець файлу може бути ще не досягнутий. Щоб вирішити цю проблему, у мові С передбачена функція **feof()**, яка використовується для визначення факту досягнення кінця файлу під час зчитування двійкових даних. Її прототип має такий вигляд:

```
int feof(FILE *fp);
```

У цьому записі параметр *fp* ідентифікує файл. Функція **feof()** повертає ненульове значення, якщо кінець файлу був-таки досягнутий; інакше нуль. Таким чином, у процесі виконання такої настанови буде зчитаний весь вміст двійкового файлу:

```
while(!feof(fp)) ch = fgetc(fp);
```

Безумовно, цей метод застосовний і до текстових файлів.

Функція **fclose()** закриває потік, який був відкрито у результаті звернення до функції **fopen()**. Вона записує у файл усі дані, що ще залишилися у дисковому буфері, і закриває файл на рівні операційної системи. Під час виклику функції **fclose()** звільняється блок керування файлом, що пов'язаний з потоком, що робить його доступним для повторного використання. Ймовірно, Вам відомо про існування обмеження операційної системи на кількість файлів, які можна тримати відкритими у будь-який момент часу, тому, перш ніж відкривати наступний файл, рекомендується закрити всі файли, вже непотрібні для роботи.

Функція **fclose()** має такий прототип:

```
int fclose(FILE *fp);
```

У цьому записі параметр *fp* файловий покажчик, що повертається функцією **fopen()**. При успішному виконанні функція **fclose()** повертає нуль; інакше повертається значення **EOF**. Спроба закрити вже закритий файл розцінюється як помилка. Під час видалення носія даних до закриття файлу згенерує помилка, як і у разі недоліку вільного простору на диску.

Функції **fopen()**, **fgetc()**, **fputc()** і **fclose()** становлять мінімальний набір операцій з файлами. Їх використання продемонстровано у наведеному нижче коді програми, яка здійснює копіювання файлу. Зверніть увагу на те, що файли відкриваються у двійковому режимі і що для перевірки досягнення кінця файлу використовується функція **feof()**.

```

#include <stdio.h>           // Для введення/виведення
int main(int argc, char *argv[])
{
    FILE *in, *out;
    char ch;
    if(argc != 3) {
        printf("Ви забули ввести ім'я файлу" << endl); return 1;
    }
    if((in=fopen(argv[1], "rb")) == NULL) {
        printf("Не вдається відкрити початковий файл" << endl); return 1;
    }
    if((out=fopen(argv[2], "wb")) == NULL) {
        printf("Не вдається відкрити файл-приймач" << endl); return 1;
    }
    // Код копіювання вмісту файлу
    while(!feof(in)) {
        ch = fgetc(in);
        if(!feof(in)) fputc(ch, out);
    }
    fclose(in); fclose(out);
    getch(); return 0;
}

```

Функція **ferror()** використовують для визначення факту виникнення помилки у процесі виконання операції з файлом. Її прототип має такий вигляд:

```
int ferror(FILE *fp);
```

У цьому записі параметр *fp* дійсний файловий покажчик. Функція **ferror()** повертає значення **true**, якщо у процесі виконання останньої файлової операції відбулася помилка; інакше значення **false**. Оскільки виникнення помилки можливе у процесі виконання будь-якої операції з файлом, функцію **ferror()** необхідно викликати відразу після кожної функції оброблення файлів; інакше інформацію про помилку можна просто втратити.

Функція **rewind()** переміщає індикатор позиції файлу у початок файлу, що задається як аргумент. Її прототип має такий вигляд:

```
void rewind(FILE *fp);
```

У цьому записі параметр *fp* дійсний файловий покажчик.

У файловій системі мови С передбачено дві функції, **fread()** і **fwrite()**, які дають змогу зчитувати і записувати блоки даних. Ці функції подібні С++функціям **read()** і **write()**. Їх прототипи мають такий вигляд:

```
size_t fread(void *buffer, size_t num_bytes, size_t count, FILE *fp);
```

```
size_t fwrite(const void *buffer, size_t num_bytes size_t count, FILE *fp);
```

Під час виклику функції **fread()** параметр *buffer* є покажчик на область пам'яті, яка призначена для прийому даних, які зчитуються з файлу. Функція зчитує *count* об'єктів завдовжки *num_bytes* з потоку, яка адресується файловим покажчиком *fp*. Функція **fread()** повертає кількість зчитаних об'єктів, яка може опинитися менше заданого значення *count*, якщо у процесі виконання цієї операції виникла помилка або був досягнуто кінець файлу.

Під час виклику функції **fwrite()** параметр *buffer* є покажчик на інформацію, яка підлягає запису у файл. Ця функція записує *count* об'єктів завдовжки *num_bytes* у потік, яка адресується файловим покажчиком *fp*. Функція **fwrite()** повертає кількість записаних об'єктів, яка буде дорівнює значенню *count*, якщо у процесі виконання цієї операції не було помилки.

Якщо під час виклику функцій **fread()** і **fwrite()** файл був відкрито для виконання двійкової операції, то вони можуть зчитувати або записувати дані будь-якого типу. Наприклад, наведений нижче код програми записує у дисковий файл значення типу **float**.

Запитання для контролю знань:

1. Який режим доступу слід обрати, якщо потрібно додати нові дані в кінець існуючого файлу, не видаляючи його попередній вміст?
2. Що станеться, якщо спробувати відкрити для читання файл, якого не існує в заданій директорії?
3. У контексті цифрової етики та безпеки, що важливо зробити після завершення роботи з файлом?
4. Який символ зазвичай використовується для переходу на новий рядок у текстових файлах більшості сучасних систем?
5. Що таке 'кодування' (encoding) текстового файлу?

Література:

1. Васильєв О. М. Програмування на С++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.

2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Змістовий модуль 10. Динамічні структури даних

Тема 10.1. Списки. Зв'язаний список.

Навчальна мета: полягає у формуванні в учнів фундаментального розуміння принципів організації лінійних структур даних. Головним завданням є опанування концепції динамічного керування пам'яттю, де особлива увага приділяється відмінностям між статичними масивами та зв'язаними списками. Учень має навчитися ідентифікувати базовий елемент списку — вузол — та розуміти механізм зв'язку між елементами через вказівники або посилання.

Розвиваюча мета: спрямована на активізацію аналітичних здібностей учнів через занурення в архітектуру динамічних структур даних.

Виховна мета: зосереджена на формуванні професійної етики та культури програмування, що базується на відповідальному ставленні до ресурсів і системності мислення.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

Лінійні списки - найпростіша форма організації даних з рекурсивною структурою. Цю форму застосовують найчастіше тоді, коли треба обробити деяку сукупність даних, кількість елементів яких заздалегідь не визначена, а взаємний порядок проходження відіграє важливу або вирішальну роль.

Лінійний односпрямований (однозв'язний) список – дані динамічної структури, що являють собою сукупність лінійно зв'язаних однорідних елементів, до яких дозволяється додавати елементи на голови (початку) або в кінець (хвіст) списку, між будь-якими двома іншими і видаляти будь-який елемент (рис.1.).

Даний тип можна описати в такий спосіб:

```
struct List
{
    typeelem Inf;
    List *Link;
};
```

де **typeelem** - будь-який тип C++;

Inf – інформаційна частина;

Link – посилальна частина (вказівник на наступний елемент).

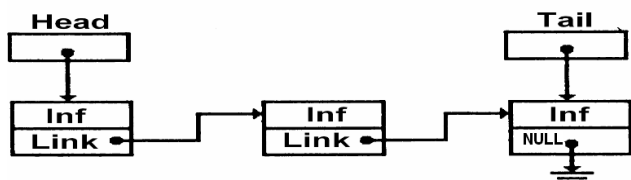


Рис.1. Схематичне зображення однозв'язного лінійного списку

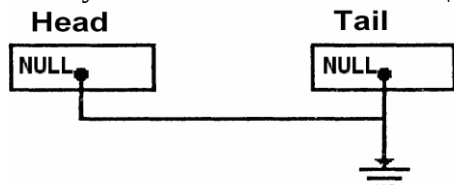
Для роботи з лінійним однозв'язним списком потрібні такі вказівники:

- 1) на голову списку **Head**;
- 2) кінець списку **Tail**, але можна обійтися й без нього;
- 3) **k**-й елемент списку;
- 4) тимчасовий для виділення пам'яті під елементи, які додаються, і для звільнення елементів, що видаляються (ідентифікатор **p**).

Розглянемо основні дії над однозв'язним списком.

Ініціалізація списку

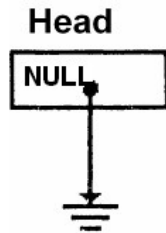
У разі застосування вказівника на кінець списку ініціалізація буде мати такий вигляд (рис.2.):



```
List *Head, *Tail; Head=NULL;
    Tail=NULL;
    або коротшим записом:
    List *Head=NULL, *Tail=NULL;
```

Рис.2. Ініціалізація списку з використанням двох вказівників (на голову та кінець списку)

Якщо вказівник на кінець списку не використовують, ініціалізувати треба тільки один вказівник (рис.3).



List *Head; Head=NULL;

або одним рядком:
List *Head = NULL;

Рис.3. Ініціалізація списку з використанням одного вказівника (на голову списку)

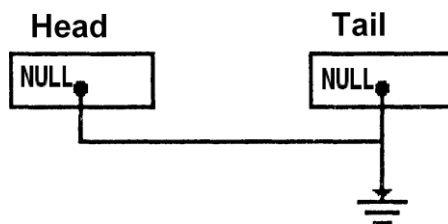
Таким чином, список ініціалізований, але не містить жодного елемента. Тепер можна заповнювати список з голови (додаючи знову створений елемент списку після попередніх) або з кінця (додаючи знову створений елемент перед попереднім).

Додавання елементів у список з використанням двох вказівників (на голову і кінець списку)

Додавання першого елемента в список (рис.4) та додавання елемента в кінець списку (рис.5) відрізняється. У будь-якому випадку необхідний ще один вказівник **p** на поточний (новий) елемент.

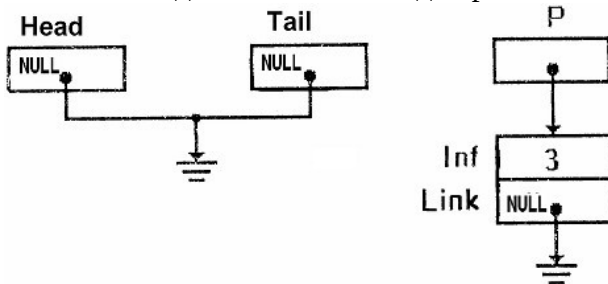
Додавання першого елемента в список

1. Вихідний стан:



//Head==NULL;
//Tail==NULL;
List *p;

2. Виділення пам'яті під перший елемент списку й занесення інформації до нього:

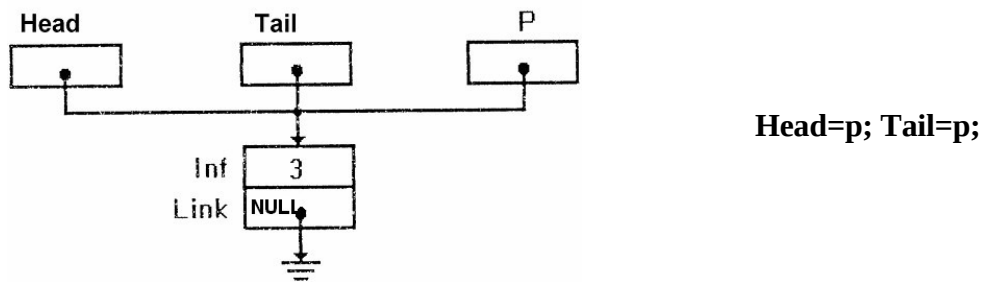


p= new List;
//виділення пам'яті під
//новий елемент
p->Inf=3;
//присвоєння значення
//інформаційній частині
p->Link=NULL;
//присвоєння вказівнику
//на наступний елемент
//значення ознаки кінця

Опис вказівника на поточний елемент і виділення пам'яті краще записувати одним рядком:

List* p= new List;

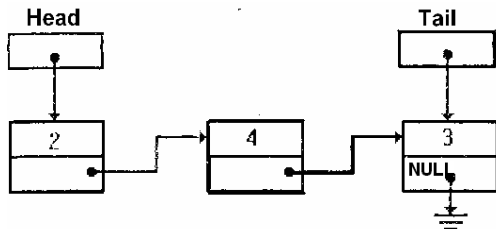
3. Встановлення вказівників **Head**, **Tail** на створений перший елемент:



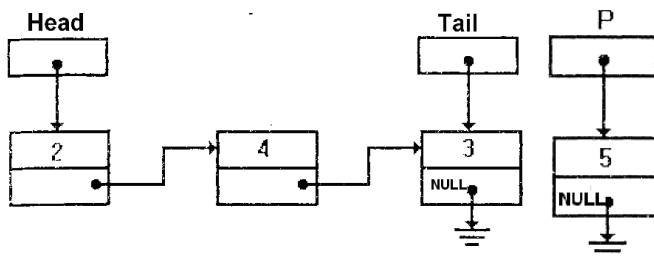
**Рис.4. Додавання першого елемента в список
(з використанням вказівників на голову і кінець)**

Додавання елемента в кінець списку

1. Вихідний стан:

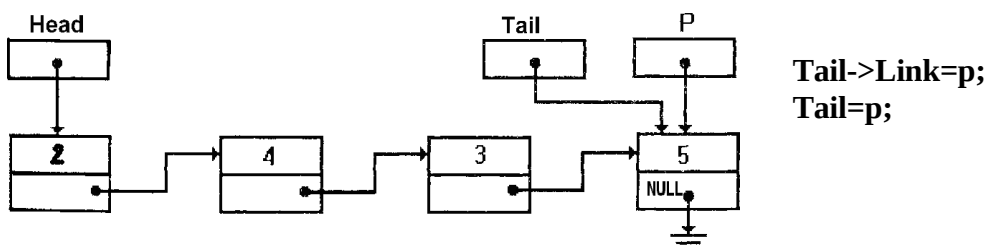


2. Виділення пам'яті під новий елемент списку й занесення інформації до нього:



```
List *p= new List; p->Inf=5;
//присвоєння значення
//інформаційній частині
p->Link=NULL;
//присвоєння вказівнику
//на наступний елемент
//значення ознаки кінця
```

3. Встановлення зв'язку між останнім елементом списку й новим, а також переміщення вказівника кінця списку на новий елемент:



**Рис.5. Додавання елемента в кінець списку
(з використанням вказівників на голову і кінець)**

Функція додавання елемента в кінець списку з використанням вказівників **Head**, **Tail** може мати такий вигляд:

```

void addNode (List * & Head, List * & Tail)
{
//виділення пам'яті під новий елемент списку
List *p = new List;
//заповнення інформаційної частини
cout<<"Enter value: "; cin>>p->Inf;
//встановлення посилання останнього елемента
p->Link = NULL;
//якщо список порожній
if (Head==NULL) //або if (!Head)
//встановлення вказівника Head на перший елемент
Head=p;
//інакше встановлення зв'язку між останнім
//елементом списку й новим
else Tail->Link=p;
//встановлення вказівника кінця списку
//на новий елемент
Tail=p;
}

```

Формальні параметри можна передавати за адресою через вказівники, тоді заголовок функції **addNode** буде мати такий вигляд:

```

void addNode (List * *Head, List * *Tail),

```

а в разі застосування у тілі функції **Head** або **Tail** необхідна операція розіменування:

```

...
if ((*Head)==NULL) //якщо список порожній
//встановлення вказівника Head на перший елемент
(*Head)=p;
//інакше встановлення зв'язку між останнім
//елементом списку й новим
else (*Tail)->link=p;
//встановлення вказівника кінця списку
//на новий елемент
(*Tail)=p;
...

```

Також під час виклику функції треба передавати адресу фактичних параметрів:

```

addNode (&Head, &Tail);.

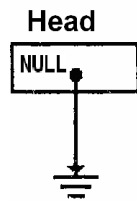
```

Додавання елементів у список з використанням одного вказівника (на голову списку)

Додавання першого елемента в список майже не відрізняється від аналогічної операції з використанням двох вказівників (рис.6), а в разі додавання елемента в кінець списку виникає необхідність додатково визначати останній елемент (рис.7).

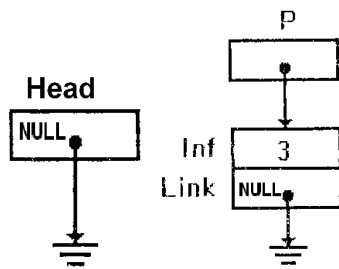
Додавання першого елемента в список

1. Вихідний стан:



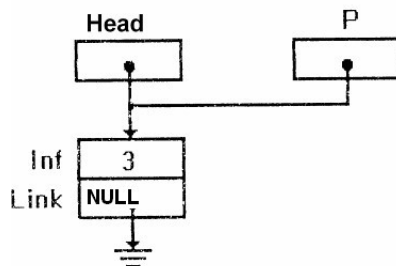
//Head==NULL;

2. Виділення пам'яті під перший елемент списку й занесення інформації до нього:



List *p= new List; p->Inf=3;
p->Link=NULL;

3. Встановлення вказівника **Head** на створений перший елемент:

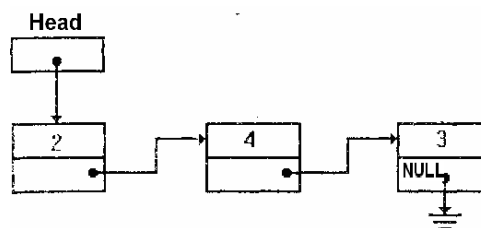


Head=p;

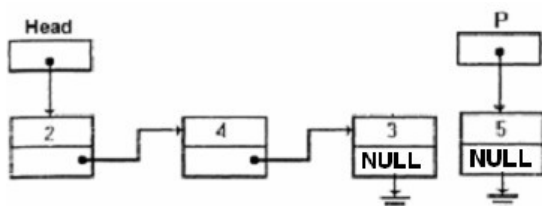
Рис.6. Додавання першого елемента в список (з використанням вказівника на голову)

Додавання елемента в кінець списку

1. Вихідний стан:



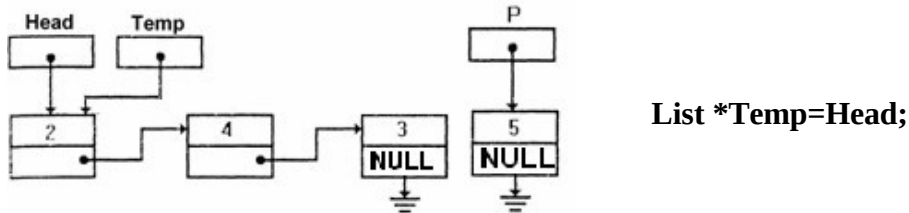
2. Виділення пам'яті під новий елемент списку й занесення інформації до нього:



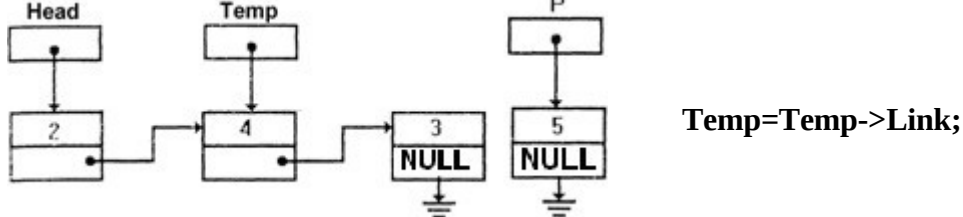
List *p= new List; p->Inf=5;
p->Link=NULL;

3. Знаходження останнього елемента списку й збереження посилання на нього в змінній **Temp**:

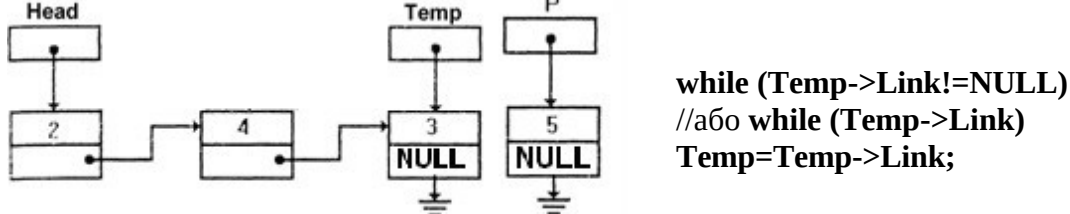
а) встановлення тимчасового вказівника **Temp** на перший елемент списку:



б) переміщення тимчасового вказівника **Temp** на другий елемент списку:



в) встановлення тимчасового вказівника **Temp** на останній елемент списку:



4. Встановлення зв'язку між останнім елементом списку й новим:

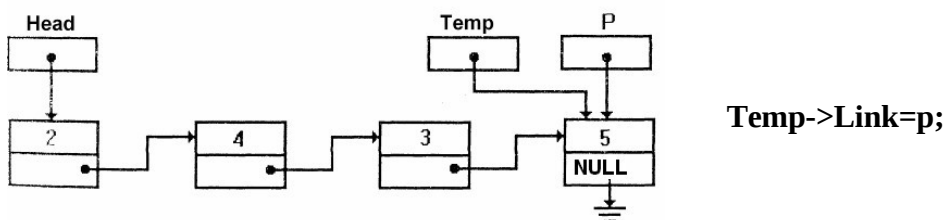


Рис.7. Додавання елемента в кінець списку (з використанням вказівника на голову)

Функція додавання елемента в список з використанням тільки вказівника **Head** може мати такий вигляд:

```
void addNodeEnd(List * & Head)
{
    //виділення пам'яті під новий елемент списку
    List *p = new nodeList;
    //заповнення інформаційної частини
    cout<<"Enter value:"; cin>>p->Inf;
    //встановлення посилання останнього елемента в NULL
    p->Link = NULL;
    //якщо список порожній
    if (Head==NULL)
    //встановлення вказівника Head на перший елемент
    Head=p;
```

```

//інакше знаходження адреси останнього елемента
else
{ List *Temp=Head; while(Temp->Link!= NULL)
Temp = Temp->Link;
//встановлення зв'язку між останнім
//та новим елементом
Temp->Link=p;
}
}

```

Знаходження останнього елемента списку можна описати окремою функцією, яка буде повертати адресу через ім'я функції:

```

List* findLast(List* node)
{
while(node->Link != NULL) node = node->Link;
return node;
}

```

тоді гілка **else** буде мати такий вигляд:

```

...
else
findLast(Head)->Link = node;
...

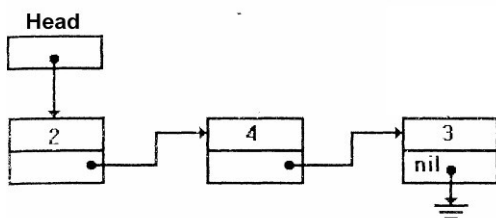
```

Зверніть увагу, що передача вказівника на голову списку **Head** як фактичного параметра не зіпсує його значення, тому що ми передаємо його не за адресою, а за значенням, тобто будемо працювати з його копією.

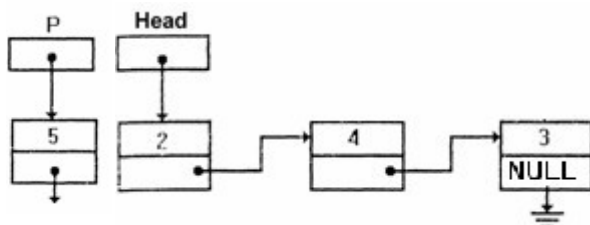
Додавати елементи можна й до вже існуючого списку: в голову списку (рис.8), всередину після заданого (рис.9) та перед заданим (рис.10) елементом.

Додавання елемента в голову списку

1. Вихідний стан:

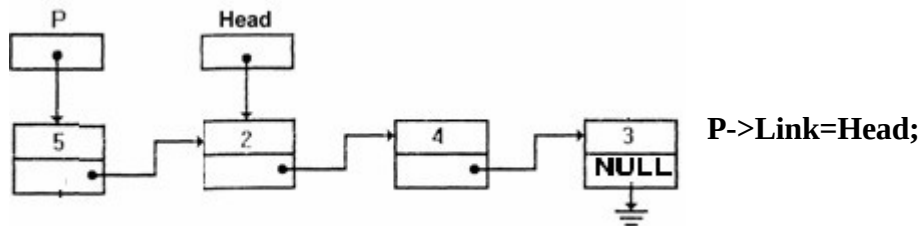


2. Виділення пам'яті під новий елемент списку й заповнення інформаційного поля:



```
List *p= new List; p->Inf=5;
```

3. Встановлення зв'язку між першим елементом списку й новим:



4. Переміщення вказівника на голову списку на новий елемент:

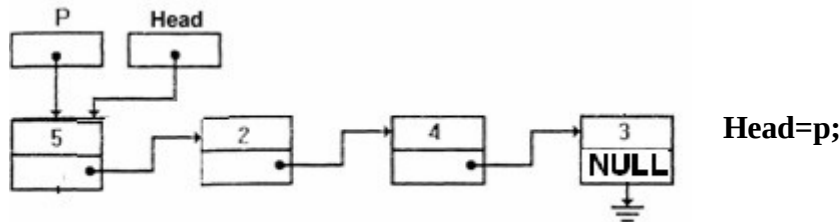


Рис.8. Додавання елемента в голову списку

Функція додавання елемента в голову списку може мати такий вигляд:

```
void addNodeBeg(List * & Head)
{
//виділення пам'яті під новий елемент списку
List *p = new nodeList;
//заповнення інформаційної частини
cout<<"Enter value: "; cin>>p->Inf;
if (Head!=NULL) //якщо список не порожній
//встановлення зв'язку між першим елементом списку
//й новим
p->Link=Head;
//Переміщення вказівника на голову
//на новий елемент
Head=p;
}
```

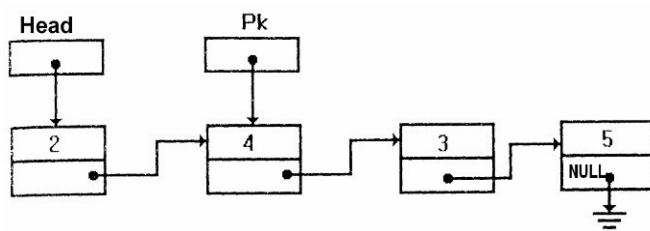
Дану функцію можна затосовувати для створення списку із кінця.

Додавання елемента всередину списку після заданого k-го елемента

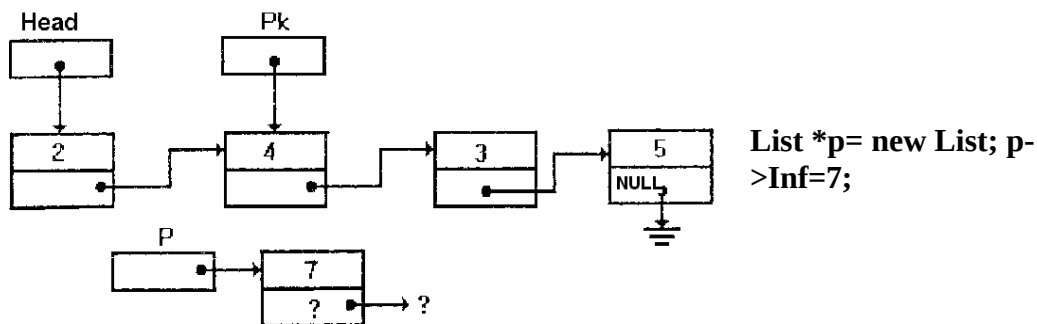
Вважаємо, що адреса заданого **k**-го елемента відома і зберігається у вказівнику **Pk**. Для додавання нового елемента після заданого необхідно:

- 1) створити новий динамічний об'єкт (нову ланку списку);
- 2) у поле **Inf** об'єкта занести задану інформаційну частину;
- 3) у поле **Link** даного об'єкта занести посилання, взяте з відповідного поля тієї ланки, за якою повинна йти нова ланка (вказівник **Pk**);
- 4) у поле **Link** тієї ланки, за якою повинна слідувати нова ланка, занести посилання на цю ланку (вказівник **Pk**).

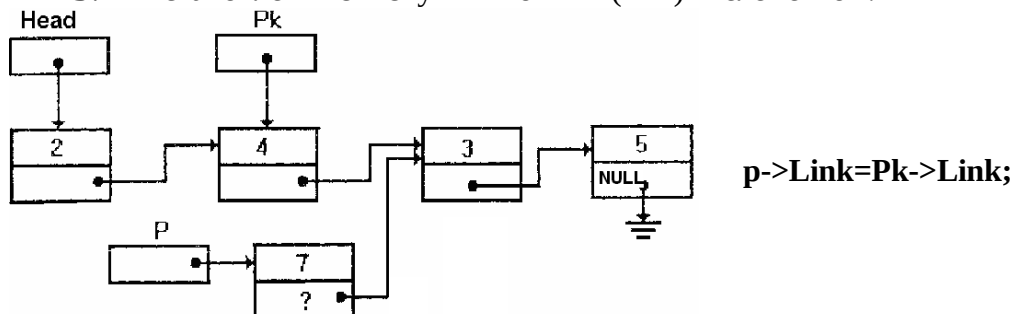
1. Вихідний стан:



2. Виділення пам'яті під новий елемент списку й заповнення інформаційного поля:



3. Встановлення зв'язку між новим і (k+1)-м елементом:



4. Перестановка зв'язку k-го елемента на новий елемент:

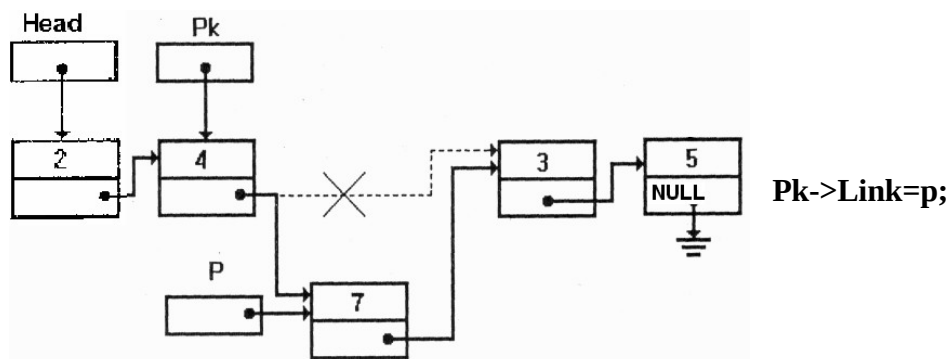


Рис.9. Додавання елемента всередину списку після заданого

Таким чином, опис функції додавання в список заданого елемента після визначеного може мати такий вигляд:

```

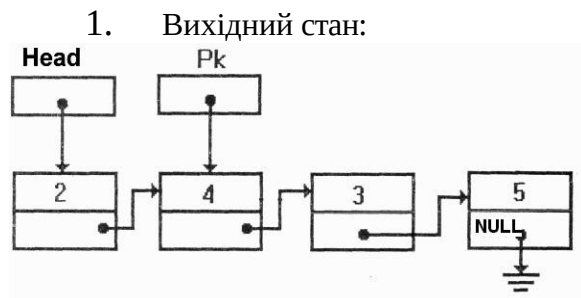
void addNodeBeg(List * & Pk)
{
//створення нового динамічного об'єкта
List *p= new List;
//запис інформаційної частини
p->Inf=Elem;
//заповнення посилання на наступний елемент
p->Link=Pk->Link;
//додавання нової ланки в список
Pk->Link=p;
}

```

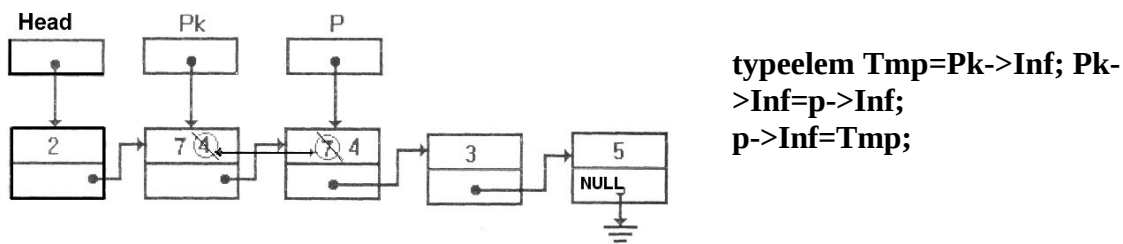
Додавання елемента всередину списку перед заданим k-м елементом

Якщо є адреса (k-1)-го елемента, то необхідну дію можна виконати так, як описано вище. Але якщо є адреса тільки k-го елемента, то задача ускладнюється. У цьому випадку замість того, щоб ще раз від початку списку шукати (k-1)-й елемент, значно простіше виконати вставку *перед* k-м елементом у такий спосіб (вважаємо, що адреса k-го елемента відома і зберігається у вказівнику Pk) (рис.10):

- 1) зробити вставку нового елемента після k-го елемента (таким чином, як це описано вище);
- 2) поміняти місцями значення інформаційних полів k-го й нового елементів;
- 3) переставити вказівник Pk на новий вставлений елемент, який вже містить значення k-го елемента.



3. Обмін місцями значень інформаційних полів **k**-го й нового елементів (змінна **Tmp** повинна мати той же тип, що й інформаційне поле елемента списку):



4. Перестановка вказівника **Pk** на новий елемент:

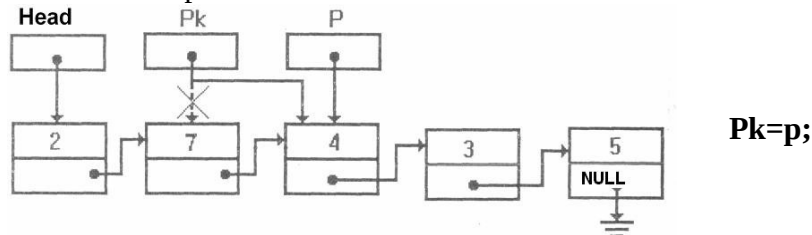


Рис.10. Додавання елемента всередину списку перед заданим

Друк елементів списку

Друк елементів списку від голови до кінця

Виведення на друк елементів списку від голови до кінця виконується аналогічно до описаного вище установа вказівника на останній елемент із єдиною відмінністю: одночасно з переміщенням вказівника виводиться на друк значення інформаційного поля кожного елемента списку.

```
void viewList(List* node)
```

```
{
//перевірка списку на пустоту
if(!node)
//якщо список порожній, виведення повідомлення про це
//та закінчення виконання функції
{
cout<<"List empty!"<<endl; return;
}
cout<<"-----"<<endl;
//поки не останній елемент списку
while(node)
{
//друк поточного елемента
cout << " - Value - " << node->Val << endl;
//перехід на наступний елемент
node = node->Link;
}
}
```

Для виведення всього списку дану функцію треба викликати з фактичним параметром **Head**, який зберігає адресу першого елемента. Функція придатна

також і для виведення на екран частини списку, для цього як фактичний параметр треба передати адресу елемента, з якого буде друкуватися список.

Друк елементів списку від кінця до початку

Вивід на друк елементів списку від кінця до початку не можна виконати так просто, як у попередньому випадку, оскільки зв'язки спрямовані тільки в одну сторону й неможливо просуватися по них у протилежну. У такому разі потрібно використовувати рекурсію або ще один додатковий список (можна просто інвертувати даний список, застосовуючи функцію додавання елемента в голову списку) для тимчасового зберігання елементів вихідного списку.

```
void viewListReverse(List * node)
{
//перевірка списку на пустоту
if(!node)
//якщо досягнуто кінця списку –
//рекурсивне повернення
return;
cout<<"-----"<<endl;
//рекурсивне «занурення»: функція викликає себе
//з адресою наступного елемента списку
viewListReverse(node->Link);
//друк поточного елемента
cout << " - Value - " << node->Val << endl;
}
```

У процесі виконання цієї рекурсивної функції елементи списку будуть виводитися у зворотному порядку. Якщо передавати як фактичний параметр вказівник на голову, то виводитися буде весь список. Функція придатна також і для виведення на екран частини списку, у цьому випадку як параметр передається адреса елемента, з якого треба починати «занурення» у список.

Пошук елемента в списку

Пошук елемента з певними властивостями

Функція пошуку елемента в списку, текст якої наведено нижче, повертає вказівник на той елемент списку, що містить у своїй інформаційній частині значення, задане користувачем; якщо ж такий елемент не знайдено, функція повертає **NULL**. Функції передають два параметри: вказівник на голову списку, у якому буде відбуватися пошук, і значення інформаційної частини елемента списку, яке необхідно знайти.

Алгоритм пошуку дуже простий: будемо послідовно переглядати ланки списку й порівнювати значення інформаційного поля із заданим значенням. Цей процес закінчується у двох випадках:

- чергова ланка списку містить заданий елемент, тоді функція повертає посилання на дану ланку та припиняє свою роботу;

- список було вичерпано, тобто повністю переглянуто, але заданий елемент не знайдено; тоді функція повертає «порожнє» посилання NULL.

```

List* findNode(List* Head, typeelem x)
{
//указання на перший елемент списку
List* node=Head
           while(node != NULL)//або while(node)
               //якщо заданий елемент знайдено
{
if (node->Inf==x)
//закінчення пошуку і повернення вказівника
//на цей елемент
return node;
//у протилежному випадку
else
//перехід на наступну ланку списку
node = node->Link;
}
//якщо список вичерпано,
//то шуканий елемент не знайдено,
//і повернення функцією «порожнього» значення
return NULL;
}

```

Дана функція повертає перше входження заданого елемента в список. Застосовуючи цю ж функцію і починаючи пошук зі знайденої ланки, можна знайти наступне входження заданого елемента й т.д., а також підрахувати кількість входжень елемента в список, злегка змінивши текст функції.

Пошук попереднього елемента в списку

Якщо нам необхідно знайти адресу елемента, який стоїть перед елементом адреса якого відома, можна застосувати таку функцію:

```

List* findPred(List* Head, List* target)
{
//поки не знайдено шуканий елемент
while(Head->Link != target)
{
//перевіряємо, якщо список вичерпаний,
if(!Head)
//повертаємо порожнє значення
return 0; //або return NULL;
//або переходимо на наступний елемент
Head = Head->Link;
}
//повертаємо шукану адресу
return Head;
}

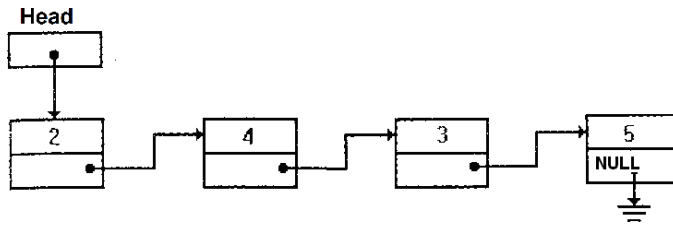
```

Видалення елементів списку

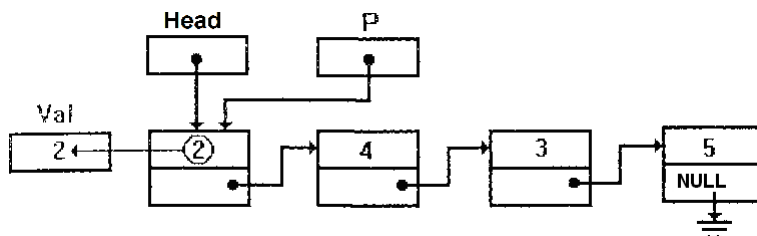
Під час видалення першого (рис.11) та останнього елемента (рис.13) списку необхідно не загубити значення вказівника на голову та ознаку кінця списку.

Видалення першого елемента

1. Вихідний стан:

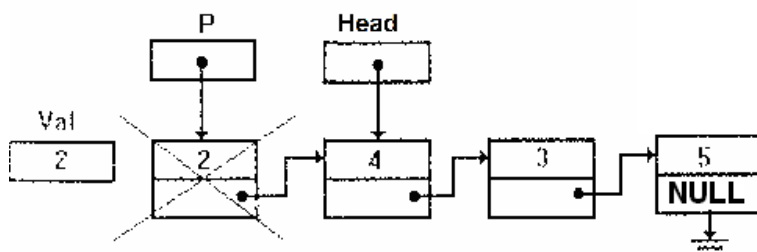


2. Встановлення додаткового вказівника **p** на елемент, який видаляють, і вибирання з нього інформації:



```
List *p=Head;  
Val=p->Inf;
```

3. Перестановка вказівника на голову списку на наступний елемент, звільнення пам'яті першого елемента списку:



```
Head=p->Link;  
//або  
//Head=Head->Link; delete  
p;
```

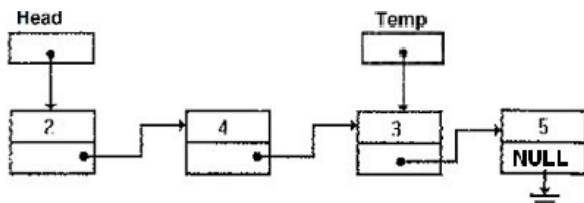
Рис.11. Видалення першого елемента списку

Функцію, що реалізує видалення першого елемента списку, наведено нижче.

```
void delFirst(List* &Head)  
{  
    //зберігаємо адресу елемента,  
    //який потрібно видалити  
    List* p = Head;  
    //отримуємо з нього інформацію  
    typeelem Val=p->Inf;  
    //встановлюємо голову списку  
    //на наступний елемент  
    Head=p->Link;  
    //або Head=Head->Link;  
    //видаляємо перший елемент  
    delete p;  
}
```

Видалення останнього елемента списку

Для видалення останнього елемента списку необхідно знати адресу передостаннього елемента для збереження в його посилальній частині ознаки кінця списку **NULL**. Такий пошук аналогічний до пошуку останнього елемента, змінюється тільки умова в заголовку циклу (рис.12):

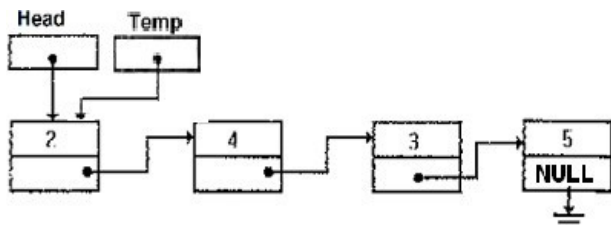


```
List * Temp=Head;
while (Temp->Link->Link!=NULL)
Temp=Temp->Link;
```

Рис.12. Знаходження передостаннього елемента списку

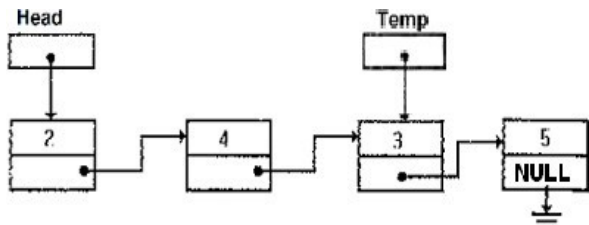
Виконуємо цикл, поки не дійдемо до елемента, для якого посилальне поле наступного за ним елемента дорівнює **NULL**.

1. Вихідний стан:



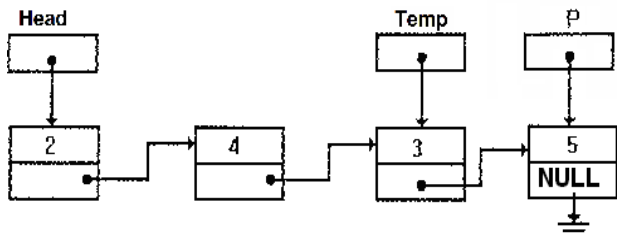
```
List * Temp=Head;
```

2. Знаходження адреси передостаннього елемента:



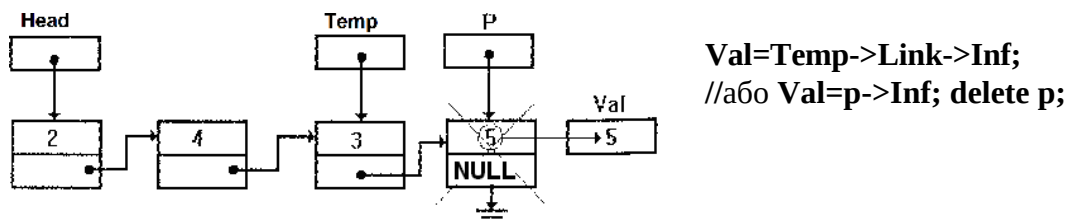
```
while (Temp->Link->Link!=NULL)
Temp=Temp->Link;
```

3. Занесення адреси останнього елемента у вказівник P:



```
List *p=Temp->Link;
```

4. Вибірання інформації з останнього елемента й звільнення пам'яті:



5. Фіксація кінця списку (встановлення посилального поля останнього елемента в NULL):

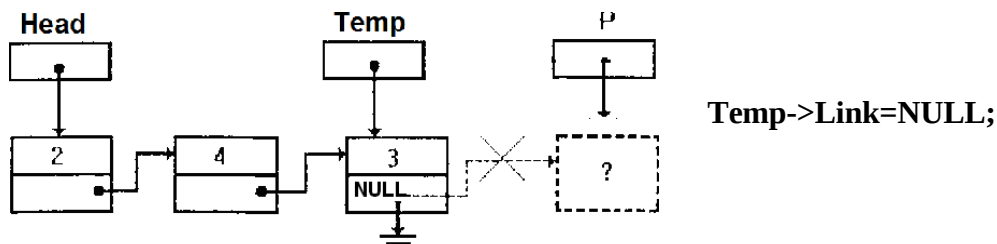


Рис.13. Видалення останнього елемента списку

Функцію, що реалізує видалення останнього елемента списку, описано нижче.

```
void delLast(List* Head)
{
//знаходження передостаннього елемента списку
List * Temp=Head;
while (Temp->Link->Link!=NULL) Temp=Temp-
>Link;
//збереження адреси останнього елемента
List p=Temp->Link;
//вибирання з нього інформації
typeelem Val=Temp->Link->Inf; //або Val=p->Inf;
//видалення останнього елемента
delete p;
//збереження ознаки кінця списку
Temp->Link=NULL;
}
```

Видалення елемента, що стоїть після заданого

Щоб видалити елемент зі списку, насамперед необхідно вирішити питання про те, як варто його задавати, оскільки це можна зробити різними способами. Наприклад, елемент, який видаляємо, можна задати його порядковим номером у списку, але тоді неможливий безпосередній доступ до нього і для його пошуку треба послідовно перебирати ланки списку, починаючи із заголовної. Набагато зручніше задати елемент, що видаляємо, за допомогою посилання на ту ланку, за якою він іде. До того ж і на практиці найбільш типовий випадок, коли треба видалити елемент, що йде за елементом, знайденим за допомогою розглянутої вище функції пошуку.

Ідею видалення ланки списку можна проілюструвати в такий спосіб. Нехай є список, із якого потрібно видалити число, що іде за 4 (рис.14).

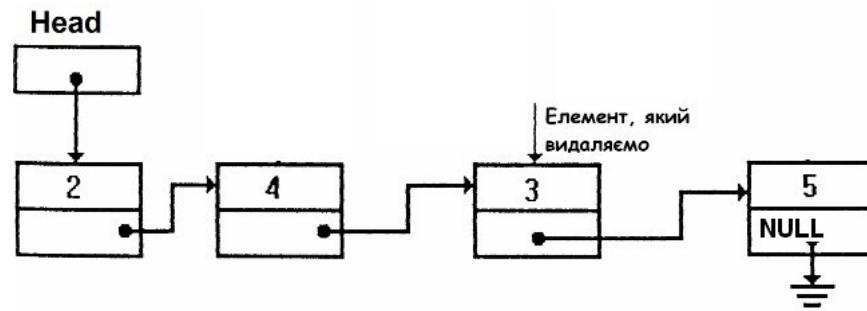


Рис.14. Список перед видаленням елемента

З огляду на зв'язок ланок за допомогою посилань ланку з числом 3 буде вилучено зі списку, якщо ланка з числом 4 буде посилатися на ланку з числом 5, оминаючи ланку з числом 3 (рис.15):

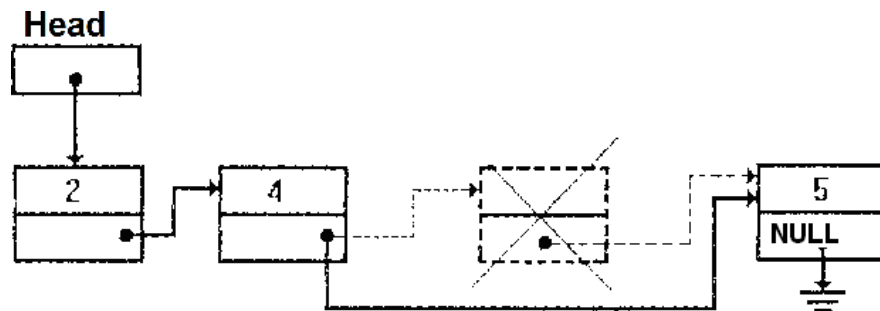
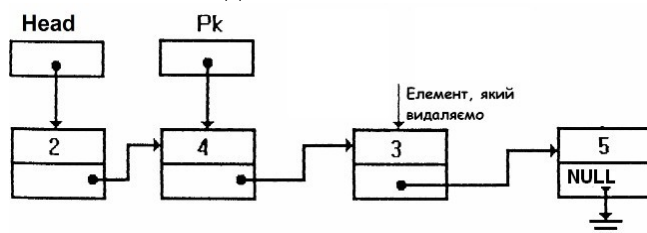


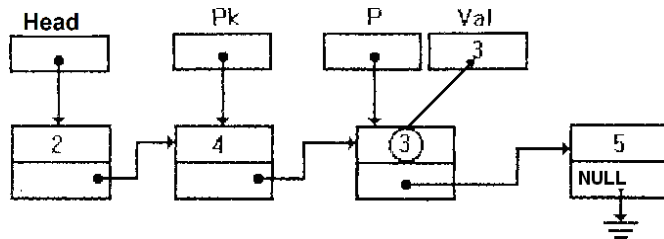
Рис.15. Зміна посилання на елемент, що видаляється

Отже, для видалення елемента зі списку достатньо змінити посилання попереднього йому елемента, причому як нове посилання цього елемента треба прийняти посилання елемента, який видаляємо. Варто звернути увагу на те, що в результаті виконання даної операції виключена зі списку ланка продовжує існувати й займати місце в пам'яті комп'ютера, хоча й стає недоступною для використання. Як бачимо, такий спосіб може призвести до неефективного використання пам'яті через зберігання в ній виключених елементів списку. Для усунення цього недоліку в описі функції видалення потрібно обов'язково передбачити знищення виключеної зі списку ланки (рис.16).

1. Вихідний стан:

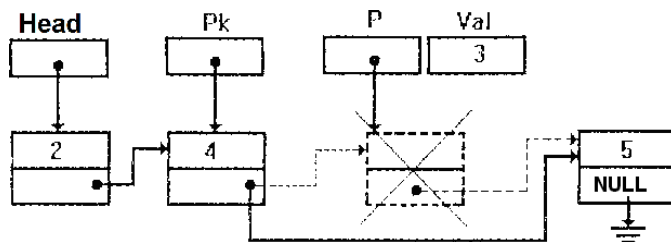


2. Встановлення додаткового вказівника **p** на елемент списку, який видаляємо, і вибирання з нього інформації:



```
List *p=Pk->Link; typeelem  
Val=p->Inf;
```

3. Встановлення зв'язку між **k**-м і **(k+2)**-м елементами та звільнення пам'яті **(k+1)**-го елемента, який видаляють:



```
Pk->Link=p->Link;  
//або  
//Pk->Link=Pk->Link->Link; delete  
p;
```

Рис.16. Видалення елемента списку після заданого

Функцію, що реалізує видалення елемента, описано нижче.

який стоїть після заданого,

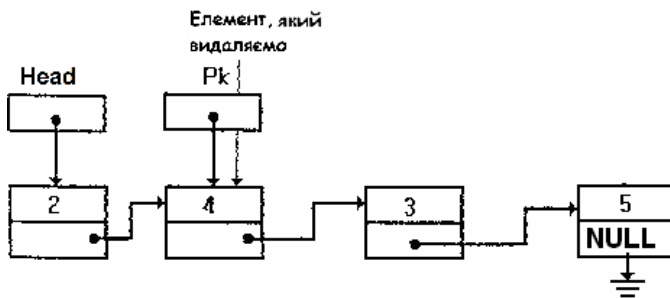
```
void delNode(List* Pk)
{
//збереження посилання на елемент,
//який видаляємо
List *p=Pk->Link;
//збереження інформації
typeelem Val=Pk->Inf;
//змінюємо посилання, виключаючи
//елемент зі списку
Pk->Link=Pk->Link->Link;
//або Pk->Link=p->Link;
//звільняємо пам'ять
delete p;
}
```

Видалення **k**-го елемента списку

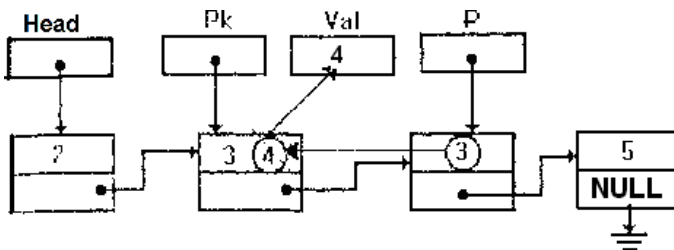
Видалення **k**-го елемента можна здійснити також, як описано вище, якщо відома адреса **(k-1)**-го елемента списку. Для цього можна застосувати вже наведену функцію **findPred**. Однак можна обійтися й без додаткового пошуку, для чого потрібно:

- 1) скопіювати значення інформаційного поля "потрібного" **(k+1)**-го елемента в інформаційне поле **k**-го елемента, який видаляємо;
- 2) видалити **(k+1)**-й елемент замість **k**-го (рис.17).

1. Вихідний стан:

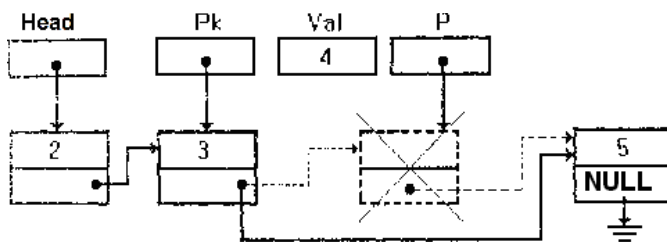


2. Встановлення додаткового вказівника **p** на $(k+1)$ -й елемент списку, вибирання інформації із k -го елемента, який видаляємо, і копіювання корисної інформації з $(k+1)$ -го елемента в k -й елемент:



```
List *p=Pk->Link; typeelem
Val=Pk->Inf; Pk->Inf=p->Inf;
```

3. Встановлення зв'язку між k -м і $(k+2)$ -м елементами й звільнення пам'яті $(k+1)$ -го елемента, який видаляємо замість k -го:



```
Pk->Link=p->Link;
//або
//Pk->Link=Pk->Link->Link; delete p;
```

Рис.17. Видалення k -го елемента списку

Функцію, що реалізує видалення k -го елемента, описано нижче.

```
void delNodeK(List* Pk)
{
    //збереження посилання на елемент,
    //який видаляємо
    List *p=Pk->Link;
    //збереження інформації
    typeelem Val=Pk->Inf;
    //обмін елементів місцями
    Pk->Inf=p->Inf;
    //змінюємо посилання, виключаючи
    //елемент зі списку
    Pk->Link=Pk->Link->Link;
    //або Pk->Link=p->Link;
    //звільняємо пам'ять
    delete p;
}
```

Функція видалення списку має такий вигляд:

```
void dropList1(List * & Head)  
{  
  List * p;  
  //поки список непорожній  
  while(Head)  
  {  
    //збереження посилання на перший елемент,  
    //який видаляємо  
    p = Head;  
    //встановлення вказівника голови списку  
    //на наступний елемент списку  
    Head = p->Link; //або Head = Head ->Link;  
    //видалення першого елемента  
    delete(p);  
  }  
}
```

або

```
void dropList2(List * & Head)  
{  
  List * p;  
  while(Head)  
  {  
    p = Head->Link;  
    delete(Head); Head = p;  
  }  
}
```

Запитання для контролю знань:

1. Яка головна особливість зв'язаного списку порівняно з масивом щодо розміщення в пам'яті?
2. З яких двох основних частин складається типовий вузол (node) однозв'язного списку?
3. Яка часова складність операції вставки нового елемента в початок зв'язаного списку?
4. Яку головну перевагу має двозв'язний список над однозв'язним?
5. Що зазвичай містить поле вказівника останнього вузла в лінійному однозв'язному списку?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Тема 10.2. Стеки, черги, деки.

Навчальна мета: формування цілісного розуміння фундаментальних структур даних лінійного типу, таких як стеки, черги та деки, через вивчення їхньої логічної архітектури та принципів взаємодії з елементами. Студенти мають засвоїти специфіку доступу до даних у кожній структурі, зокрема механізми LIFO та FIFO, а також опанувати практичні навички реалізації базових операцій додавання, видалення та перегляду елементів. Особлива увага приділяється аналізу ефективності використання цих структур у програмуванні для оптимізації алгоритмів обробки інформації та вирішення прикладних задач, де порядок надходження даних має критичне значення.

Розвиваюча мета: полягає у стимулюванні аналітичного мислення та здатності до абстрагування шляхом перенесення реальних процесів упорядкування об'єктів у площину алгоритмічного моделювання. Вивчення стеків, черг та деків сприяє розвитку логічної пам'яті та системного підходу до вирішення комплексних проблем, де важливо передбачати наслідки обраної стратегії керування потоками даних. Окрім цього, робота з цими структурами формує вміння оцінювати ресурсомісткість програмних рішень, розвиває когнітивну гнучкість при виборі оптимального інструменту для конкретної ситуації та закладає фундамент для майбутнього проектування складних інтелектуальних систем.

Виховна мета: спрямована на формування професійної відповідальності та культури програмування через розуміння важливості суворої впорядкованості та дисципліни при роботі з інформаційними потоками. Процес опанування структур даних виховує в учнів наполегливість у пошуку найбільш елегантних рішень та уважність до деталей, що безпосередньо впливає на надійність майбутніх програмних продуктів. Крім того, вивчення принципів взаємодії елементів у чергах та стеках сприяє розвитку почуття етики цифрової взаємодії, усвідомленню значущості послідовності дій та вихованню поваги до інтелектуальної праці як основи створення ефективних технологічних рішень.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання
- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

Стек (Stack) як модель LIFO

Стек — це абстрактна структура даних, що працює за принципом «останнім прийшов — першим пішов» (Last-In, First-Out). Уявним аналогом стека є стопка тарілок або магазин пістолета: ви можете покласти новий об'єкт лише зверху і забрати теж тільки той об'єкт, який було покладено останнім.

У мові C++ стек можна реалізувати власноруч за допомогою масивів або зв'язних списків, проте стандартна бібліотека (STL) пропонує готовий контейнер-адаптер `std::stack`. Основні операції включають `push` для додавання елемента на вершину, `pop` для видалення верхнього елемента, `top` для отримання значення вершини без її видалення, та `empty` для перевірки наявності даних.

Важливо розуміти, що стек не дозволяє довільний доступ до елементів. Ви не можете звернутися до другого чи третього елемента, не видаливши всі ті, що знаходяться над ними. Це робить стек ідеальним інструментом для задач рекурсії, обробки вкладених дужок у виразах або реалізації функції «скасувати дію» (Undo) у програмному забезпеченні.

```
#include <iostream>
#include <stack>

int main() {
    std::stack<int> s;
    s.push(10); // Додаємо 10
    s.push(20); // Додаємо 20 (тепер це вершина)

    std::cout << "Вершина стека: " << s.top() << std::endl; // Виведе 20
    s.pop(); // Видаляємо 20
    std::cout << "Нова вершина: " << s.top() << std::endl; // Виведе 10

    return 0;
}
```

Черга (Queue) як модель FIFO

На відміну від стека, черга базується на принципі «першим прийшов — першим пішов» (First-In, First-Out). Це структура з двома кінцями: «хвостом» (back), куди додаються нові елементи, та «головою» (front), звідки елементи вилучаються. Це ідентично черзі в магазині: той, хто прийшов раніше, обслуговується першим.

У C++ STL черга представлена адаптером `std::queue`. Вона використовує методи `push` для додавання в кінець, `pop` для видалення з початку, а також `front` та `back` для доступу до крайніх елементів. Реалізація черги на базі звичайного масиву потребує особливої уваги, оскільки після видалення елементів з початку виникає порожній простір, що призводить до необхідності використання «кільцевого буфера».

Черги є незамінними в операційних системах для керування чергою завдань на друк, планування процесів процесора або буферизації мережевих пакетів, де важливо зберігати хронологічний порядок надходження запитів.

```
#include <iostream>
#include <queue>

int main() {
    std::queue<std::string> q;
    q.push("Перший");
    q.push("Другий");
}
```

```

std::cout << "Перший у черзі: " << q.front() << std::endl; // Перший
q.pop(); // Видаляємо "Перший"
std::cout << "Тепер перший: " << q.front() << std::endl; // Другий

return 0;
}

```

Дек (Deque) — двостороння черга

Дек (Double-Ended Queue) є більш гнучкою структурою, яка об'єднує властивості і стека, і черги. Вона дозволяє додавати та видаляти елементи з обох кінців — як з початку, так і з кінця. Це робить дек універсальним інструментом для складних алгоритмів.

Контейнер `std::deque` у C++ забезпечує швидкий доступ не лише до країв, а й до довільних елементів за індексом (оператор `[]`), що наближає його до векторів. Проте, на відміну від вектора, дек не гарантує зберігання всіх елементів у безперервному блоці пам'яті; він зазвичай складається з набору окремих блоків. Це дозволяє ефективно нарощувати структуру в обох напрямках без повного копіювання даних при переповненні. Використання деку виправдане в задачах, де потік даних може змінювати пріоритет або вимагає перевірки з обох сторін, наприклад, при пошуку паліндромів або в алгоритмах ковзного вікна для обробки сигналів.

```

#include <iostream>
#include <deque>

int main() {
    std::deque<int> d;
    d.push_back(10); // Додаємо в кінець: [10]
    d.push_front(5); // Додаємо в початок: [5, 10]
    d.push_back(15); // Додаємо в кінець: [5, 10, 15]

    std::cout << "Елемент за індексом 1: " << d[1] << std::endl; // 10
    d.pop_front(); // Видаляємо з початку: [10, 15]

    return 0;
}

```

Запитання для контролю знань:

1. Який основний принцип роботи структури даних "стек" і як він розшифровується?
2. Чому для реалізації черги на базі масиву часто використовують модель "кільцевого буфера"?
3. Чи підтримує `std::stack` доступ до елементів за індексом? Обґрунтуйте відповідь.

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

Тема 10.3. Дерева

Навчальна мета: полягає у формуванні в учнів цілісного розуміння ієрархічних структур даних як ефективного способу організації інформації. Заняття спрямоване на засвоєння фундаментальних понять, таких як корінь, вузли, листки та рівні дерева, а також на вивчення властивостей бінарних дерев пошуку. У процесі навчання особлива увага приділяється опануванню алгоритмів обходу дерев, розумінню принципів рекурсивного доступу до даних та усвідомленню переваг використання деревних структур для оптимізації операцій пошуку, вставки та видалення елементів у порівнянні з лінійними структурами. Зрештою, це має підготувати студентів до самостійної реалізації дерев у програмуванні та їхнього застосування для розв'язання складних прикладних задач.

Розвиваюча мета: спрямована на активізацію аналітичного мислення учнів через виявлення логічних зв'язків у складних ієрархічних системах. Вона передбачає розвиток здатності до абстрагування, що дозволяє бачити структуру дерева у повсякденних явищах, від генеалогії до файлових систем комп'ютера. Особливий акцент ставиться на вдосконаленні навичок алгоритмічного підходу до розв'язання задач, де рекурсія стає інструментом для декомпозиції великої проблеми на менші, аналогічні частини. Також заняття сприяє розвитку когнітивної гнучкості, привчаючи студентів оцінювати ефективність різних шляхів обробки даних та прогнозувати результати виконання операцій у динамічних структурах.

Виховна мета: зосереджена на формуванні системного погляду на світ та розумінні важливості порядку й ієрархії в будь-якій структурованій діяльності. Вона сприяє вихованню культури програмування та інформаційної охайності, привчаючи студентів до усвідомленого вибору найбільш раціональних шляхів розв'язання задач замість хаотичного пошуку варіантів. Через роботу з розгалуженими структурами у студентів плекається інтелектуальна наполегливість, уважність до деталей та відповідальність за архітектурну чистоту створеного коду. Крім того, вивчення дерев дозволяє провести паралелі з природними та соціальними процесами, що виховує повагу до логіки побудови навколишнього світу та стимулює розвиток етичного ставлення до використання інтелектуальних ресурсів.

Тип: заняття повідомлення нових знань

Структура заняття:

- Організаційний момент (перевірка відвідування та готовності до заняття)
- Повідомлення теми, мети і завдань заняття
- Мотивація навчальної діяльності студентів (практичне застосування знань з теми, міжпредметні зв'язки)
- Виклад нового матеріалу
- Підведення підсумків (узагальнення, систематизація)
- Контрольні питання

- Пояснення домашнього завдання, рекомендація додаткових матеріалів

Теоретичний матеріал:

Ієрархічні структури даних: Дерева в C++

На відміну від лінійних структур, таких як стеки чи черги, дерева представляють ієрархічний спосіб організації даних. Це дозволяє моделювати реальні взаємозв'язки, такі як файлові системи, організаційні структури або логічні розгалуження в алгоритмах штучного інтелекту.

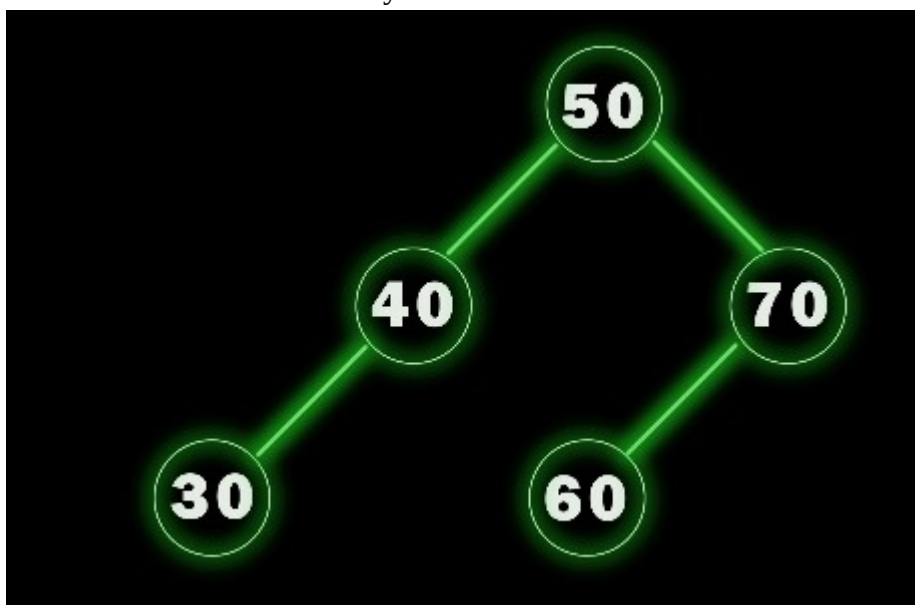
Концепція та термінологія дерев

Дерево — це сукупність вузлів, з'єднаних ребрами, де один вузол є головним і називається «коренем» (root). Кожен вузол може мати «дітей» (похідні вузли), а сам бути «батьком» для них. Вузли, які не мають дітей, називаються «листками».

Найбільш вживаним типом у програмуванні є **бінарне дерево**, де кожен вузол може мати не більше двох нащадків (лівого та правого). Особливе місце посідає **бінарне дерево пошуку (BST)**. Його головна властивість: для будь-якого вузла всі значення у лівому піддереві менші за значення самого вузла, а в правому — більші. Це забезпечує надзвичайно швидкий пошук, додавання та видалення даних — у середньому за логарифмічний час $O(\log n)$.

Програмна реалізація вузла дерева

У мові C++ дерево будується на основі структур або класів, що використовують покажчики для зв'язку між елементами. Кожен вузол містить корисне навантаження (дані) та два покажчики на об'єкти того ж типу



Приклад опису вузла:

```
struct Node {  
    int data;          // Значення вузла  
    Node* left;        // Покажчик на лівого нащадка  
    Node* right;       // Покажчик на правого нащадка  
  
    // Конструктор для зручного створення вузла  
    Node(int value) : data(value), left(nullptr), right(nullptr) {}  
};
```

Операції в бінарних деревах пошуку

Основними операціями є вставка нового елемента та обхід дерева. При вставці ми порівнюємо нове значення з поточним вузлом: якщо воно менше — переходимо ліворуч, якщо більше — праворуч. Цей процес повторюється рекурсивно, доки не знайдемо вільне місце (nullptr).

Приклад функції вставки:

```
Node* insert(Node* root, int value) {  
    // Якщо піддерево порожнє, створюємо новий вузол  
    if (root == nullptr) {  
        return new Node(value);  
    }  
    // Рекурсивний спуск у потрібну сторону  
    if (value < root->data) {  
        root->left = insert(root->left, value);  
    } else {  
        root->right = insert(root->right, value);  
    }  
    return root;  
}
```

Методи обходу дерева

Оскільки дерево не є лінійним, існує кілька способів відвідати всі його вузли:

1. **In-order (Центрований):** Ліве піддерево -> Вузол -> Праве піддерево. Для BST цей обхід повертає всі елементи у відсортованому порядку.
2. **Pre-order (Прямий):** Вузол -> Ліве піддерево -> Праве піддерево. Використовується для копіювання дерев.
3. **Post-order (Зворотний):** Ліве піддерево -> Праве піддерево -> Вузол. Зручний для видалення дерева з пам'яті.

Приклад In-order обходу:

```
void printInOrder(Node* root) {  
    if (root != nullptr) {  
        printInOrder(root->left);    // Рекурсія ліворуч  
        std::cout << root->data << " "; // Обробка кореня  
        printInOrder(root->right);   // Рекурсія праворуч  
    }  
}
```

Переваги та використання

Дерева дозволяють ефективно керувати великими обсягами даних. Наприклад, асоціативні контейнери в C++ STL, такі як std::set та std::map, зазвичай реалізовані на основі збалансованих бінарних дерев (червоно-чорних дерев). Це гарантує, що навіть при великій кількості елементів операції пошуку залишатимуться стабільно швидкими, що неможливо забезпечити у звичайних масивах без попереднього сортування.

Запитання для контролю знань:

1. Чим бінарне дерево відрізняється від довільного дерева?
2. Сформулюйте основну властивість бінарного дерева пошуку (BST).
3. Яка часова складність пошуку в збалансованому бінарному дереві?
4. Опишіть призначення покажчиків у структурі вузла дерева.
5. Який тип обходу (traversal) дозволяє отримати відсортовану послідовність елементів BST?

Література:

1. Васильєв О. М. Програмування на C++ в прикладах і задачах : Навчальний посібник / О. М. Васильєв. — Київ : Видавництво Ліра-К, 2017. — 382 с.
2. Грицюк Ю., Рак Т. Програмування мовою C++: Навчальний посібник / Грицюк Ю., Рак Т. — Львів : Видавництво ЛДУ БЖД, 2011. — 292 с.

ІНСТРУКТИВНО-МЕТОДИЧНІ МАТЕРІАЛИ ДО ВИКОНАННЯ ЛАБОРАТОРНИХ ТА ПРАКТИЧНИХ ЗАНЯТЬ

Тема роботи 1.2 : Компілятор, інтерпретатор, компоувальник. Структура і способи опису мов програмування високого рівня.

Мета роботи: вивчити етапи обробки коду в C++, зрозуміти різницю між типами трансляторів та опанувати базову структуру програми на мові високого рівня.

Порядок виконання роботи:

- Ознайомтеся з лекційним матеріалом.
- Запустіть середовище розробки (Visual Studio, Code::Blocks, CLion або OnlineGDB).
- Створіть проєкт та файл main.cpp.
- Реалізуйте програму відповідно до вашого варіанту.
- Дайте відповіді на контрольні запитання у звіті.
- Здати завдання(звіт та посилання на програмний проєкт) через віртуальний клас Google Classroom.

Результат виконання роботи: репозиторій з вихідним кодом або посилання на проєкт, звіт з відповідями на контрольні запитання.

Варіанти завдань для виконання практичної роботи:

1. Сума двох чисел: Запитати числа a та b , вивести їх суму ($a + b$).
2. Різниця двох чисел: Запитати числа a та b , вивести результат віднімання ($a - b$).
3. Добуток двох чисел: Запитати числа a та b , вивести результат множення ($a * b$).
4. Подвоєння: Запитати число x , вивести його значення, помножене на 2.
5. Наступне число: Запитати ціле число n , вивести число, яке йде за ним ($n + 1$).
6. Попереднє число: Запитати ціле число n , вивести число, яке стоїть перед ним ($n - 1$).
7. Квадрат числа: Запитати число a , вивести його квадрат ($a * a$).
8. Периметр квадрата: Запитати довжину сторони a , вивести периметр ($4 * a$).
9. Площа прямокутника: Запитати сторони a та b , вивести площу ($a * b$).
10. Потрійна сума: Запитати три числа, вивести їх загальну суму ($a + b + c$).
11. Вартість цукерок: Запитати ціну за 1 кг та кількість кілограмів, вивести загальну суму.
12. Обчислення залишку: Запитати кількість грошей у користувача та ціну

квитка, вивести скільки залишиться після покупки.

13. Різниця віку: Запитати рік народження користувача та поточний рік, вивести кількість років.
14. Поділ навпіл: Запитати парне число, вивести результат його ділення на 2.
15. Трикутник: Запитати сторону рівностороннього трикутника, вивести його периметр ($3 * a$).
16. Збільшення на 10: Запитати будь-яке число, вивести його значення, збільшене на 10.
17. Зменшення на 5: Запитати будь-яке число, вивести його значення, зменшене на 5.
18. Множення на нуль: Запитати число, вивести результат його множення на 0 (для перевірки логіки).
19. Кілометри в метри: Запитати кількість кілометрів, вивести скільки це в метрах ($km * 1000$).
20. Метри в сантиметри: Запитати кількість метрів, вивести скільки це в сантиметрах ($m * 100$).
21. Сума однакових чисел: Запитати число a , вивести результат додавання $a + a$.
22. Крок назад: Запитати число, вивести результат віднімання від нього числа 2.
23. П'ять копій: Запитати число, вивести його значення, помножене на 5.
24. Склянки води: Запитати кількість людей, вивести скільки склянок води потрібно (по 2 на кожного).
25. Дві дії: Запитати число, додати до нього 1, а потім результат помножити на 2.

Контрольні запитання для звіту:

- Яка роль компонувальника (linker) у створенні програми?
- Чим відрізняється об'єктний файл від виконуваного файлу?
- Що робить директива `#include <iostream>`?
- Чому C++ вважається мовою високого рівня?
- Який етап обробки коду відповідає за перевірку синтаксичних помилок?

Приклад виконання роботи:

```
#include <iostream>
#include <cmath>
```

```
/**
```

```
 * Практична робота №1
```

```
 * Тема: Структура програми C++
```

```
 * Завдання: Обчислення площі прямокутника
```

```

*/

int main() {
    // Оголошення змінних для сторін прямокутника
    double sideA, sideB;
    double area;

    // Запит даних у користувача
    std::cout << "--- Prohrama dlya obchyslennya ploshchi pryamokutnyka
---" << std::endl;

    std::cout << "Vvedit storonu A: ";
    std::cin >> sideA;

    std::cout << "Vvedit storonu B: ";
    std::cin >> sideB;

    // Перевірка на коректність введення
    if (sideA <= 0 || sideB <= 0) {
        std::cout << "Pomylka: storony mayut' buty dodatnymi!" <<
std::endl;
    } else {
        // Обчислення площі за формулою  $S = a * b$ 
        area = sideA * sideB;

        // Виведення результату
        std::cout << "Ploshcha pryamokutnyka: " << area << std::endl;
    }

    return 0;
}

```

Тема роботи 2.1 : Вступ в програмування. Найпростіші програми.

Мета роботи: Ознайомитися з середовищем програмування, структурою програми на мові C++, навчитися використовувати стандартні потоки введення-виведення (cin, cout) та виконувати прості арифметичні розрахунки.

Порядок виконання роботи:

- Ознайомитися з теоретичними відомостями та структурою мови C++.
- Запустити середовище розробки (наприклад, Visual Studio, Code::Blocks або онлайн-компілятор).
- Створити новий проект або файл із розширенням .cpp.
- Набрати та протестувати приклад програми.
- Отримати у викладача номер варіанта (за списком групи).
- Розробити алгоритм розв'язання задачі згідно з варіантом.
- Написати програмний код, скомпілювати його та виправити можливі помилки.
- Оформити звіт, який має містити: титульну сторінку, мету, текст програми (код), скріншот результату виконання та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

1. Квадрат: Ввести сторону a . Обчислити периметр $P = 4a$.
2. Прямокутник: Ввести сторони a та b . Обчислити площу $S = ab$.
3. Коло: Ввести радіус r . Обчислити довжину $L = 6.28 * r$.
4. Куб: Ввести ребро куба a . Обчислити об'єм $V = a * a * a$.
5. Числа: Ввести три числа a , b , c . Знайти їх суму $S = a + b + c$.
6. Валюта: Ввести суму в доларах. Перевести в гривні (курс 41.5).
7. Шлях: Ввести швидкість v та час t . Обчислити відстань $S = v * t$.
8. Круг: Ввести радіус r . Обчислити площу $S = 3.14 * r * r$.
9. Закон Ома: Ввести напругу U та опір R . Знайти силу струму $I = U / R$.
10. Маса: Ввести густину ρ та об'єм V . Знайти масу $m = \rho * V$.
11. Сила: Ввести масу m та прискорення a . Знайти силу $F = m * a$.
12. Ціна: Ввести ціну товару та кількість. Обчислити загальну вартість.
13. Зріст: Ввести зріст у метрах. Перевести його у сантиметри.
14. Температура: Ввести градуси Цельсія C . Знайти Кельвіни $K = C + 273$.
15. Трикутник: Ввести основу a та висоту h . Знайти площу $S = 0.5 * a * h$.
16. Гіпотенуза: Ввести катети a та b . Знайти суму їх квадратів $c^2 = a^2 + b^2$.
17. Відсоток: Ввести число x . Знайти 10% від цього числа ($0.1 * x$).
18. Периметр: Ввести три сторони трикутника a , b , c . Знайти $P = a + b + c$.
19. Знижка: Ввести ціну. Обчислити ціну зі знижкою 5% (помножити на 0.95).
20. Бензин: Ввести відстань (км). Знайти розхід при нормі 8 л на 100 км.
21. Час: Ввести кількість хвилин. Перевести їх у секунди.
22. Середнє: Ввести два числа x та y . Знайти їх середнє арифметичне.
23. Тиск: Ввести силу F та площу S . Обчислити тиск $P = F / S$.
24. Вік: Ввести рік народження та поточний рік. Обчислити кількість років.

25. Стіна: Ввести висоту та ширину стіни. Знайти її площу.

Контрольні запитання для звіту:

- Яка бібліотека відповідає за стандартне введення-виведення в C++?
- Опишіть призначення об'єктів cin та cout.
- Що означає оператор >> та в яких випадках він використовується?
- Навіщо потрібна директива #include?
- Яку роль відіграє функція main() у програмі?
- Як вивести повідомлення у новий рядок консолі?

Приклад виконання роботи:

```
#include <iostream>
using namespace std;

int main() {
    // Встановлення кодування для коректного відображення кирилиці
    setlocale(LC_ALL, "Ukrainian");

    double a, b, sum, average;

    cout << "Введіть перше число: ";
    cin >> a;
    cout << "Введіть друге число: ";
    cin >> b;

    sum = a + b;
    average = sum / 2;

    cout << "Сума: " << sum << endl;
    cout << "Середнє арифметичне: " << average << endl;

    return 0;
}
```


Тема роботи 2.2 : Типи даних і змінні. Знаходження суми двох чисел (ввід і вивід).

Мета роботи:

- Ознайомитися з базовими типами даних мови C++ (int, float, double).
- Навчитися оголошувати змінні та присвоювати їм значення.
- Освоїти роботу з потоками стандартного вводу (cin) та виводу (cout).
- Навчитися складати лінійні алгоритми для розв'язання арифметичних задач.

Порядок виконання роботи:

- Вивчити правила іменування змінних та особливості цілочисельних і дійсних типів даних.
- Запустити середовище розробки (Visual Studio, Code::Blocks, CLion або онлайн-компілятор).
- Створити новий проект або .cpp файл.
- Згідно з номером варіанта (за журналом) написати програмний код для трьох завдань.
- Провести компіляцію та тестування програми.
- Оформити звіт, додавши код програми та скріншоти результатів виконання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

- А: Сума двох цілих чисел.
- Б: Добуток двох дійсних чисел.
- В: Периметр квадрата зі стороною a.

Варіант 2

- А: Різниця двох цілих чисел.
- Б: Середнє арифметичне двох дійсних чисел.
- В: Площа прямокутника зі сторонами a та b.

Варіант 3

- А: Добуток трьох цілих чисел.
- Б: Знайти $1/3$ від дійсного числа x.
- В: Довжина кола радіуса r ($L = 2 * 3.14 * r$).

Варіант 4

- А: Остача від ділення a на b.
- Б: Знайти 20% від суми дійсних чисел x + y.
- В: Периметр прямокутника ($P = 2(a+b)$).

Варіант 5

- А: Сума квадратів двох цілих чисел.
- Б: Частка двох дійсних чисел.
- В: Площа круга ($S = 3.14 * r^2$).

Варіант 6

- А: Знайти $x = (a + b)^2$.
- Б: Перевести кілограми у грами (дійсне число).
- В: Об'єм куба зі стороною a.

Варіант 7

- А: Різниця квадратів $a^2 - b^2$.
- Б: Середнє арифметичне трьох дійсних чисел.
- В: Площа прямокутного трикутника ($S = 0.5 a b$).

Варіант 8

- А: Знайти $y = 3a - b$.
- Б: Обчислити ціну N кг товару за заданою ціною.
- В: Периметр рівностороннього трикутника зі стороною a .

Варіант 9

- А: Добуток a і b с.
- Б: Знайти результат виразу $x/4 + 10.5$.
- В: Об'єм паралелепіпеда ($V = a b c$).

Варіант 10

- А: Сума трьох послідовних цілих чисел.
- Б: Знайти довжину гіпотенузи c (звід катетів a, b).
- В: Знайти вартість покупки зі знижкою 5%.

Варіант 11

- А: Знайти ціле середнє значення $(a+b)/2$.
- Б: Перевести метри в сантиметри.
- В: Площа поверхні куба ($6a^2$).

Варіант 12

- А: Остача від ділення цілого числа на 10.
- Б: Знайти значення виразу $2.5x + y$.
- В: Периметр трапеції (звід чотирьох сторін).

Варіант 13

- А: Збільшити ціле число на 15.
- Б: Перевести долари в гривні за заданим курсом.
- В: Відстань $S = v t$ (звід швидкості та часу).

Варіант 14

- А: Подвоєна сума трьох цілих чисел.
- Б: Знайти квадрат дійсного числа.
- В: Площа бічної поверхні циліндра ($2 \pi r h$).

Варіант 15

- А: Різниця $(a+b) - c$.
- Б: Знайти результат виразу $x - 0.5y$.
- В: Середнє геометричне двох дійсних чисел.

Варіант 16

- А: Квадрат суми двох цілих чисел $(a+b)^2$.
- Б: Знайти 15% від дійсного числа.
- В: Периметр ромба зі стороною a .

Варіант 17

- А: Сума цифр двоцифрового цілого числа.
- Б: Перевести температуру з Цельсія у Фаренгейти.
- В: Довжина діагоналі прямокутника за сторонами a, b .

Варіант 18

- А: Добуток цілих чисел $(a-1)(b+1)$.
- Б: Обчислити результат $x/y + z$.
- В: Площа рівнобічної трапеції за основами та висотою.

Варіант 19

- А: Частка a/b та остача $a \% b$ для цілих чисел.
- Б: Знайти суму трьох дійсних чисел.
- В: Сила струму $I = U/R$ (звід напруги та опору).

Варіант 20

- А: Знайти результат $2a + 3b$.
- Б: Знайти значення $x \cdot y / 2.0$.
- В: Об'єм циліндра ($V = 3.14 r^2 h$).

Варіант 21

- А: Різниця $100 - (a+b+c)$.
- Б: Знайти суму дробів $1/x + 1/y$.
- В: Шлях при рівноприскореному русі ($at^2/2$).

Варіант 22

- А: Сума першого та третього введених цілих чисел.
- Б: Перевести хвилини в години (дійсне число).
- В: Площа паралелограма ($S = a \cdot h$).

Варіант 23

- А: Знайти куб цілого числа a^3 .
- Б: Обчислити значення $x + y - 0.75$.
- В: Кінетична енергія ($mv^2/2$).

Варіант 24

- А: Знайти результат $(a \cdot b) + (c \cdot d)$.
- Б: Знайти результат виразу $x/2 + y/4$.
- В: Периметр паралелограма за двома сторонами.

Варіант 25

- А: Сума чотирьох цілих чисел.
- Б: Знайти суму $0.1x + 0.2y$.
- В: Площа кільця за двома радіусами ($3.14 (R^2 - r^2)$).

Контрольні запитання для звіту:

- Яке призначення директиви `#include <iostream>`?
- Чим відрізняється тип `int` від типу `double`? Скільки байт вони займають у пам'яті?
- Які правила іменування змінних у C++ (що дозволено, а що заборонено)?
- Поясніть роботу операторів `<<` та `>>`.
- Що таке інкремент і декремент?

Приклад виконання роботи:

```
#include <iostream>

using namespace std;

int main() {
    // Оголошення змінних цілого типу
    int first_number, second_number, result;

    cout << "--- Program to find the sum of two numbers ---" << endl;

    // Введення даних
    cout << "Enter the first integer: ";
    cin >> first_number;

    cout << "Enter the second integer: ";
    cin >> second_number;

    // Арифметична операція
    result = first_number + second_number;
```

```
    // Виведення результату
    cout << "The sum of " << first_number << " and " << second_number << "
is: " << result << endl;

    return 0;
}
```

Тема роботи 2.3 : Арифметичні вирази. Формати для виводу даних.

Мета роботи:

- Ознайомитися з базовими типами даних для збереження чисел (int, float, double).
- Навчитися реалізовувати складні математичні формули за допомогою бібліотеки `<cmath>`.
- Опанувати маніпулятори виводу бібліотеки `<iomanip>` для налаштування точності та вигляду результатів.

Порядок виконання роботи:

- Аналіз завдання: Отримайте номер варіанту та розберіть математичні формули. Визначте, які змінні є вхідними, а які — результатом.
- Написання коду: Підключіть необхідні бібліотеки: `iostream` (введення-виведення), `cmath` (математика), `iomanip` (форматування).
 - Оголосіть змінні відповідних типів (рекомендується `double` для високої точності).
 - Реалізуйте введення даних користувачем за допомогою `cin`.
 - Обчисліть значення виразів, дотримуючись пріоритетів операцій.
- Форматування: Використовуйте `fixed` та `setprecision(n)` для виводу результату з заданою кількістю знаків після коми.
- Тестування: Запустіть програму та перевірте правильність обчислень на калькуляторі.
- Звіт: Підготуйте документ, що містить код програми, скріншот результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. $z = \sqrt{x^2 + \sin(y)}$
2. $k = |a - b| + \log(x)$
3. $f = e^{x+y} / (a + 1)$
4. $w = \cos(x^2) + \tan(y)$
5. $p = (x + y) \sqrt[3]{a}$

Варіант 2

1. $z = \sin(x) + \cos(y) - e^x$
2. $k = \sqrt[3]{a^2 + b^2}$
3. $f = \log_{10}(x + y)$
4. $w = (x \cdot y) / (a + b)$
5. $p = |x - \tan(y)|$

Варіант 3

1. $z = 1/\sqrt{x^2 + 5}$
2. $k = \sin^2(a) + \cos(b)$
3. $f = \exp(x - y)$
4. $w = \sqrt{|a \cdot b|}$
5. $p = \tan(x)/(y + 1)$

Варіант 4

1. $z = \sqrt{x \cdot \sin(a)}$
2. $k = \ln(x^2 + 1)$
3. $f = (a + b)/(x - y)$
4. $w = \cos^3(x)$
5. $p = e^{-x} + \sqrt{y}$

Варіант 5

1. $z = \log(a + b^2)$
2. $k = \sqrt{x^2 + y^2}$
3. $f = \sin(x + y)/\cos(x - y)$
4. $w = |a^2 - b^2|$
5. $p = 1/x + 1/y$

Варіант 6

1. $z = a^b - \cos(x)$
2. $k = \sqrt{x + \sqrt{}}$
3. $f = \sin(a)/(b + 1)$
4. $w = \ln |x - y|$
5. $p = e^{a+b}$

Варіант 7

1. $z = \sqrt[4]{x^2 + a^2}$
2. $k = \tan(x) + \sin(y)$
3. $f = (a + b)/(x \cdot y)$

4. $w = |x|^y$

5. $p = \cos(a - b)^2$

Варіант 8

1. $z = e^x \cdot \sin(y)$

2. $\sqrt{k} = a^2 + b^2/2$

3. $f = \log(x/y)$

4. $w = (x + y)/(a - b)$

5. $p = \sin(x)^2 + \cos(y)$

Варіант 9

1. $z = \sqrt{a \cdot b} + \cos(x)$

2. $k = |x^2 - y^2|$

3. $f = e^{-a} \cdot \tan(b)$

4. $w = \ln(x + 1)/y$

5. $p = (a + b)^x$

Варіант 10

1. $z = \sin(x/y)$

2. $k = \sqrt{a^2 + 1}$

3. $f = \exp(x)/\cos(y)$

4. $w = |a - b| \cdot \sin(x)$

5. $p = \log_{10}(x^2 + y^2)$

Варіант 11

1. $\sqrt{z} = x + y/a$

2. $k = \cos(a + b)^2$

3. $f = \ln |x| + e^y$

4. $w = \tan(x/a)$

5. $p = x^y + a$

Варіант 12

1. $z = |x| + \sin^2(a)$
2. $k = \sqrt{x^2 + y^2 + a^2}$
3. $f = e^{x \cdot y}$
4. $w = (a + b) / \ln(x)$
5. $p = \cos(x) / (y + 1)$

Варіант 13

1. $z = \log(\sqrt{x} + a)$
2. $k = \sin(x) \cdot \cos(y)$
3. $f = (a^2 + b^2)^x$
4. $w = \sqrt{|x - y|}$
5. $p = e^x / (a + b)$

Варіант 14

1. $z = \tan(x + a)$
2. $k = \ln(x^2) + y$
3. $\sqrt{f} = a + b/x$
4. $w = e^{-y} + \cos(x)$
5. $p = |a^3 - b^3|$

Варіант 15

1. $z = \sin(a) / b + \sqrt{x}$
2. $k = \log_{10}(x + y)$
3. $f = (x + y) \cdot e^a$
4. $w = \cos(x^2) - \sin(y^2)$
5. $p = \sqrt[3]{a \cdot b \cdot \dots}$

Контрольні запитання для звіту:

- Яке призначення бібліотеки <cmath> ? Назвіть 3-4 функції з цієї бібліотеки.
- Чим відрізняється операція / для цілих чисел (int) та для дійсних чисел (double)?
- Яка функція використовується для піднесення числа до степеня?
- Для чого потрібен маніпулятор fixed у поєднанні з setprecision ?
- Як обчислити корінь n-ного степеня, якщо в бібліотеці є лише квадратний корінь sqrt() ?

Приклад виконання роботи:

```
#include <iostream>
#include <cmath>      // Для sqrt, sin, cos
#include <iomanip> // Для setprecision

using namespace std;

int main() {
    double x, a, y, z;
    cout << "Введіть x: "; cin >> x;
    cout << "Введіть a: "; cin >> a;
    // Обчислення
    y = sqrt(pow(x, 2) + a);
    z = sin(x) / (cos(x) + 1);
    // Вивід результатів
    cout << "---- Результати ----" << endl;
    cout << "y = " << fixed << setprecision(3) << y << endl; cout << "z = " << fixed <<
    setprecision(3) << z << endl;

    return 0;
}
```

Тема роботи 3.1 :Алгоритмічний вибір альтернатив (Умовний оператор if — else. Множинний вибір switch).

Мета роботи:

- Ознайомитися з принципами розгалужених алгоритмів.
- Навчитися використовувати умовний оператор if для перевірки логічних умов.
- Опанувати оператор множинного вибору switch для обробки дискретних значень.
- Розвинути навички побудови логічних виразів за допомогою операторів порівняння та логічних операцій (&&, ||, !).

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо структури та синтаксису операторів if, else if, else та switch.
- Отримати номер варіанта від викладача.
- Розробити алгоритми для розв'язання 5-ти завдань свого варіанта.
- Написати програмний код мовою C++ у середовищі розробки (наприклад, Visual Studio, Code::Blocks або онлайн-компілятор).
- Протестувати програму на різних вхідних даних (враховуючи межові значення).
- Оформити звіт, що містить код програм та скриншоти результатів виконання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Дано два числа. Вивести найбільше з них.
2. Перевірити, чи є введене число парним та додатним.
3. Визначити, чи лежить точка $A(x, y)$ у першій чверті координат.
4. Ввести номер місяця, вивести назву пори року.
5. Реалізувати простий калькулятор (+, -, *, /) для двох чисел.

Варіант 2

1. Визначити, чи є трикутник зі сторонами a, b, c рівностороннім.
2. Дано три числа. Знайти суму двох найбільших.
3. Перевірити, чи ділиться число N на 3 та 5 одночасно.
4. Ввести оцінку (1-5), вивести її текстовий опис (незадовільно... відмінно).
5. За номером місяця вивести кількість днів у ньому (невисокосний рік).

Варіант 3

1. З'ясувати, чи є число X трицифровим.
2. Порівняти площі кола радіуса R та квадрата зі стороною A.
3. Знайти мінімальне серед чотирьох чисел.
4. За кодом країни (1 - Україна, 2 - Польща, 3 - Німеччина) вивести назву столиці.
5. Ввести номер пальця руки (1-5), вивести його назву.

Варіант 4

1. Дано вік людини. Вивести: "Дитина", "Підліток", "Дорослий" або "Літня людина".
2. Перевірити, чи є рік високосним.
3. Визначити, чи можна побудувати трикутник зі сторін a, b, c.
4. За номером дня тижня вивести графік роботи (н-р, 1-3: 09:00-18:00).
5. Ввести символ. Якщо це голосна буква (a, e, i, o, u) — повідомити про це.

Варіант 5

1. Дано число. Вивести його модуль без використання функції `abs()`.
2. Перевірити, чи закінчується число цифрою 7.
3. Визначити, чи є точка (x, y) всередині кола радіуса R з центром у (0,0).
4. Ввести номер квартири. Визначити під'їзд (9-поверховий будинок, 4 квартири на поверсі).
5. За кодом операції вивести результат логічного "І", "АБО" для двох бітів.

Варіант 6

1. Знайти корені квадратного рівняння $ax^2 + bx + c = 0$.
2. Дано три числа. Перевірити, чи є серед них хоча б одне від'ємне.
3. Визначити найбільше значення серед a^2 та b^3 .
4. Ввести номер пори року (1-4), вивести назви місяців, що до неї належать.
5. За першою літерою кольору (r, g, b, y) вивести повну назву англійською.

Варіант 7

1. Перевірити, чи є введене число кратним 10.
2. Дано координати двох точок. Яка з них ближче до початку координат?
3. Визначити, чи є чотирикутник зі сторонами a, b, c, d квадратом.
4. Ввести номер вагону. Визначити тип (1-5: СВ, 6-10: Купе, 11-18: Плацкарт).
5. Реалізувати меню вибору напою (1 - Кава, 2 - Чай, 3 - Сік).

Варіант 8

1. Дано ціле число. Додати до нього 1, якщо воно додатне, і відняти 2, якщо від'ємне.
2. Перевірити, чи є трикутник прямокутним за трьома сторонами.
3. Визначити, чи є введений символ цифрою.
4. За номером місяця вивести кількість днів до кінця кварталу.
5. Ввести номер планети від Сонця (1-8), вивести її назву.

Варіант 9

1. Порівняти два дроби $a/$ та c/d .
2. Дано три числа. Вивести їх у порядку зростання.
3. Визначити, чи потрапляє число x у діапазон [A, B].
4. За типом геометричної фігури (1-трикутник, 2-квадрат, 3-коло) обрати формулу площі.
5. Ввести оцінку в системі ECTS (A, B, C, D, E, F), вивести бал (5, 4, 3, 2).

Варіант 10

1. Перевірити, чи є сума цифр двоцифрового числа парною.
2. Дано температуру води. Визначити її стан (лід, рідина, пара).
3. З'ясувати, чи є число паліндромом (для трицифрових чисел).
4. За номером шкільного класу вивести ступінь навчання (молодша, середня, старша).
5. Ввести код валюти (USD, EUR, UAH), вивести назву.

Варіант 11

1. Дано радіус кулі та ребро куба. У кого об'єм більший?
2. Перевірити, чи є число X дільником числа Y .
3. Визначити, чи є точка (x, y) на осях координат.
4. За номером години (0-23) вивести привітання (Добрий ранок, день...).
5. Ввести назву геометричного тіла, вивести кількість його вершин (куб, піраміда).

Варіант 12

1. Дано вартість товару. Якщо вона > 1000 грн, надати знижку 5%.
2. Перевірити, чи є серед трьох чисел хоча б одна пара рівних.
3. Визначити чверть координатної площини для точки (x, y) .
4. За номером місяця вивести наступний за ним місяць.
5. Ввести номер ноти (1-7), вивести її назву (До, Ре...).

Варіант 13

1. Обчислити значення функції $y = \sqrt{x}$, якщо $x \geq 0$, інакше вивести помилку.
2. Порівняти периметри прямокутника та рівностороннього трикутника.
3. Перевірити, чи є число N чотирицифровим.
4. За номером поверху визначити, чи потрібна зупинка ліфта (н-р, тільки парні).
5. Ввести символ операції над множинами, вивести опис ($\cup$ - об'єднання).

Варіант 14

1. Дано два числа. Замінити менше число їх сумою, а більше — різницею.
2. Визначити, чи є символ великою латинською літерою.
3. Перевірити, чи є добуток трьох чисел додатним.
4. За зростом людини (в см) визначити розмір одягу (S, M, L, XL).
5. Ввести номер хімічного елемента (1-5), вивести назву (H, He, Li...).

Варіант 15

1. Визначити, чи є введений час (години, хвилини) коректним.
2. Дано три відрізки. Чи можна з них скласти рівнобедрений трикутник?
3. Перевірити, чи є число N степенем двійки (для $N < 100$).
4. За номером місця в поїзді визначити: верхнє чи нижнє (парні/непарні).
5. Ввести назву шахової фігури, вивести її "цінність" у балах.

Варіант 16

1. Порівняти ціну за 1 кг продукту у двох різних упаковках.
2. Перевірити, чи є сума двох чисел більшою за їх добуток.
3. Визначити, чи є трикутник тупокутним.
4. За номером радіус-вектора вивести значення кута (0, 90, 180, 270).
5. Ввести літеру латинського алфавіту, вивести її порядковий номер (A-E).

Варіант 17

1. Дано оцінку студента. Визначити, чи отримує він стипендію.
2. Перевірити, чи лежать три точки на одній прямій.
3. Визначити, чи є число "щасливим" (сума перших двох цифр дорівнює сумі останніх двох для 4-значного числа).
4. За кодом помилки HTTP (200, 404, 500) вивести опис.
5. Ввести номер пальця ноги, вивести назву.

Варіант 18

1. Дано швидкість у км/год та м/с. Яка швидкість більша?
2. Перевірити, чи є число кратним і 2, і 3, і 5.
3. Визначити, чи перетинаються два відрізки на прямій.
4. За номером століття вивести його приналежність до ери.
5. Ввести назву операційної системи, вивести розробника (Windows, macOS).

Варіант 19

1. Дано x та y . Обчислити $\max(x, y) / \min(x, y)$.
2. Перевірити, чи є введений текст словом "Hello" (ігноруючи регістр).
3. Визначити, чи є вказаний день (число, місяць) днем народження користувача.
4. За першою цифрою поштового індексу вивести область України.
5. Ввести номер ігрового рівня (1-3), вивести складність.

Варіант 20

1. Дано три числа. Знайти середнє за значенням (не $\min$ і не $\max$).
2. Перевірити, чи входить точка у заштриховану область (квадрат 2×2).
3. Визначити, чи є сума цифр трицифрового числа трицифровим числом.
4. За назвою тварини вивести клас (ссавці, птахи, рептилії).
5. Ввести номер сезону серіалу, вивести кількість серій у ньому.

Варіант 21

1. Дано x . Обчислити $f(x) = x^2$, якщо $x > 0$, та $f(x) = x + 5$, якщо $x \leq 0$.
2. Перевірити, чи є квадрат числа рівним кубу іншого числа.
3. Визначити, чи є рядок порожнім.
4. За кодом кольору світлофора вивести дію водія.
5. Ввести тип двигуна (бензин, дизель, електро), вивести податок.

Варіант 22

1. Дано ціну товару та кількість грошей. Чи вистачить на покупку?
2. Перевірити, чи є трикутник гострокутним.
3. Визначити, чи є число N простим (для $N < 20$).
4. За номером поверху вивести назву відділу в ТЦ.
5. Ввести розширення файлу (.cpp, .txt, .exe), вивести тип файлу.

Варіант 23

1. Дано масу тіла та зріст. Обчислити індекс маси тіла (ІМТ) та дати оцінку.
2. Перевірити, чи є число дзеркальним (наприклад, 121).
3. Визначити, чи є введений символ розділовим знаком.
4. За номером групи вивести спеціальність.
5. Ввести назву соцмережі, вивести рік заснування.

Варіант 24

1. Порівняти довжину кола та периметр трикутника.
2. Перевірити, чи є сума двох кутів трикутника меншою за 90 градусів.
3. Визначити, чи є вказаний рік роком Дракона за китайським календарем.
4. За номером квартири визначити парність сторони коридору.
5. Ввести назву місяця англійською, вивести переклад.

Варіант 25

1. Дано два інтервали $[a, b]$ та $[c, d]$. Знайти їх перетин.
2. Перевірити, чи є добуток цифр двоцифрового числа більшим за саме число.
3. Визначити, чи знаходиться точка всередині прямокутника.
4. За типом кузова авто вивести кількість дверей (седан, купе...).
5. Ввести код клавіші (W, A, S, D), вивести напрямок руху в грі.

Контрольні запитання для звіту:

- У чому полягає різниця між повною та неповною формою оператора `if`?
- Які логічні операції використовуються для об'єднання кількох умов у одному операторі `if`?
- Яка роль ключового слова `break` в операторі `switch`? Що станеться, якщо його пропустити?
- Для яких типів даних можна використовувати оператор `switch`?
- Що таке "тернарний оператор" і як він пов'язаний з темою розгалуження?

Приклад виконання роботи:

```
#include <iostream>
#include <string>

using namespace std;

int main() {
    int day;
    cout << "Введіть номер дня тижня (1-7): ";
    cin >> day;
```

```

// Використання switch для множинного вибору
switch (day) {
    case 1: cout << "Понеділок" << endl; break;
    case 2: cout << "Вівторок" << endl; break;
    case 3: cout << "Середа" << endl; break;
    case 4: cout << "Четвер" << endl; break;
    case 5: cout << "П'ятниця" << endl; break;
    case 6: cout << "Субота" << endl; break;
    case 7: cout << "Неділя" << endl; break;
    default:
        cout << "Помилка: дня з таким номером не існує!" << endl;
}

// Приклад використання if для перевірки на вихідний
if (day == 6 || day == 7) {
    cout << "Це вихідний день!" << endl;
} else if (day >= 1 && day <= 5) {
    cout << "Це робочий день." << endl;
}

return 0;
}

```

Тема роботи 3.2 : Циклічні алгоритми та програми (Цикл з відомою кількістю кроків (for). Цикл з передумовою (while). Цикл з післяумовою (do — while). Достроковий вихід з циклу. Розрахунок сум послідовностей.)

Мета роботи: Ознайомитися з принципами побудови циклічних алгоритмів; навчитися використовувати оператори циклу for, while, do-while та оператори керування циклом (break, continue) для розв'язання задач обчислення сум послідовностей та ітераційних процесів.

Порядок виконання роботи:

- Ознайомитися з теоретичним матеріалом щодо циклів у мові C++.
- Отримати варіант завдання (згідно з номером у списку групи).
- Розробити алгоритми для кожного з 5 завдань варіанту.
- Створити програмний проєкт у середовищі розробки (напр., Visual Studio, Code::Blocks, CLion або онлайн-компілятор).
- Написати код програм, провести їх налагодження та тестування.
- Оформити звіт про виконання роботи.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Обчислити суму чисел від 1 до N (цикл for).
2. Знайти перше число Фібоначчі, що перевищує 1000 (while).
3. Вводити числа, поки не буде введено 0; знайти їх добуток (do-while).
4. Вивести всі числа від 1 до 50, крім тих, що діляться на 5 (continue).
5. Вивести всі трицифрові числа, сума цифр яких дорівнює 15.

Варіант 2

1. Обчислити суму квадратів парних чисел від 2 до N.
2. Знайти найменший дільник числа K, відмінний від 1.
3. Організувати перевірку введення пароля (цикл до правильного вводу).
4. Знайти суму перших 10 додатних чисел у довільній послідовності.
5. Підрахувати, скільки разів цифра 3 зустрічається в усіх числах від 1 до 100.

Варіант 3

1. Обчислити факторіал числа N.
2. Визначити кількість цифр у цілому числі.
3. Вводити числа, поки сума не перевищить 100.
4. Вивести таблицю квадратів чисел від 1 до 20, перервати при $x^2 > 300$.
5. Знайти всі двоцифрові числа, які діляться на суму своїх цифр.

Варіант 4

1. Обчислити суму непарних чисел у діапазоні від А до В.
2. Знайти суму цифр введеного числа.
3. Реалізувати меню програми з виходом по натисканню '0'.
4. Вивести числа від 100 до 1, які діляться на 7.
5. Визначити кількість цілих чисел у діапазоні від 1 до 1000, що діляться на 13 і на 17.

Варіант 5

1. Обчислити середнє арифметичне чисел від 1 до N.
2. Перевірити, чи є число Р простим.
3. Знайти максимальне число серед введених (кількість чисел задається на початку).
4. Вивести парні числа від 20 до 0.
5. Вивести всі дільники кожного числа в діапазоні від 10 до 20.

Варіант 6

1. Обчислити суму кубів перших n натуральних чисел.
2. Знайти найбільшу цифру в числі.
3. Вводити символи, поки не буде введено символ '*'.
4. Пошук першого від'ємного елемента в послідовності.
5. Знайти всі трицифрові числа, які при діленні на 2, 3, 4 та 5 дають у остачі 1.

Варіант 7

1. Обчислити суму степенів двійки $2^1 + 2^2 + \dots + 2^n$.
2. Знайти НСД двох чисел методом віднімання.
3. Обчислити добуток n випадкових чисел.
4. Вивести всі числа від 1 до N, пропустивши число 13.
5. Надрукувати всі числа від 1 до 200, у яких остання цифра дорівнює 7.

Варіант 8

1. Обчислити суму $1! + 2! + \dots + n!$.
2. Визначити, чи є число N степенем двійки.
3. Знайти мінімальне число серед введених (введення до нуля).
4. Знайти суму чисел у діапазоні [10, 50], що не кратні 3.
5. Визначити кількість "щасливих" чотирицифрових квитків (сума перших двох цифр дорівнює сумі двох останніх).

Варіант 9

1. Знайти кількість парних чисел у діапазоні [1, N].
2. Перевернути число (реверс цифр).
3. Обчислити x^n без використання стандартної функції.
4. Вийти з циклу, якщо введено від'ємне число.
5. Знайти суму всіх парних чисел у діапазоні від 1 до 500, які не діляться на 10.

Варіант 10

1. Обчислити суму всіх двозначних чисел.
2. Знайти найменшу цифру числа.
3. Підрахувати кількість голосів "за" та "проти" (введення 'у'/'п' до символу 'q').
4. Вивести всі числа від 1 до 100, кратні 11.
5. Знайти всі трицифрові числа-паліндроми (наприклад, 121, 545).

Варіант 11

1. Обчислити добуток непарних чисел від 1 до 15.
2. Перевірити, чи містить число цифру 5.
3. Обчислити суму чисел, поки користувач не введе від'ємне.
4. Знайти перше число, квадрат якого більший за 500.
5. Вивести таблицю множення для числа 7 від 1 до 10 у форматі " $7 * i = \text{результат}$ ".

Варіант 12

1. Вивести таблицю множення на число K.
2. Підрахувати кількість парних цифр у числі.
3. Організувати повторне обчислення задачі за бажанням користувача.
4. Вивести числа від A до B, що закінчуються на 3.
5. Знайти суму всіх двоцифрових чисел, які кратні 5, але не кратні 10.

Варіант 13

1. Обчислити суму перших n елементів арифметичної прогресії.
2. Перевірити, чи є число паліндромом.
3. Знайти середнє геометричне введених чисел (до нуля).
4. Пропустити вивід чисел у діапазоні від 10 до 20.
5. Вивести всі числа від 1 до 100, сума цифр яких парна.

Варіант 14

1. Обчислити суму $1^k + 2^k + \dots + n^k$.
2. Знайти всі дільники числа N.
3. Знайти суму чисел, цифри яких у сумі дають 10 (діапазон 1-500).
4. Перервати підсумовування, якщо сума перевищила заданий поріг.
5. Підрахувати кількість непарних цифр у всіх числах від 1 до 50.

Варіант 15

1. Обчислити суму значень функції $y=2x+3$ для $x \in [1, 10]$ з кроком 1.
2. Визначити, чи є число досконалим (сума дільників дорівнює самому числу).
3. Вводити радіуси кіл, обчислювати площі, поки $R > 0$.
4. Вивести ASCII таблицю для символів від 'A' до 'Z'.
5. Знайти всі двоцифрові числа, у яких перша цифра вдвічі більша за другу.

Варіант 16

1. Знайти суму чисел від 1 до N , що діляться на 4.
2. Знайти другу цифру числа.
3. Обчислити добуток цифр числа.
4. Вивести числа від 1 до N , які не є квадратами інших чисел.
5. Знайти суму чисел від 1 до 100, у яких хоча б одна цифра дорівнює 0.

Варіант 17

1. Обчислити суму $1/2 + 2/3 + 3/4 + \dots + n/(n+1)$.
2. Знайти кількість входжень цифри X у число Y .
3. Вводити числа, поки не буде введено два однакових підряд.
4. Пошук першого числа в послідовності, кратне 17.
5. Вивести всі числа від 10 до 100, у яких обидві цифри однакові.

Варіант 18

1. Обчислити суму квадратів непарних чисел від 1 до N .
2. Знайти суму першої та останньої цифри числа.
3. Знайти кількість додатних і від'ємних чисел серед введених.
4. Вивести всі числа від 1 до 100, що не діляться на 2 і на 3.
5. Знайти добуток усіх чисел від 1 до 20, що діляться на 3.

Варіант 19

1. Обчислити добуток $1 \cdot 3 \cdot 5 \cdot \dots \cdot (2n-1)$.
2. Перевірити, чи є число "сильним" (сума факторів цифр дорівнює числу).
3. Знайти суму кубів введених чисел (до першого 0).
4. Вийти з циклу при введенні символу 'q'.
5. Вивести всі трицифрові числа, у яких середня цифра дорівнює сумі крайніх.

Варіант 20

1. Обчислити $1 + 1/2 + 1/4 + 1/8 + \dots$ (до n -го доданка).
2. Визначити кількість непарних цифр у числі.
3. Гра "Вгадай число" (цикл до вгадування випадкового числа).
4. Пропустити вивід чисел, кратних 4.
5. Підрахувати суму чисел від 1 до 100, які закінчуються на 5 або на 0.

Варіант 21

1. Обчислити суму $S = 1/3 + 1/9 + 1/27 + \dots + 1/3^n$.
2. Знайти всі прості числа до 100.
3. Вводити цілі числа, знайти суму лише тих, що закінчуються на 0.
4. Вивести числа від N до 1 з кроком -2.
5. Вивести всі числа від 1 до 50, квадрат яких не перевищує 1000.

Варіант 22

1. Обчислити суму чисел від А до В, кратних 3.
2. Визначити, чи є число степенем 3.
3. Обчислити суму x^2 для введених x , поки $x \neq 0$.
4. Пошук першого числа, що ділиться на 13 і більше 100.
5. Знайти суму всіх парних цифр у числах від 1 до 30.

Варіант 23

1. Обчислити добуток $1/1 \cdot 1/2 \cdot \dots \cdot 1/n$.
2. Знайти середнє арифметичне цифр числа.
3. Організувати введення координат (x, y) до введення (0,0).
4. Вивести всі парні числа від 1 до 50, крім чисел 22, 24, 26.
5. Знайти всі трицифрові числа, які складаються з однакових цифр.

Варіант 24

1. Обчислити суму $10 + 20 + 30 + \dots + 10n$.
2. Перевірити, чи є число числом Армстронга (3-цифрове).
3. Знайти добуток від'ємних введених чисел.
4. Вийти з циклу, якщо введено число, більше за 1000.
5. Підрахувати кількість дільників кожного парного числа від 1 до 20.

Варіант 25

1. Обчислити суму парних кубів від 2 до N.
2. Визначити, чи є в числі хоча б одна цифра 0.
3. Знайти кількість чисел, що діляться на 5 (введення до нуля).
4. Вивести кожне п'яте число від 0 до 100.
5. Вивести всі числа від 1 до 100, у яких цифра десятків більша за цифру одиниць.

Контрольні запитання для звіту:

- Яка основна відмінність між циклами `while` та `do-while`?
- У якому випадку цикл `for` може не виконатися жодного разу?
- Як реалізувати нескінченний цикл за допомогою оператора `while`?
- Яка роль оператора `break` у циклічних алгоритмах?
- Що таке ітерація циклу?
- Які типи даних найкраще підходять для лічильників циклу і чому?

Приклад виконання роботи:

```
#include <iostream>
#include <iomanip>

using namespace std;

int main() {
    int n;
    double sum = 0.0;
```

```

cout << "Введіть кількість елементів n: ";
cin >> n;

if (n <= 0) {
    cout << "Помилка: n має бути додатним!" << endl;
    return 1;
}

// Використання циклу for для розрахунку суми
for (int i = 1; i <= n; ++i) {
    sum += 1.0 / (i * i);
}

cout << "Результат обчислення суми: " << fixed << setprecision(6) << sum
<< endl;

return 0;
}

```

Тема роботи 4.1 : Підпрограми, їх різновиди та способи використання (Процедури. Функції користувача).

Мета роботи:

- Ознайомитися з концепцією модульного програмування.
- Навчитися створювати та використовувати функції користувача (процедури та функції, що повертають значення).
- Вивчити способи передачі параметрів у підпрограми (за значенням, за посиланням, за вказівником).
- Набути навичок декомпозиції складних задач на простіші складові.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо оголошення, визначення та виклику функцій у C++.
- Отримати варіант завдання від викладача.
- Розробити алгоритми для кожного з 5 завдань вашого варіанту.
- Написати програмний код, використовуючи підпрограми для виконання основних обчислень.
- Протестувати програму на різних вхідних даних.
- Оформити звіт згідно з вимогами.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Обчислити довжину кола за радіусом.
2. Знайти суму цифр трицифрового числа.
3. Функція для перевірки, чи є число парним.
4. Знайти найбільше з трьох чисел.
5. Обчислити факторіал числа.

Варіант 2

1. Знайти об'єм куба за його ребром.
2. Перевірити, чи є символ голосною літерою.
3. Знайти середнє арифметичне двох дійсних чисел.
4. Функція для переведення температури з Цельсія у Фаренгейти.
5. Знайти суму елементів масиву з 5 чисел.

Варіант 3

1. Обчислити периметр прямокутника.
2. Знайти квадратний корінь суми квадратів двох чисел.
3. Функція, що повертає знак числа (1, -1 або 0).
4. Обчислити n-ту степінь числа x.
5. Процедура для виведення рядка "Hello" n разів.

Варіант 4

1. Обчислити площу круга.
2. Знайти найменше з чотирьох чисел.
3. Перевірити, чи ділиться число на 5 без остачі.
4. Обчислити гіпотенузу прямокутного трикутника.
5. Функція для знаходження НСД двох чисел.

Варіант 5

1. Знайти площу поверхні кулі.
2. Функція для обчислення модуля числа.
3. Перевірити, чи є рік високосним.
4. Знайти останню цифру числа.
5. Обчислити суму чисел від 1 до N.

Варіант 6

1. Обчислити об'єм циліндра.
2. Функція для обміну значень двох змінних (за посиланням).
3. Визначити чверть координатної площини за точкою (x, y).
4. Перевірити, чи є число простим.
5. Знайти добуток цифр двоцифрового числа.

Варіант 7

1. Обчислити відстань між двома точками на площині.
2. Знайти суму двох дробів (результат у вигляді чисельника і знаменника).
3. Перевірити, чи є число трицифровим.
4. Знайти максимум у масиві.
5. Обчислити площу трапеції.

Варіант 8

1. Знайти довжину діагоналі квадрата.
2. Перевірити, чи є трикутник рівностороннім.
3. Функція для підрахунку кількості пробілів у рядку.
4. Знайти середнє геометричне двох чисел.
5. Обчислити суму парних чисел у діапазоні від 1 до 100.

Варіант 9

1. Обчислити площу паралелограма.
2. Функція для перевертання числа (наприклад, 123 -> 321).
3. Знайти мінімум у масиві.
4. Перевірити, чи є число досконалим.
5. Знайти кількість дільників числа.

Варіант 10

1. Знайти об'єм конуса.
2. Перевірити, чи є слово паліндромом.
3. Знайти суму елементів масиву, більших за задане число.
4. Функція для обчислення логарифма за основою 2.
5. Процедура малювання прямокутника символами '*'.

Варіант 11

1. Обчислити довжину дуги кола.
2. Знайти суму непарних цифр числа.
3. Визначити, чи лежить точка всередині кола.
4. Знайти друге за величиною число з трьох.
5. Обчислити n-те число Фібоначчі.

Варіант 12

1. Знайти площу ромба за діагоналями.
2. Функція для переведення секунд у години, хвилини та секунди.
3. Знайти кількість від'ємних елементів у масиві.
4. Перевірити, чи є число ступенем двійки.
5. Обчислити значення функції $y = x^2 + 5x - 3$.

Варіант 13

1. Обчислити об'єм піраміди.
2. Перевірити, чи є символ цифрою.
3. Знайти індекс максимального елемента в масиві.
4. Обчислити суму квадратів перших n натуральних чисел.
5. Функція для конкатенації двох рядків.

Варіант 14

1. Знайти периметр правильного n-кутника.
2. Функція для перевірки, чи є трикутник прямокутним.
3. Знайти кількість входжень заданого символу в рядок.
4. Перетворити кілометри в милі.
5. Обчислити суму цифр числа за допомогою рекурсії.

Варіант 15

1. Знайти радіус вписаного кола в квадрат.
2. Обчислити добуток парних елементів масиву.
3. Перевірити, чи є число паліндромом.
4. Знайти НСК (найменше спільне кратне) двох чисел.
5. Функція для визначення дня тижня за номером.

Варіант 16

1. Обчислити площу сектора кола.
2. Знайти кількість голосних у рядку.
3. Перевірити, чи є число кратним 3 і 7 одночасно.
4. Обчислити середнє арифметичне елементів масиву.
5. Функція для піднесення матриці 2×2 до квадрата.

Варіант 17

1. Знайти об'єм паралелепіпеда.
2. Функція для знаходження коренів квадратного рівняння.
3. Знайти кількість нулів у масиві.
4. Перевірити, чи є рік початком століття.
5. Обчислити суму ряду $1 + 1/2 + 1/3 + \dots + 1/n$.

Варіант 18

1. Знайти довжину медіани трикутника.
2. Функція для переведення числа з десяткової в двійкову систему.
3. Знайти суму цифр у шістнадцятковому представленні числа.
4. Перевірити рівність двох масивів.
5. Обчислити площу кільця (зовнішній і внутрішній радіуси).

Варіант 19

1. Знайти площу бічної поверхні циліндра.
2. Функція для визначення типу символу (літера, цифра, знак).
3. Знайти добуток елементів масиву з непарними індексами.
4. Перевірити, чи є число числом Армстронга.
5. Обчислити біноміальний коефіцієнт $C(n, k)$.

Варіант 20

1. Знайти довжину бісектриси кута трикутника.
2. Функція для сортування трьох чисел за зростанням.
3. Знайти суму елементів на головній діагоналі матриці 3×3 .
4. Перевірити, чи є рядок порожнім.
5. Обчислити синус кута через ряд Тейлора.

Варіант 21

1. Обчислити об'єм тора.
2. Функція для пошуку підрядка в рядку.
3. Знайти кількість чисел у масиві, що закінчуються на 0.
4. Обчислити значення полінома n-го степеня.
5. Перевірити, чи є число від'ємним, додатним чи нулем.

Варіант 22

1. Знайти площу еліпса.
2. Функція для генерації випадкового числа в заданому діапазоні.
3. Знайти суму елементів масиву між першим і останнім додатним елементом.
4. Перевірити, чи є графік функції симетричним.
5. Обчислити периметр довільного чотирикутника за координатами вершин.

Варіант 23

1. Обчислити довжину вектора в 3D просторі.
2. Функція для видалення всіх пробілів із рядка.
3. Знайти найбільший спільний дільник трьох чисел.
4. Перевірити, чи є матриця симетричною.
5. Обчислити суму непарних чисел у масиві.

Варіант 24

1. Знайти площу поверхні піраміди.
2. Функція для перевірки, чи є число степенем трійки.
3. Знайти перше число в масиві, що перевищує 100.
4. Обчислити скалярний добуток двох векторів.
5. Функція для перетворення великих літер у малі.

Варіант 25

1. Знайти об'єм призми.
2. Функція для підрахунку слів у реченні.
3. Перевірити, чи є послідовність чисел арифметичною прогресією.
4. Знайти суму елементів масиву, індекси яких є парними.
5. Обчислити відстань від точки до прямої на площині.

Контрольні запитання для звіту:

- Що таке підпрограма і які переваги її використання?
- Чим відрізняється функція `void` від функцій інших типів?
- Що таке формальні та фактичні параметри?
- Поясніть різницю між передачею параметрів за значенням та за посиланням (&).
- Де в програмі зазвичай розташовують прототипи функцій?
- Що таке рекурсивна функція?

Приклад виконання роботи:

```
#include <iostream>
```

```

#include <cmath>
#include <iomanip>

using namespace std;

// Функція користувача, що повертає значення (double)
double calculateArea(double a, double b, double c) {
    if (a + b <= c || a + c <= b || b + c <= a) {
        return -1.0; // Повертаємо помилку, якщо трикутник не існує
    }
    double p = (a + b + c) / 2.0;
    return sqrt(p * (p - a) * (p - b) * (p - c));
}

// Процедура (функція типу void) для виведення результату
void displayResult(double area) {
    if (area < 0) {
        cout << "Помилка: Трикутник із такими сторонами не існує!" << endl;
    } else {
        cout << "Площа трикутника: " << fixed << setprecision(2) << area << "
кв. од." << endl;
    }
}

int main() {
    double x, y, z;
    cout << "Введіть три сторони трикутника: ";
    cin >> x >> y >> z;

    // Виклик підпрограм
    double result = calculateArea(x, y, z);
    displayResult(result);

    return 0;
}

```

Тема роботи 4.2 : Структура програм. Складові частини програми. Глобальні та локальні змінні.

Мета роботи:

- Ознайомитися з базовою структурою програми на мові C++.
- Вивчити правила оголошення та використання локальних і глобальних змінних.
- Навчитися розділяти код на функціональні блоки.
- Зрозуміти область видимості ідентифікаторів.

Порядок виконання роботи:

- Ознайомитися з теоретичним матеріалом щодо структури C++ програми.
- Отримати номер варіанта від викладача.
- Розробити алгоритми для розв'язання 5 завдань свого варіанта.
- Написати програмний код, використовуючи як локальні, так і (де доречно) глобальні змінні.
- Протестувати програму на різних вхідних даних.
- Оформити звіт, що містить код програм, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Обчислити периметр квадрата за заданою стороною.
2. Знайти середнє арифметичне двох цілих чисел.
3. Перевести температуру з градусів Цельсія у Фаренгейти.
4. Вивести повідомлення "Hello" та ім'я користувача, яке він ввів.
5. Обчислити добуток трьох дробових чисел.

Варіант 2

1. Знайти площу прямокутника за двома сторонами.
2. Обчислити об'єм куба за довжиною його ребра.
3. Конвертувати суму в гривнях у долари за глобальним курсом валют.
4. Знайти остачу від ділення першого цілого числа на друге.
5. Обчислити результат множення введеного числа на два та додати п'ять.

Варіант 3

1. Обчислити довжину кола, знаючи його радіус (використати число π).
2. Знайти суму цифр введеного двоцифрового числа.
3. Обчислити гіпотенузу прямокутного трикутника за двома відомими катетами.
4. Вивести квадрат введеного цілого числа.
5. Знайти різницю між найбільшим та найменшим із трьох введених чисел.

Варіант 4

1. Обчислити площу трикутника за його основою та висотою.
2. Знайти швидкість об'єкта, якщо відомі подоланий шлях та час у дорозі.
3. Перевести довжину із сантиметрів у дюйми (вважати, що 1 дюйм дорівнює 2.54 см).
4. Обчислити результат піднесення числа до квадрата та віднімання від нього четвірки.
5. Вивести назву навчального закладу, збережену в глобальній змінній-рядку.

Варіант 5

1. Знайти периметр прямокутника за його довжиною та шириною.
2. Обчислити силу струму, поділивши напругу на опір (закон Ома).
3. Визначити повну кількість хвилин за введеною кількістю секунд.
4. Обчислити середню вартість товару на основі трьох введених цін.
5. Вивести символ на екран, використовуючи його числовий код ASCII.

Варіант 6

1. Обчислити об'єм циліндра за його радіусом та висотою.
2. Знайти відстань між двома точками на числовій прямій.
3. Розрахувати суму знижки у розмірі 15 відсотків від вартості товару.
4. Вивести таблицю множення для числа п'ять (від 1 до 10).
5. Знайти суму квадратів двох введених користувачем чисел.

Варіант 7

1. Знайти площу трапеції за її основами та висотою.
2. Обчислити кінетичну енергію тіла за його масою та швидкістю.
3. Конвертувати відстань із кілометрів у милі.
4. Вивести на екран номер поточного року, використовуючи глобальну змінну.
5. Обчислити корінь квадратний із введеного додатного числа.

Варіант 8

1. Обчислити довжину діагоналі квадрата за його стороною.
2. Знайти суму трьох довільних чисел.
3. Розрахувати суму податку у розмірі 18 відсотків від заробітної плати.
4. Перевірити, чи є число парним (вивести результат як істину або хибу).
5. Обчислити об'єм кулі за введеним радіусом.

Варіант 9

1. Знайти площу ромба за довжинами його діагоналей.
2. Обчислити шлях, який пройшло тіло при рівномірному русі за певний час.
3. Перевести об'єм рідини з літрів у галони.
4. Знайти середнє геометричне двох додатних чисел.
5. Вивести число, яке є оберненим до введеного (одиниця поділена на число).

Варіант 10

1. Обчислити периметр рівностороннього трикутника за його стороною.
2. Знайти тиск, поділивши силу на площу поверхні.
3. Конвертувати кількість днів у відповідну кількість годин та хвилин.
4. Додати два числа та поділити отриману суму на третє число.
5. Вивести ініціали користувача (перші літери імені та прізвища) окремо.

Варіант 11

1. Обчислити площу паралелограма за його стороною та висотою.
2. Знайти густину речовини, поділивши її масу на об'єм.
3. Розрахувати загальну вартість поїздки (витрачене паливо помножити на ціну).
4. Вивести абсолютне значення (модуль) числа, не використовуючи стандартні функції.
5. Обчислити куб введеного числа (число у третьому степені).

Варіант 12

1. Знайти об'єм прямокутного паралелепіпеда за трьома його вимірами.
2. Обчислити прискорення тіла, поділивши зміну швидкості на час.
3. Перевести вагу тіла з кілограмів у фунти.
4. Знайти суму першої та останньої цифри введеного трицифрового числа.
5. Перевірити, чи входить число в діапазон від нуля до десяти.

Варіант 13

1. Обчислити повну площу поверхні куба за довжиною його ребра.
2. Знайти потужність, поділивши виконану роботу на час.
3. Розрахувати індекс маси тіла (вага поділена на квадрат зросту).
4. Вивести числовий код ASCII для введеного користувачем символу.
5. Обчислити добуток цифр введеного двоцифрового числа.

Варіант 14

1. Знайти довжину медіани у рівносторонньому трикутнику за його стороною.
2. Обчислити потенційну енергію тіла (маса на прискорення вільного падіння та висоту).
3. Конвертувати величину кута з градусів у радіани.
4. Порівняти два числа та вивести результат (яке з них більше).
5. Знайти цілу частину від ділення першого числа на друге.

Варіант 15

1. Обчислити площу еліпса за його великою та малою півосьми.
2. Знайти частоту коливань, знаючи період (одиниця поділена на період).
3. Розрахувати нову ціну товару після нарахування податку у розмірі 20 відсотків.
4. Вивести версію програми, що зберігається у глобальній константі.
5. Знайти середнє арифметичне трьох введених користувачем чисел.

Варіант 16

1. Знайти периметр правильного шестикутника за його стороною.
2. Обчислити механічну роботу як добуток сили на шлях.
3. Конвертувати об'єм даних із байтів у мегабайти.
4. Обчислити суму числа, помноженого на п'ять, та його квадрата, помноженого на три.
5. Ввести три символи та вивести суму їхніх числових кодів.

Варіант 17

1. Обчислити об'єм конуса за радіусом основи та висотою.
2. Знайти електричний опір, поділивши напругу на силу струму.
3. Розрахувати необхідну кількість плиток для підлоги (площа кімнати поділена на площу однієї плитки).
4. Перевірити, чи ділиться введене число на три без остачі.
5. Вивести назву навчальної групи, використовуючи глобальну змінну-рядок.

Варіант 18

1. Знайти площу кільця як різницю площ зовнішнього та внутрішнього кругів.
2. Обчислити імпульс тіла як добуток його маси на швидкість.
3. Конвертувати час із хвилин у секунди.
4. Знайти різницю між сумою двох чисел та їхнім добутком.
5. Обчислити суму модуля числа та його квадрата.

Варіант 19

1. Обчислити площу бічної поверхні циліндра за радіусом та висотою.
2. Знайти момент сили як добуток сили на плече важеля.
3. Розрахувати кількість днів, що залишилися до кінця місяця (ввести поточне число).
4. Вивести введений цілий номер у вісімковій системі числення.
5. Обчислити середню швидкість руху на двох різних ділянках шляху.

Варіант 20

1. Знайти довжину діагоналі прямокутника за його двома сторонами.
2. Обчислити період коливань математичного маятника за його довжиною.
3. Конвертувати відстань із метрів у ярди.
4. Обчислити добуток суми двох чисел на їхню різницю.
5. Вивести прізвище користувача, перетворивши всі літери на великі.

Варіант 21

1. Обчислити об'єм піраміди за площею її основи та висотою.
2. Знайти напруженість електричного поля, поділивши силу на величину заряду.
3. Розрахувати вартість спожитої електроенергії (кіловат-години помножити на тариф).
4. Знайти залишок від ділення суми двох чисел на два.
5. Перевірити умови: чи є число парним і чи воно більше за сто.

Варіант 22

1. Знайти радіус кола, вписаного у квадрат із заданою стороною.
2. Обчислити кутову швидкість, поділивши лінійну швидкість на радіус.
3. Конвертувати суму грошей із гривень у євро.
4. Обчислити довжину вектора на площині за його координатами.
5. Вивести результат перевірки: чи не дорівнює перше число другому.

Варіант 23

1. Обчислити площу сегмента круга (за спрощеним текстовим описом).
2. Знайти кількість речовини, поділивши масу на молярну масу.
3. Розрахувати приблизний час завантаження файлу (розмір поділити на швидкість).
4. Знайти суму трьох останніх цифр введеного номера телефону.
5. Обчислити результат піднесення числа два до степеня, який ввів користувач.

Варіант 24

1. Знайти периметр паралелограма за двома сусідніми сторонами.
2. Обчислити коефіцієнт корисної дії (ККД), поділивши корисну роботу на витрачену.
3. Конвертувати площу ділянки з гектарів у квадратні метри.
4. Обчислити значення одиниці, поділеної на різницю числа та одиниці.
5. Вивести поточну дату, використовуючи дві глобальні змінні (день та місяць).

Варіант 25

1. Обчислити об'єм тора (фігури у формі бублика) за його радіусами.
2. Знайти силу тертя як добуток коефіцієнта тертя на силу нормальної реакції.
3. Розрахувати відсоток успішності (кількість присутніх поділити на загальну кількість).
4. Знайти найбільше з двох чисел, використовуючи скорочену умовну конструкцію.
5. Вивести фінальне повідомлення про завершення роботи через спеціальну глобальну функцію.

Контрольні запитання для звіту:

- З яких основних частин складається програма на C++?
- Що таке препроцесорні директиви і для чого вони потрібні?
- У чому головна різниця між глобальними та локальними змінними?
- Що станеться, якщо локальна змінна має таке ж ім'я, як і глобальна?
- Який оператор використовується для доступу до глобальної змінної, якщо її ім'я перекрите локальною?

Приклад виконання роботи:

```
#include <iostream>
```



```

// Глобальна змінна (константа)
const double PI = 3.14159;

void displayHeader() {
    // Використання глобальної змінної у функції
    std::cout << "--- Програма розрахунку площі (PI = " << PI << ") ---" <<
std::endl;
}

int main() {
    // Локальна змінна
    double radius;

    displayHeader();

    std::cout << "Введіть радіус: ";
    std::cin >> radius;

    if (radius < 0) {
        std::cout << "Помилка: радіус не може бути від'ємним!" << std::endl;
    } else {
        // Локальна змінна для результату
        double area = PI * radius * radius;
        std::cout << "Площа круга: " << area << std::endl;
    }

    return 0;
}

```

Тема роботи 5.1 : Алгоритми і величини. Лінійні обчислювальні алгоритми

Мета роботи:

- Ознайомитися з базовою структурою програми на C++.
- Навчитися оголошувати змінні різних типів даних.
- Опанувати введення даних з клавіатури (cin) та виведення результатів на екран (cout).
- Навчитися реалізовувати лінійні алгоритми для розв'язання математичних задач.
- Використовувати математичну бібліотеку <cmath>.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо типів даних та операторів C++.
- Отримати номер варіанта від викладача.
- Проаналізувати умову завдань та скласти математичні формули.
- Написати програмний код для кожного з 5 завдань варіанта.
- Протестувати програму на різних вхідних даних.
- Оформити звіт, що містить: тему, мету, код програм та скріншоти результатів виконання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Визначити місткість кубічного резервуара, якщо відома довжина його ребра.
2. Знайти середнє арифметичне трьох довільних дійсних чисел.
3. Розрахувати відстань, яку подолає потяг, рухаючись стабільно протягом певного часу з постійною швидкістю.
4. Обчислити довжину найбільшої сторони прямокутного трикутника за відомими довжинами інших двох сторін.
5. Знайти значення виразу: квадрат аргументу плюс п'ятикратне значення аргументу мінус три.

Варіант 2

1. Розрахувати загальну довжину огорожі для квадратної ділянки, знаючи довжину однієї сторони.
2. Визначити площу підлоги в прямокутній кімнаті за її довжиною та шириною.
3. Сконвертувати показники температури зі шкали Цельсія у шкалу Фаренгейта.
4. Визначити лінійний розмір ребра куба, маючи значення загальної площі всіх його граней.
5. Знайти суму значень синуса та косинуса для введеного кута.

Варіант 3

1. Обчислити площу поверхні трикутного вітрила за його горизонтальною основою та вертикальною висотою.
2. Розрахувати підсумкову суму до сплати за покупку певної маси продукту за встановленою ціною за одиницю маси.
3. Вивести суму та результат множення двох введених користувачем дійсних чисел.
4. Обчислити корінь квадратний із суми квадратів двох довільних значень.
5. Знайти суму квадратного кореня від аргументу, збільшеного на одиницю, та подвоєного значення цього ж аргументу.

Варіант 4

1. Визначити повну довжину межі кола, якщо відома відстань між його протилежними точками через центр.
2. Розрахувати об'єм циліндричної колони за її висотою та радіусом основи.
3. Визначити густину речовини, якщо відомі її маса та займаний простір.
4. Знайти довжину відрізка на координатній осі між двома точками.
5. Обчислити суму експоненти та натурального логарифма для введеного додатного числа.

Варіант 5

1. Обчислити площу земельної ділянки у формі трапеції за довжинами паралельних сторін та відстанню між ними.
2. Визначити середнє геометричне двох введених додатних чисел.
3. Розрахувати силу електричного струму в провіднику за відомою різницею потенціалів та електричним опором.
4. Визначити тривалість підйому об'єкта, підкинутого вгору, знаючи його початкову швидкість.
5. Обчислити результат піднесення до куба суми числа та двійки, після чого розділити отримане на чотири.

Варіант 6

1. Визначити сумарну довжину всіх сторін прямокутної рами.
2. Розрахувати об'єм іграшкової кулі за її радіусом.
3. Обчислити енергію руху об'єкта, знаючи його масу та швидкість переміщення.
4. Розрахувати суму окремих цифр для введеного цілого двозначного числа.
5. Знайти суму результату тригонометричного тангенса та величини, що є оберненою до аргументу.

Варіант 7

1. Визначити загальну площу поверхні граней куба з ребром заданої довжини.
2. Обчислити еквівалентний опір ділянки кола, де два пристрої з'єднані паралельно.
3. Перерахувати швидкість об'єкта з кілометрів на годину у метри на секунду.
4. Обчислити величину останнього кута трикутника, якщо відомі два інших кути.
5. Знайти значення математичного виразу: п'ять помножити на аргумент у четвертому степені, мінус подвоєний квадрат аргументу плюс один.

Варіант 8

1. Визначити довжину сторони квадрата, якщо відома його внутрішня площа.
2. Розрахувати місткість конічної ємності за її радіусом та глибиною.
3. Обчислити вартість пального для подорожі, знаючи дистанцію, середній розхід на одиницю шляху та вартість пального.
4. Визначити довжину вертикальної лінії, проведеної з вершини рівностороннього трикутника до його основи.
5. Обчислити суму результатів логарифмування за основою десять та синуса від квадрата введенного значення.

Варіант 10

1. Обчислити площу поверхні плоского кільця, знаючи два його радіуси.
2. Визначити відсоткове відношення одного введенного числа до іншого.
3. Розрахувати тиск на поверхню, знаючи силу впливу та розмір контактної зони.
4. Визначити площу правильного трикутника за довжиною його сторони.
5. Обчислити результат ділення виразу (квадрат числа плюс один) на вираз (число мінус один).

Варіант 11

1. Знайти довжину прямої лінії, що з'єднує протилежні кути прямокутника.
2. Обчислити довжину лінії, що з'єднує вершину трикутника з серединою протилежної сторони.
3. Перевести кутову міру з радіанної системи у градусну.
4. Визначити об'єм матеріалу в порожнистій кулі за її двома радіусами.
5. Знайти суму квадрата синуса та косинуса від квадрата введенного аргументу.

Варіант 12

1. Обчислити площу бічної поверхні круглого стовпа (циліндра).
2. Розрахувати суму третіх степенів двох введених дійсних чисел.
3. Визначити зміну швидкості за одиницю часу (прискорення), знаючи початкову швидкість, фінальну швидкість та тривалість процесу.
4. Визначити повну кількість годин у заданій кількості секунд.
5. Обчислити суму абсолютного значення (модуля) числа та цього ж числа, піднесеного до п'ятого степеня.

Варіант 13

1. Визначити радіус кола, маючи дані про його площу.
2. Обчислити енергію спокою об'єкта, піднятого над землею, за його масою та висотою.
3. Знайти суму послідовності чисел, що змінюються на сталу величину.
4. Визначити точку, що розташована точно посередині між двома заданими координатами на площині.
5. Обчислити значення функції: одиниця поділена на корінь квадратний із суми квадрата числа та заданої сталої.

Варіант 14

1. Визначити периметр симетричної трапеції за довжинами її основ та бічної грані.
2. Розрахувати об'єм піраміди за площею її нижньої грані та висотою.
3. Обчислити силу взаємного тяжіння двох космічних об'єктів за їх масами та відстанню між ними.
4. Знайти результат множення всіх цифр введеного цілого тризначного числа.
5. Обчислити результат множення експоненти від від'ємного аргументу на його синус.

Варіант 15

1. Визначити площу ромбовидної ділянки за довжинами ліній, що з'єднують його кути.
2. Розрахувати середній темп руху на маршруті, що складається з двох відрізків з різними швидкостями.
3. Визначити точну кількість дрібних одиниць пам'яті (байтів) у великому обсязі (кілобайтах).
4. Визначити радіус кола, яке торкається всіх сторін квадрата зсередини.
5. Обчислити натуральний логарифм від величини, що дорівнює квадрату числа плюс одиниця.

Варіант 16

1. Розрахувати площу частини круга (сектора) за радіусом та кутовим розміром цієї частини.
2. Визначити подолану дистанцію під час розгону за певний проміжок часу.
3. Обчислити загальну площу поверхні конуса.
4. Знайти число, що вказує на кількість десятків у заданому цілому числі.
5. Обчислити половину від суми куба числа та його квадратного кореня.

Варіант 17

1. Визначити місткість правильної багатогранної призми за її висотою та площею основи.
2. Обчислити довжину лінії, що ділить кут трикутника навпіл, за відомими сторонами.
3. Визначити потужність електричного пристрою за силою струму та його опором.
4. Перевести лінійний розмір з дюймів у загальноприйняті сантиметри.
5. Обчислити частку від ділення косинуса числа на суму цього числа та п'ятірки.

Варіант 18

1. Обчислити сумарну довжину сторін паралелограма за двома різними сторонами.
2. Визначити об'єм піраміди з квадратною основою.
3. Розрахувати механічну роботу, знаючи прикладену силу та шлях, пройдений під її дією.
4. Виділити саму праву цифру введеного цілого числа.
5. Знайти суму результату видобування кореня третього степеня та квадрата введеного значення.

Варіант 19

1. Обчислити площу овалу (еліпса) за його головними розмірами (півосями).
2. Знайти модуль різниці (відстань) між двома дійсними числами.
3. Визначити сумарний опір ланцюга, що складається з трьох послідовних елементів.
4. Обчислити довжину похилої сторони трикутника через катет та кут, що до нього прилягає.
5. Знайти результат віднімання одиниці від квадрата тригонометричного тангенса.

Варіант 20

1. Визначити площу похилих граней правильної піраміди.
2. Обчислити час одного повного коливання підвішеного вантажу (маятника) за його довжиною.
3. Знайти масу газу в балоні, знаючи його об'єм та густину за певних умов.
4. Визначити цифру, що позначає сотні у введеному числі.
5. Знайти суму експоненти від подвоєного числа та кореня квадратного з цього числа.

Варіант 21

1. Розрахувати довжину внутрішньої діагоналі куба за довжиною його ребра.
2. Обчислити об'єм кільцеподібної фігури (тора) за її внутрішнім та зовнішнім радіусом.
3. Визначити остаточну ціну продукту після віднімання відсоткової знижки.
4. Обчислити площу відтятої частини круга (сегмента) за його основними параметрами.
5. Знайти величину, обернену до суми синуса та косинуса введеного кута.

Варіант 22

1. Визначити довжину лінії, що з'єднує прямий кут трикутника з серединою найбільшої сторони.
2. Розрахувати інтенсивність потоку електронів (густину струму) через провідник певної площі.
3. Перерахувати об'єм рідини з побутових літрів у системні кубічні метри.
4. Обчислити результат додавання квадратів двох введених дійсних чисел.
5. Знайти результат ділення натурального логарифма числа на саме це число.

Варіант 23

1. Обчислити площу зовнішньої поверхні кулі за її радіусом.
2. Розрахувати швидкість обертання в кутових одиницях, знаючи лінійну швидкість точки на колі.
3. Визначити числові характеристики вектора, знаючи координати двох його точок.
4. Обчислити площу паралелограма через дві його сторони та нахил між ними.
5. Знайти результат віднімання числа від четвертого степеня суми цього числа та одиниці.

Варіант 24

1. Визначити радіус кола, в яке вписано прямокутник з відомими сторонами.
2. Обчислити кількість повторюваних подій за одиницю часу (частоту), знаючи період однієї події.
3. Перевести морську дистанцію з миль у стандартні метри.
4. Розрахувати місткість призми, що має в основі правильний шестикутник.
5. Знайти абсолютне значення (модуль) різниці між квадратом та кубом введеного числа.

Варіант 25

1. Обчислити площу рівнобедреного трикутника за довжиною його низу та бічної грані.
2. Розрахувати силу, що діє на занурене в рідину тіло, за об'ємом витісненої рідини.
3. Перевести часовий інтервал, заданий у годинах, у повну кількість хвилин.
4. Визначити довжину вектора у просторі за трьома його компонентами.
5. Обчислити результат множення синуса суми двох кутів на косинус їхньої різниці.

Контрольні запитання для звіту:

- Що таке лінійний алгоритм?
- Які типи даних у C++ використовуються для роботи з дробовими числами?
- Як підключити математичну бібліотеку для використання функції `sqrt()` або `pow()`?
- Яка різниця між операторами `/` для цілих та дійсних чисел?
- Як забезпечити перехід на новий рядок при виведенні тексту?

Приклад виконання роботи:

```
#include <iostream>
#include <cmath> // Для використання константи M_PI та функції pow

using namespace std;

int main() {
    double r, area, circumference;
    const double PI = 3.141592653589793;

    cout << "Введіть радіус кола: ";
    cin >> r;

    // Лінійні обчислення
    area = PI * pow(r, 2);
    circumference = 2 * PI * r;

    cout << "Площа круга: " << area << endl;
    cout << "Довжина кола: " << circumference << endl;

    return 0;
}
```

Тема роботи 5.2 : Розгалуження і цикли в обчислювальних алгоритмах. Допоміжні алгоритми і процедури.

Мета роботи:

- Оволодіти навичками використання операторів розгалуження (if-else, switch) та циклів (for, while, do-while).
- Навчитися декомпонувати складні задачі за допомогою допоміжних алгоритмів (функцій/процедур).
- Отримати досвід передачі параметрів у функції та повернення результатів.

Порядок виконання роботи:

- Ознайомитися з теоретичними відомостями щодо синтаксису розгалужень та циклів у C++.
- Отримати варіант завдання від викладача.
- Розробити алгоритми для кожного з 5 завдань варіанту.
- Написати програмний код у середовищі розробки (наприклад, Visual Studio, Code::Blocks або онлайн-компілятор).
- Протестувати програму на різних вхідних даних.
- Оформити звіт, що включає код програм та результати їх виконання (скриншоти).
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Вивести всі числа від 1 до 50, які діляться на 3.
2. Користувач вводить вік. Програма має сказати, чи він повнолітній.
3. Порахувати суму чисел, які вводить користувач, доки не буде введено 0.
4. Створити функцію, яка повертає назву місяця за його номером.
5. Написати функцію, яка перевіряє, чи є символ голосною літерою.

Варіант 2

1. Вивести алфавіт від 'A' до 'Z' у консоль.
2. Порівняти два рядки за довжиною та вивести довший.
3. Знайти найбільшу цифру у введеному числі.
4. Створити функцію для визначення мінімального з трьох чисел.
5. Написати функцію, яка перевіряє, чи є число парним.

Варіант 3

1. Порахувати кількість пробілів у введеному реченні.
2. Визначити, чи є введене число тризначним.
3. Вивести таблицю символів та їх кодів від 32 до 127.
4. Функція, яка повертає назву дня тижня за його номером.
5. Функція, яка "подвоює" введений рядок (приймає "A", повертає "AA").

Варіант 4

1. Вивести всі парні числа від 100 до 1 у зворотному порядку.
2. Перевірити, чи містить введене число цифру 5.
3. Користувач вводить пароль. Повторювати запит, доки пароль не буде вірним.
4. Функція, яка визначає, чи є рік високосним.
5. Процедура, яка виводить рядок символів '-' заданої довжини.

Варіант 5

1. Знайти кількість непарних чисел у діапазоні від A до B.
2. Визначити, чи є перша літера введеного слова великою.
3. Вивести всі числа від 1 до N, які закінчуються на 7.
4. Функція, яка повертає максимальну з двох сум (приймає 4 числа).
5. Функція, яка перевіряє, чи є введений символ цифрою.

Варіант 6

1. Порахувати кількість цифр у числі.
2. Вивести повідомлення "Доброго ранку", "День" або "Ніч" залежно від години.
3. Знайти перше число в послідовності, яке ділиться на 13.
4. Функція для перевірки, чи є пароль достатньо довгим (більше 8 символів).
5. Функція, яка повертає назву пори року за номером місяця.

Варіант 7

1. Вивести всі дільники числа N.
2. Визначити, яке з двох слів довше.
3. Порахувати кількість від'ємних чисел серед 10 введених.
4. Функція, яка замінює малі літери 'a' на великі 'A' у рядку.
5. Процедура, яка виводить рамку з символів '#' навколо тексту.

Варіант 8

1. Знайти середнє арифметичне 5 введених чисел.
2. Перевірити, чи закінчується число на 0.
3. Вивести ряд чисел: 1, 2, 4, 8, 16... до 1024.
4. Функція, яка визначає тип введеного символу (літера, цифра чи знак).
5. Функція, яка повертає "Так" або "Ні" залежно від логічного значення.

Варіант 9

1. Знайти суму цифр введеного числа.
2. Визначити, чи є введена назва кольору "червоним", "синім" або "зеленим".
3. Порахувати кількість слів у реченні (за пробілами).
4. Функція, яка перевертає рядок (читає задом наперед).
5. Функція, яка знаходить найменшу цифру числа.

Варіант 10

1. Вивести всі числа від 1 до 100, що кратні 5 і 3 одночасно.
2. Перевірити, чи є введений символ знаком пунктуації.
3. Знайти суму лише додатних чисел серед введених користувачем.
4. Функція, яка видаляє всі пробіли з рядка.
5. Процедура, яка виводить "Привіт, [Ім'я]" залежно від статі (пане/пані).

Варіант 11

1. Вивести квадрати чисел від 1 до 10.
2. Визначити, чи є число X дільником числа Y.
3. Порахувати, скільки разів у слові зустрічається літера 'о'.
4. Функція для перевірки, чи є число дзеркальним (наприклад, 121).
5. Функція, яка повертає найдовше слово з двох запропонованих.

Варіант 12

1. Вивести числа від N до M з кроком K.
2. Визначити, чи є введений символ латинською літерою.
3. Знайти добуток цифр числа.
4. Функція, яка перетворює хвилини у формат "години:хвилини".
5. Процедура, яка виводить сходинки з символів '*' (1 зірка, 2 зірки...).

Варіант 13

1. Порахувати кількість приголосних літер у слові.
2. Визначити, чи лежить число в межах від 10 до 20.
3. Знайти суму парних цифр у числі.
4. Функція, яка повертає оцінку словом (A -> "Відмінно").
5. Функція, яка перевіряє, чи є два числа однаковими за знаком.

Варіант 14

1. Вивести таблицю множення для введеного числа N.
2. Визначити, чи починається і закінчується слово на однакову літеру.
3. Знайти кількість великих літер у рядку.
4. Функція, яка повертає "Парне" або "Непарне" для числа.
5. Процедура, яка імітує завантаження (виводить крапки з затримкою).

Варіант 15

1. Знайти суму всіх непарних чисел від 1 до 100.
2. Перевірити, чи є число N двозначним.
3. Вивести зворотний порядок цифр числа.
4. Функція, яка визначає, чи є слово паліндромом.
5. Функція, яка знаходить середню цифру тризначного числа.

Варіант 16

1. Вивести всі числа від 1 до N , крім тих, що діляться на 4.
2. Використати `switch` для вибору мови привітання.
3. Порахувати суму чисел, які вводить користувач, до першого від'ємного.
4. Функція, яка повертає символ за його ASCII-кодом.
5. Функція, яка визначає кількість слів у реченні.

Варіант 17

1. Вивести символи від 'z' до 'a' (зворотний алфавіт).
2. Визначити, чи є число степенем двійки.
3. Знайти кількість входжень символу '!' у тексті.
4. Функція, яка замінює всі цифри у рядку на символ '*'.
5. Процедура, яка малює квадрат із символів '+' заданого розміру.

Варіант 18

1. Знайти суму чисел на проміжку від 1 до N , що закінчуються на 0.
2. Перевірити, чи є введене слово назвою дня тижня.
3. Порахувати кількість крапок у тексті.
4. Функція, яка повертає індекс першої знайденої літери у слові.
5. Функція, яка визначає, чи є число простим.

Варіант 19

1. Вивести кожне друге число від 1 до 20.
2. Визначити, чи є введена оцінка додатною (більше 0).
3. Знайти добуток непарних цифр числа.
4. Функція, яка генерує "сильний" пароль (додає випадкові знаки).
5. Функція, яка повертає суму двох найбільших чисел з трьох.

Варіант 20

1. Знайти кількість голосних у реченні.
2. Перевірити, чи є число кратним 7.
3. Вивести числа Фібоначчі до заданого N .
4. Функція, яка конвертує малі літери у великі.
5. Процедура, яка виводить список справ (масив рядків).

Варіант 21

1. Знайти суму чисел від 1 до N , які не діляться на 2 і 3.
2. Визначити, чи містить рядок цифри.
3. Вивести всі числа від 10 до 100, у яких цифри однакові (11, 22...).
4. Функція, яка повертає останню цифру числа.
5. Функція, яка порівнює два числа і повертає більшу суму їх цифр.

Варіант 22

1. Порахувати кількість речень у тексті (за знаками '.', '!', '?').
2. Визначити, чи є число паліндромом.
3. Вивести всі тризначні числа, сума цифр яких дорівнює 5.
4. Функція, яка видаляє останній символ рядка.
5. Процедура, яка виводить "Завантаження..." і прогрес у %.

Варіант 23

1. Знайти суму чисел, що вводить користувач, доки сума не перевищить 100.
2. Визначити, чи є слово "адмін" (регістр не має значення).
3. Вивести таблицю множення від 2 до 5.
4. Функція, яка повертає кількість дільників числа.
5. Функція, яка перевіряє, чи є введений текст числом.

Варіант 24

1. Порахувати кількість символів у слові без урахування літери 'a'.
2. Визначити, чи є введений час (години та хвилини) коректним.
3. Вивести всі числа від A до B, що є парними та додатними.
4. Функція, яка об'єднує два рядки через пробіл.
5. Процедура, яка малює трикутник із цифр.

Варіант 25

1. Знайти добуток усіх чисел від 1 до 10.
2. Визначити, чи є перша і остання цифри числа однаковими.
3. Порахувати кількість слів, що починаються на літеру 'S'.
4. Функція, яка повертає середнє арифметичне цифр числа.
5. Функція, яка визначає сезон за назвою місяця.

Контрольні запитання для звіту:

- У чому різниця між операторами if та switch?
- Які існують типи циклів у C++ та коли краще використовувати кожен із них?
- Що таке прототип функції і навіщо він потрібен?
- Чим відрізняється передача параметрів за значенням від передачі за посиланням?
- Що таке рекурсивний допоміжний алгоритм?

Приклад виконання роботи:

```
#include <iostream>
#include <iomanip>

// Допоміжний алгоритм (функція) для обчислення факторіалу
long long calculateFactorial(int n) {
    long long fact = 1;
    for (int i = 1; i <= n; i++) {
        fact *= i;
    }
    return fact;
}
```

```

// Допоміжний алгоритм (процедура/void-функція) для виведення статусу числа
void checkParity(int n) {
    if (n % 2 == 0) {
        std::cout << n << " - парне число." << std::endl;
    } else {
        std::cout << n << " - непарне число." << std::endl;
    }
}

int main() {
    int limit;
    std::cout << "Введіть верхню межу (N): ";
    std::cin >> limit;

    std::cout << "\nРезультати обчислень:\n";
    for (int i = 1; i <= limit; i++) {
        checkParity(i);
        std::cout << "Факторіал " << i << " = " << calculateFactorial(i) <<
"\n";
        std::cout << "-----\n";
    }

    return 0;
}

```

Тема роботи 6.1 : Одновимірні масиви

Мета роботи: Ознайомитися з поняттям одновимірного масиву як структурованого типу даних. Навчитися оголошувати масиви, ініціалізувати їх, вводити дані з клавіатури та виводити на екран. Опанувати базові алгоритми обробки масивів: пошук суми, максимуму/мінімуму, фільтрацію елементів та зміну їхнього порядку.

Порядок виконання роботи:

- Ознайомитися з теоретичним матеріалом щодо оголошення та індексації масивів у C++.
- Отримати номер варіанта від викладача.
- Розробити блок-схему алгоритму для одного із завдань (за вимогою викладача).
- Написати програмний код для кожного з 5 завдань свого варіанта.
- Протестувати програму на різних вхідних даних.
- Оформити звіт, що містить: титульну сторінку, мету, код програм, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Кожен варіант передбачає роботу з масивом цілих або дійсних чисел розміром $N=10$ (якщо не вказано інше).

Варіант 1

1. Знайти кількість парних елементів.
2. Обчислити добуток елементів з непарними індексами.
3. Знайти мінімальний елемент.
4. Замінити всі елементи менші за 5 числом 0.
5. Вивести масив у зворотному порядку.

Варіант 2

1. Знайти суму елементів, що діляться на 3.
2. Знайти максимальний елемент масиву.
3. Порахувати кількість від'ємних чисел.
4. Подвоїти всі додатні елементи.
5. Знайти середнє арифметичне всіх значень.

Варіант 3

1. Знайти різницю між максимальним і мінімальним елементами.
2. Вивести індекси всіх нульових елементів.
3. Обчислити суму квадратів усіх елементів.
4. Замінити від'ємні числа на їх абсолютне значення (модуль).
5. Перевірити, чи є в масиві число, введене користувачем.

Варіант 4

1. Знайти добуток усіх ненульових елементів.
2. Порахувати кількість елементів, більших за середнє арифметичне.
3. Знайти другий за величиною елемент масиву.
4. Поміняти місцями перший і останній елементи.
5. Замінити всі непарні числа на 1.

Варіант 5

1. Знайти суму елементів, що стоять на парних позиціях.
2. Визначити кількість додатних елементів.
3. Знайти індекс першого мінімального елемента.
4. Обнулити всі елементи між першим і останнім.
5. Вивести елементи, які є кратними 5.

Варіант 6

1. Обчислити середнє значення додатних чисел.
2. Знайти суму елементів масиву до першого від'ємного.
3. Знайти кількість елементів, рівних максимальному.
4. Збільшити кожен елемент на величину мінімального.
5. Вивести тільки ті числа, що входять у діапазон $[A, B]$.

Варіант 7

1. Порахувати кількість пар чисел з однаковими значеннями, що стоять поруч.
2. Знайти добуток від'ємних елементів.
3. Знайти індекс останнього максимального елемента.
4. Переставити елементи масиву в дзеркальному відображенні.
5. Замінити елементи з парними індексами на їх квадрати.

Варіант 8

1. Знайти суму модулів елементів масиву.
2. Визначити, скільки разів у масиві зустрічається число X .
3. Знайти мінімальний додатний елемент.
4. Всі додатні елементи перемістити в початок масиву.
5. Поділити всі елементи на максимальне значення.

Варіант 9

1. Знайти добуток елементів масиву, розташованих між макс. і мін.
2. Порахувати кількість чисел, що закінчуються на 3.
3. Знайти середнє арифметичне парних чисел.
4. Замінити всі від'ємні числа на середнє значення масиву.
5. Перевірити, чи масив впорядкований за зростанням.

Варіант 10

1. Знайти кількість елементів, що лежать поза заданим інтервалом.
2. Обчислити суму елементів з індексами, що є степенями двійки.
3. Знайти найбільший парний елемент.
4. Обміняти місцями сусідні пари елементів (1-й з 2-м, 3-й з 4-м...).
5. Видалити всі нулі з масиву (зсунути елементи).

Варіант 11

1. Сума елементів масиву, що стоять після мінімального.
2. Кількість елементів, значення яких більше за їх індекс.
3. Знайти найменший непарний елемент.
4. Замінити всі числа, більші за 10, на число 10.
5. Вивести масив, спочатку парні числа, потім непарні.

Варіант 12

1. Добуток додатних елементів з непарними індексами.
2. Кількість локальних мінімумів (елемент менший за обох сусідів).
3. Знайти різницю між сумою парних і сумою непарних чисел.
4. Замінити кожен елемент сумою його сусідів.
5. Знайти номер першого додатного елемента.

Варіант 13

1. Знайти середнє геометричне додатних елементів.
2. Кількість елементів, що мають парні значення та непарні індекси.
3. Знайти суму цифр усіх чисел масиву (якщо числа двозначні).
4. Замінити мінімальний елемент на протилежне число.
5. Вивести елементи, які зустрічаються в масиві лише один раз.

Варіант 14

1. Сума від'ємних елементів, що стоять на непарних місцях.
2. Кількість елементів, що дорівнюють заданому числу Z .
3. Знайти максимальний серед від'ємних чисел.
4. Зсунути всі елементи масиву циклічно вправо на одну позицію.
5. Обчислити суму елементів, що знаходяться між двома першими нулями.

Варіант 15

1. Добуток елементів масиву, які є простими числами.
2. Кількість елементів, чиї значення потрапляють у проміжок $[-5, 5]$.
3. Знайти суму значень масиву, ігноруючи максимальне і мінімальне.
4. Помножити всі елементи з непарними індексами на 3.
5. Перевірити, чи є в масиві хоча б одне від'ємне число.

Варіант 16

1. Знайти кількість чисел, що діляться на 2 і на 3 одночасно.
2. Сума квадратів від'ємних елементів.
3. Знайти індекс елемента, найближчого до середнього арифметичного.
4. Замінити всі елементи, менші за середнє, на -1.
5. Вивести елементи масиву через один (0, 2, 4...).

Варіант 17

1. Добуток ненульових елементів, що стоять після максимального.
2. Кількість парних елементів на непарних позиціях.
3. Знайти суму модулів елементів після першого від'ємного.
4. Поставити нуль на місце максимального елемента.
5. Перевірити, чи всі елементи масиву однакові.

Варіант 18

1. Сума елементів, чий індекс кратний 3.
2. Кількість додатних чисел, що стоять перед мінімальним.
3. Знайти найбільше від'ємне число.
4. Кожен елемент масиву замінити його квадратом.
5. Вивести індекси від'ємних елементів.

Варіант 19

1. Середнє арифметичне елементів, що мають непарні значення.
2. Кількість елементів, які більші за попередній елемент.
3. Знайти суму першої та другої половини масиву окремо.
4. Замінити всі додатні елементи на їх індекси.
5. Знайти добуток елементів масиву до першого нуля.

Варіант 20

1. Сума елементів масиву з непарними номерами.
2. Кількість від'ємних елементів, що стоять на парних місцях.
3. Знайти суму елементів, що знаходяться між двома від'ємними.
4. Збільшити від'ємні елементи на 5, а додатні зменшити на 5.
5. Знайти найменший за модулем елемент.

Варіант 21

1. Добуток чисел масиву, що не перевищують 10.
2. Кількість нульових елементів у масиві.
3. Знайти суму індексів усіх додатних чисел.
4. Замінити всі парні елементи на їхню половину.
5. Вивести масив у форматі: "Елемент [i] = значення".

Варіант 22

1. Сума елементів, що мають парні індекси та додатні значення.
2. Кількість елементів, що менші за останній елемент масиву.
3. Знайти добуток першого та останнього додатного елементів.
4. Замінити всі числа, що кратні 4, на 0.
5. Знайти кількість змін знака в масиві.

Варіант 23

1. Середнє арифметичне всіх елементів, крім нульових.
2. Кількість чисел, що є парними та більшими за 10.
3. Знайти суму елементів масиву до першого максимального.
4. Поміняти місцями максимальний та мінімальний елементи.
5. Перевірити, чи є в масиві дублікати.

Варіант 24

1. Добуток модулів від'ємних елементів.
2. Кількість елементів, що закінчуються на 0 або 5.
3. Знайти суму кубів додатних чисел.
4. Відняти від кожного елемента значення першого елемента.
5. Знайти номер (індекс) останнього від'ємного елемента.

Варіант 25

1. Сума елементів масиву, значення яких лежать у межах [1, 100].
2. Кількість елементів, що стоять після останнього нуля.
3. Знайти добуток елементів з парними індексами.
4. Замінити всі від'ємні елементи сумою додатних.
5. Вивести елементи масиву, які діляться на власні індекси (індекс > 0).

Контрольні запитання для звіту:

- Що таке масив і чим він відрізняється від звичайної змінної?
- З якого індексу починається нумерація елементів у C++?
- Як визначити розмір масиву, якщо він не був заданий явно через константу?
- Що станеться, якщо звернутися до елемента за межами масиву (наприклад, `arr[10]` при розмірі 10)?
- Як ініціалізувати всі елементи масиву нулями при оголошенні?

Приклад виконання роботи:

```
#include <iostream>

#include <vector>

using namespace std;

int main() {

    const int n = 10;

    double arr[n];

    double sum = 0;
```

```

// Введення даних
cout << "Enter " << n << " numbers:" << endl;
for (int i = 0; i < n; i++) {
    cin >> arr[i];
}

// Обробка масиву
for (int i = 0; i < n; i++) {
    if (arr[i] > 0) {
        sum += arr[i];
    }
    if (arr[i] < 0) {
        arr[i] = 0;
    }
}

// Виведення результатів
cout << "Sum of positive: " << sum << endl;
cout << "Modified array: ";
for (int i = 0; i < n; i++) {
    cout << arr[i] << " ";
}
return 0;
}

```

Тема роботи 6.2 : Символьні строки. Стандартний ввід-вивід. Функції для роботи зі строками. Строки в функціях і процедурах.

Мета роботи:

- Ознайомитися з особливостями представлення символьних рядків у форматі C-style (`char[]`).
- Опанувати функції стандартного вводу-виводу для рядків.
- Вивчити основні функції бібліотеки `<cstring>` (`strlen`, `strcpy`, `strcat`, `strcmp` тощо).
- Навчитися передавати рядки як аргументи у функції та процедури.

Порядок виконання роботи:

- Ознайомитися з теоретичними відомостями щодо масивів символів.
- Проаналізувати приклад виконання роботи.
- Отримати варіант завдання у викладача.
- Розробити алгоритм розв'язання задач свого варіанту.
- Написати програмний код, забезпечивши коректне розбиття на функції.
- Протестувати програму на різних вхідних даних.
- Скласти звіт, що містить код, результати роботи та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Кожен варіант містить 5 завдань. Виконання має бути реалізоване у вигляді окремих функцій, що викликаються з `main`.

Варіант 1

1. Визначити довжину найдовшого слова в рядку.
2. Замінити всі крапки на знаки оклику.
3. Підрахувати кількість цифр у тексті.
4. Перевірити, чи починається рядок з великої літери.
5. Написати функцію, що копіює рядок у зворотному порядку.

Варіант 2

1. Підрахувати кількість слів у реченні.
2. Перетворити всі літери рядка у верхній регістр.
3. Видалити всі знаки пунктуації.
4. Знайти індекс першого входження символу 'a'.
5. Об'єднати два рядки в один без використання `strcat`.

Варіант 3

1. Перевірити, чи є рядок паліндромом.
2. Замінити кожен символ символом '*'.
3. Знайти кількість слів, що починаються на 'b'.
4. Вивести рядок задом наперед.
5. Порівняти два рядки за довжиною та вмістом.

Варіант 4

1. Видалити зайві пробіли між словами (залишити по одному).
2. Порахувати кількість приголосних літер.
3. Знайти найкоротше слово.
4. Замінити слово "хто" на слово "що".
5. Визначити, скільки разів у рядку зустрічається символ 'z'.

Варіант 5

1. Перетворити кожен перший символ слова на велику.
2. Видалити всі цифри з рядка.
3. Підрахувати кількість речень у тексті (за крапками).
4. Перевірити, чи містить рядок підрядок "C++".
5. Дублювати кожен символ у рядку.

Варіант 6

1. Вивести слова, довжина яких більше 5 символів.
2. Поміняти місцями першу та останню літеру рядка.
3. Підрахувати кількість латинських літер.
4. Видалити частину рядка між дужками.
5. Перетворити рядок у формат "CamelCase".

Варіант 7

1. Визначити кількість слів, які закінчуються на 'ing'.
2. Знайти суму всіх цифр, що зустрічаються в рядку.
3. Розвернути кожне слово в реченні окремо.
4. Перевірити правильність розстановки круглих дужок.
5. Видалити всі символи, крім літер.

Варіант 8

1. Порахувати кількість великих літер у тексті.
2. Замінити всі пробіли символом підкреслення.
3. Вивести рядок вертикально (по одному символу на рядок).
4. Видалити кожне друге слово.
5. Знайти найчастіший символ у рядку.

Варіант 9

1. Перевірити, чи є рядок анаграмою іншого рядка.
2. Видалити всі голосні літери.
3. Знайти кількість слів, що складаються лише з 3 літер.
4. Замінити всі входження "1" на "one".
5. Обчислити довжину рядка без пробілів.

Варіант 10

1. Переставити слова у зворотному порядку.
2. Знайти довжину найкоротшого слова.
3. Видалити всі символи з парними індексами.
4. Перевірити, чи закінчується рядок крапкою.
5. Підрахувати кількість спеціальних символів (#, , %, &).

Варіант 11

1. Замінити всі малі літери на великі, а великі на малі.
2. Видалити повторювані символи (залишити по одному).
3. Порахувати середню довжину слова в реченні.
4. Знайти позицію підрядка в рядку без `strstr`.
5. Вивести всі слова, що містять літеру 'e'.

Варіант 12

1. Визначити кількість слів, що починаються і закінчуються на одну літеру.
2. Замінити "apple" на "orange".
3. Сформувати новий рядок лише з цифр вихідного рядка.
4. Перевірити, чи є рядок ідентифікатором (починається не з цифри).
5. Видалити останнє слово.

Варіант 13

1. Знайти кількість символів, що не є літерами чи цифрами.
2. Вивести слова у алфавітному порядку.
3. Видалити всі слова, що містять цифри.
4. Продублювати всі голосні в рядку.
5. Визначити, чи є рядок числом (цілим).

Варіант 14

1. Підрахувати кількість входжень кожного символу.
2. Видалити символи з 5-го по 10-й.
3. Знайти слово з максимальною кількістю голосних.
4. Замінити всі коми на крапку з комою.
5. Перевірити, чи є в рядку послідовність "abc".

Варіант 15

1. Стиснути рядок (наприклад, "aaabb" -> "a3b2").
2. Видалити всі слова, довжина яких менше 3.
3. Порахувати кількість символів у найдовшому слові.
4. Змінити порядок букв у кожному слові на випадковий.
5. Перевірити, чи є рядок штрих-кодом (лише 13 цифр).

Варіант 16

1. Перевірити рядок на вміст нецензурних слів (із заданого списку).
2. Вивести лише унікальні слова з тексту.
3. Замінити символи табуляції на 4 пробіли.
4. Порахувати кількість слів, написаних великими літерами.
5. Розділити рядок на два за певним символом-розділювачем.

Варіант 17

1. Перевірити, чи є рядок правильною електронною поштою (наявність @ та крапки).
2. Видалити всі літери 'x' та 'y'.
3. Знайти друге за довжиною слово.
4. Зашифрувати рядок шифром Цезаря (зсув на 3).
5. Визначити кількість пробілів на початку рядка.

Варіант 18

1. Вивести кожне слово з нового рядка.
2. Порахувати кількість знаків оклику та знаків питання.
3. Видалити всі слова, що починаються з великої літери.
4. Замінити всі подвійні пробіли одинарними.
5. Знайти суму довжин усіх слів.

Варіант 19

1. Визначити, чи є в рядку слова-дублікати.
2. Видалити всі символи, які не є пробілами.
3. Перетворити рядок у "змійку" (snake_case).
4. Підрахувати кількість слів, що закінчуються на голосну.
5. Замінити першу букву кожного слова символом ".

Варіант 20

1. Знайти найдовшу послідовність однакових символів.
2. Видалити перше входження слова "error".
3. Порахувати кількість цифр у кожному слові.
4. Перевірити, чи є рядок датою у форматі DD.MM.YYYY.
5. Вивести рядок у зворотному алфавітному порядку.

Варіант 21

1. Перетворити рядок у "kebab-case".
2. Видалити всі слова, що містять більше 2 голосних.
3. Порахувати кількість абзаців (за відступами або символом \n).
4. Замінити всі великі літери на символ '#'.
5. Знайти позицію останнього слова.

Варіант 22

1. Видалити всі знаки пунктуації в кінці кожного слова.
2. Перевірити, чи є в рядку прихований номер телефону (10 цифр).
3. Вивести слова, що складаються лише з великих літер.
4. Замінити кожен літеру на наступну за алфавітом.
5. Обчислити кількість символів між першою та останньою літерою 'a'.

Варіант 23

1. Сформувати рядок з останніх літер кожного слова.
2. Видалити всі слова, що починаються на приголосну.
3. Підрахувати кількість слів, що зустрічаються більше одного разу.
4. Замінити кожне слово "так" на "ні".
5. Перевірити, чи є рядок паліндромом без урахування регістру.

Варіант 24

1. Знайти слово з найбільшою кількістю приголосних.
2. Видалити всі символи, крім цифр та знаків '+', '-'.
3. Перетворити рядок так, щоб слова йшли за довжиною.
4. Замінити всі голосні великими літерами.
5. Порахувати кількість символів у рядку до першої крапки.

Варіант 25

1. Створити акронім з перших літер слів (наприклад, "НТУУ КПІ").
2. Видалити слова, що мають парну кількість символів.
3. Перевірити рядок на наявність лише латинських літер.
4. Замінити всі символи на їх ASCII коди через пробіл.
5. Знайти відстань між найдовшим та найкоротшим словом.

Контрольні запитання для звіту:

- Чим відрізняється символ \0 від символу '0'?
- Яка різниця між `cin >> str` та `cin.getline(str, size)`?
- Як визначити довжину рядка без використання `strlen`?
- Що станеться, якщо спробувати скопіювати довший рядок у коротший масив за допомогою `strcpy`?
- Як передати рядок у функцію так, щоб вона не могла його змінити?

Приклад виконання роботи:

```
#include <iostream>
```



```

#include <cstring>
#include <cctype>

using namespace std;

// Функція для підрахунку голосних
int countVowels(const char* str) {
    int count = 0;
    const char* vowels = "aeiouAEIOUaеіоуАЕІОУЯЄІ";
    for (int i = 0; i < strlen(str); i++) {
        if (strchr(vowels, str[i])) {
            count++;
        }
    }
    return count;
}

// Процедура для видалення пробілів (змінює рядок на місці)
void removeSpaces(char* str) {
    int n = strlen(str);
    int j = 0;
    for (int i = 0; i < n; i++) {
        if (str[i] != ' ') {
            str[j++] = str[i];
        }
    }
    str[j] = '\0'; // Встановлюємо новий термінальний нуль
}

int main() {
    char buffer[256];

    cout << "Введіть рядок: ";
    cin.getline(buffer, 256);

    cout << "Кількість голосних: " << countVowels(buffer) << endl;

    removeSpaces(buffer);
    cout << "Рядок без пробілів: " << buffer << endl;

    return 0;
}

```

Тема роботи 6.3 : Багатовимірні масиви

Мета роботи:

- Ознайомитися з особливостями оголошення та ініціалізації двовимірних масивів у C++.
- Опанувати алгоритми опрацювання елементів матриць (пошук, сортування, обчислення сум/середніх значень).
- Навчитися використовувати вкладені цикли для доступу до елементів багатовимірних структур.
- Розвинути навички алгоритмічного мислення при розв'язанні задач лінійної алгебри.

Порядок виконання роботи:

- Ознайомитися з теоретичним матеріалом щодо багатовимірних масивів.
- Отримати варіант завдання від викладача.
- Розробити блок-схему алгоритму для кожного завдання (за вимогою).
- Написати програмний код на мові C++.
- Протестувати програму на різних вхідних даних.
- Підготувати звіт, що містить код програм, результати їх виконання та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Знайти максимальний елемент у кожному рядку матриці.
2. Обчислити добуток ненульових елементів матриці.
3. Поміняти місцями перший та останній стовпці.
4. Знайти середнє арифметичне елементів побічної діагоналі.
5. Перевірити, чи є матриця симетричною відносно головної діагоналі.

Варіант 2

1. Знайти суму елементів у кожному стовпці матриці.
2. Порахувати кількість парних елементів у матриці.
3. Знайти мінімальний елемент серед тих, що лежать вище головної діагоналі.
4. Впорядкувати елементи першого рядка за зростанням.
5. Створити одномірний масив із сум елементів кожного рядка.

Варіант 3

1. Знайти індекси (рядок, стовпець) першого від'ємного елемента.
2. Обчислити суму квадратів елементів останнього рядка.
3. Транспонувати квадратну матрицю.
4. Знайти найбільший елемент серед від'ємних значень матриці.
5. Обнулити елементи, що знаходяться на перетині парних рядків та непарних стовпців.

Варіант 4

1. Знайти суму модулів елементів матриці.
2. Порахувати кількість елементів, рівних заданому числу X .
3. Вивести номери рядків, у яких немає жодного нульового елемента.
4. Поміняти місцями два рядки з індексами K та L .
5. Знайти добуток елементів побічної діагоналі.

Варіант 5

1. Знайти середнє арифметичне додатних елементів матриці.
2. Знайти мінімальний елемент у кожному стовпці.
3. Перевірити, чи містить матриця хоча б один нульовий елемент.
4. Відняти перший рядок від усіх інших рядків матриці.
5. Знайти суму елементів у "шаховому" порядку (де сума індексів парна).

Варіант 6

1. Знайти добуток від'ємних елементів матриці.
2. Знайти суму елементів, що знаходяться над побічною діагоналлю.
3. Поміняти місцями максимальний і мінімальний елементи всієї матриці.
4. Вивести кількість додатних елементів у кожному рядку.
5. Побудувати вектор, елементи якого дорівнюють найбільшим значенням відповідних стовпців.

Варіант 7

1. Знайти суму елементів, кратних числу 3.
2. Обчислити добуток елементів головної діагоналі.
3. Впорядкувати елементи кожного рядка за спаданням.
4. Знайти суму елементів у рядках, де перший елемент додатний.
5. Видалити рядок з заданим номером (зсунути інші рядки вгору).

Варіант 8

1. Знайти найбільший за модулем елемент матриці.
2. Обчислити кількість елементів у кожному рядку, що перевищують середнє значення цього рядка.
3. Поміняти місцями головну та побічну діагоналі.
4. Знайти суму парних елементів під головною діагоналлю.
5. Сформувати масив з елементів матриці, що лежать в інтервалі $[A, B]$.

Варіант 9

1. Обчислити суму елементів по периметру матриці.
2. Знайти індекс стовпця з найбільшою сумою елементів.
3. Замінити всі непарні елементи матриці на їх квадрати.
4. Знайти кількість локальних мінімумів (елементів, менших за всіх сусідів).
5. Вивести всі елементи матриці у "змісподібному" порядку.

Варіант 10

1. Знайти добуток додатних елементів вище головної діагоналі.
2. Порахувати кількість рядків, що містять хоча б один від'ємний елемент.
3. Поділити всі елементи кожного рядка на максимальний елемент цього рядка.
4. Знайти суму елементів у тих стовпцях, де є нулі.
5. Побудувати логічний вектор: true, якщо в рядку є дублікати елементів.

Варіант 11

1. Знайти суму елементів другого та передостаннього рядків.
2. Визначити, чи є матриця магічним квадратом (суми в усіх рядках, стовпцях та діагоналях рівні).
3. Замінити від'ємні елементи на 1, а додатні на 0.
4. Знайти середнє геометричне додатних елементів матриці.
5. Поміняти місцями два стовпці, що містять мінімальний та максимальний елементи.

Варіант 12

1. Знайти суму елементів з непарними обома індексами.
2. Порахувати кількість елементів, більших за суму решти елементів свого рядка.
3. Віддзеркалити матрицю відносно вертикальної осі.
4. Знайти добуток елементів у стовпцях з парними номерами.
5. Створити масив з найменших елементів діагоналей, паралельних головній.

Варіант 13

1. Знайти мінімальний елемент серед додатних.
2. Обчислити різницю між сумою елементів головної та побічної діагоналей.
3. Впорядкувати стовпці за зростанням їхніх перших елементів.
4. Знайти кількість елементів, що закінчуються цифрою 5.
5. Збільшити всі елементи матриці на величину мінімального елемента.

Варіант 14

1. Знайти суму квадратів від'ємних елементів.
2. Визначити номер рядка, у якому знаходиться найдовша послідовність однакових елементів.
3. Поміняти місцями елементи i -го рядка та i -го стовпця (транспонування за умови квадратної матриці).
4. Знайти середнє значення елементів, розташованих під побічною діагоналлю.
5. Вивести матрицю, замінивши всі парні числа на символ '*'.

Варіант 15

1. Знайти добуток ненульових елементів по периметру.
2. Визначити кількість від'ємних елементів у кожному стовпці.
3. Змістити всі елементи матриці циклічно на один рядок вниз.
4. Знайти максимальний елемент серед тих, що повторюються.
5. Перевірити, чи є всі рядки матриці впорядкованими.

Варіант 16

1. Знайти суму елементів матриці, чиї індекси складають парне число.
2. Знайти номер першого стовпця, що не містить від'ємних чисел.
3. Сортувати тільки елементи головної діагоналі.
4. Порахувати кількість "сідлових точок" (мінімум у рядку і максимум у стовпці одночасно).
5. Заповнити матрицю числами від 1 до N^2 по спіралі.

Варіант 17

1. Знайти середнє арифметичне елементів, що не лежать на діагоналях.
2. Знайти індекси всіх мінімальних елементів матриці.
3. Переставити рядки так, щоб їх суми елементів зростали.
4. Обчислити суму кубів елементів другого стовпця.
5. Знайти кількість елементів, які більші за всіх своїх правих сусідів у рядку.

Варіант 18

1. Знайти добуток елементів у рядках, що мають парний номер.
2. Знайти різницю між максимальним елементом вище діагоналі та мінімальним під нею.
3. Замінити всі елементи, менші за середнє арифметичне, на нуль.
4. Порахувати кількість стовпців, сума елементів яких від'ємна.
5. Сформувати масив з діагональних елементів (головної).

Варіант 19

1. Знайти суму елементів, що знаходяться між першим та останнім додатним елементом у кожному рядку.
2. Знайти номер рядка з найменшим добутком елементів.
3. Поміняти місцями елементи над головною та під побічною діагоналями.
4. Знайти кількість пар сусідніх однакових елементів у стовпцях.
5. Зробити всі елементи на діагоналях рівними 1, а інші — 0.

Варіант 20

1. Знайти добуток елементів, чиє значення лежить в діапазоні $[1, 10]$.
2. Вивести індекси елементів, які дорівнюють сумі своїх індексів.
3. Відняти від кожного елемента матриці середнє арифметичне всієї матриці.
4. Знайти суму найбільших елементів рядків.
5. Перевірити, чи є матриця ортогональною.

Варіант 21

1. Обчислити суму елементів матриці, крім кутових елементів.
2. Знайти кількість стовпців, що містять хоча б один нуль.
3. Впорядкувати стовпці за спаданням кількості від'ємних елементів у них.
4. Знайти максимальний елемент у "рамці" матриці товщиною 1.
5. Створити нову матрицю як результат множення початкової на саму себе.

Варіант 22

1. Знайти суму елементів i -го рядка, якщо i — парне, та добуток, якщо i — непарне.
2. Знайти кількість елементів, які є степенями двійки.
3. Замінити всі нулі значенням максимального елемента матриці.
4. Знайти суму елементів у зонах "пісочного годинника".
5. Визначити, чи є в матриці два однакові рядки.

Варіант 23

1. Знайти добуток модулів елементів нижче побічної діагоналі.
2. Знайти середнє арифметичне кожного другого стовпця.
3. Повернути матрицю на 90 градусів за годинниковою стрілкою.
4. Порахувати кількість елементів, чиї сусіди (зліва і справа) менші за них.
5. Замінити елементи останнього стовпця сумами елементів відповідних рядків.

Варіант 24

1. Знайти суму елементів, що стоять на перетині непарних рядків та непарних стовпців.
2. Знайти максимальне значення серед мінімальних значень рядків.
3. Очистити (обнулити) всі стовпці, у яких сума елементів парна.
4. Порахувати кількість від'ємних елементів на обох діагоналях.
5. Створити вектор з елементів, що трапляються в матриці лише один раз.

Варіант 25

1. Знайти добуток елементів матриці, крім елементів головної діагоналі.
2. Вивести номери стовпців, де всі елементи додатні.
3. Поміняти місцями перший рядок та рядок з максимальним елементом.
4. Знайти суму квадратів елементів вище побічної діагоналі.
5. Перевірити, чи утворюють елементи матриці арифметичну прогресію при читанні по рядках.

Контрольні запитання для звіту:

- Як оголошується двовимірний масив у C++?
- Яким чином дані розташовуються в пам'яті комп'ютера при використанні багатовимірних масивів?
- Чим відрізняється головна діагональ матриці від побічної з точки зору індексів?
- Як передати двовимірний масив у функцію як параметр?
- Які переваги та недоліки використання статичних масивів порівняно з динамічними?

Приклад виконання роботи:

```
#include <iostream>
#include <iomanip>
#include <ctime>

using namespace std;

int main() {
    srand(time(0));
    int n;
    cout << "Enter matrix size (n): ";
    cin >> n;

    int matrix[10][10]; // Оголошення статичного масиву

    // Заповнення та виведення початкової матриці
    cout << "Original Matrix:" << endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            matrix[i][j] = rand() % 21 - 10; // Діапазон [-10, 10]
            cout << setw(4) << matrix[i][j];
        }
        cout << endl;
    }

    int sumDiagonal = 0;
    for (int i = 0; i < n; i++) {
        sumDiagonal += matrix[i][i]; // Головна діагональ: індекси i == j
    }

    // Опрацювання елементів під головною діагоналлю (i > j)
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            if (i > j && matrix[i][j] < 0) {
                matrix[i][j] = 0;
            }
        }
    }

    cout << "\nSum of main diagonal: " << sumDiagonal << endl;
    cout << "Modified Matrix (negative elements below diagonal replaced):" <<
endl;
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            cout << setw(4) << matrix[i][j];
        }
        cout << endl;
    }

    return 0;
}
```

Тема роботи 6.4: Массиви символьних строк. Оголошення і ініціалізація. Ввід-вивід. Сортування

Мета роботи:

- Ознайомитися з особливостями оголошення та ініціалізації масивів символів (`char[]`).
- Отримати навички використання функцій вводу-виводу рядків.
- Навчитися реалізовувати алгоритми сортування масивів рядків.
- Вивчити основні функції бібліотеки `<cstring>` (або `<string.h>`).

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо символьних масивів (рядків у стилі C).
- Отримати варіант завдання у викладача.
- Розробити алгоритми для вирішення 5 задач свого варіанту.
- Написати програмний код на C++.
- Протестувати програму на різних вхідних даних.
- Підготувати звіт, що включає код програм, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Оголосити масив з 10 імен та ініціалізувати його.
2. Підрахувати загальну кількість голосних літер у введеному рядку.
3. Знайти найкоротший рядок у масиві з 5 слів.
4. Відсортувати 5 назв міст за спаданням (від Я до А).
5. Видалити всі цифри з введеного символьного рядка.

Варіант 2

1. Створити масив назв фруктів та вивести ті, що починаються на 'A'.
2. Перевернути (реверс) кожен рядок у масиві з 4 елементів.
3. Порівняти два введені рядки без урахування регістру.
4. Відсортувати список студентів за кількістю літер у прізвищі.
5. Замінити всі пробіли в рядку на символ підкреслення.

Варіант 3

1. Ввести рядок і вивести його символи через один.
2. Знайти кількість слів у реченні (розділювач — пробіл).
3. Визначити, чи є введений рядок паліндромом.
4. Відсортувати масив з 6 назв країн за алфавітом.
5. Перевести всі літери рядка у верхній регістр.

Варіант 4

1. Порахувати кількість знаків пунктуації в рядку.
2. Знайти найдовше слово серед 5 введених.
3. Злити два масиви рядків в один третій.
4. Відсортувати масив назв мов програмування за довжиною.
5. Вивести масив рядків у зворотному порядку.

Варіант 5

1. Знайти номер першого входження символу 'e' у масиві рядків.
2. Підрахувати кількість приголосних у тексті.
3. Перевірити, чи закінчується рядок на ".txt".
4. Відсортувати 5 назв тварин за останньою літерою.
5. Продублювати кожен літеру в рядку.

Варіант 6

1. Створити масив назв місяців, вивести тільки літні.
2. Видалити всі подвійні пробіли в рядку.
3. Знайти середнє арифметичне довжин 5 введених рядків.
4. Відсортувати масив прізвищ за другою літерою.
5. Замінити кожен цифру в рядку на символ '*'.

Варіант 7

1. Вивести символи рядка, які мають парні індекси.
2. Порахувати, скільки разів слово "C++" зустрічається в тексті.
3. Визначити, який рядок у масиві з 5 елементів має найбільше голосних.
4. Відсортувати список товарів за алфавітом.
5. Видалити останнє слово з речення.

Варіант 8

1. Перетворити перший символ кожного слова в реченні на велику літеру.
2. Знайти рядок-мінімум за алфавітом у масиві з 7 слів.
3. Обчислити кількість символів, що не є літерами чи цифрами.
4. Відсортувати масив назв планет за зростанням довжини.
5. Вставити символ '!' після кожного речення в масиві рядків.

Варіант 9

1. Вивести рядок вертикально (кожен символ з нового рядка).
2. Порівняти довжину першого та останнього рядка масиву.
3. Знайти кількість великих літер у тексті.
4. Відсортувати масив кольорів за алфавітом.
5. Видалити всі голосні з рядка.

Варіант 10

1. Перевірити, чи містить рядок тільки цифри.
2. Склеїти (конкатенація) 3 коротких рядка в один довгий.
3. Знайти слово з максимальною кількістю літер 'o'.
4. Відсортувати список марок автомобілів за алфавітом.
5. Поміняти місцями перший і останній символи в рядку.

Варіант 11

1. Порахувати кількість речень у тексті (за крапками).
2. Вивести масив рядків, замінюючи 'a' на 'A'.
3. Знайти найдовший рядок і вивести його індекс.
4. Відсортувати назви професій за кількістю голосних.
5. Видалити всі латинські літери з рядка.

Варіант 12

1. Визначити частоту появи символу 's' у масиві рядків.
2. Перевірити, чи є введений рядок анаграмою іншого рядка.
3. Вивести рядки масиву, довжина яких більша за 10.
4. Відсортувати масив імен за спаданням довжини.
5. Скопіювати один масив рядків в інший без використання `strcpy`.

Варіант 13

1. Підрахувати кількість слів, що починаються на велику літеру.
2. Створити масив назв днів тижня, вивести тільки вихідні.
3. Визначити сумарну кількість символів у масиві з 5 рядків.
4. Відсортувати масив назв хім. елементів за алфавітом.
5. Видалити всі пробіли на початку та в кінці рядка.

Варіант 14

1. Знайти в масиві рядків ті, що містять хоча б одну цифру.
2. Перетворити рядок у формат "camelCase" (видалити пробіли, наступна літера велика).
3. Порахувати кількість знаків '+' та '-' у виразі.
4. Відсортувати список фільмів за алфавітом у зворотньому порядку.
5. Замінити кожне слово "bad" на "good".

Варіант 15

1. Вивести всі слова рядка, довжина яких дорівнює 3.
2. Знайти кількість входжень підрядка в рядок.
3. Перевірити, чи є рядок дзеркальним (як "123321").
4. Відсортувати масив назв квітів за алфавітом.
5. Видалити з рядка всі символи, що не є пробілами.

Варіант 16

1. Порахувати кількість слів у реченні, що закінчуються на 'у'.
2. Знайти індекс першого рядка, що починається з цифри.
3. Вивести символи рядка в алфавітному порядку.
4. Відсортувати 5 прізвищ авторів за алфавітом.
5. Додати в кінець кожного рядка масиву символ '*'.

Варіант 17

1. Створити масив назв предметів, вивести ті, де більше 2 складів (умовно).
2. Порахувати кількість слів, що складаються лише з великих літер.
3. Знайти найкоротше слово в реченні.
4. Відсортувати масив назв сузір'їв за алфавітом.
5. Видалити з рядка всі символи з кодом ASCII менше 48.

Варіант 18

1. Визначити, чи містить масив рядків дублікати.
2. Замінити всі малі літери на великі, а великі на малі.
3. Порахувати суму цифр, що зустрічаються в рядку.
4. Відсортувати список назв страв за довжиною назви.
5. Видалити кожне друге слово в реченні.

Варіант 19

1. Вивести рядки, в яких кількість голосних дорівнює кількості приголосних.
2. Об'єднати всі рядки масиву в один через кому.
3. Знайти найдовшу послідовність однакових символів у рядку.
4. Відсортувати масив назв річок за алфавітом.
5. Замінити всі крапки на знаки оклику.

Варіант 20

1. Перевірити, чи є в рядку символи '(', ')', і чи вони збалансовані.
2. Вивести масив рядків у порядку зростання кількості пробілів у них.
3. Підрахувати кількість слів, що мають парну довжину.
4. Відсортувати список назв гір за висотою (якщо висота — частина рядка).
5. Видалити всі розділові знаки.

Варіант 21

1. Вивести масив рядків, де кожен рядок записаний задом наперед.
2. Знайти слово з найбільшою кількістю унікальних символів.
3. Порахувати кількість входжень символу, введенного користувачем.
4. Відсортувати масив назв музичних інструментів за алфавітом.
5. Видалити з тексту всі слова коротші за 3 символи.

Варіант 22

1. Створити масив назв валют, вивести ті, що мають 3 символи (код).
2. Перевірити, чи починається рядок і закінчується одним і тим самим символом.
3. Визначити позицію найдовшого слова в реченні.
4. Відсортувати масив назв геометричних фігур за алфавітом.
5. Замінити всі цифри на їх назви (1 -> "one").

Варіант 23

1. Порахувати кількість слів, що містять літеру 'х'.
2. Вивести масив рядків, видаливши з кожного рядка перший символ.
3. Знайти рядок з мінімальною кількістю голосних.
4. Відсортувати назви міст за кількістю літер 'а'.
5. Розділити один довгий рядок на масив підрядків за розділювачем ';'.

Варіант 24

1. Визначити, чи є рядок коректним e-mail (містить @ та крапку).
2. Порахувати кількість слів у реченні, довжина яких > 5.
3. Вивести масив рядків у порядку спадання ASCII-коду їх перших символів.
4. Відсортувати масив назв дерев за алфавітом.
5. Видалити всі символи, що стоять на непарних позиціях.

Варіант 25

1. Знайти найдовший спільний префікс у двох рядках.
2. Вивести масив рядків, де кожне слово починається з великої літери.
3. Порахувати кількість слів, що збігаються з першим словом речення.
4. Відсортувати назви вітамінів (A, B1, C...) за алфавітом.
5. Замінити кожну літеру 'e' на '3', а 'a' на '4'.

Контрольні запитання для звіту:

- Чим відрізняється масив символів від звичайного числового масиву?
- Яка роль символу \0 у рядках?
- Яка функція використовується для порівняння двох рядків у стилі C?
- Як зчитати рядок, що містить пробіли?
- Яка різниця між sizeof() та strlen() при роботі з рядками?

Приклад виконання роботи:

```
#include <iostream>
#include <cstring>
#include <algorithm>

using namespace std;

int main() {
    const int N = 5;
    const int MAX_LEN = 50;
    char names[N][MAX_LEN];

    cout << "Введіть " << N << " прізвищ:" << endl;
    for (int i = 0; i < N; i++) {
        cout << i + 1 << ": ";
```

```

        cin >> names[i];
    }

    // Сортування методом бульбашки
    for (int i = 0; i < N - 1; i++) {
        for (int j = 0; j < N - i - 1; j++) {
            if (strcmp(names[j], names[j + 1]) > 0) {
                char temp[MAX_LEN];
                strcpy(temp, names[j]);
                strcpy(names[j], names[j + 1]);
                strcpy(names[j + 1], temp);
            }
        }
    }

    cout << "\nВідсортований список:" << endl;
    for (int i = 0; i < N; i++) {
        cout << names[i] << endl;
    }

    return 0;
}

```

Тема роботи 7.1: Структури.

Мета роботи:

- Ознайомитися з поняттям структур у мові C++.
- Навчитися створювати власні типи даних для опису складних об'єктів.
- Отримати навички роботи з масивами структур, функціями та базовими операціями введення-виведення даних.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо оголошення структур та доступу до їх полів.
- Отримати варіант завдання у викладача.
- Розробити алгоритм розв'язання задачі.
- Написати програмний код мовою C++, використовуючи структури.
- Протестувати програму на різних вхідних даних.
- Оформити звіт, що містить код програми та скріншоти результатів роботи.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

У кожному варіанті необхідно:

- Описати структуру згідно з предметною областю.
- Створити масив структур (мінімум 5 елементів).
- Реалізувати функції пошуку, фільтрації та виведення даних.

Варіант 1: Склад магазину

1. Створити структуру **Product** (назва, ціна, кількість, термін придатності).
2. Вивести товари, ціна яких перевищує 500 грн.
3. Обчислити загальну вартість усіх товарів на складі.
4. Знайти товар з найменшою кількістю.
5. Вивести список товарів, термін придатності яких закінчується через 3 дні.

Варіант 2: Бібліотека

1. Створити структуру **Book** (автор, назва, рік видання, кількість сторінок).
2. Вивести книги певного автора (вводиться з клавіатури).
3. Знайти найдавнішу книгу в списку.
4. Порахувати загальну кількість сторінок у всіх книгах разом.
5. Відсортувати книги за роком видання.

Варіант 3: Кадровий відділ

1. Створити структуру Employee (ПІБ, посада, зарплата, стаж роботи).
2. Вивести співробітників зі стажем понад 5 років.
3. Розрахувати середню зарплату в компанії.
4. Знайти працівника з найвищою зарплатою.
5. Вивести список працівників на посаді "Менеджер".

Варіант 4: Автосалон

1. Створити структуру Car (марка, модель, рік випуску, об'єм двигуна).
2. Вивести автомобілі випущені після 2020 року.
3. Знайти машину з найбільшим об'ємом двигуна.
4. Порахувати кількість автомобілів конкретної марки.
5. Вивести список авто, відсортованих за маркою.

Варіант 5: Спортивна команда

1. Створити структуру Athlete (прізвище, вид спорту, вік, кількість нагород).
2. Вивести спортсменів, яким менше 20 років.
3. Знайти спортсмена з найбільшою кількістю нагород.
4. Порахувати середній вік команди.
5. Вивести список спортсменів у заданому виді спорту.

Варіант 6: Аптека

1. Створити структуру Medicine (назва, виробник, ціна, дозування).
2. Вивести ліки від конкретного виробника.
3. Знайти найдешевші ліки.
4. Порахувати кількість препаратів дорожче 200 грн.
5. Вивести список ліків у порядку зростання ціни.

Варіант 7: Авіарейси

1. Створити структуру Flight (номер рейсу, пункт призначення, час вильоту, ціна квитка).
2. Вивести всі рейси до вказаного міста.
3. Знайти рейс з найнижчою ціною квитка.
4. Порахувати кількість рейсів, що вилітають вранці (до 12:00).
5. Вивести список рейсів, відсортованих за часом вильоту.

Варіант 8: Відеотека

1. Створити структуру Movie (назва, режисер, жанр, тривалість у хвилинах).
2. Вивести всі фільми жанру "Бойовик".
3. Знайти найдовший фільм.
4. Обчислити загальну тривалість усіх фільмів.
5. Пошук фільму за назвою.

Варіант 9: Студентський деканат

1. Створити структуру `Course` (назва предмету, викладач, кількість годин, тип контролю).
2. Вивести предмети, де годин більше 40.
3. Порахувати кількість іспитів у семестрі.
4. Знайти предмети конкретного викладача.
5. Відсортувати предмети за кількістю годин.

Варіант 10: Нерухомість

1. Створити структуру `Apartment` (кількість кімнат, площа, поверх, ціна).
2. Вивести квартири на 1-му та останньому поверхах.
3. Знайти квартиру з найбільшою площею.
4. Розрахувати вартість 1 кв. метра для кожної квартири.
5. Пошук квартир за ціною, нижчою за вказану.

Варіант 11: Готель

1. Створити структуру `HotelRoom` (номер, тип (стандарт/люкс), кількість місць, ціна за добу).
2. Вивести всі вільні номери типу "люкс".
3. Знайти номер з мінімальною ціною.
4. Порахувати загальну кількість спальних місць у готелі.
5. Відсортувати номери за ціною.

Варіант 12: Комп'ютерний магазин

1. Створити структуру `Component` (назва, тип (CPU/GPU/RAM), частота, ціна).
2. Вивести всі відеокарти (GPU).
3. Знайти компонент з найбільшою частотою.
4. Обчислити середню ціну оперативної пам'яті.
5. Вивести список деталей у порядку спадання ціни.

Варіант 13: Зоопарк

1. Створити структуру `Animal` (вид, ім'я, вік, раціон).
2. Вивести тварин, яким понад 10 років.
3. Порахувати кількість тварин певного виду.
4. Знайти наймолодшу тварину.
5. Пошук тварини за іменем.

Варіант 14: Розклад потягів

1. Створити структуру **Train** (номер потяга, маршрут, час прибуття, кількість зупинок).
2. Вивести потяги, що мають понад 10 зупинок.
3. Пошук потяга за номером.
4. Вивести маршрути, що проходять через вказану станцію.
5. Відсортувати потяги за часом прибуття.

Варіант 15: Музичний альбом

1. Створити структуру **Song** (назва, виконавець, тривалість, альбом).
2. Вивести пісні тривалістю понад 4 хвилини.
3. Порахувати загальну тривалість альбому.
4. Знайти всі пісні певного виконавця.
5. Відсортувати пісні за назвою.

Варіант 16: Ресторанне меню

1. Створити структуру **Dish** (назва, вага порції, калорійність, ціна).
2. Вивести страви калорійністю менше 300 ккал.
3. Знайти найдорожчу страву в меню.
4. Обчислити середню вартість страви.
5. Вивести страви вагою понад 400г.

Варіант 17: Туристична агенція

1. Створити структуру **Tour** (країна, тривалість у днях, тип харчування, вартість).
2. Вивести тури до вказаної країни.
3. Знайти тур з максимальною тривалістю.
4. Порахувати кількість турів з харчуванням "All Inclusive".
5. Відсортувати тури за вартістю.

Варіант 18: Смартфони

1. Створити структуру **Phone** (бренд, модель, об'єм пам'яті, ємність акумулятора).
2. Вивести моделі з пам'яттю 256 ГБ і більше.
3. Знайти телефон з найпотужнішим акумулятором.
4. Порахувати кількість смартфонів конкретного бренду.
5. Пошук моделі за назвою.

Варіант 19: Шкільний журнал

1. Створити структуру **Pupil** (прізвище, клас, оцінка з математики, оцінка з фізики).
2. Вивести учнів, які мають 12 балів з будь-якого предмету.
3. Обчислити середній бал класу з математики.
4. Знайти учня з найвищим середнім балом.
5. Вивести список учнів вказаного класу.

Варіант 20: Поліклініка

1. Створити структуру `Patient` (ПІБ, вік, діагноз, дата прийому).
2. Вивести список пацієнтів з конкретним діагнозом.
3. Порахувати кількість пацієнтів старше 60 років.
4. Знайти пацієнта, який був на прийомі останнім.
5. Відсортувати пацієнтів за віком.

Варіант 21: Інтернет-провайдер

1. Створити структуру `Tariff` (назва, швидкість, об'єм трафіку, абонплата).
2. Вивести тарифи зі швидкістю понад 100 Мбіт/с.
3. Знайти найдешевший тариф.
4. Порахувати кількість безлімітних тарифів.
5. Відсортувати тарифи за швидкістю.

Варіант 22: Кафедра університету

1. Створити структуру `Lecturer` (ПІБ, науковий ступінь, кількість публікацій, зарплата).
2. Вивести викладачів зі ступенем "Професор".
3. Знайти викладача з найбільшою кількістю публікацій.
4. Розрахувати загальну суму виплат кафедри.
5. Вивести список викладачів у алфавітному порядку.

Варіант 23: Служба доставки

1. Створити структуру `Parcel` (номер замовлення, вага, вартість доставки, статус).
2. Вивести посилки вагою понад 20 кг.
3. Знайти найдорожчу доставку.
4. Порахувати кількість доставлених посилок (статус "Delivered").
5. Пошук за номером замовлення.

Варіант 24: Метеостанція

1. Створити структуру `Weather` (дата, температура, вологість, швидкість вітру).
2. Вивести дні, коли температура була нижчою за 0 градусів.
3. Знайти день з найвищою швидкістю вітру.
4. Обчислити середню температуру за тиждень.
5. Вивести дні з вологістю понад 90%.

Варіант 25: Книжковий магазин

1. Створити структуру `Author` (ім'я, країна, кількість виданих книг, основний жанр).
2. Вивести авторів, які написали понад 50 книг.
3. Знайти автора з найбільшою кількістю книг.
4. Вивести авторів з вказаної країни.
5. Пошук за основним жанром.

Контрольні запитання для звіту:

- Що таке структура в C++ і чим вона відрізняється від масиву?

- Як здійснюється доступ до елементів (полів) структури?
- Чи можна використовувати структури як елементи масивів?
- Яка різниця між передачею структури у функцію за значенням та за посиланням?
- Що таке вкладені структури?

Приклад виконання роботи:

```
#include <iostream>
#include <string>
#include <vector>

using namespace std;

// Оголошення структури
struct Student {
    string lastName;
    string group;
    double averageGrade;
};

int main() {
    const int N = 3;
    Student students[N];

    // Введення даних
    for (int i = 0; i < N; i++) {
        cout << "Введіть прізвище студента " << i + 1 << ": ";
        cin >> students[i].lastName;
        cout << "Введіть групу: ";
        cin >> students[i].group;
        cout << "Введіть середній бал: ";
        cin >> students[i].averageGrade;
    }

    // Обробка та виведення
    cout << "\nСтуденти-відмінники (бал > 4.5):" << endl;
    bool found = false;
    for (int i = 0; i < N; i++) {
        if (students[i].averageGrade > 4.5) {
            cout << students[i].lastName << " (" << students[i].group << ")"
<< endl;
            found = true;
        }
    }

    if (!found) cout << "Таких студентів не знайдено." << endl;

    return 0;
}
```

Тема роботи 7.2: Масиви структур.

Мета роботи:

- Ознайомитися з поняттям структури (struct) у мові C++.
- Навчитися створювати масиви структур та працювати з ними.
- Опанувати методи введення, виведення, пошуку та сортування даних, організованих у вигляді структур.
- Розвинути навички алгоритмічного мислення при роботі з комбінованими типами даних.

Порядок виконання роботи:

- Ознайомитися з теоретичним матеріалом щодо оголошення структур та масивів структур.
- Проаналізувати приклад виконання роботи, наведений у цій інструкції.
- Отримати номер варіанта (відповідно до списку в журналі або за вказівкою викладача).
- Розробити алгоритми для вирішення 5-ти завдань свого варіанта.
- Написати програму на мові C++, що реалізує завдання варіанта.
- Протестувати програму на різних вхідних даних.
- Скласти звіт, що включає: тему, мету, код програми, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Створити структуру `Book` (назва, автор, рік). Вивести книги автора "Шевченко".
2. Знайти найстарішу книгу в масиві `Book`.
3. Порахувати кількість книг, виданих після 2010 року.
4. Відсортувати книги за назвою (алфавітно).
5. Вивести всі книги, назва яких починається на букву 'A'.

Варіант 2

1. Створити структуру `Car` (марка, колір, ціна). Вивести всі червоні машини.
2. Знайти найдорожчий автомобіль.
3. Обчислити середню вартість автомобілів у списку.
4. Вивести машини певної марки, яку вводить користувач.
5. Відсортувати масив структур за ціною (спадання).

Варіант 3

1. Створити структуру **Product** (назва, кількість, ціна за одиницю). Вивести товари, яких менше 5 одиниць.
2. Обчислити загальну вартість кожного товару (ціна * кількість).
3. Знайти товар з найменшою ціною.
4. Вивести товари, назва яких довша за 10 символів.
5. Збільшити ціну всіх товарів на 10%.

Варіант 4

1. Створити структуру **Employee** (прізвище, посада, зарплата). Вивести менеджерів.
2. Знайти працівника з найбільшою зарплатою.
3. Порахувати сумарну зарплату всіх працівників.
4. Вивести працівників, чиє прізвище починається на 'К'.
5. Змінити посаду конкретному працівнику (пошук за прізвищем).

Варіант 5

1. Створити структуру **Movie** (назва, режисер, рейтинг). Вивести фільми з рейтингом > 8.0.
2. Знайти фільм з найнижчим рейтингом.
3. Вивести всі фільми конкретного режисера.
4. Порахувати середній рейтинг усіх фільмів.
5. Відсортувати фільми за рейтингом (зростання).

Варіант 6

1. Створити структуру **City** (назва, населення, площа). Вивести міста-мільйонники.
2. Знайти місто з найбільшою площею.
3. Обчислити щільність населення для кожного міста.
4. Вивести міста, назва яких містить більше 2-х слів.
5. Відсортувати міста за кількістю населення.

Варіант 7

1. Створити структуру **Laptop** (модель, обсяг ОЗП, ціна). Вивести ноутбуки з ОЗП >= 16 ГБ.
2. Знайти найдешевший ноутбук.
3. Порахувати кількість ноутбуків дорожче 20000 грн.
4. Вивести моделі ноутбуків Apple.
5. Знизити ціну на всі моделі на 5%.

Варіант 8

1. Створити структуру **Air Lines** (номер рейсу, пункт призначення, час вильоту). Вивести рейси до "Київ".
2. Вивести всі ранкові рейси (час до 12:00).
3. Порахувати загальну кількість рейсів у списку.
4. Знайти рейс за номером.
5. Відсортувати рейси за часом вильоту.

Варіант 9

1. Створити структуру `Medicine` (назва, термін придатності, ціна). Вивести ліки, що дорожчі за 100 грн.
2. Знайти ліки, термін придатності яких закінчується у 2024 році.
3. Обчислити середню ціну ліків.
4. Вивести ліки, назва яких складається з одного слова.
5. Відсортувати за назвою.

Варіант 10

1. Створити структуру `Sport` (назва команди, кількість перемог, кількість поразок). Вивести команди без поразок.
2. Знайти команду з найбільшою кількістю перемог.
3. Обчислити відсоток перемог для кожної команди.
4. Вивести команди, у яких перемог більше ніж поразок.
5. Відсортувати за відсотком перемог.

Варіант 11

1. Створити структуру `Animal` (вид, кличка, вік). Вивести тварин віком понад 5 років.
2. Знайти наймолодшу тварину.
3. Порахувати скільки у списку "Котів".
4. Вивести тварин, чії клички починаються на "М".
5. Збільшити вік усіх тварин на 1 рік (день народження).

Варіант 12

1. Створити структуру `Subject` (назва предмету, викладач, кількість годин). Вивести предмети з обсягом > 40 годин.
2. Знайти предмет з найменшою кількістю годин.
3. Вивести предмети конкретного викладача.
4. Обчислити загальну кількість годин усіх предметів.
5. Відсортувати за кількістю годин.

Варіант 13

1. Створити структуру `Phone` (бренд, модель, заряд батареї у %). Вивести телефони, де заряд < 20%.
2. Знайти телефон з максимальним зарядом.
3. Порахувати скільки телефонів бренду "Samsung".
4. Вивести моделі, де назва бренду збігається з назвою моделі.
5. Відсортувати за брендом.

Варіант 14

1. Створити структуру `Music` (виконавець, альбом, рік випуску). Вивести альбоми випущені до 2000 року.
2. Знайти найновіший альбом.
3. Вивести всі альбоми гурту "The Beatles".
4. Порахувати кількість альбомів у масиві.
5. Відсортувати за роком випуску.

Варіант 15

1. Створити структуру `Recipe` (назва страви, час приготування, калорійність). Вивести страви швидше 30 хв.
2. Знайти найбільш калорійну страву.
3. Обчислити середню калорійність усіх страв.
4. Вивести страви, у назві яких є слово "салат".
5. Відсортувати за часом приготування.

Варіант 16

1. Створити структуру `Worker` (ім'я, стаж роботи, цех). Вивести працівників зі стажем > 10 років.
2. Знайти працівника з найменшим стажем.
3. Вивести список працівників 1-го цеху.
4. Порахувати середній стаж по підприємству.
5. Відсортувати за цехами.

Варіант 17

1. Створити структуру `Planet` (назва, відстань від Сонця, наявність атмосфери). Вивести планети з атмосферою.
2. Знайти найвіддаленішу планету.
3. Порахувати планети без атмосфери.
4. Вивести планети, назва яких закінчується на 'a'.
5. Відсортувати за відстанню.

Варіант 18

1. Створити структуру `Game` (назва, жанр, ціна). Вивести ігри жанру "RPG".
2. Знайти безкоштовні ігри (ціна = 0).
3. Обчислити сумарну вартість всієї бібліотеки ігор.
4. Вивести ігри, ціна яких у діапазоні 100-500 грн.
5. Відсортувати за жанром.

Варіант 19

1. Створити структуру `Building` (адреса, кількість поверхів, тип). Вивести будинки вище 9 поверхів.
2. Знайти найвищий будинок.
3. Порахувати кількість "цегляних" будинків.
4. Вивести будинки на вулиці "Центральна".

5. Відсортувати за поверховістю.

Варіант 20

1. Створити структуру `Journal` (назва, тираж, ціна). Вивести журнали з тиражем понад 10000.
2. Знайти журнал з найбільшою ціною.
3. Обчислити дохід від продажу всього тиражу кожного журналу.
4. Вивести журнали, назва яких складається з 3-х букв.
5. Відсортувати за тиражем.

Варіант 21

1. Створити структуру `Tool` (назва, матеріал, вага). Вивести металеві інструменти.
2. Знайти найважчий інструмент.
3. Порахувати середню вагу інструментів.
4. Вивести інструменти легше 1 кг.
5. Відсортувати за матеріалом.

Варіант 22

1. Створити структуру `Furniture` (назва, матеріал, ціна). Вивести меблі з дерева.
2. Знайти найдешевші меблі.
3. Збільшити ціну всіх меблів на 200 грн.
4. Вивести меблі, ціна яких вище середньої.
5. Відсортувати за ціною.

Варіант 23

1. Створити структуру `Driver` (прізвище, категорія, стаж). Вивести водіїв категорії "С".
2. Знайти водія з найбільшим стажем.
3. Порахувати водіїв зі стажем менше 2 років.
4. Вивести водіїв, чиє прізвище має більше 7 літер.
5. Відсортувати за прізвищем.

Варіант 24

1. Створити структуру `Event` (назва події, дата, кількість учасників). Вивести події з > 100 учасниками.
2. Знайти подію з найменшою кількістю людей.
3. Порахувати загальну кількість учасників усіх подій.
4. Вивести події, що відбудуться у грудні.
5. Відсортувати за датою (якщо дата — число).

Варіант 25

1. Створити структуру `Course` (назва, тривалість у місяцях, вартість). Вивести курси тривалістю 3 місяці.
2. Знайти найдорожчий курс.
3. Порахувати вартість одного місяця навчання для кожного курсу.
4. Вивести курси, назва яких починається на "Java".
5. Відсортувати за тривалістю.

Контрольні запитання для звіту:

- Що таке структура в C++ і чим вона відрізняється від масиву?
- Як здійснюється доступ до окремих полів структури?
- Яким чином можна ініціалізувати масив структур при його оголошенні?
- Чи можна використовувати структуру як поле іншої структури (вкладені структури)?
- Як передати масив структур у функцію?

Приклад виконання роботи:

```
#include <iostream>
#include <string>
#include <vector>
#include <windows.h> // Для підтримки кирилиці в консолі

using namespace std;

// Оголошення структури
struct Student {
    string surname;
    string group;
    double averageScore;
};

int main() {
    // Налаштування кодування для виводу українською
    SetConsoleCP(1251);
    SetConsoleOutputCP(1251);

    const int N = 3;
    Student students[N]; // Масив структур

    // Введення даних
    cout << "Введіть дані про " << N << " студентів:" << endl;
    for (int i = 0; i < N; i++) {
        cout << "Студент #" << i + 1 << endl;
        cout << "Прізвище: ";
        cin >> students[i].surname;
        cout << "Група: ";
        cin >> students[i].group;
        cout << "Середній бал: ";
        cin >> students[i].averageScore;
    }

    // Виведення результатів
    cout << "\nСтуденти з балом вище 4.0:" << endl;
    bool found = false;
    for (int i = 0; i < N; i++) {
        if (students[i].averageScore > 4.0) {
            cout << students[i].surname << " (Група: " << students[i].group
            << ") - " << students[i].averageScore << endl;
            found = true;
        }
    }
}
```

```
    }  
}  
  
if (!found) {  
    cout << "Таких студентів не знайдено." << endl;  
}  
return 0;  
}
```

Тема роботи 8.1: Оголошення і ініціалізація змінної покажчика

Мета роботи: Ознайомитися з концепцією покажчиків у мові програмування C++. Навчитися оголошувати змінні-покажчики, ініціалізувати їх адресами існуючих змінних, використовувати операції розіменування (*) та взяття адреси (&), а також виконувати базові операції з пам'яттю.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо покажчиків та адресації.
- Проаналізувати наведений приклад виконання роботи.
- Відповідно до свого варіанта (номер у списку групи), виконати 5 практичних завдань.
- Написати програмний код для кожного завдання.
- Оформити звіт, що містить: титульну сторінку, код програм, скріншоти результатів виконання та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Кожне завдання передбачає: оголошення відповідних змінних, ініціалізацію покажчиків, вивід адрес та значень, виконання операцій згідно з умовою.

Варіант 1

1. Створіть `double` змінну та покажчик на неї. Виведіть адресу.
2. Збільште ціле число на 10, використовуючи лише покажчик.
3. Створіть два покажчики на одну змінну типу `char`.
4. Знайдіть суму двох чисел через їх покажчики.
5. Ініціалізуйте покажчик значенням `nullptr` і перевірте його в `if`.

Варіант 2

1. Оголосіть `int` змінну та виведіть її розмір у байтах через покажчик.
2. Поміняйте місцями значення двох `float` змінних, використовуючи покажчики.
3. Створіть покажчик на константу.
4. Обчисліть добуток трьох чисел через покажчики.
5. Створіть масив з 3 елементів і покажчик на його перший елемент.

Варіант 3

1. Створіть покажчик на тип `bool`. Виведіть логічне значення через покажчик.
2. Відніміть 5 від змінної через розіменування покажчика.
3. Виведіть адресу покажчика (покажчик на покажчик).
4. Порівняйте дві адреси покажчиків у консолі.
5. Присвойте покажчику адресу другого елемента масиву.

Варіант 4

1. Створіть змінну `long` та ініціалізуйте її через покажчик.
2. Обчисліть середнє арифметичне двох чисел, звертаючись до них через покажчики.
3. Виведіть шістнадцяткове представлення адреси змінної.
4. Створіть масив символів та покажчик на його початок.
5. Оголосіть порожній покажчик і надайте йому адресу динамічної змінної.

Варіант 5

1. Оголосіть `short` змінну та покажчик. Змініть її значення на протилежне.
2. Перевірте, чи два покажчики вказують на одну й ту саму адресу.
3. Виведіть значення символьного масиву посимвольно через покажчик.
4. Обчисліть квадрат числа через покажчик.
5. Продемонструйте різницю між `&` (адреса) та `*` (значення).

Варіант 6

1. Створіть покажчик на структуру (наприклад, "Точка" з `x`, `y`).
2. Напишіть програму, де покажчик змінює значення глобальної змінної.
3. Використайте оператор `sizeof` для покажчика на `char` та `double`. Порівняйте.
4. Створіть константний покажчик на змінну.
5. Знайдіть мінімальне з двох чисел через їх адреси.

Варіант 7

1. Створіть масив цілих чисел і виведіть третій елемент за допомогою покажчика.
2. Збільште значення змінної `int` вдвічі через покажчик.
3. Оголосіть покажчик на покажчик на `float`.
4. Створіть змінну типу `unsigned int` та покажчик на неї.
5. Виведіть адресу кожного елемента масиву `int[5]` через покажчик.

Варіант 8

1. Ініціалізуйте покажчик на `char` буквою 'A'. Змініть на 'Z' через покажчик.
2. Обчисліть периметр прямокутника (сторони `a`, `b`), використовуючи покажчики.
3. Продемонструйте ініціалізацію покажчика нулем.
4. Створіть два числа. Виведіть те, чия адреса більша.
5. Використайте `void*` для зберігання адреси `int` (з приведенням типів).

Варіант 9

1. Створіть змінну `double` та два покажчики, що вказують на неї.
2. Виведіть значення змінної через "покажчик на покажчик".
3. Додайте до змінної через покажчик значення іншої змінної.
4. Виведіть адресу локальної змінної всередині функції через покажчик.
5. Використайте покажчик для ініціалізації елемента масиву користувачем.

Варіант 10

1. Опишіть роботу оператора `&` на прикладі трьох різних типів даних.
2. Створіть покажчик на масив структур "Студент" (ім'я, бал).
3. Змініть значення `bool` з `true` на `false` через адресу.
4. Обчисліть об'єм куба через покажчик на сторону.
5. Виведіть адресу першого та останнього елемента масиву.

Варіант 11

1. Створіть динамічну змінну за допомогою `new` та покажчик на неї.
2. Порівняйте значення двох змінних, звертаючись до них через покажчики.
3. Змініть символ у рядку (масиві `char`) через покажчик.
4. Виведіть адресу системної функції (наприклад, `main`).
5. Створіть масив покажчиків на цілі числа.

Варіант 12

1. Створіть покажчик на `long long` та ініціалізуйте великим числом.
2. Поділіть одне число на інше, використовуючи лише їх покажчики.
3. Виведіть розмір пам'яті, який займає сам покажчик (не змінна).
4. Напишіть програму для обміну значень `char` змінних через покажчики.
5. Передайте адресу змінної у функцію для зміни її значення.

Варіант 13

1. Створіть змінну типу `float`. Оголосіть покажчик. Присвойте йому адресу.
2. Виведіть результат логічного виразу (`ptr == &var`).
3. Знайдіть залишок від ділення двох чисел через покажчики.
4. Продемонструйте арифметику покажчиків: `ptr + 1`.
5. Створіть покажчик на константний рядок.

Варіант 14

1. Оголосіть масив з 10 елементів та виведіть парні елементи через покажчик.
2. Створіть покажчик на `int`. Присвойте йому адресу елемента масиву.
3. Обчисліть факторіал числа 5, використовуючи покажчик у циклі.
4. Створіть константну змінну та спробуйте створити на неї звичайний покажчик (поясніть помилку).
5. Виведіть адресу масиву двома способами.

Варіант 15

1. Створіть покажчик на структуру "Координати" (`x, y, z`).
2. Виконайте операцію `ptr++` для покажчика на масив `double`. Що змінилося?
3. Обчисліть суму елементів масиву через покажчик.
4. Використайте покажчик для введення даних у змінну через `cin`.
5. Створіть два покажчики і поміняйте їх адреси (не значення змінних).

Варіант 16

1. Оголосіть `int a = 5; int* p = &a;`. Виведіть `*p + 1` та `*(p + 1)`. Поясніть різницю.
2. Створіть динамічний масив через покажчик.
3. Виведіть значення `char` через покажчик на `int` (приведення типів).
4. Обчисліть довжину кола через покажчик на радіус.
5. Знайдіть адресу максимального елемента в масиві з 3 чисел.

Варіант 17

1. Створіть покажчик на `short`. Присвойте йому адресу `static` змінної.
2. Перевірте, чи дорівнює значення за покажчиком нулю.
3. Створіть масив покажчиків на рядки (імена друзів).
4. Виведіть різницю між двома адресами елементів масиву.
5. Змініть значення другого поля структури через покажчик на структуру.

Варіант 18

1. Продемонструйте ініціалізацію покажчика адресою іншого покажчика.
2. Створіть програму, що виводить таблицю множення на 5 через покажчик.
3. Використайте `const int *ptr` та спробуйте змінити значення.
4. Знайдіть суму квадратів двох чисел через покажчики.
5. Виведіть адресу першого елемента масиву та адресу самого масиву.

Варіант 19

1. Створіть покажчик на тип `unsigned char`.
2. Використайте покажчик для обходу масиву у зворотному порядку.
3. Обчисліть площу трикутника через покажчики на основу та висоту.
4. Присвойте покажчику результат функції, що повертає адресу.
5. Створіть "порожній" покажчик і пізніше ініціалізуйте його.

Варіант 20

1. Оголосіть масив та покажчик. Використовуйте нотацію `ptr[i]` замість `*(ptr+i)`.
2. Створіть покажчик на структуру "Дата" (день, місяць, рік).
3. Виведіть адресу змінної в десятковому форматі (приведення до `long`).
4. Поміняйте два символи в слові через покажчики.
5. Створіть покажчик на `void` і приведіть його до `float*`.

Варіант 21

1. Оголосіть змінну `int x = 10` та покажчик `p`. Зробіть `*p = *p * *p`.
2. Виведіть адреси трьох послідовно оголошених змінних.
3. Створіть масив `double` і покажчик на його середину.
4. Обчисліть суму цифр двозначного числа через покажчик.
5. Створіть константний покажчик на константні дані.

Варіант 22

1. Створіть покажчик на `int`. Виведіть значення, адресу та адресу самого покажчика.
2. Напишіть функцію, яка приймає покажчик і обнуляє змінну.
3. Виведіть кожен байт числа `int` окремо через `char*`.
4. Знайдіть середнє значення елементів масиву через арифметику покажчиків.
5. Створіть два числа. Якщо перше більше, змініть його через покажчик.

Варіант 23

1. Створіть структуру "Книга" (назва, ціна) та покажчик на неї.
2. Виведіть адресу нульового елемента масиву двома різними способами.
3. Збільште покажчик на масив `float` на 2 позиції. Виведіть значення.
4. Обчисліть гіпотенузу трикутника через покажчики на катети.
5. Перевірте через покажчик, чи є число парним.

Варіант 24

1. Створіть покажчик на тип `long double`.
2. Використайте покажчик для заміни всіх пробілів у рядку на підкреслення.
3. Порівняйте значення за двома різними покажчиками.
4. Оголосіть покажчик всередині циклу `for`.
5. Продемонструйте видалення динамічної пам'яті через `delete` після використання покажчика.

Варіант 25

1. Створіть масив з 5 значень. Виведіть адреси елементів у циклі.
2. Оголосіть покажчик на функцію (опційно) або покажчик на змінну `size_t`.
3. Обчисліть потужність (через покажчики на напругу та струм).
4. Створіть два покажчика на різні змінні та спробуйте їх відняти.
5. Змініть значення константи через "брудний хак" з покажчиком (приведення типів).

Контрольні запитання для звіту:

- Що таке покажчик і що він зберігає?
- Яка різниця між операторами `*` (при оголошенні) та `*` (при використанні в коді)?
- Що таке оператор взяття адреси `&`?
- Скільки байтів займає покажчик на тип `char` та покажчик на тип `double` у 64-бітній системі?
- Чим небезпечне використання неініціалізованих покажчиків?
- Що таке `nullptr` і чому його варто використовувати?
- Як змінити значення змінної, маючи лише покажчик на неї?

Приклад виконання роботи:

```
#include <iostream>
```

```

int main() {
    // 1. Оголошення звичайної змінної
    int number = 42;

    // 2. Оголошення та ініціалізація покажчика адресою змінної number
    int* ptr = &number;

    std::cout << "Значення number: " << number << std::endl;
    std::cout << "Адреса number (&number): " << &number << std::endl;
    std::cout << "Значення покажчика ptr (адреса): " << ptr << std::endl;
    std::cout << "Значення за адресою, яку зберігає ptr (*ptr): " << *ptr <<
std::endl;

    // 3. Зміна значення змінної через покажчик (розіменування)
    *ptr = 100;

    std::cout << "\nПісля зміни через покажчик:" << std::endl;
    std::cout << "Нове значення number: " << number << std::endl;

    return 0;
}

```


Тема роботи 8.2: Масиви покажчиків. Покажчики на функцію.

Мета роботи:

- Оволодіти навичками роботи з покажчиками на різні типи даних.
- Навчитися створювати та використовувати масиви покажчиків.
- Ознайомитися з концепцією покажчиків на функції як засобу реалізації функціонального поліморфізму та створення гнучких інтерфейсів програм.

Порядок виконання роботи:

- Вивчити теоретичний матеріал за темою.
- Отримати варіант завдання у викладача.
- Розробити алгоритми для вирішення кожної з 5 задач варіанту.
- Написати програмний код мовою C++, використовуючи сучасні стандарти (C++11/14/17/20).
- Протестувати програму на різних вхідних даних.
- Підготувати звіт, що включає код, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

У кожному варіанті необхідно реалізувати 5 задач, використовуючи масиви покажчиків або покажчики на функції де це доречно.

Варіант 1

1. Створити масив покажчиків на рядки та відсортувати їх за алфавітом.
2. Реалізувати функцію обчислення інтегралу методом прямокутників, передаючи функцію як аргумент.
3. Знайти максимальний елемент у масиві, використовуючи покажчик на масив.
4. Створити меню програми через масив покажчиків на функції.
5. Написати функцію, що приймає покажчик на масив структур "Студент" і фільтрує їх за оцінкою.

Варіант 2

1. Створити масив покажчиків на цілі числа та знайти суму значень, на які вони вказують.
2. Реалізувати алгоритм пошуку кореня рівняння методом бісекції, де рівняння передається як покажчик.
3. Переставити рядки двовимірного масиву за допомогою масиву покажчиків.
4. Створити функцію зворотного виклику (callback) для обробки подій масиву.
5. Порівняти два масиви символів, використовуючи тільки покажчики.

Варіант 3

1. Динамічно виділити пам'ять під масив покажчиків на об'єкти класу "Книга".
2. Написати універсальну функцію сортування масиву, що приймає покажчик на функцію-компаратор.
3. Обчислити добуток елементів масиву, використовуючи арифметику покажчиків.
4. Створити масив покажчиків на функції, що обчислюють різні тригонометричні значення.
5. Реалізувати пошук підрядка в рядку через покажчики.

Варіант 4

1. Сформувати масив покажчиків на парні елементи вхідного числового масиву.
2. Виконати табулювання довільної функції $f(x)$ на відрізку $[a, b]$ через покажчик на функцію.
3. Реалізувати стек на базі масиву покажчиків.
4. Створити систему команд (командний рядок), де кожна команда — це функція в масиві покажчиків.
5. Перевернути рядок (реверс), використовуючи два покажчики (на початок і кінець).

Варіант 5

1. Знайти мінімальний елемент у кожному рядку матриці за допомогою масиву покажчиків.
2. Обчислити похідну функції в точці, передаючи функцію як параметр.
3. Створити масив покажчиків на константні рядки.
4. Реалізувати функцію `map`, яка застосовує задану через покажчик функцію до кожного елемента масиву.
5. Написати програму обліку товарів, використовуючи масив покажчиків на структури.

Варіант 6

1. Створити масив покажчиків на масиви різної довжини (зубчастий масив).
2. Реалізувати чисельне диференціювання функції через покажчик.
3. Написати функцію пошуку елемента в масиві, яка повертає покажчик на нього.
4. Використати масив покажчиків на функції для реалізації простого калькулятора.
5. Підрахувати кількість голосних у рядку через покажчик на символ.

Варіант 7

1. Сортування масиву структур за декількома полями (через зміну компараторів-покажчиків).
2. Реалізувати функцію `filter` для масиву чисел.
3. Вивести адресу та значення кожного елемента масиву через покажчики.
4. Створити масив покажчиків на методи класу (використовуючи `static` або звичайні функції).
5. Об'єднати два рядки без використання стандартної бібліотеки `string.h`.

Варіант 8

1. Створити масив покажчиків на рядки-назви місяців та вивести їх.
2. Обчислити суму ряду, де загальний член ряду задається функцією через покажчик.
3. Знайти середнє арифметичне елементів масиву через арифметику покажчиків.
4. Реалізувати паттерн "Стратегія" через покажчики на функції.
5. Визначити довжину рядка, не використовуючи `str len`.

Варіант 9

1. Реалізувати двовимірний масив як масив покажчиків на рядки.
2. Написати функцію, що знаходить екстремуми (`min/max`) довільної функції.
3. Перевірити, чи є масив симетричним, використовуючи покажчики.
4. Побудувати таблицю істинності для логічних функцій, переданих через покажчик.
5. Скопіювати один масив в інший за допомогою покажчиків.

Варіант 10

1. Створити масив покажчиків на функції обробки тексту (`caps`, `lower`, `reverse`).
2. Обчислити площу фігури під кривою (метод трапецій).
3. Виділити пам'ять під масив структур та заповнити його через покажчик.
4. Реалізувати диспетчер задач, де задачі — функції в масиві покажчиків.
5. Знайти перше входження символу в рядок за допомогою покажчика.

Варіант 11

1. Сортування масиву цілих чисел за допомогою покажчика на функцію порівняння (зростання/спадання).
2. Реалізувати функцію для знаходження суми елементів масиву, що відповідають певній умові (умова через покажчик).
3. Створити масив покажчиків на рядки і замінити в них всі пробіли на підкреслення.
4. Реалізувати функцію, яка повертає покажчик на найбільший рядок у масиві покажчиків.
5. Написати програму для знаходження суми двох матриць, використовуючи покажчики на елементи.

Варіант 12

1. Створити динамічний масив покажчиків на структури "Автомобіль".
2. Обчислити значення складного виразу, де окремі математичні функції передаються як аргументи.
3. Реалізувати циклічний зсув елементів масиву через покажчики.
4. Створити меню для керування базою даних студентів за допомогою масиву покажчиків на функції.
5. Перевірити, чи є рядок паліндромом, використовуючи покажчики на початок та кінець.

Варіант 13

1. Створити масив покажчиків на функції, що виконують різні типи сортування (bubble, insertion, selection).
2. Реалізувати функцію `reduce` (або `accumulate`), яка приймає операцію через покажчик.
3. Знайти кількість входжень певного числа в масив через покажчики.
4. Створити масив покажчиків на функції для малювання різних геометричних фігур (символами в консолі).
5. Виділити всі слова в рядку і зберегти їх як масив покажчиків на початок кожного слова.

Варіант 14

1. Реалізувати "зубчастий" масив, де кожен рядок має різну кількість елементів, за допомогою покажчиків.
2. Написати функцію для знаходження кореня рівняння $f(x)=0$ методом хорд.
3. Створити масив покажчиків на рядки-цитати та вивести випадкову цитату.
4. Реалізувати функцію обробки масиву, де тип обробки (`min`, `max`, `avg`) вибирається через покажчик.
5. Порівняти два рядки без урахування регістру, використовуючи покажчики.

Варіант 15

1. Створити масив покажчиків на функції, що конвертують валюту (USD в UAH, EUR в UAH тощо).
2. Реалізувати пошук елемента в масиві структур за ключовим полем через покажчик.
3. Написати функцію, що приймає покажчик на масив і повертає покажчик на його середину.
4. Створити систему фільтрації списку товарів за ціною або категорією через покажчики на функції.
5. Перетворити число в рядок (`itoa`), використовуючи покажчики для побудови рядка.

Варіант 16

1. Створити масив покажчиків на функції перевірки умов (чи парне, чи просте, чи додатне).
2. Реалізувати функцію обчислення добутку двох функцій $f(x) * g(x)$ у заданій точці.
3. Знайти суму елементів головної діагоналі матриці через покажчики.
4. Створити масив покажчиків на рядки, що містять назви днів тижня, та вивести їх у зворотному порядку.
5. Видалити всі цифри з рядка за допомогою покажчиків.

Варіант 17

1. Створити динамічний масив покажчиків на об'єкти типу "Працівник".
2. Реалізувати функцію, що обчислює значення n -ї похідної функції в точці.
3. Знайти другий за величиною елемент у масиві через покажчики.
4. Створити масив покажчиків на функції, що обробляють виключні ситуації.
5. Замінити одне слово в рядку іншим словом, використовуючи покажчики.

Варіант 18

1. Реалізувати масив покажчиків на функції для обчислення статистичних характеристик (медіана, мода).
2. Написати функцію сортування масиву рядків за їхньою довжиною.
3. Створити масив покажчиків на елементи масиву, що більші за середнє значення.
4. Реалізувати функцію зворотного виклику для таймера (імітація).
5. Визначити, чи містить один рядок інший (strstr), використовуючи покажчики.

Варіант 19

1. Створити масив покажчиків на функції, що виконують різні типи шифрування рядка (Цезар, XOR).
2. Реалізувати функцію для знаходження точки перетину двох кривих, заданих покажчиками на функції.
3. Знайти суму елементів масиву, розташованих між першим і останнім від'ємними елементами.
4. Створити масив покажчиків на рядки-команди ОС і виконати їх через system().
5. Перетворити рядок у число (atoi), використовуючи покажчики.

Варіант 20

1. Створити масив покажчиків на масиви структур "Товар" для різних відділів магазину.
2. Реалізувати функцію для знаходження локальних мінімумів функції через покажчик.
3. Повернути масив на 90 градусів через перестановку покажчиків.
4. Створити масив покажчиків на функції для генерації різних типів шуму (випадкових чисел).
5. Порахувати кількість слів у рядку через покажчик.

Варіант 21

1. Створити масив покажчиків на функції обчислення площ різних геометричних фігур.
2. Реалізувати функцію сортування структур "Книга" за роком видання або автором через компаратор.
3. Обчислити скалярний добуток двох векторів, використовуючи покажчики.
4. Створити масив покажчиків на рядки-повідомлення про помилки.
5. Перенести всі великі літери в початок рядка, використовуючи покажчики.

Варіант 22

1. Створити масив покажчиків на функції, що реалізують різні методи наближення функцій.
2. Реалізувати функцію "пошуку та заміни" в масиві покажчиків на рядки.
3. Знайти найдовшу послідовність однакових елементів у масиві через покажчики.
4. Створити масив покажчиків на функції для форматування тексту (ліворуч, по центру).
5. Перевірити, чи є одна назва анаграмою іншої, використовуючи покажчики.

Варіант 23

1. Створити масив покажчиків на функції для обробки сигналів датчиків (фільтр, підсилення).
2. Реалізувати функцію для знаходження об'єму тіла обертання через покажчик на функцію.
3. Об'єднати два впорядковані масиви в один впорядкований через покажчики.
4. Створити масив покажчиків на рядки, що містять назви міст, та відсортувати їх за кількістю букв.
5. Розбити рядок на частини за роздільником (split), повернувши масив покажчиків.

Варіант 24

1. Створити масив покажчиків на функції, що виконують арифметичні дії над комплексними числами.
2. Реалізувати функцію для знаходження максимуму функції двох змінних $f(x, y)$.
3. Знайти мінімальну відстань між двома числами в масиві через покажчики.
4. Створити масив покажчиків на об'єкти "Користувач" та реалізувати вхід у систему.
5. Видалити зайві пробіли в рядку (залишити тільки по одному між словами) через покажчики.

Варіант 25

1. Створити масив покажчиків на функції для перетворення одиниць виміру (кг в фунти, км в милі).
2. Реалізувати функцію сортування масиву структур "Літак" за часом вильоту.
3. Знайти кількість локальних максимумів у масиві цілих чисел через покажчики.
4. Створити масив покажчиків на рядки-назви мов програмування та виконати пошук за назвою.
5. Визначити, чи є рядок коректним e-mail адресом (спрощено), використовуючи покажчики.

Контрольні запитання для звіту:

- Що таке покажчик і як він оголошується?
- Як отримати адресу змінної та як отримати значення за адресою?
- У чому різниця між масивом покажчиків та покажчиком на масив?
- Як оголосити покажчик на функцію, що приймає два цілих числа та повертає void?
- Які переваги надає використання масивів покажчиків на функції при створенні меню?
- Як передати функцію як аргумент іншій функції?
- Що таке арифметика покажчиків і які операції вона дозволяє?

Приклад виконання роботи:

```
#include <iostream>
```

```

#include <cmath>

// Функції, на які будуть посилатися покажчики
double add(double a, double b) { return a + b; }
double sub(double a, double b) { return a - b; }
double mul(double a, double b) { return a * b; }
double div_op(double a, double b) {
    return (b != 0) ? a / b : 0;
}

int main() {
    // Оголошення типу покажчика на функцію
    typedef double (*MathFunc)(double, double);

    // Масив покажчиків на функції
    MathFunc operations[] = { add, sub, mul, div_op };
    const char* names[] = { "Додавання", "Віднімання", "Множення",
        "Ділення" };

    double x = 10.5, y = 2.5;
    int choice;

    std::cout << "Оберіть операцію (0-3): ";
    std::cin >> choice;

    if (choice >= 0 && choice < 4) {
        // Виклик функції через масив покажчиків
        double result = operations[choice](x, y);
        std::cout << names[choice] << " (" << x << ", " << y << ") = " <<
result << std::endl;
    } else {
        std::cout << "Невірний вибір!" << std::endl;
    }

    return 0;
}

```

Тема роботи 9.1: Робота з текстовими файлами.

Мета роботи:

- Ознайомитися з бібліотекою `fstream`.
- Навчитися створювати, записувати, читати та редагувати текстові файли.
- Отримати навички обробки текстової інформації (пошук, фільтрація, підрахунок).

Порядок виконання роботи:

- Ознайомитися з теоретичними відомостями щодо використання класів `ifstream` та `ofstream`.
- Вивчити приклад виконання роботи.
- Отримати варіант завдання у викладача.
- Написати програму на C++ для вирішення поставлених завдань.
- Протестувати програму на різних вхідних даних.
- Скласти звіт, що містить код програми, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Створити файл `data1.txt` і записати в нього 5 довільних речень.
2. Прочитати файл і вивести на екран лише ті слова, що починаються на голосну літеру.
3. Порахувати загальну кількість розділових знаків у файлі.
4. Знайти найдовше слово в першому реченні.
5. Скопіювати зміст файлу в новий файл, замінивши всі пробіли на символи підкреслення `_`.

Варіант 2

1. Записати у файл список з 10 цілих чисел.
2. Прочитати числа з файлу та знайти їх середнє арифметичне.
3. Створити новий файл, куди записати тільки парні числа з першого файлу.
4. Знайти мінімальне число у файлі та його позицію.
5. Дописати в кінець файлу поточну дату (у вигляді рядка).

Варіант 3

1. Створити файл з текстом. Вивести на екран кожне друге слово.
2. Визначити кількість рядків у файлі.
3. Знайти та вивести рядки, довжина яких перевищує 20 символів.
4. Замінити всі цифри у файлі на символ `*`.
5. Порахувати кількість входжень заданого користувачем слова.

Варіант 4

1. Записати у файл 5 прізвищ студентів та їхні оцінки.
2. Вивести список студентів, чия оцінка вища за 75 балів.
3. Знайти студента з найвищим балом.
4. Відсортувати список за прізвищами (в пам'яті) і записати результат у новий файл.
5. Видалити з файлу запис про студента з найнижчим балом.

Варіант 5

1. Прочитати текстовий файл і вивести його зміст у зворотному порядку (по літерах).
2. Порахувати кількість великих літер у файлі.
3. Знайти кількість речень (рахувати за крапками, знаками оклику та питання).
4. Видалити всі зайві пробіли між словами (залишити по одному).
5. Створити копію файлу, де кожне нове речення починається з нового рядка.

Варіант 6

1. Створити файл з цінами на товари (назва та ціна).
2. Порахувати сумарну вартість усіх товарів.
3. Знайти найдорожчий товар.
4. Зменшити всі ціни на 10% і оновити файл.
5. Вивести товари, назва яких починається на літеру 'А'.

Варіант 7

1. Записати у файл матрицю чисел 3×3 .
2. Прочитати матрицю та знайти суму елементів головної діагоналі.
3. Записати в інший файл транспоновану матрицю.
4. Знайти максимальний елемент у кожному рядку.
5. Перевірити, чи є у файлі від'ємні числа.

Варіант 8

1. Створити файл з англійським текстом.
2. Вивести статистику: скільки разів зустрічається кожна літера алфавіту.
3. Перетворити весь текст у верхній регістр.
4. Видалити всі слова, що складаються менше ніж з 3-х літер.
5. Знайти слово з максимальною кількістю голосних.

Варіант 9

1. Записати у файл координати 5 точок (x, y).
2. Знайти точку, найвіддаленішу від початку координат.
3. Обчислити відстань між першою та останньою точками.
4. Записати у новий файл точки, що знаходяться в першій чверті.
5. Додати нову точку в кінець файлу.

Варіант 10

1. Створити файл-словник (слово — переклад).
2. Реалізувати пошук перекладу за введеним словом.
3. Додати можливість додавання нових слів у файл.
4. Видалити обране слово зі словника.
5. Порахувати кількість унікальних слів у словнику.

Варіант 11

1. Прочитати файл і визначити, чи є він порожнім.
2. Вивести слова, що закінчуються на літеру 'я'.
3. Замінити всі знаки оклику на крапки.
4. Створити новий файл, що містить лише цифри з оригінального файлу.
5. Вивести останнє слово кожного рядка.

Варіант 12

1. Записати у файл 10 випадкових чисел від 1 до 100.
2. Знайти суму першої та останньої цифри у файлі.
3. Визначити, скільки чисел є простими.
4. Записати числа у зворотному порядку в інший файл.
5. Знайти число, що найближче до середнього значення.

Варіант 13

1. Створити файл з розкладом занять (день, назва пари, аудиторія).
2. Вивести пари лише для вказаного користувачем дня тижня.
3. Знайти всі пари, що проходять у конкретній аудиторії.
4. Оновити номер аудиторії для конкретної дисципліни.
5. Порахувати загальну кількість пар у тижні.

Варіант 14

1. Прочитати текст і вивести слова, що зустрічаються більше одного разу.
2. Перевірити текст на наявність балансу дужок () (відкриті мають дорівнювати закритим).
3. Видалити всі коментарі виду // з текстового файлу.
4. Замінити кожен цифру її назвою (1 -> "один").
5. Вивести текст, де кожне слово починається з великої літери.

Варіант 15

1. Записати у файл дані про книги (автор, назва, рік видання).
2. Вивести книги, видані після 2000 року.
3. Знайти найстарішу книгу у списку.
4. Пошук книг за автором.
5. Відсортувати книги за роком видання та зберегти у новий файл.

Варіант 16

1. Створити файл зі списком IP-адрес.
2. Перевірити правильність формату кожної адреси (4 числа через крапку, 0-255).
3. Вивести лише адреси, що починаються на 192.168..
4. Порахувати кількість адрес у файлі.
5. Записати унікальні адреси в інший файл.

Варіант 17

1. Створити файл з логами (час, подія, статус).
2. Вивести події зі статусом "Error".
3. Порахувати кількість подій за останню годину (якщо формат часу дозволяє).
4. Очистити файл від подій старше 30 днів.
5. Знайти найчастішу подію.

Варіант 18

1. Записати у файл математичний вираз (наприклад, $12 + 45$).
2. Прочитати вираз, обчислити результат і додати його в кінець файлу.
3. Обробити файл з декількома такими виразами (по одному на рядок).
4. Знайти вираз з найбільшим результатом.
5. Створити файл з помилками (вирази, що не можна обчислити, наприклад ділення на 0).

Варіант 19

1. Створити файл з іменами та телефонами.
2. Реалізувати пошук телефону за іменем.
3. Форматувати всі номери телефонів до єдиного вигляду +38(0XX)XXX-XX-XX.
4. Видалити дублікати контактів.
5. Відсортувати контакти за іменами.

Варіант 20

1. Прочитати файл з кодом C++.
2. Підрахувати кількість ключових слів int, float, double.
3. Вивести всі назви функцій (рядки, що закінчуються на ()).
4. Порахувати кількість фігурних дужок.
5. Створити копію файлу без пустих рядків.

Варіант 21

1. Створити файл з даними про погоду за місяць (день, температура).
2. Знайти день з найвищою та найнижчою температурою.
3. Обчислити середню температуру за місяць.
4. Вивести дні, коли була від'ємна температура.
5. Записати дані у файл у вигляді графіка (використовуючи символи *).

Варіант 22

1. Записати у файл назви міст та кількість населення.
2. Вивести міста-мільйонники.
3. Знайти місто з найменшою кількістю населення.
4. Порахувати загальну кількість людей у списку міст.
5. Створити файл, де міста відсортовані за спаданням населення.

Варіант 23

1. Створити файл, що містить зашифрований текст (шифр Цезаря, зсув на 3).
2. Прочитати файл і розшифрувати його.
3. Записати розшифрований текст у новий файл.
4. Порахувати кількість символів, які не вдалося розшифрувати.
5. Змінити зсув шифру за бажанням користувача.

Варіант 24

1. Записати у файл результати забігу (прізвище, час у секундах).
2. Визначити переможця (мінімальний час).
3. Обчислити середній час усіх учасників.
4. Вивести список учасників, які пробігли швидше ніж за 10 хвилин.
5. Додати нового учасника в середину списку (перезаписати файл).

Варіант 25

1. Створити файл з текстом пісні.
2. Знайти слово, яке повторюється найчастіше.
3. Видалити всі розділові знаки.
4. Порахувати кількість складів у першому рядку (за голосними).
5. Вивести текст, замінивши всі літери 'a' на 'o'.

Контрольні запитання для звіту:

- Яку бібліотеку потрібно підключити для роботи з файлами?
- У чому різниця між класами `ifstream`, `ofstream` та `fstream`?
- Які є режими відкриття файлів (наприклад, `ios::app`, `ios::out`)?
- Як перевірити, чи успішно було відкрито файл?
- Чим відрізняється читання файлу за допомогою оператора `>>` від методу `getline()`?

Приклад виконання роботи:

```
#include <iostream>
```

```

#include <fstream>
#include <string>
#include <sstream>

using namespace std;

int main() {
    string filename = "test.txt";

    // 1. Запис у файл
    ofstream outFile(filename);
    if (outFile.is_open()) {
        outFile << "Це приклад тексту для практичної роботи.\n";
        outFile << "C++ дозволяє легко працювати з файлами.\n";
        outFile.close();
        cout << "Файл успішно створено та записано." << endl;
    } else {
        cerr << "Помилка відкриття файлу для запису!" << endl;
        return 1;
    }

    // 2. Читання та обробка
    ifstream inFile(filename);
    string word;
    int wordCount = 0;

    if (inFile.is_open()) {
        while (inFile >> word) {
            wordCount++;
        }
        inFile.close();
        cout << "Кількість слів у файлі: " << wordCount << endl;
    } else {
        cerr << "Помилка відкриття файлу для читання!" << endl;
        return 1;
    }

    return 0;
}

```

Тема роботи 9.2: Створення файлу послідовного доступу.

Мета роботи:

- Ознайомитися з основами файлового введення-виведення у мові C++.
- Навчитися використовувати класи `ifstream`, `ofstream` та `fstream` бібліотеки `<fstream>`.
- Набути навичок створення файлів послідовного доступу та виконання операцій над структурованими даними.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо роботи з файлами в C++ (режими відкриття, методи `read`, `write`, `open`, `close`).
- Створити проект у середовищі розробки (наприклад, Visual Studio, Code::Blocks або CLion).
- Відповідно до свого варіанта, оголосити структуру (`struct`) для зберігання даних.
- Написати програму, що реалізує:
 - Створення файлу та запис у нього початкових даних.
 - Читання даних з файлу та їх виведення на екран.
 - Виконання специфічних завдань обробки (пошук, фільтрація, обчислення).
- Протестувати програму на різних наборах даних.
- Скласти звіт, до якого включити код програми, скріншоти роботи та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Кожен варіант складається з 5 обов'язкових завдань:

1. Визначення структури.
2. Наповнення файлу.
3. Повний перегляд.
4. Фільтрація за умовою.
5. Розрахунок підсумкового показника.

Варіант 1

- Структура: "Книга" (Автор, Назва, Рік видання, Ціна, Кількість сторінок).
- Створити файл з даними про 10 книг.
- Вивести всі книги, видані після 2010 року.
- Знайти книгу з найменшою кількістю сторінок.
- Обчислити загальну вартість усіх книг у файлі.

Варіант 2

- Структура: "Студент" (Прізвище, Група, Оцінка1, Оцінка2, Оцінка3).
- Створити файл зі списком студентів.
- Вивести студентів, які мають середній бал більше 4.5.
- Знайти студента з конкретної групи (ввід користувачем).
- Підрахувати кількість студентів-"відмінників".

Варіант 3

- Структура: "Автомобіль" (Марка, Модель, Рік випуску, Об'єм двигуна, Ціна).
- Сформувати базу даних автомобілів автосалону.
- Вивести всі авто конкретної марки (наприклад, "BMW").
- Знайти найдорожчий автомобіль.
- Обчислити середню ціну автомобіля в салоні.

Варіант 4

- Структура: "Товар" (Код, Назва, Категорія, Кількість, Вага).
- Створити файл обліку складу.
- Вивести товари певної категорії (наприклад, "Продукти").
- Знайти товар з найбільшою вагою.
- Підрахувати загальну кількість одиниць товару на складі.

Варіант 5

- Структура: "Рейс" (Номер рейсу, Пункт призначення, Час вильоту, Вільні місця, Ціна квитка).
- Записати розклад авіарейсів у файл.
- Вивести рейси до заданого міста.
- Знайти рейс з найбільшою кількістю вільних місць.
- Обчислити виручку, якщо всі квитки на певний рейс будуть продані.

Варіант 6

- Структура: "Працівник" (Прізвище, Посада, Стаж, Зарплата, Стать).
- Створити файл штату працівників фірми.
- Вивести працівників зі стажем понад 5 років.
- Знайти працівника з найменшою зарплатою.
- Підрахувати сумарну зарплату всіх жінок-працівників.

Варіант 7

- Структура: "Пацієнт" (Прізвище, Вік, Діагноз, Номер палати, Дата госпіталізації).
- Сформувати список пацієнтів відділення.
- Вивести пацієнтів конкретної палати.
- Знайти найстаршого пацієнта.
- Підрахувати кількість пацієнтів з діагнозом "Грип".

Варіант 8

- Структура: "Телефон" (Бренд, Модель, Пам'ять, Колір, Ціна).
- Записати прайс-лист магазину електроніки.
- Вивести телефони з об'ємом пам'яті 128 ГБ і більше.
- Знайти найдешевший телефон білого кольору.
- Підрахувати середню вартість телефонів певного бренду.

Варіант 9

- Структура: "Країна" (Назва, Континент, Населення, Площа, Мова).
- Створити файл з даними про країни світу.
- Вивести країни, що знаходяться в "Європі".
- Знайти країну з найбільшою площею.
- Обчислити загальне населення всіх країн у файлі.

Варіант 10

- Структура: "Спортсмен" (Прізвище, Вид спорту, Вік, Кількість медалей, Розряд).
- Сформувати файл реєстру спортсменів.
- Вивести спортсменів, що займаються "Плаванням".
- Знайти наймолодшого спортсмена.
- Підрахувати загальну кількість медалей у команді.

Варіант 11

- Структура: "Фільм" (Назва, Режисер, Рік, Жанр, Тривалість).
- Створити файл каталогу фільмів.
- Вивести фільми жанру "Комедія".
- Знайти найдовший фільм.
- Підрахувати кількість фільмів, знятих до 2000 року.

Варіант 12

- Структура: "Абонент" (Прізвище, Номер, Баланс, Тариф, Дата підключення).
- Створити файл бази даних мобільного оператора.
- Вивести абонентів з від'ємним балансом.
- Знайти абонента за номером телефону.
- Обчислити середній баланс усіх користувачів.

Варіант 13

- Структура: "Тварина" (Вид, Кличка, Вік, Вага, Раціон).
- Сформувати список мешканців зоопарку.
- Вивести всіх тварин виду "Лев".
- Знайти тварину з найбільшою вагою.
- Підрахувати середній вік тварин.

Варіант 14

- Структура: "Учень" (Прізвище, Клас, Бал з математики, Бал з фізики, Бал з мови).
- Створити файл результатів іспитів.
- Вивести учнів 11-х класів.
- Знайти учня з найвищим балом з математики.
- Обчислити середній бал класу з фізики.

Варіант 15

- Структура: "Квартира" (Номер, Поверх, Кількість кімнат, Площа, Ціна).
- Записати базу даних агентства нерухомості.
- Вивести всі 3-кімнатні квартири.
- Знайти квартиру з найбільшою площею на 1-му поверсі.
- Обчислити вартість 1 квадратного метра для кожної квартири.

Варіант 16

- Структура: "Замовлення" (Номер, Клієнт, Назва товару, Дата, Сума).
- Створити файл журналу замовлень інтернет-магазину.
- Вивести замовлення конкретного клієнта.
- Знайти замовлення на найбільшу суму.
- Підрахувати загальну суму продажів за певну дату.

Варіант 17

- Структура: "Ноутбук" (Марка, Процесор, ОЗП, Ціна, Наявність).
- Сформувати перелік товарів комп'ютерного магазину.
- Вивести ноутбуки з ОЗП 16 ГБ.
- Знайти найдешевший ноутбук марки "ASUS".
- Підрахувати кількість доступних (у наявності) моделей.

Варіант 18

- Структура: "Потяг" (Номер, Маршрут, Час відправлення, Кількість вагонів, Тип).
- Записати розклад руху потягів.
- Вивести потяги, що курсують за маршрутом "Київ-Львів".
- Знайти потяг, що відправляється найраніше.
- Підрахувати загальну кількість вагонів у всіх потягах.

Варіант 19

- Структура: "Викладач" (Прізвище, Кафедра, Дисципліна, Кількість годин, Зарплата).
- Створити файл навантаження викладачів.
- Вивести викладачів кафедри "Програмування".
- Знайти викладача з найбільшою кількістю годин.
- Обчислити середню зарплату викладачів кафедри.

Варіант 20

- Структура: "Ліки" (Назва, Виробник, Термін придатності, Ціна, Кількість).
- Створити файл асортименту аптеки.
- Вивести ліки певного виробника.
- Знайти ліки з найменшим терміном придатності.
- Підрахувати вартість усіх упаковок конкретного препарату.

Варіант 21

- Структура: "Дерево" (Вид, Вік, Висота, Стан, Місце посадки).
- Сформувати реєстр зелених насаджень парку.
- Вивести всі дерева виду "Каштан".
- Знайти найвище дерево в парку.
- Підрахувати кількість дерев, вік яких перевищує 50 років.

Варіант 22

- Структура: "Музичний інструмент" (Назва, Тип, Матеріал, Ціна, Виробник).
- Створити файл товарів музичного магазину.
- Вивести всі інструменти типу "Струнні".
- Знайти найдорожчий інструмент з дерева.
- Підрахувати середню ціну інструментів певного виробника.

Варіант 23

- Структура: "Деталь" (Назва, Матеріал, Вага, Ціна за одиницю, Номер цеху).
- Записати дані про продукцію заводу.
- Вивести деталі, виготовлені з "Алюмінію".
- Знайти деталь з мінімальною вагою.
- Підрахувати загальну вартість деталей, виготовлених у цеху №5.

Варіант 24

- Структура: "Журнал" (Назва, Тираж, Періодичність, Ціна передплати, Тематика).
- Створити файл каталогу періодичних видань.
- Вивести журнали тематики "Наука".
- Знайти журнал з найбільшим тиражем.
- Обчислити дохід від передплати всіх журналів (Тираж * Ціна).

Варіант 25

- Структура: "Планета" (Назва, Маса, Діаметр, Кількість супутників, Відстань від Сонця).
- Сформувати файл астрономічних даних.
- Вивести планети, що мають більше 5 супутників.
- Знайти планету з найбільшим діаметром.
- Підрахувати середню відстань усіх планет від Сонця.

Контрольні запитання для звіту:

- Яка різниця між текстовими та бінарними файлами?
- Які класи використовуються для читання та для запису файлів?
- Що означає прапорець `ios::app` при відкритті файлу?
- Навіщо потрібно використовувати метод `close()`?
- Як перевірити, чи досягнуто кінець файлу під час читання?

Приклад виконання роботи:

```
#include <iostream>
#include <fstream>
#include <string>
#include <vector>

using namespace std;

struct Item {
    char name[50];
    double price;
};

int main() {
    // 1. Запис даних у файл
    ofstream outFile("Inventory.bin", ios::binary);
    if (!outFile) return 1;

    Item items[] = { {"Phone", 15000.0}, {"Cable", 85.5}, {"Case", 120.0} };
    for (const auto& i : items) {
        outFile.write((char*)&i, sizeof(Item));
    }
    outFile.close();

    // 2. Читання та фільтрація
    ifstream inFile("Inventory.bin", ios::binary);
    Item temp;
    cout << "Items more expensive than 100 UAH:\n";
    while (inFile.read((char*)&temp, sizeof(Item))) {
        if (temp.price > 100.0) {
            cout << temp.name << " - " << temp.price << " UAH" << endl;
        }
    }
    inFile.close();
    return 0;
}
```

Тема роботи 9.3: Створення файлу довільного доступу

Мета роботи:

- Ознайомитися з принципами організації файлів довільного доступу.
- Навчитися використовувати функції позиціювання (`seekg`, `seekp`) та методи читання/запису блоків даних (`read`, `write`).
- Отримати навички роботи зі структурами даних при збереженні їх у бінарні файли.

Порядок виконання роботи:

- Ознайомтеся з методами класу `std::fstream` для переміщення вказівника у файлі.
- Оберіть варіант згідно зі списком групи.
- Опишіть структуру даних (`struct`) для зберігання запису.
- Реалізуйте функції для створення файлу, додавання записів, пошуку за індексом (довільний доступ) та модифікації конкретного запису.
- Перевірте роботу програми на коректність введення та виведення даних.
- Оформіть звіт з кодом програми та результатами виконання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Кожен варіант передбачає створення програми для керування файлом (додавання, перегляд за номером запису, редагування за номером запису, видалення шляхом позначення, підрахунок кількості).

Варіант 1: Спортивний інвентар

1. Запис даних про товар: назва, категорія, ціна, кількість, термін придатності.
2. Знайти товар за номером запису та подвоїти його кількість.
3. Вивести всі товари, ціна яких вище заданої.
4. Реалізувати перехід до останнього запису файлу.
5. Порахувати загальну вартість всього інвентарю.

Варіант 2: Студентський деканат

1. Дані: Прізвище, група, номер заліковки, 5 оцінок, середня оцінка.
2. Виправити оцінки студента за вказаним номером запису.
3. Знайти студента за номером заліковки.
4. Перейти до середини файлу та вивести запис.
5. Вивести список відмінників.

Варіант 3: Аптека

1. Дані: Назва ліків, виробник, дата виготовлення, ціна, рецептурний відпуск (`bool`).
2. Змінити ціну ліків на позиції `N`.
3. Пошук ліків за назвою.
4. Перейти на 2 записи вперед від поточної позиції.
5. Вивести всі безрецептурні препарати.

Варіант 4: Автосалон

1. Дані: Марка, модель, рік випуску, об'єм двигуна, ціна.
2. Оновити дані про авто за індексом.
3. Знайти авто з найбільшим об'ємом двигуна.
4. Перейти до першого запису.
5. Вивести авто певної марки.

Варіант 5: Бібліотека

1. Дані: Автор, назва, рік, кількість сторінок, наявність у залі.
2. Позначити книгу як "видану" за номером запису.
3. Пошук книг автора.
4. Перейти на K записів назад від кінця файлу.
5. Знайти найтовстішу книгу.

Варіант 6: Залізнична каса

1. Дані: Номер потяга, станція призначення, час відправлення, вільні місця, ціна квитка.
2. Зменшити кількість місць на 1 за номером запису (купівля).
3. Вивести потяги до конкретної станції.
4. Визначити позицію вказівника після читання 3-го запису.
5. Знайти потяг з найдешевшим квитком.

Варіант 7: Кадри підприємства

1. Дані: Прізвище, посада, стаж, зарплата, відділ.
2. Змінити посаду працівника за номером запису.
3. Знайти працівників зі стажем понад 10 років.
4. Перейти до запису перед поточним.
5. Обчислити середню зарплату по відділу.

Варіант 8: Склад електроніки

1. Дані: Тип пристрою, бренд, потужність, ціна, гарантія (місяці).
2. Оновити термін гарантії за номером запису.
3. Пошук пристроїв за брендом.
4. Перемістити вказівник на початок і прочитати перший запис.
5. Вивести товари з ціною в діапазоні [A, B].

Варіант 9: Зоомагазин

1. Дані: Вид тварини, кличка, вік, ціна, особливості догляду.
2. Змінити ціну тварини за індексом.
3. Знайти всіх тварин певного виду.
4. Розрахувати кількість записів у файлі через `tellg()`.
5. Знайти найстарішу тварину.

Варіант 10: Авіарейси

1. Дані: Номер рейсу, авіакомпанія, модель літака, тривалість польоту, статус.
2. Змінити статус рейсу за номером запису.
3. Вивести всі рейси певної авіакомпанії.
4. Перейти до запису N та вивести його на екран.
5. Знайти найкоротший за часом рейс.

Варіант 11: Готель

1. Дані: Номер кімнати, тип (люкс/стандарт), кількість місць, вартість за добу, статус зайнятості.
2. Змінити статус (зайнято/вільно) для кімнати N.
3. Вивести всі вільні номери типу "люкс".
4. Перейти до останнього запису файлу.
5. Знайти найдешевший вільний номер.

Варіант 12: Музична колекція

1. Дані: Виконавець, альбом, рік випуску, жанр, кількість треків.
2. Оновити кількість треків в альбомі за індексом.
3. Пошук альбомів певного жанру.
4. Вивести 3-й запис з кінця файлу.
5. Знайти альбом з найбільшою кількістю треків.

Варіант 13: Магазин одягу

1. Дані: Артикул, тип, розмір, матеріал, ціна.
2. Змінити ціну для товару за номером запису.
3. Знайти одяг певного розміру.
4. Перейти на початок файлу.
5. Вивести всі товари з матеріалу "бавовна".

Варіант 14: Страхова компанія

1. Дані: ПІБ клієнта, вид страхування, сума виплати, термін дії, статус оплати.
2. Подовжити термін дії за номером запису.
3. Пошук клієнта за ПІБ.
4. Перейти до середини списку клієнтів.
5. Обчислити загальну суму всіх страхових виплат.

Варіант 15: Станція ТО

1. Дані: Прізвище власника, марка авто, вид поломки, вартість ремонту, дата здачі.
2. Встановити вартість ремонту для авто на позиції N.
3. Вивести всі авто однієї марки.
4. Визначити поточну позицію вказівника запису.
5. Знайти ремонт з найвищою вартістю.

Варіант 16: Комп'ютерний клуб

1. Дані: Номер ПК, конфігурація, ціна за годину, статус (вільний/зайнятий), час старту.
2. Зайняти/звільнити ПК за номером запису.
3. Вивести всі вільні комп'ютери.
4. Перейти на 5-й запис від початку.
5. Порахувати прибуток за зміну (якщо додано поле суми).

Варіант 17: Туристична агенція

1. Дані: Країна, готель, зірковість, вартість туру, тривалість.
2. Змінити вартість туру за номером запису.
3. Вивести всі тури в обрану країну.
4. Перейти до передостаннього запису.
5. Знайти найдовший тур.

Варіант 18: Прокат авто

1. Дані: Марка, держномер, ціна прокату, застава, статус наявності.
2. Змінити статус авто за індексом.
3. Пошук авто за держномером.
4. Розрахувати розмір файлу в байтах.
5. Знайти авто з мінімальною заставою.

Варіант 19: Ветеринарна клініка

1. Дані: Власник, вид тварини, діагноз, дата прийому, ціна послуги.
2. Оновити діагноз за номером запису.
3. Вивести записи за конкретну дату.
4. Перейти до першого запису і вивести його.
5. Знайти найдорожчу послугу.

Варіант 20: Магазин іграшок

1. Дані: Назва, вікова група, матеріал, ціна, виробник.
2. Змінити виробника для іграшки на позиції N.
3. Вивести іграшки для вікової групи "3+".
4. Перейти на 2 записи вперед.
5. Знайти найдешевшу іграшку.

Варіант 21: Служба доставки

1. Дані: Номер посилки, вага, пункт призначення, вартість, статус (доставлено/в дорозі).
2. Змінити статус посилки за номером запису.
3. Пошук посилки за номером.
4. Перейти до запису K.
5. Знайти посилку з найбільшою вагою.

Варіант 22: Кафедра університету

1. Дані: Прізвище викладача, посада, науковий ступінь, кількість публікацій, зарплата.
2. Оновити кількість публікацій за індексом.
3. Вивести всіх професорів кафедри.
4. Повернутися на початок файлу після пошуку.
5. Знайти викладача з найбільшою зарплатою.

Варіант 23: Ремонт взуття

1. Дані: Тип взуття, вид робіт, термін виконання, ціна, замовник.
2. Змінити ціну роботи за номером запису.
3. Вивести замовлення з терміном виконання "сьогодні".
4. Перейти до останнього запису.
5. Порахувати загальну кількість замовлень.

Варіант 24: Парковка

1. Дані: Номер місця, держномер авто, час заїзду, тариф, оплачено (bool).
2. Позначити оплату за номером запису.
3. Знайти авто за держномером.
4. Перейти на запис назад від поточного.
5. Вивести список всіх неоплачених місць.

Варіант 25: Книжковий склад

1. Дані: Назва, видавництво, кількість примірників, оптова ціна, роздрібна ціна.
2. Оновити кількість примірників за номером запису.
3. Вивести книги конкретного видавництва.
4. Перейти до 10-го запису файлу.
5. Обчислити різницю між роздрібною та оптовою ціною для обраної книги.

Контрольні запитання для звіту:

- Чим файл довільного доступу відрізняється від файлу послідовного доступу?
- Які режими відкриття файлу необхідні для одночасного читання та запису у бінарному режимі?
- Поясніть призначення методів `seekp()` та `seekg()`. Що означають параметри `ios::beg`, `ios::cur`, `ios::end`?
- Як розрахувати позицію N-го запису у файлі, якщо відомий розмір структури?
- Чому при роботі з бінарними файлами не рекомендується використовувати типи зі змінним розміром (наприклад, `std::string`) всередині структур?

Приклад виконання роботи:


```

#include <iostream>
#include <fstream>
#include <cstring>

using namespace std;

struct Book {
    int id;
    char title[50];
    double price;
};

void add_book(const char* filename) {
    ofstream out(filename, ios::binary | ios::app);
    Book b;
    cout << "ID: "; cin >> b.id;
    cout << "Title: "; cin.ignore(); cin.getline(b.title, 50);
    cout << "Price: "; cin >> b.price;
    out.write((char*)&b, sizeof(Book));
    out.close();
}

void update_book(const char* filename, int index) {
    fstream file(filename, ios::binary | ios::in | ios::out);
    if (!file) return;

    file.seekp(index * sizeof(Book), ios::beg);
    Book b;
    cout << "New Title: "; cin.ignore(); cin.getline(b.title, 50);
    cout << "New Price: "; cin >> b.price;
    b.id = index + 1; // Припустимо, ID відповідає позиції

    file.write((char*)&b, sizeof(Book));
    file.close();
}

int main() {
    const char* db = "library.dat";
    // Логіка меню...
    return 0;
}

```

Тема роботи 9.4: Робота з двійковими файлами.

Мета роботи:

- Ознайомитися з особливостями організації двійкових файлів.
- Навчитись використовувати об'єктно-орієнтовані засоби C++ (`std::fstream`) для запису та читання структур даних у двійковому режимі.
- Отримати навички позиціювання у файлі (довільний доступ) та редагування записів без повного перезапису файлу.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо відкриття файлів у режимі `std::ios::binary`.
- Ознайомитися з методами `write()` та `read()`.
- Отримати варіант завдання у викладача.
- Розробити алгоритм розв'язання задачі:
 - o Визначити структуру даних.
 - o Реалізувати функцію створення початкового файлу.
 - o Реалізувати функцію виводу вмісту файлу на екран.
 - o Реалізувати специфічну логіку обробки (пошук, видалення, редагування) згідно з варіантом.
- Написати та відлагодити програму.
- Оформити звіт та відповісти на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Створити двійковий файл зі списком студентів (Прізвище, група, середній бал).
2. Вивести студентів, чий бал вище 4.5.
3. Порахувати загальну кількість студентів у заданій групі.
4. Змінити прізвище студента за його порядковим номером у файлі.
5. Видалити останній запис у файлі.

Варіант 2

1. Створити файл "Inventory" (Назва товару, артикул, ціна, складський залишок).
2. Знайти товар за артикулом.
3. Вивести товари, сумарна вартість яких на складі перевищує 10000 грн.
4. Збільшити ціну всіх товарів на 10%.
5. Сформувати новий файл, куди переписати лише дефіцитні товари (залишок < 5).

Варіант 3

1. Створити файл "Library" (Автор, назва книги, рік видання, статус: в наявності/видана).
2. Вивести всі книги конкретного автора.
3. Змінити статус книги на "видана" за її назвою.
4. Порахувати кількість книг, виданих раніше 2000 року.
5. Видалити книгу з файлу за її позицією.

Варіант 4

1. Файл "Personnel" (ПІБ, посада, рік прийняття на роботу, зарплата).
2. Вивести працівників зі стажем понад 10 років.
3. Знайти найвищу зарплату серед менеджерів.
4. Оновити посаду працівника за його ПІБ.
5. Видалити запис про працівника з найменшою зарплатою.

Варіант 5

1. Файл "Airlines" (Номер рейсу, пункт призначення, час вильоту, ціна квитка).
2. Вивести рейси до заданого міста.
3. Знайти найдешевший квиток.
4. Перенести час вильоту конкретного рейсу.
5. Створити копію файлу, відсортовану за номером рейсу.

Варіант 6

1. Файл "Hospital" (Прізвище пацієнта, номер палати, діагноз, дата госпіталізації).
2. Вивести список пацієнтів у конкретній палаті.
3. Порахувати пацієнтів з певним діагнозом.
4. Виписати пацієнта (видалити запис) за прізвищем.
5. Перевести пацієнта в іншу палату (редагування).

Варіант 7

1. Файл "Cars" (Марка, модель, рік випуску, об'єм двигуна, ціна).
2. Вивести авто, випущені після 2020 року.
3. Порахувати середню ціну автомобілів заданої марки.
4. Змінити ціну авто за його моделлю.
5. Видалити всі авто з об'ємом двигуна менше 1.2 л.

Варіант 8

1. Файл "Weather" (Дата, температура, вологість, опади у мм).
2. Знайти день з максимальною температурою.
3. Розрахувати середню кількість опадів за місяць.
4. Виправити дані за конкретну дату.
5. Вивести дати, коли була від'ємна температура.

Варіант 9

1. Файл "Students_Scholarship" (Прізвище, номер картки, сума стипендії, тип стипендії).
2. Вивести студентів, що отримують підвищену стипендію.
3. Змінити суму стипендії всім студентам (індексація +5%).
4. Порахувати загальну суму виплат за файлом.
5. Видалити студента за номером картки.

Варіант 10

1. Файл "Train_Station" (Номер потяга, маршрут, час прибуття, вільні місця).
2. Вивести потяги, що мають понад 20 вільних місць.
3. Знайти час прибуття потяга за його номером.
4. Бронювання місця (зменшення кількості вільних місць на 1).
5. Видалити скасовані рейси з файлу.

Варіант 11

1. Файл "Bank_Accounts" (Номер рахунку, власник, баланс, тип валюти).
2. Вивести рахунки з балансом понад 1 000 000.
3. Переказати кошти (змінити баланс) за номером рахунку.
4. Порахувати кількість рахунків у валюті "USD".
5. Закрити рахунок (видалити запис).

Варіант 12

1. Файл "Cinema" (Назва фільму, жанр, тривалість, рейтинг IMDb).
2. Вивести всі бойовики тривалістю понад 120 хв.
3. Знайти фільм з найвищим рейтингом.
4. Оновити рейтинг фільму.
5. Видалити фільми з рейтингом нижче 5.0.

Варіант 13

1. Файл "Hardware" (Назва деталі, тип, виробник, ціна, гарантійний термін).
2. Вивести деталі з гарантією понад 24 місяці.
3. Знайти найдорожчу деталь конкретного виробника.
4. Змінити виробника для групи деталей.
5. Видалити застарілі деталі за списком.

Варіант 14

1. Файл "Sport_Results" (Прізвище спортсмена, вид спорту, результат, місце).
2. Вивести всіх призерів (місце 1, 2 або 3).
3. Пошук результату за прізвищем.
4. Оновити результат спортсмена після апеляції.
5. Порахувати кількість учасників у виді спорту "Плавання".

Варіант 15

1. Файл "Music_Album" (Виконавець, назва альбому, рік, кількість треків).
2. Вивести альбоми, випущені в 90-х роках.
3. Знайти альбом з найбільшою кількістю треків.
4. Змінити назву альбому за ПІБ виконавця.
5. Видалити запис за порядковим номером.

Варіант 16

1. Файл "Real_Estate" (Адреса, кількість кімнат, площа, ціна).
2. Вивести 3-кімнатні квартири площею понад 80 кв.м.
3. Розрахувати ціну за 1 кв.м для кожного об'єкта.
4. Змінити ціну об'єкта за адресою.
5. Видалити записи про продані квартири.

Варіант 17

1. Файл "Pharmacy" (Назва ліків, діюча речовина, термін придатності, ціна).
2. Вивести ліки, термін придатності яких закінчується в поточному місяці.
3. Порахувати середню ціну ліків з певною діючою речовиною.
4. Знизити ціну на 20% для ліків, що виходять з терміну.
5. Видалити препарат за назвою.

Варіант 18

1. Файл "University_Staff" (ПІБ, кафедра, науковий ступінь, кількість публікацій).
2. Вивести професорів кафедри програмної інженерії.
3. Знайти співробітника з максимальною кількістю публікацій.
4. Змінити науковий ступінь за ПІБ.
5. Додати новий запис у середину файлу (шляхом перезапису).

Варіант 19

1. Файл "Zoo" (Вид тварини, кличка, вік, норма корму на день).
2. Вивести тварин, старших за 10 років.
3. Розрахувати загальну вагу корму для всіх тварин виду "Лев".
4. Змінити норму корму за кличкою тварини.
5. Видалити запис про тварину, яку передали в інший зоопарк.

Варіант 20

1. Файл "Mobile_Plans" (Назва тарифу, абонплата, об'єм ГБ, хвилини).
2. Вивести тарифи з безлімітним інтернетом (наприклад, ГБ > 100).
3. Знайти найвигідніший тариф за ціною за 1 ГБ.
4. Змінити абонплату для конкретного тарифу.
5. Створити файл-архів для застарілих тарифів.

Варіант 21

1. Файл "Hotel" (Номер кімнати, клас, кількість місць, ціна за добу).
2. Вивести вільні номери класу "Люкс".
3. Порахувати загальну місткість готелю.
4. Змінити ціну номера за його типом.
5. Видалити інформацію про номери, що закриті на ремонт.

Варіант 22

1. Файл "E-Shop_Orders" (ID замовлення, клієнт, сума, статус: оплачено/очікує).
2. Вивести всі неоплачені замовлення на суму понад 5000 грн.
3. Змінити статус замовлення за його ID.
4. Порахувати сумарний виторг (оплачені замовлення).
5. Видалити замовлення, що були скасовані.

Варіант 23

1. Файл "Delivery" (Номер посилки, вага, пункт призначення, вартість).
2. Вивести посилки вагою понад 30 кг.
3. Знайти посилку за номером.
4. Змінити пункт призначення для посилки.
5. Видалити запис після успішної доставки.

Варіант 24

1. Файл "Game_Library" (Назва гри, розробник, рік, об'єм на диску в ГБ).
2. Вивести ігри конкретного розробника.
3. Знайти гру, що займає найменше місця.
4. Оновити версію (рік випуску) гри за назвою.
5. Видалити ігри, випущені до 2010 року.

Варіант 25

1. Файл "Conference" (Прізвище учасника, назва доповіді, секція, тривалість).
2. Вивести доповідачів секції "Штучний інтелект".
3. Порахувати сумарний час усіх доповідей у файлі.
4. Змінити секцію для учасника.
5. Видалити учасника, що відмовився від виступу.

Контрольні запитання для звіту:

- Чим двійковий файл відрізняється від текстового з точки зору зберігання чисел?
- Яке призначення прапорця `ios::binary` при відкритті файлу?
- Чому при роботі з двійковими файлами рекомендується використовувати масиви `char[]` замість `std::string` всередині структур?
- Для чого використовуються методи `seekg()` та `seekp()`?
- Як визначити кількість записів у двійковому файлі, знаючи розмір структури?

Приклад виконання роботи:

```

#include <iostream>
#include <fstream>
#include <string>
#include <cstring>

using namespace std;

// Використовуємо структуру з фіксованим розміром полів для коректного запису
в файл
struct Product {
    int id;
    char name[40];
    double price;
    int quantity;
};

// Функція для додавання запису в файл
void addProduct(const char* filename) {
    ofstream outFile(filename, ios::binary | ios::app);
    Product p;
    cout << "Введіть ID: "; cin >> p.id;
    cout << "Введіть назву: "; cin.ignore(); cin.getline(p.name, 40);
    cout << "Введіть ціну: "; cin >> p.price;
    cout << "Введіть кількість: "; cin >> p.quantity;

    outFile.write(reinterpret_cast<char*>(&p), sizeof(Product));
    outFile.close();
}

// Функція для читання всіх записів
void displayAll(const char* filename) {
    ifstream inFile(filename, ios::binary);
    if (!inFile) {
        cout << "Файл не знайдено!" << endl;
        return;
    }

    Product p;
    cout << "\n--- Вміст файлу ---\n";
    while (inFile.read(reinterpret_cast<char*>(&p), sizeof(Product))) {
        cout << "ID: " << p.id << " | Назва: " << p.name
            << " | Ціна: " << p.price << " | К-сть: " << p.quantity << endl;
    }
    inFile.close();
}

int main() {
    const char* file = "storage.dat";
    int choice;
    do {
        cout << "\n1. Додати товар\n2. Вивести всі\n0. Вихід\nВибір: ";
        cin >> choice;
        if (choice == 1) addProduct(file);
        else if (choice == 2) displayAll(file);
    } while (choice != 0);
    return 0;
}

```

Тема роботи 10.1: Списки. Зв'язаний список.

Мета роботи:

- Ознайомитися з принципами функціонування динамічних структур даних.
- Навчитися створювати та маніпулювати однозв'язними (Single Linked List) та двозв'язними (Double Linked List) списками.
- Реалізувати базові операції: додавання, видалення, пошук та сортування елементів у списку.

Порядок виконання роботи:

- Вивчити теоретичний матеріал щодо динамічного розподілу пам'яті та структури вузла списку.
- Отримати варіант завдання від викладача.
- Розробити алгоритм розв'язання задач свого варіанту.
- Написати програмний код на мові C++.
- Протестувати програму на різних вхідних даних.
- Оформити звіт, що містить: титульну сторінку, мету, постановку задачі свого варіанту, текст програми з коментарями, скріншоти результатів та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

У кожному завданні слід реалізувати окремий однозв'язний або двозв'язний список відповідно до умови.

Варіант 1

1. Створити список цілих чисел. Знайти суму парних елементів.
2. Видалити перший елемент списку.
3. Додати новий елемент після вузла з заданим значенням.
4. Знайти максимальний елемент у списку.
5. Перевернути список (реверс).

Варіант 2

1. Створити список дійсних чисел. Обчислити середнє арифметичне.
2. Видалити останній елемент.
3. Вставити число на 3-ю позицію в списку.
4. Порахувати кількість від'ємних елементів.
5. Перевірити, чи є список відсортованим.

Варіант 3

1. Створити список символів (char). Порахувати кількість голосних букв.
2. Видалити елемент за значенням, яке вводить користувач.
3. Додати елемент у початок списку.
4. Знайти мінімальний елемент.
5. Склеїти два списки в один.

Варіант 4

1. Створити список рядків (string). Знайти найдовший рядок.
2. Видалити всі елементи з парними індексами.
3. Додати елемент перед заданим вузлом.
4. Замінити всі входження заданого числа на нуль.
5. Відсортувати список методом бульбашки.

Варіант 5

1. Створити двозв'язний список цілих чисел. Вивести його у зворотному порядку.
2. Видалити кожний другий елемент.
3. Додати елемент в середину списку.
4. Знайти індекс першого входження заданого числа.
5. Розділити список на два: з додатними та від'ємними числами.

Варіант 6

1. Створити список студентів (ПІБ, оцінка). Знайти кращого студента.
2. Видалити студентів з оцінкою нижче 60.
3. Додати студента в алфавітному порядку.
4. Порахувати загальну кількість вузлів.
5. Змінити місцями перший і останній елементи.

Варіант 7

1. Створити список товарів (назва, ціна). Обчислити загальну вартість.
2. Видалити товар за назвою.
3. Збільшити ціну всіх товарів на 10%.
4. Знайти товар з мінімальною ціною.
5. Скопіювати список в інший список.

Варіант 8

1. Створити список температур за тиждень. Знайти дні з температурою вище нуля.
2. Видалити всі нульові значення.
3. Вставити середнє значення між кожним елементом.
4. Перевірити наявність дублікатів.
5. Очистити весь список (звільнити пам'ять).

Варіант 9

1. Створити двозв'язний список слів. Знайти слова, що починаються на 'А'.
2. Видалити елемент за вказаним індексом.
3. Додати копію першого елемента в кінець.
4. Знайти передостанній елемент.
5. Циклічно зсунути список вправо на 1 позицію.

Варіант 10

1. Створити список цілих чисел. Знайти добуток всіх ненульових елементів.
2. Видалити всі елементи, більші за 100.
3. Додати елемент X після кожного парного числа.
4. Визначити, чи є в списку хоча б одне число 7.
5. Видалити дублікати зі списку.

Варіант 11

1. Створити список координат (x, y). Знайти точку, найближчу до початку координат.
2. Видалити всі точки, що лежать у 2-й чверті.
3. Додати нову точку в початок.
4. Порахувати кількість точок на осях координат.
5. Знайти відстань між першою і останньою точкою.

Варіант 12

1. Створити список книг (автор, назва, рік). Знайти книги автора "Шевченко".
2. Видалити книги, видані до 2000 року.
3. Відсортувати книги за роком видання.
4. Знайти найстарішу книгу.
5. Змінити назву книги за запитом користувача.

Варіант 13

1. Створити список оцінок. Визначити відсоток "п'ятірок".
2. Видалити всі "двійки".
3. Додати "бонусну" оцінку в кінець.
4. Знайти моду (найчастіше значення) у списку.
5. Перетворити однозв'язний список на двозв'язний.

Варіант 14

1. Створити список працівників (прізвище, стаж). Знайти досвідчених (стаж > 10 років).
2. Видалити працівника з найменшим стажем.
3. Додати нового працівника після працівника з конкретним прізвищем.
4. Обчислити середній стаж.
5. Вивести список у форматі "Прізвище - Стаж".

Варіант 15

1. Створити двозв'язний список чисел. Знайти суму елементів на непарних позиціях.
2. Видалити елементи, значення яких дорівнює сумі сусідів.
3. Вставити 0 перед кожним від'ємним числом.
4. Знайти кількість елементів, що закінчуються на 5.
5. Перевірити список на симетрію (паліндром).

Варіант 16

1. Створити список номерів телефонів (рядки). Знайти номери певного оператора (за префіксом).
2. Видалити номер за запитом.
3. Додати код країни до кожного номера.
4. Порахувати кількість унікальних номерів.
5. Відсортувати номери за зростанням.

Варіант 17

1. Створити список авто (марка, рік, ціна). Знайти найдорожче авто.
2. Видалити авто певної марки.
3. Додати авто в початок списку.
4. Порахувати кількість авто, випущених після 2020 року.
5. Вивести всі авто в зворотному порядку (через двозв'язний список).

Варіант 18

1. Створити список символів. Перевірити, чи утворюють вони слово "Hello".
2. Видалити всі цифри зі списку символів.
3. Замінити всі малими літерами великі.
4. Знайти індекс останнього входження символу 'z'.
5. Розділити список на два: літери та інші символи.

Варіант 19

1. Створити список планет (назва, відстань до Сонця). Знайти найближчу планету.
2. Видалити планету за назвою.
3. Додати нову планету, зберігши сортування за відстанню.
4. Знайти планету з найдовшою назвою.
5. Вивести назви планет у стовпчик.

Варіант 20

1. Створити список цілих чисел. Знайти середнє геометричне.
2. Видалити всі парні числа.
3. Додати суму всіх елементів у кінець списку.
4. Порахувати кількість елементів у діапазоні [A, B].
5. Змінити знак у всіх від'ємних елементів.

Варіант 21

1. Створити список інгредієнтів (назва, вага). Знайти інгредієнт з найбільшою вагою.
2. Видалити інгредієнт, вага якого $< 10\text{г}$.
3. Додати новий інгредієнт в середину.
4. Порахувати загальну вагу страви.
5. Перейменувати інгредієнт.

Варіант 22

1. Створити двозв'язний список міст (назва, населення). Знайти мільйонники.
2. Видалити місто за назвою.
3. Додати місто в кінець списку.
4. Обчислити загальне населення всіх міст.
5. Відсортувати міста за кількістю населення.

Варіант 23

1. Створити список дат (день, місяць, рік). Знайти найпізнішу дату.
2. Видалити всі дати зимових місяців.
3. Додати одну добу до кожної дати.
4. Порахувати кількість дат у поточному році.
5. Вивести дати у форматі DD.MM.YYYY.

Варіант 24

1. Створити список фільмів (назва, рейтинг). Знайти фільми з рейтингом > 8.5 .
2. Видалити фільм з найнижчим рейтингом.
3. Додати фільм у список.
4. Знайти фільм за назвою (пошук).
5. Збільшити рейтинг усім фільмам на 0.5.

Варіант 25

1. Створити список логів (час, повідомлення). Знайти логи з помилками (слово "Error").
2. Видалити логи, старіші за певну годину.
3. Додати новий лог у кінець.
4. Порахувати кількість критичних помилок.
5. Вивести список повідомлень у хронологічному порядку.

Контрольні запитання для звіту:

- Чим відрізняється зв'язаний список від масиву?
- Які переваги має однозв'язний список перед масивом у контексті вставки елементів?
- Що таке "вузол" (node) у списку і які поля він зазвичай містить?
- Опишіть алгоритм видалення елемента з середини однозв'язного списку.
- Навіщо у двозв'язному списку потрібен вказівник prev?
- Що станеться, якщо при видаленні вузла не використовувати оператор delete?
- Яким чином можна реалізувати циклічний список?

Приклад виконання роботи:

```
#include <iostream>

// Структура вузла
struct Node {
    int data;
    Node* next;
};

// Функція додавання в кінець
void addToEnd(Node*& head, int value) {
    Node* newNode = new Node;
    newNode->data = value;
    newNode->next = nullptr;

    if (head == nullptr) {
        head = newNode;
    } else {
        Node* temp = head;
        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newNode;
    }
}

// Функція виведення
void printList(Node* head) {
    Node* temp = head;
    while (temp != nullptr) {
        std::cout << temp->data << " -> ";
        temp = temp->next;
    }
    std::cout << "NULL" << std::endl;
}

int main() {
    Node* list = nullptr;
    addToEnd(list, 10);
    addToEnd(list, 20);
    addToEnd(list, 30);

    std::cout << "Список елементів: ";
    printList(list);

    return 0;
}
```

Тема роботи 10.2: Стеки, черги, деки.

Мета роботи: Вивчити принципи організації та функціонування лінійних структур даних: стека, черги та дека. Оволодіти навичками реалізації цих структур за допомогою масивів або зв'язних списків мовою C++, а також навчитися використовувати стандартну бібліотеку шаблонів (STL).

Порядок виконання роботи:

- Ознайомитися з теоретичними відомостями про структури даних LIFO (Stack) та FIFO (Queue, Deque).
- Отримати номер варіанта у викладача.
- Розробити алгоритм розв'язання задач свого варіанта.
- Реалізувати програму мовою C++. Дозволяється використовувати як власні класи/структури, так і контейнери `std::stack`, `std::queue`, `std::deque`.
- Протестувати програму на різних вхідних даних.
- Оформити звіт та підготувати відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Стек: Вивести символи рядка у зворотному порядку.
2. Черга: Імітація черги на друк документів.
3. Дек: Перевірка, чи є рядок паліндромом.
4. Стек: Перетворення десяткового числа в двійкове.
5. Черга: Сортування черги цілих чисел за допомогою допоміжного стека.

Варіант 2

1. Стек: Перевірка балансу дужок {}, [], ().
2. Черга: Обчислення середнього значення елементів черги.
3. Дек: Додавання елементів в обидва кінці та видалення парних чисел.
4. Стек: Знаходження максимального елемента в стеку без його руйнування.
5. Черга: Об'єднання двох відсортованих черг в одну.

Варіант 3

1. Стек: Обчислення виразу у постфіксній формі (Зворотна польська нотація).
2. Черга: Реалізація циклічної черги.
3. Дек: Розділення дека на два: з додатними та від'ємними числами.
4. Стек: Видалення кожного другого елемента стека.
5. Черга: Знаходження мінімального елемента в черзі.

Варіант 4

1. Стек: Перетворення інфіксного запису у постфіксний.
2. Черга: Моделювання обслуговування клієнтів у банку.
3. Дек: Пошук елемента за значенням у деку.
4. Стек: Копіювання вмісту одного стека в інший із збереженням порядку.
5. Черга: Інвертування перших K елементів черги.

Варіант 5

1. Стек: Перевірка правильності вкладеності HTML-тегів.
2. Черга: Визначення кількості елементів між мінімальним і максимальним.
3. Дек: Видалення всіх повторюваних елементів з дека.
4. Стек: Знаходження суми парних елементів стека.
5. Черга: Переміщення від'ємних чисел у кінець черги.

Варіант 6

1. Стек: Реалізація функцій Undo/Redo для текстового редактора.
2. Черга: Реалізація пріоритетної черги на базі масиву.
3. Дек: Вставка елемента в середину дека.
4. Стек: Порівняння двох стеків на рівність вмісту.
5. Черга: Видалення дублікатів, що йдуть підряд.

Варіант 7

1. Стек: Сортування стека за зростанням за допомогою іншого стека.
2. Черга: Знаходження другого за величиною елемента.
3. Дек: Реалізація "ковзного вікна" для пошуку максимуму.
4. Стек: Видалення середнього елемента стека.
5. Черга: Обмін місцями першого та останнього елементів.

Варіант 8

1. Стек: Пошук найдовшого підрядка з правильним балансом дужок.
2. Черга: Розбиття черги на дві: парні та непарні позиції.
3. Дек: Ротація елементів дека на N позицій вправо.
4. Стек: Перетворення числа з 10-ї системи у 16-ту.
5. Черга: Перевірка, чи є черга відсортованою.

Варіант 9

1. Стек: Оцінка виразу в префіксній формі.
2. Черга: Імітація процесу передачі пакетів у мережі.
3. Дек: Видалення елементів з парними індексами.
4. Стек: Рекурсивне інвертування стека.
5. Черга: Знаходження добутку всіх ненульових елементів.

Варіант 10

1. Стек: Реалізація двох стеків в одному масиві (з різних кінців).
2. Черга: Реалізація стека за допомогою двох черг.
3. Дек: Перевірка симетрії значень у деку.
4. Стек: Підрахунок кількості входжень заданого числа.
5. Черга: Вилучення кожного N-го елемента (задача Йосипа Флавія).

Варіант 11

1. Стек: Видалення всіх чисел, що більші за середнє арифметичне.
2. Черга: Генерація двійкових чисел від 1 до N.
3. Дек: Злиття двох деків у один.
4. Стек: Перевірка, чи є послідовність чисел зростаючою.
5. Черга: Підрахунок кількості унікальних елементів.

Варіант 12

1. Стек: Реалізація калькулятора (додавання, віднімання).
2. Черга: Сортуння методом вибору з використанням черги.
3. Дек: Видалення мінімального елемента з дека.
4. Стек: Розділення стека на два за знаком числа.
5. Черга: Перевірка, чи містить черга задану підпослідовність.

Варіант 13

1. Стек: Отримання N-го елемента з дна стека.
2. Черга: Визначення довжини найдовшої серії однакових елементів.
3. Дек: Створення дека з символів і сортування їх за алфавітом.
4. Стек: Видалення всіх елементів, що дорівнюють X.
5. Черга: Копіювання черги в масив.

Варіант 14

1. Стек: Обчислення факторіала за допомогою стека.
2. Черга: Реалізація черги за допомогою двох стеків.
3. Дек: Заміна всіх входжень числа A на число B.
4. Стек: Пошук другого максимуму.
5. Черга: Видалення всіх елементів менших за X.

Варіант 15

1. Стек: Реалізація стека з підтримкою операції `getMin()` за $O(1)$.
2. Черга: Пошук циклів у послідовності за допомогою черги.
3. Дек: Об'єднання елементів дека в один рядок.
4. Стек: Збереження історії відвідувань веб-сторінок.
5. Черга: Перестановка елементів черги у шаховому порядку (парний/непарний).

Варіант 16

1. Стек: Перевірка правильності вкладеності лапок у коді.
2. Черга: Реалізація черги з обмеженим пріоритетом.
3. Дек: Знаходження суми елементів між першим і останнім нулем.
4. Стек: Побудова гістограми та пошук найбільшого прямокутника.
5. Черга: Виділення з черги підчерги з заданою сумою.

Варіант 17

1. Стек: Вилучення дублікатів з рядка за допомогою стека.
2. Черга: Імітація роботи світлофора на перехресті.
3. Дек: Переміщення всіх нулів у початок дека.
4. Стек: Створення дзеркальної копії стека.
5. Черга: Обчислення медіани черги.

Варіант 18

1. Стек: Виконання арифметичних операцій над великими числами.
2. Черга: Моделювання черги покупців у супермаркеті.
3. Дек: Інвертування порядку елементів у деку.
4. Стек: Знаходження глибини вкладеності дужок.
5. Черга: Порівняння двох черг на ідентичність.

Варіант 19

1. Стек: Перевірка, чи є рядок анаграмою іншого рядка.
2. Черга: Планувальник завдань для процесора (Round Robin).
3. Дек: Отримання списку всіх унікальних елементів.
4. Стек: Пошук суми цифр у стеку.
5. Черга: Очищення черги від від'ємних значень.

Варіант 20

1. Стек: Реалізація стека, що автоматично розширюється.
2. Черга: Реалізація черги з можливістю перегляду останнього елемента.
3. Дек: Сортування дека вставками.
4. Стек: Підрахунок кількості голосних у стеку символів.
5. Черга: Розділення черги на дві за парністю значень.

Варіант 21

1. Стек: Перетворення десяткового числа в систему числення з основою N.
2. Черга: Знаходження найчастішого елемента в черзі.
3. Дек: Перевірка, чи є вміст дека дзеркальним.
4. Стек: Сортування стека методом "бульбашки" (з доп. стеком).
5. Черга: Додавання елемента після кожного від'ємного значення.

Варіант 22

1. Стек: Моделювання роботи стека викликів функцій.
2. Черга: Побудова черги з випадкових чисел і пошук середнього.
3. Дек: Видалення елементів, що потрапляють у діапазон [A, B].
4. Стек: Копіювання елементів, що стоять на непарних місцях.
5. Черга: Об'єднання декількох черг в одну.

Варіант 23

1. Стек: Перевірка балансу тегів у XML файлі.
2. Черга: Визначення часу очікування клієнта в черзі.
3. Дек: Знаходження добутку максимального та мінімального елементів.
4. Стек: Рекурсивне видалення елементів за умовою.
5. Черга: Знаходження суми квадратів елементів черги.

Варіант 24

1. Стек: Пошук найменшого елемента під стеком.
2. Черга: Реалізація черги з двома кінцями (через масив).
3. Дек: Стиснення дека шляхом видалення дублікатів.
4. Стек: Робота з символьним рядком: видалення пробілів.
5. Черга: Зміна порядку елементів на зворотний.

Варіант 25

1. Стек: Визначення пріоритету операцій у математичному виразі.
2. Черга: Генерація послідовності Фібоначчі за допомогою черги.
3. Дек: Розділення дека на три частини: <0 , 0 , >0 .
4. Стек: Знаходження кількості елементів, більших за X .
5. Черга: Перевірка черги на наявність нулів.

Контрольні запитання для звіту:

- Дайте визначення стека. Який принцип лежить в його основі (LIFO чи FIFO)?
- Які основні операції виконуються над стеком і чергою?
- Чим дек відрізняється від черги та стека?
- Опишіть переваги та недоліки реалізації черги на базі масиву порівняно зі зв'язним списком.
- Що таке переповнення стека (stack overflow) і як його уникнути?
- Як реалізувати чергу за допомогою двох стеків?
- Поясніть призначення стандартних контейнерів `std::stack`, `std::queue` та `std::deque` в C++.

Приклад виконання роботи:

```
#include <iostream>
```

```

#include <string>

// Проста реалізація стека на основі масиву
class Stack {
private:
    static const int MAX = 100;
    int top;
    char arr[MAX];

public:
    Stack() { top = -1; }

    bool push(char x) {
        if (top >= (MAX - 1)) return false;
        arr[++top] = x;
        return true;
    }

    char pop() {
        if (top < 0) return 0;
        return arr[top--];
    }

    bool isEmpty() { return (top < 0); }
};

int main() {
    std::string expression;
    std::cout << "Введіть вираз з дужками: ";
    std::getline(std::cin, expression);

    Stack s;
    bool balanced = true;

    for (char c : expression) {
        if (c == '(') {
            s.push(c);
        } else if (c == ')') {
            if (s.isEmpty()) {
                balanced = false;
                break;
            }
            s.pop();
        }
    }

    if (balanced && s.isEmpty()) {
        std::cout << "Баланс дужок дотримано." << std::endl;
    } else {
        std::cout << "Помилка в балансі дужок." << std::endl;
    }

    return 0;
}

```

Тема роботи 10.3: Дерева

Мета роботи:

- Ознайомитися з концепцією нелінійних структур даних.
- Навчитися реалізовувати бінарні дерева пошуку (BST).
- Опанувати методи обходу дерев (прямий, симетричний, зворотний).
- Реалізувати алгоритми вставки, видалення та пошуку елементів у дереві.

Порядок виконання роботи:

- Ознайомитися з теоретичним матеріалом щодо бінарних дерев.
- Отримати варіант завдання від викладача.
- Розробити алгоритм розв'язання задач свого варіанту.
- Написати програму на мові C++, використовуючи структури або класи.
- Протестувати програму на різних вхідних даних.
- Підготувати звіт, що містить код програми, результати тестування та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Знайти кількість листків у дереві.
2. Знайти суму значень усіх вузлів.
3. Визначити максимальну глибину дерева.
4. Вивести всі парні елементи дерева.
5. Написати функцію пошуку мінімального елемента.

Варіант 2

1. Обчислити кількість внутрішніх вузлів (не листків).
2. Знайти середнє арифметичне значень вузлів.
3. Реалізувати зворотний обхід (Post-order).
4. Вивести всі вузли, що мають лише одного нащадка.
5. Знайти вузол із заданим значенням X.

Варіант 3

1. Знайти кількість вузлів на заданому рівні K.
2. Перевірити, чи є дерево порожнім.
3. Знайти максимальний елемент у дереві.
4. Вивести значення вузлів у порядку спадання.
5. Видалити всі листки дерева.

Варіант 4

1. Порахувати загальну кількість вузлів у дереві.
2. Знайти різницю між максимальним та мінімальним значенням.
3. Реалізувати прямий обхід (Pre-order).
4. Вивести значення вузлів, які більші за число N.
5. Визначити, чи є дерево збалансованим за висотою.

Варіант 5

1. Знайти кількість правих нащадків у дереві.
2. Обчислити добуток значень усіх листків.
3. Вивести шлях від кореня до максимального елемента.
4. Визначити ширину дерева (макс. кількість вузлів на рівні).
5. Видалити корінь дерева та перебудувати його.

Варіант 6

1. Знайти кількість лівих нащадків.
2. Вивести всі непарні елементи дерева.
3. Знайти батька для вузла із заданим значенням.
4. Обчислити висоту лівого піддерева.
5. Перевірити, чи є дерево дзеркальним відносно іншого дерева.

Варіант 7

1. Знайти кількість вузлів, що мають двох нащадків.
2. Обчислити суму елементів на рівні N.
3. Вивести вузли, що знаходяться на непарних рівнях.
4. Знайти другий за величиною елемент у BST.
5. Очистити дерево (видалити всі вузли).

Варіант 8

1. Знайти кількість вузлів, значення яких кратні 3.
2. Визначити висоту правого піддерева.
3. Вивести всі значення вузлів, що лежать у діапазоні [A, B].
4. Перевірити, чи є дерево повним бінарним деревом.
5. Скопіювати структуру дерева в інше дерево.

Варіант 9

1. Порахувати кількість вузлів, що мають значення менше за корінь.
2. Знайти суму значень усіх "лівих" листків.
3. Вивести всі вузли, що є "одинаками" (один нащадок).
4. Визначити рівень, на якому знаходиться мінімальний елемент.
5. Видалити вузол із двома нащадками.

Варіант 10

1. Знайти кількість вузлів із парними значеннями.
2. Обчислити медіану значень у бінарному дереві пошуку.
3. Вивести дерево графічно (у вигляді ієрархії в консолі).
4. Перевірити, чи є дерево "строгим" (кожен вузол має 0 або 2 нащадки).
5. Знайти найменшого спільного предка (LCA) для двох вузлів.

Варіант 11

1. Знайти суму значень усіх внутрішніх вузлів.
2. Визначити кількість рівнів у дереві.
3. Вивести значення листків у порядку зліва направо.
4. Перевірити, чи є дерево піддеревом іншого дерева.
5. Знайти шлях найбільшої довжини між двома листками.

Варіант 12

1. Порахувати вузли, значення яких закінчуються на 5.
2. Знайти суму елементів на останньому рівні.
3. Вивести всі вузли, висота яких дорівнює H .
4. Реалізувати ітеративний пошук у дереві.
5. Створити дзеркальне відображення дерева.

Варіант 13

1. Знайти кількість вузлів, у яких ліве піддерево порожнє.
2. Обчислити середнє значення листків.
3. Вивести вузли, значення яких є простими числами.
4. Знайти глибину конкретного вузла.
5. Видалити всі вузли з від'ємними значеннями.

Варіант 14

1. Порахувати вузли, у яких праве піддерево порожнє.
2. Знайти максимальний елемент серед листків.
3. Вивести всі вузли, що мають хоча б одного нащадка.
4. Перевірити, чи всі значення в дереві унікальні.
5. Побудувати BST з відсортованого масиву.

Варіант 15

1. Знайти кількість вузлів, значення яких більші за середнє арифметичне.
2. Обчислити суму значень вузлів на парних рівнях.
3. Вивести шлях від кореня до заданого елемента X .
4. Визначити ступінь кожного вузла.
5. Перетворити дерево на зв'язаний список.

Варіант 16

1. Знайти кількість вузлів на самому нижньому рівні.
2. Знайти мінімальний елемент серед внутрішніх вузлів.
3. Вивести дерево по рівнях (обхід у ширину).
4. Перевірити, чи є дерево збалансованим по AVL-критерію.
5. Збільшити всі значення вузлів на константу K .

Варіант 17

1. Порахувати вузли, що мають значення в інтервалі (10, 50).
2. Знайти суму значень "правих" листків.
3. Вивести батьків усіх листків.
4. Визначити кількість вузлів, що мають тільки лівого нащадка.
5. Видалити всі вузли, що мають значення менше за X .

Варіант 18

1. Знайти суму значень вузлів, що не є листками.
2. Визначити кількість вузлів, що мають тільки правого нащадка.
3. Вивести значення вузлів у зворотному алфавітному порядку (якщо дані - символи).
4. Перевірити, чи є дерево симетричним.
5. Знайти вузол з максимальним значенням у лівому піддереві.

Варіант 19

1. Порахувати кількість вузлів з непарними значеннями на 3-му рівні.
2. Знайти добуток усіх вузлів, що мають двох нащадків.
3. Вивести найкоротший шлях від кореня до будь-якого листка.
4. Перевірити, чи є дане дерево деревом пошуку.
5. Видалити піддерево, корінь якого має значення X .

Варіант 20

1. Знайти кількість вузлів, значення яких рівні заданому V .
2. Обчислити суму значень вузлів, що мають рівно одного нащадка.
3. Вивести всі вузли, які знаходяться на максимальній глибині.
4. Визначити коефіцієнт збалансованості кореня.
5. Виконати ротацію дерева вправо навколо кореня.

Варіант 21

1. Порахувати кількість вузлів, чиї значення є квадратами цілих чисел.
2. Знайти суму значень усіх вузлів, крім кореня.
3. Вивести всі шляхи від кореня до кожного листка.
4. Порівняти два дерева на ідентичність.
5. Знайти елемент, що є наступником (successor) для X .

Варіант 22

1. Знайти кількість вузлів, що мають значення, кратне їх глибині.
2. Обчислити суму значень вузлів на непарних рівнях.
3. Вивести всі вузли, які мають значення більше за своїх батьків.
4. Знайти глибину самого "лівого" вузла.
5. Виконати ротацію дерева вліво навколо кореня.

Варіант 23

1. Порахувати кількість вузлів, що мають значення менше за середнє.
2. Знайти суму від'ємних значень у дереві.
3. Вивести листки в порядку спадання їх значень.
4. Перевірити, чи є дерево "деревом-паліндромом" (структурно).
5. Об'єднати два бінарні дерева пошуку в одне.

Варіант 24

1. Знайти кількість вузлів на передостанньому рівні.
2. Обчислити добуток значень усіх парних вузлів.
3. Вивести вузли, які є "братами" для заданого вузла X.
4. Визначити довжину найдовшої гілки.
5. Знайти елемент, що є попередником (predecessor) для X.

Варіант 25

1. Порахувати кількість вузлів, що є коренями піддерев з висотою 1.
2. Знайти суму значень вузлів, які є лівими нащадками.
3. Вивести значення вузлів, що мають парну кількість нащадків у своєму піддереві.
4. Визначити, скільки разів число X зустрічається в дереві.
5. Перетворити бінарне дерево на масив.

Контрольні запитання для звіту:

- Що таке бінарне дерево пошуку (BST)?
- Чим відрізняється дерево від графа?
- Яка часова складність пошуку елемента в ідеально збалансованому бінарному дереві?
- Опишіть три основні типи обходу бінарного дерева.
- Що таке "висота дерева" та як її обчислити рекурсивно?

Приклад виконання роботи:

```
#include <iostream>

struct Node {
    int data;
    Node* left;
    Node* right;

    Node(int val) : data(val), left(nullptr), right(nullptr) {}
};

// Функція вставки
Node* insert(Node* root, int val) {
    if (root == nullptr) return new Node(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
}

// Симетричний обхід (In-order) - виводить елементи у зростаючому порядку
void inOrder(Node* root) {
    if (root == nullptr) return;
    inOrder(root->left);
    std::cout << root->data << " ";
    inOrder(root->right);
}

int main() {
    Node* root = nullptr;
    int values[] = {50, 30, 70, 20, 40, 60, 80};

    for (int v : values) {
        root = insert(root, v);
    }

    std::cout << "In-order traversal: ";
    inOrder(root);
    return 0;
}
```

Тема роботи 10.4: Графи

Мета роботи:

- Ознайомитися з основними поняттями теорії графів (вершини, ребра, ваги, орієнтованість).
- Вивчити способи представлення графів у пам'яті комп'ютера (матриця суміжності, список суміжності).
- Набути навичок реалізації базових алгоритмів обходу графів (BFS, DFS) та пошуку специфічних характеристик мовою C++.

Порядок виконання роботи:

- Ознайомтеся з 5-ма завданнями свого варіанту.
- Виберіть оптимальну структуру збереження графа для ваших завдань. Для задач на пошук шляхів рекомендується список суміжності (`std::vector<std::vector<int>>`), для задач на перевірку властивостей ребер — матриця суміжності.
- Напишіть програмний код на C++. Кожне завдання варіанту може бути реалізоване як окрема функція в одній програмі або як окремі файли.
- Підготуйте тестові дані (набори ребер) та перевірте коректність роботи алгоритмів.
- Підготуйте звіт, що містить титульну сторінку, код програми, результати виконання та відповіді на контрольні запитання.
- Здати завдання через віртуальний клас Google Classroom.

Результат виконання роботи: заповнений звіт.

Варіанти завдань для виконання практичної роботи:

Варіант 1

1. Реалізувати введення орієнтованого графа через матрицю суміжності.
2. Вивести всі вершини, які є ізольованими.
3. Реалізувати алгоритм DFS для перевірки наявності шляху між двома заданими вершинами.
4. Порахувати загальну кількість ребер у графі.
5. Знайти вершину з найбільшим напівстепенем виходу.

Варіант 2

1. Реалізувати введення неорієнтованого графа через список суміжності.
2. Знайти всі "висячі" вершини (ступінь дорівнює 1).
3. Реалізувати алгоритм BFS для пошуку найкоротшої відстані (в ребрах).
4. Перевірити, чи є граф повним.
5. Вивести матрицю суміжності на основі введеного списку.

Варіант 3

1. Реалізувати введення зваженого орієнтованого графа.
2. Знайти всі пари вершин, між якими існує пряме ребро в обох напрямках.
3. Реалізувати алгоритм Дейкстри для однієї вершини.
4. Визначити щільність графа.
5. Перевірити наявність петлі у графа.

Варіант 4

1. Створити матрицю інцидентності для неорієнтованого графа.
2. Знайти компоненти зв'язності графа.
3. Реалізувати пошук у глибину для виявлення циклів.
4. Вивести всі вершини, суміжні з вершиною X.
5. Знайти сумарну вагу всіх ребер у зваженому графі.

Варіант 5

1. Реалізувати алгоритм побудови доповнення графа (complement graph).
2. Використати BFS для перевірки графа на зв'язність.
3. Знайти ексцентриситет заданої вершини.
4. Порахувати кількість трикутників (циклів довжини 3) у графі.
5. Визначити, чи є граф регулярним (всі степені рівні).

Варіант 6

1. Реалізувати перетворення матриці суміжності у список ребер.
2. Знайти джерела та стоки в орієнтованому графі.
3. Реалізувати топологічне сортування графа.
4. Обчислити радіус графа.
5. Перевірити, чи є задана послідовність вершин шляхом у графі.

Варіант 7

1. Побудувати граф за списком ребер та знайти його діаметр.
2. Реалізувати алгоритм Прима для знаходження мінімального кістякового дерева.
3. Знайти вершину-центр графа.
4. Вивести всі вершини, степені яких є парними числами.
5. Перевірити, чи є граф деревом.

Варіант 8

1. Реалізувати алгоритм Краскала для зваженого графа.
2. Знайти напівстепені заходження для кожної вершини.
3. Перевірити граф на наявність Ейлерового циклу.
4. Видалити задану вершину та всі інцидентні ребра з графа.
5. Знайти всі вершини на відстані k від заданої.

Варіант 9

1. Реалізувати алгоритм Флойда-Воршелла для пошуку всіх найкоротших шляхів.
2. Перевірити, чи є граф двочастковим.
3. Знайти кількість ребер, які потрібно додати, щоб граф став повним.
4. Вивести всі цикли довжини 4.
5. Визначити транзитивне замикання графа.

Варіант 10

1. Реалізувати алгоритм Беллмана-Форда.
2. Знайти мости у неорієнтованому графі.
3. Перевірити граф на сильну зв'язність.
4. Побудувати конденсацію орієнтованого графа.
5. Знайти максимальний степінь серед усіх вершин.

Варіант 11

1. Знайти точки зчленування графа.
2. Реалізувати BFS для пошуку всіх вершин у тій же компоненті зв'язності, що й задана.
3. Визначити, чи є граф ациклічним.
4. Обчислити середній степінь вершини графа.
5. Знайти ребро з максимальною вагою.

Варіант 12

1. Реалізувати алгоритм Косарайю для пошуку компонент сильної зв'язності.
2. Знайти відстань між найбільш віддаленими вершинами.
3. Перевірити, чи містить граф Гамільтонів шлях (спрощений пошук).
4. Вивести список усіх ребер, впорядкований за вагою.
5. Обчислити кількість шляхів довжиною 2 між двома вершинами.

Варіант 13

1. Реалізувати представлення графа через динамічний масив об'єктів `Vertex`.
2. Знайти медіану графа.
3. Визначити, чи є граф планарним (за формулою Ейлера для малих графів).
4. Реалізувати операцію об'єднання двох графів.
5. Порахувати кількість вершин з непарним степенем.

Варіант 14

1. Знайти найкоротший шлях у зваженому графі за допомогою DFS (з обмеженням ваги).
2. Визначити, чи є граф орієнтованим деревом.
3. Побудувати інвертований граф (зміна напрямку всіх ребер).
4. Вивести матрицю суміжності після піднесення її до квадрату.
5. Знайти всі вершини, що не входять до жодного циклу.

Варіант 15

1. Реалізувати алгоритм Едмондса-Карпа для пошуку максимального потоку.
2. Визначити обхват графа (довжина найкоротшого циклу).
3. Перевірити, чи є граф ізоморфним іншому графу (для 4-5 вершин).
4. Знайти всі кліки розміру 3.
5. Обчислити кількість компонент зв'язності після видалення ребра.

Варіант 16

1. Реалізувати алгоритм пошуку в глибину з використанням стека (без рекурсії).
2. Знайти всі вершини, що мають самопетлі.
3. Визначити, чи є граф зіркою.
4. Знайти суму ваг ребер, що виходять з заданої вершини.
5. Перевірити, чи є граф підграфом іншого заданого графа.

Варіант 17

1. Реалізувати алгоритм "найближчого сусіда" для задачі комівояжера.
2. Знайти периферичні вершини графа.
3. Перевірити, чи є граф сильно регулярним.
4. Вивести всі маршрути довжини 3 між вершинами A та B.
5. Визначити кількість ребер у найбільшій компоненті зв'язності.

Варіант 18

1. Реалізувати алгоритм Таріана для пошуку точок зчленування.
2. Знайти відстань від кожної вершини до центру графа.
3. Визначити, чи є орієнтований граф ациклічним (DAG).
4. Створити випадковий граф з N вершинами та M ребрами.
5. Вивести ступені заходження та виходу для кожної вершини.

Варіант 19

1. Знайти найкоротший шлях у графі з від'ємними вагами (якщо немає циклів).
2. Реалізувати перетворення списку суміжності в матрицю інцидентності.
3. Визначити число незалежності графа (для малих графів).
4. Перевірити, чи є граф критично зв'язним.
5. Знайти всі вершини, суміжні одночасно двом заданим вершинам.

Варіант 20

1. Реалізувати алгоритм Форда-Фалкерсона.
2. Знайти найдовший простий шлях у дереві.
3. Визначити, чи є граф Ейлеровим.
4. Порахувати кількість петель та кратних ребер.
5. Вивести всі вершини, які недосяжні з заданої вершини.

Варіант 21

1. Знайти мінімальне розфарбування вершин графа (жадібний алгоритм).
2. Визначити, чи є граф Гамільтоновим (за теоремою Дірака).
3. Реалізувати стиснення графа (видалення вершин степеня 0).
4. Знайти кількість ребер у доповненні до повного графа.
5. Вивести список ребер, інцидентних заданій вершині.

Варіант 22

1. Реалізувати пошук у ширину для знаходження всіх вершин на рівні L дерева.
2. Знайти найменшу вагу ребра в кожній компоненті зв'язності.
3. Визначити, чи є граф турніром.
4. Обчислити циклічне число графа ($M - N + C$).
5. Знайти вершини, видалення яких розриває граф на 3 частини.

Варіант 23

1. Реалізувати алгоритм Джонсона для всіх пар найкоротших шляхів.
2. Знайти всі орієнтовані цикли в графі.
3. Визначити, чи є граф мережею (наявність джерела та стоку).
4. Порахувати кількість "листіків" у графі.
5. Перевірити, чи сума степенів вершин дорівнює подвійній кількості ребер.

Варіант 24

1. Реалізувати пошук K -найкоротших шляхів (спрощено).
2. Знайти внутрішню стійкість графа.
3. Визначити, чи є граф самодоповнюваним.
4. Вивести матрицю суміжності для орієнтованого графа без циклів.
5. Знайти максимальну вагу шляху в дереві.

Варіант 25

1. Реалізувати алгоритм Брона-Кербоша для пошуку максимальних клік.
2. Знайти центр графа у зваженому варіанті.
3. Визначити, чи є граф скелетом іншого графа.
4. Порахувати кількість ребер у кожному степені вершини.
5. Знайти найкоротший шлях, що проходить через задане ребро.

Контрольні запитання для звіту:

- Яка різниця між матрицею суміжності та списком суміжності за витратами пам'яті?
- Дайте визначення поняттю "компонента зв'язності".
- Як за допомогою матриці суміжності визначити, чи є граф орієнтованим?
- Яка часова складність алгоритму пошуку в глибину (DFS)?
- Що таке зважений граф і як він впливає на вибір алгоритму пошуку найкоротшого шляху?

Приклад виконання роботи:

```
#include <iostream>
#include <vector>
```

```

using namespace std;

// Функція для створення матриці суміжності та підрахунку степенів
void processGraph() {
    int v, e;
    cout << "Введіть кількість вершин та ребер: ";
    cin >> v >> e;

    // Створення матриці суміжності розміром V x V, заповненої нулями
    vector<vector<int>> matrix(v, vector<int>(v, 0));

    cout << "Введіть пари вершин (ребра):" << endl;
    for (int i = 0; i < e; ++i) {
        int start, end;
        cin >> start >> end;
        // Оскільки граф неорієнтований, ставимо 1 в обох клітинках
        if (start < v && end < v) {
            matrix[start][end] = 1;
            matrix[end][start] = 1;
        }
    }

    // Виведення степенів вершин
    cout << "\nРезультати (Степені вершин):" << endl;
    for (int i = 0; i < v; ++i) {
        int degree = 0;
        for (int j = 0; j < v; ++j) {
            if (matrix[i][j] == 1) degree++;
        }
        cout << "Вершина " << i << ": степінь = " << degree << endl;
    }
}

int main() {
    processGraph();
    return 0;
}

```

МЕТОДИЧНІ МАТЕРІАЛИ, ЩО ЗАБЕЗПЕЧУЮТЬ САМОСТІЙНУ РОБОТУ СТУДЕНТІВ

З метою забезпечення самостійної роботи студентів пропонується ряд безкоштовних та вільно доступних у Мережі тренінгів, книг та сертифікаційних курсів.

1. C++ Essentials 1/ Cisco Networking Academy - <https://www.netacad.com/courses/c-plus-plus-essentials-1?courseLang=en-US>
2. C++ Essentials 2 / Cisco Networking Academy - <https://www.netacad.com/courses/c-plus-plus-essentials-2?courseLang=en-US>
3. C++ Advanced / Cisco Networking Academy - <https://www.netacad.com/courses/c-plus-plus-advanced?courseLang=en-US>
4. C++ вже сьогодні! / SpacaLAB - <https://spacelab-academy.com/uk/course/c/>
5. Learn C++ / Codecademy - <https://www.codecademy.com/enrolled/courses/learn-c-plus-plus>

НАВЧАЛЬНО-МЕТОДИЧНІ МАТЕРІАЛИ ДЛЯ ПРОМІЖНОГО КОНТРОЛЮ ТА ПЕРЕЛІК ПИТАНЬ ПІДСУМКОВОГО КОНТРОЛЮ

Проміжний та підсумковий контроль здійснюється з допомогою автоматизованого тестування. Проміжне тестування – на платформі Google Classroom з використанням Google Forms. Посилання на тести доступні у віртуальному Класі в день тестування, деякі з них наведені далі.

Підсумковий контроль - (за умови виконання всіх практичних робіт) здійснюється на платформі Google Classroom. Завдання іспиту містять питання теоретичного (60) і практичного спрямування (30) та тестове завдання(60) загальною кількістю 150 питань, з яких під час іспиту для кожного студента випадковим чином обирається 2 теоретичних , одне практичне завдання та тестове завдання (автоматично оцінюється) з 60 запитань.

Приклад тестових завдань для проміжного контролю:

Варіант 1

- Програма мовою C++ починає виконуватися з :
 - функції main
 - першої функції в програмі
 - тієї функції , яка вказана як стартова при компіляції програми
 - з кінця програми
- Яке буде значення змінної k після виконання наступного оператора $d = ++d$; якщо до його виконання d дорівнювало 5 ?
 - 6
 - 4
 - 7
 - 1
- Відзначте правильне оголошення змінної:
 - `float ; float = y ;`
 - `int 1h;`
 - `char float = 53.5 ;`
 - `int x ; int y ; int X ;`
- Вкажіть в якому виразі не використовуються ключові слова ?
 - `TStringList * S = new TStringList ;`
 - `sdf = 2 ; int r = 24 ;`
 - `x = 3 ; x = x + 4 ;`
 - `void function ( )`
- Оберіть не правильний ідентифікатор:
 - `static_value`
 - `_test2data`
 - `Switch`
 - `base-form`

6. Чому дорівнює значення цілої змінної при обчисленні виразу $16 / 5 * 3$?
- a) 9.6
 - b) інше значення
 - c) 9
 - d) 1.06
7. Який з нижче перерахованих операторів , не є циклом в C++ ?
- a) while
 - b) repeat until
 - c) do while
 - d) for
8. Який результат роботи наступного фрагмента коду ?
- ```
int x = 0 ;
switch (x)
{
 case 1 : cout << "Один" ;
 case 0 : cout << " Нуль " ;
 case 2 : cout << " Привіт світ " ;
}
```
- a) Привіт світ
  - b) Нуль
  - c) НульПривіт світ
  - d) Один
9. Яка з наступних записів - правильний коментар в C++ ?
- a) \* / Коментарі \* /
  - b) / \* коментар \* /
  - c) {коментар}
  - d) \*\* Коментар \*\*
10. Який службовий знак ставиться після оператора case?
- a);
  - b).
  - c):
  - d) –
11. Вкажіть об'єктно-орієнтовану мову програмування
- a) Всі варіанти відповідей
  - b) C++
  - c) Eiffel
  - d) Java
12. Виберіть правильний варіант оголошення константної змінної в C++, де type - тип даних в C++ variable - ім'я змінної value - константне значення
- a) const type variable = value;
  - b) const type variable: = value;
  - c) type variable=100;
  - d) const variable = value;

13. Що з'явиться на екрані, після виконання цього фрагмента коду?
- ```
int c = 3, x = 5;  
if (c == x);  
cout << c << "=" << x << endl;
```
- a) 3 = 5
 - b) синтаксична помилка
 - c) вивід на екран не виконається
 - d) c = x
14. Чому буде дорівнювати змінна Y після виконання умови, якщо x=9? Умова: if x>0 if x>0 Y=x+x; else Y=x*x;
- a) 9
 - b) 81
 - c) 18
 - d) немає правильної відповіді
15. Яку дію виконує оператор "cout"?
- a) вивід даних на екран
 - b) введення даних з клавіатури
 - c) видалення елемента масиву
 - d) немає вірної відповіді
16. Обчисліть результат виразу `int X; X = ((5 + 5 * 4) % 10) / 4;`
- a) 0
 - b) 1.25
 - c) 1
 - d) -1
17. Скільки разів буде виконуватися цикл `i=15; do i=i-2 while i>5;?`
- a) 1 раз
 - b) 5 раз
 - c) 4 рази
 - d) нескінчену кількість разів
18. Потрібно підрахувати суму натуральних чисел від 5 до 125. Яку умову потрібно використовувати в циклі While?
- a) $i > 125$
 - b) $i \geq 125$
 - c) $i < 125$
 - d) $i \leq 125$
19. Яка операція не належить до операцій з пам'яттю?
- a) new
 - b) delete
 - c) &
 - d) ()
20. Для визначення нового класу використовується ключове слово...
- a) struct або class
 - b) new
 - c) main
 - d) немає вірної відповіді

21. Конструктор, що приймає посилання на власний клас, називається...
- a) конструктор копіювання
 - b) пустий конструктор
 - c) деструктор
 - d) немає правильної відповіді
22. Оператор виділення пам'яті...
- a) delete
 - b) new
 - c) &
 - d) []
23. Що є результатом компонування програми?
- a) заголовний файл
 - b) виконуваний файл чи бібліотека
 - c) набір заголовних файлів з визначення в них усіх виконуваних функцій
 - d) немає вірної відповіді
24. Що знаходиться в запису мінімального по своїм можливостям класу?
- a) тільки деструктор та конструктор
 - b) не знаходиться нічого
 - c) не менше одного методу
 - d) не менше одного атрибуту та хоч би один метод-конструктор класу
25. Розроблений в C++ потік виводу, це...
- a) cout
 - b) cin
 - c) std
 - d) scanf
26. Що означає наступна інструкція? `cin << x;`
- a) введення значення в змінну x зі стандартного потоку cin
 - b) виведення значення змінної x в стандартний потік cout
 - c) введення двох змінних
 - d) немає вірної відповіді
27. Який маніпулятор означає, що значення змінних типу bool виводяться як true і false ?
- a) noboolalpha
 - b) boolalpha
 - c) dec
 - d) fixed
28. Для використання файлових потоків треба включити заголовний файл...
- a) <fstream>
 - b) <iostream>
 - c) <stdio.h>
 - d) <conio.h>

29. Для того, щоб перевірити, чи був досягнутий кінець файлу, використовується функція...
- a) seekg/seekp
 - b) tellg/tellp
 - c) eof
 - d) is_open
30. Що таке *методи* класу?
- a) це його змінні
 - b) це його функції
 - c) це його бібліотеки
 - d) немає вірної відповіді
31. Специфікатор класу пам'яті register...
- a) Вказує на те, що змінна є локальною для кожного виклику блоку та зникає при виході з блоку
 - b) Вказує на те, що змінна є локалізованою в блоці, але при виході з блоку вона зберігає своє значення для цього блоку
 - c) Вказує на те, що змінна зберігає своє значення на весь час виконання програми
 - d) Вказує на те, що змінна зберігається, якщо це можливо, у регістрах, для підвищення швидкості програми? Час життя – локальний.
32. Яка перевага використання ключового слова const замість директиви #define?
- a) константу, визначену за допомогою const, можна змінювати під час роботи
 - b) до константи, визначеної за допомогою const, можна застосовувати операції інкременту та декременту
 - c) константа, визначена за допомогою const, доступна в інших модулях програми
 - d) константа, визначена за допомогою const, має тип, і компілятор може прослідкувати за її використанням у відповідності з об'явленим типом
33. Що відбудеться, якщо визначення функції буде знаходитися в двох місцях?
- a) буде використано друге визначення
 - b) залежить від інших факторів
 - c) помилка компіляції
 - d) друге визначення буде проігноровано
34. Що відбудеться при використанні невірної адреси в операції delete?
- a) відбудеться аварійне завершення програми
 - b) програма видасть повідомлення, що пам'ять звільняється по невірній адресі
 - c) результат непередбачуваний
 - d) немає вірної відповіді
35. Що відбудеться після наступного прикладу:
- ```
cout << "\n\n" << "New program" << "\a";
```
- a) продзвенить дзвінок та з'явиться повідомлення "New program"
  - b) в третьому рядку з'явиться повідомлення "New program" та продзвенить дзвінок
  - c) з'явиться повідомлення "New programa"
  - d) немає вірної відповіді

36. Що відбудеться після об'ви в програмі даного набору перераховуваних значень

```
enum{N=0, E=1, S=2, W=3};?
```

- a) програма буде працювати з числовими значеннями N, E, S, W
- b) програма буде працювати з ідентифікаторами N, E, S, W
- c) програма сформує ітератори з вказівниками на N, E, S, W
- d) програма буде ігнорувати усі вказівники на N, E, S, W

37. Що описує даний програмний код?

```
struct {
char fio[30];
int date, code;
double salary;
}staff[100], *ps;
```

- a) визначення масиву структур і вказівника на структуру
- b) копіювання рядка salary в рядок staff
- c) створення вказівника на рядок date
- d) створення ітератора з вказівником на рядок date

38. Що значить запис while (false)?

- a) нескінченний цикл
- b) цикл, який не виконується жодного разу
- c) помилка компіляції
- d) аварійний вихід з програми

39. Що означає запис cout << flush?

- a) вивести переведення рядка
- b) вивести символ нового рядка
- c) вивести нульовий байт
- d) призвести виведення та очистку буферів

40. Що буде робити функція find(arr+2, arr+ARR\_SIZE, 5)?

- a) шукати число 5 на інтервалі від 2 до ARR\_SIZE
- b) шукати 5 в масиві arr починаючи від другого елементу
- c) нічого, ця запис помилкова
- d) немає вірного варіанту

### Варіант 3

1. Ключове слово void позначає що функція

- a) повертає ціле число
- b) є головною
- c) повертає число з плаваючою комою
- d) нічого не повертає

2. Вкажіть всі ключові слова в наведеному прикладі ?
- ```
int calc ( int a , int b , bool f )  
{    if ( f == 1 )  
    return a + b ;  
else  
    return a * b ;  
}
```
- a) int , bool , if , else , return
b) int , if , else , return
c) int , calc , bool , return , if , else
d) if, else
3. Оберіть правильний ідентифікатор:
- a) aB12bC
b) for
c) big\$test
d) 4matic
4. Яке буде значення змінної k після виконання наступного оператора
k = + + k ;
якщо до його виконання k дорівнювало 6 ?
- a) 6
b) 8
c) 7
d) 1
5. Чому дорівнює значення цілої змінної при обчисленні виразу $21 / 5 * 3$?
- a) 13.02
b) інше значення
c) 12
d) 1.47
6. Який результат роботи наступного фрагмента коду ?
- ```
int x = 2 ;
switch (x)
{ case 1 : cout << "Один" ;
 case 0 : cout << " Нуль " ;
 case 2 : cout << " Привіт світ " ;
}
```
- a) Привіт світ  
b) Нуль  
c) НульПривет світ  
d) Один
7. Яку функцію повинні містити всі програми на C ++?
- a) main()  
b) system()  
c) start()  
d) program()

8. Які службові символи використовуються для позначення початку і кінця блоку коду?
- a) {}
  - b) begin end
  - c) <>
  - d) ()
9. Назву C++ запропонував
- a) Бьєрн Страуструп
  - b) Рик Массітті
  - c) Кен Томпсон
  - d) Дональд Кнут
10. Виберіть правильний варіант оголошення константної змінної в C++
- a) `const int a = 10;`
  - b) `const int a: = 10;`
  - c) `int a=10;`
  - d) `const a = 10;`
11. До яких пір будуть виконуватися оператори в тілі циклу `while (x > 100)` у мові C++?
- a) Поки `x` дорівнює `стам`
  - b) Поки `x` строго менше `ста`
  - c) Поки `x` менше або дорівнює `стам`
  - d) Поки `x` строго більше `ста`
12. В якому випадку можна не використовувати фігурні скобочки в операторі вибору `if`?
- a) немає правильної відповіді
  - b) якщо в тілі оператора `if` два і більше операторів
  - c) якщо в тілі оператора `if` немає жодного оператора
  - d) якщо в тілі оператора `if` всього один оператор
13. Що з'явиться на екрані, після виконання цього фрагмента коду?
- ```
int a = 3, b = 4;  
if (a == b);  
cout << a << "=" << b << endl;
```
- a) `3 = 4`
 - b) синтаксична помилка
 - c) вивід на екран не виконається
 - d) `a = b`
14. Чому буде дорівнювати змінна `Y` після виконання умови, якщо `x=-9`? Умова: `if x>0 Y=x*x;`
`else Y=x+x;`
- a) 3
 - b) 81
 - c) 9
 - d) -18
15. Яку дію виконує оператор `"cin"`?
- a) виведення даних на екран
 - b) введення даних з клавіатури
 - c) перетворення елемента масиву
 - d) немає вірної відповіді

16. Обчисліть результат виразу `int A; A = ((19 - 3 * 2) / 3) % 4;`
- a) 1
 - b) 0
 - c) 0.33
 - d) 0.34
17. Потрібно підрахувати суму натуральних чисел від 2 до 22. Яку умову потрібно використовувати в циклі `While`?
- a) `i > 23`
 - b) `i >= 22`
 - c) `i < 22`
 - d) `i <= 22`
18. Скільки разів буде виконуватися цикл `i=12; do i=i-2 while i>4;?`
- a) 1 раз
 - b) 5 раз
 - c) 4 рази
 - d) нескінченну кількість разів
19. Яка операція не належить до операцій з пам'яттю?
- a) `new`
 - b) `delete`
 - c) `++`
 - d) `&`
20. Відкритий доступ, коли члени класу доступні з будь-якої точки програми, це...
- a) `public`
 - b) `protected`
 - c) `private`
 - d) немає вірної відповіді
21. Конструктор без аргументів, називається...
- a) конструктор копіювання
 - b) пустий конструктор
 - c) деструктор
 - d) немає правильної відповіді
22. Оператор звільнення пам'яті...
- a) `delete`
 - b) `new`
 - c) `&`
 - d) `[]`
23. Що є мінімальною областю видимості імен?
- a) модуль
 - b) блок
 - c) функція
 - d) клас
24. Що таке `cout`?
- a) об'єкт типу `iostream`
 - b) клас, котрий виводить дані на термінал

- c) змінна, яку програміст повинен створити для виведення даних
 - d) немає вірної відповіді
25. Розроблений в C++ потік вводу, це...
- a) cout
 - b) cin
 - c) std
 - d) scanf
26. Що означає наступна інструкція? `cout << x;`
- a) введення значення в змінну x зі стандартного потоку cin
 - b) виведення значення змінної x в стандартний потік cout
 - c) введення двох змінних
 - d) немає вірної відповіді
27. Який маніпулятор означає, що значення змінних типу bool виводяться як 1 і 0?
- a) noboolalpha
 - b) boolalpha
 - c) dec
 - d) fixed
28. Для відкриття файлу для читання та запису треба задати таку інструкцію...
- a) `fstream fs("f1.txt");`
 - b) `ifstream ifs("f2.txt");`
 - c) `ofstream ofs("f3.txt");`
 - d) немає вірної відповіді
29. Для того, щоб перевірити, чи файл відкритий, використовується функція...
- a) `seekg/seekp`
 - b) `tellg/tellp`
 - c) `eof`
 - d) `is_open`
30. Перерахуйте директиви обмеження видимості в порядку «збільшення відкритості»
- a) `Public, protected, private.`
 - b) `Public, private, protected.`
 - c) `Private, public, protected.`
 - d) `Private, protected, public.`
31. Що таке *властивості* класу?
- a) це його змінні
 - b) це його функції
 - c) це його бібліотеки
 - d) немає вірної відповіді
32. Специфікатор класу пам'яті `extern...`
- a) Вказує на те, що змінна є локальною для кожного виклику блоку та зникає при виході з блоку
 - b) Вказує на те, що змінна є локалізованою в блоці, але при виході з блоку вона зберігає своє значення для цього блоку
 - c) Вказує на те, що змінна зберігає своє значення на весь час виконання програми
 - d) Вказує на те, що змінна зберігається, якщо це можливо, у регістрах, для підвищення швидкості програми? Час життя – локальний

33. Визначення класу – це...
- a) об'ява усіх його методів та полів
 - b) визначення усіх його методів
 - c) ініціалізація усіх його полів та виклик конструктора
 - d) немає вірної відповіді
34. Що відбудеться, якщо операція виділення пам'яті new завершиться невдало?
- a) відбудеться аварійне завершення програми
 - b) програма видасть повідомлення про неможливість виділення пам'яті під даний об'єкт та поверне нульовий вказівник
 - c) виділення пам'яті під об'єкт не відбудеться, та операція new поверне нульовий вказівник чи буде згенеровано виключення
 - d) немає вірної відповіді
35. Що відбудеться після наступного прикладу:
- ```
cout << "\n\n\n\n " << "Test" << "\a";
```
- a) продзвенить дзвінок та з'явиться повідомлення "Test"
  - b) в п'ятому рядку з'явиться повідомлення "Test" та продзвенить дзвінок
  - c) з'явиться повідомлення "Test"
  - d) немає вірної відповіді
36. Що відбудеться після об'ви в програмі даного набору перераховуваних значень
- ```
enum{P=0, R=1, T=2, Y=3};?
```
- a) програма буде працювати з числовими значеннями P, R, T, Y
 - b) програма буде працювати з ідентифікаторами P, R, T, Y
 - c) програма сформує ітератори з вказівниками на P, R, T, Y
 - d) програма буде ігнорувати усі вказівники на P, R, T, Y
37. Що описує даний програмний код?
- ```
struct {
char fio[40];
int wart, code;
double rew;
}mas[110], *ps;
```
- a) копіювання рядка rew в рядок mas
  - b) створення вказівника на рядок wart
  - c) визначення масиву структур і вказівника на структуру
  - d) створення ітератора з вказівником на рядок wart
38. Що означає запис cout << setw(3)?
- a) ширина поля виводу встановлюється рівною 3
  - b) виводяться рядки скорочуються до 3 символів
  - c) виводяться рядки доповнюються до 3 символів
  - d) не можна ввести більше 3 символів за один раз
39. Що таке динамічне виділення пам'яті?
- a) пам'ять під об'єкт (змінну) виділяється щоразу при зверненні до змінної
  - b) пам'ять під об'єкт (змінну) може виділятися не відразу, а в процесі роботи програми, звільнення пам'яті проводиться автоматично після завершення програми

- c) пам'ять під об'єкт (змінну) може виділятися не відразу, а в процесі роботи програми, звільнення пам'яті проводиться вручну
- d) усі твердження вірні

40. Що буде виведено на екран в результаті виконання наступного коду?

```
int a=3;
if (a>1) cout << "a>1";
if(a>2) cout << "a>2";
if(a>3) cout << "a>3";
```

- a) a>1a>2a>3
- b) a>1a>2
- c) a>1
- d) a>2

## Перелік екзаменаційних білетів

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

### Завдання № 1

1. Умовний оператор.
2. Масиви. Двомірні масиви.
3. Дано двозначне число знайти суму його цифр.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_ від „\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_  
(підпис)

О.В. Бабич  
(прізвище та ініціали)

Викладач \_\_\_\_\_  
підпис

Олійник В.В.  
(прізвище та ініціали)

-----  
МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

### Завдання № 2

1. Типи даних та змінні.
2. Цикл з умовою.
3. Дано 2 числа знайти більше з них.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_ від „\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_  
(підпис)

О.В. Бабич  
(прізвище та ініціали)

Викладач \_\_\_\_\_  
підпис

Олійник В.В.  
(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 3**

1. Цикл do-while.
2. Різниця функції від процедури.
3. Написати програму що виводить на екран повідомлення «Вам більше 18 років », якщо вести значення змінної більше 18.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

**Голова циклової комісії** \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

**Викладач** \_\_\_\_\_

підпис

підпис

Олійник В.В.

(прізвище та ініціали)

(прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення

Навчальна дисципліна „Програмування”

**Завдання № 4**

1. Процедури.
2. Умовний оператор.
3. Написати програму що виводить назву тварини на відповідну літеру, що ввели з клавіатури.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

**Голова циклової комісії** \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

**Викладач** \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 5**

1. Вказівники.
2. Операції над рядками.
3. Дано ціле число. Якщо воно є позитивним, то додати до нього 1; якщо негативним, то відняти від нього 2; якщо нульовим, то замінити його на 10. Вивести отримане число.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 6**

1. Множинний вибір.
2. Складний умовний оператор.
3. Дано два числа. Вивести порядковий номер меншого з них.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст

Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення

Навчальна дисципліна „Програмування”

**Завдання № 7**

1. Структури.
2. Масиви.
3. Дано цілі числа  $K$  і  $N$  ( $N > 0$ ). Вивести  $N$  раз  $K$ .

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_  
(підпис)

О.В. Бабич  
(прізвище та ініціали)

Викладач \_\_\_\_\_  
підпис  
підпис

Олійник В.В.  
(прізвище та ініціали)  
(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст

Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення

Навчальна дисципліна „Програмування”

**Завдання № 8**

1. Робота з текстовими файлами.
2. Функції роботи з рядками.
3. Дано ціле число  $N$  ( $> 0$ ). Використовуючи один цикл, знайти суму  
 $1! + 2! + 3! + \dots + N!$

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_  
(підпис)

О.В. Бабич  
(прізвище та ініціали)

Викладач \_\_\_\_\_  
підпис

Олійник В.В.  
(прізвище та ініціали)



МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 9**

1. Робота з файлами.
2. Сортювання масиву.
3. Дано тризначне число. Використовуючи одну операцію ділення остачі, вивести першу цифру даного числа (сотні).

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_ від „\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_  
(підпис)

О.В. Бабич  
(прізвище та ініціали)

Викладач \_\_\_\_\_  
підпис

Олійник В.В.  
(прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 10**

1. Масиви в процедурах та функціях.
2. Робота з двійковими файлами.
3. Відомо, що  $X$  кг шоколадних цукерок коштує  $A$  рублів, а  $Y$  кг ірисок варто  $B$  рублів. Визначити, скільки коштує 1 кг шоколадних цукерок, 1 кг ірисок, а також у скільки разів шоколадні цукерки дорожче ірисок.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_ від „\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_  
(підпис)

О.В. Бабич  
(прізвище та ініціали)

Викладач \_\_\_\_\_  
підпис

Олійник В.В.  
(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 11**

1. Вказівники.
2. Що таке структура.
3. Дано двозначне число знайти суму його цифр.

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)

Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 12**

1. Алгоритм роботи з матрицями.
2. Рекурсія.
3. Дано 2 числа знайти більше з них.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)

Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 13**

1. Швидке сортування.
2. Що таке масиви.
3. Написати програму що виводить на екран повідомлення «Вам більше 18 років», якщо вести значення змінної більше 18.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 14**

1. Простий пошук в масивах.
2. Сортування масивів.
3. Написати програму що виводить назву тварини на відповідну літеру, що ввели з клавіатури.

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 15**

1. Що таке масиви.
2. Типи даних. Змунні.
3. Дано ціле число. Якщо воно є позитивним, то додати до нього 1; якщо негативним, то відняти від нього 2; якщо нульовим, то замінити його на 10. Вивести отримане число.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

-----  
МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 16**

1. Рекурсивний пошук.
2. Масиви структур.
3. Дано два числа. Вивести порядковий номер меншого з них.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 17**

1. Робота з рядками.
2. Робота з файлами.
3. Дано цілі числа  $K$  і  $N$  ( $N > 0$ ). Вивести  $N$  раз  $K$ .

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)

Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 18**

1. Двійкові файли.
2. Що таке рекурсія.
3. Дано ціле число  $N$  ( $> 0$ ). Використовуючи один цикл, знайти суму  
 $1! + 2! + 3! + \dots + N!$

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)

Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 19**

1. Робота з текстовими файлами.
2. Заповнення масиву випадковими цифрами.
3. Дано тризначне число. Використовуючи одну операцію ділення остачі, вивести першу цифру даного числа (сотні).

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 20**

1. Рекурсивний пошук.
2. Алгоритм роботи з матрицями.
3. Відомо, що  $X$  кг шоколадних цукерок коштує  $A$  рублів, а  $Y$  кг ірисок варто  $B$  рублів. Визначити, скільки коштує 1 кг шоколадних цукерок, 1 кг ірисок, а також у скільки разів шоколадні цукерки дорожче ірисок.

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 21**

1. Що таке рекурсія.
2. Швидке сортування.
3. Відомо, що  $X$  кг шоколадних цукерок коштує  $A$  рублів, а  $Y$  кг ірисок варто  $B$  рублів. Визначити, скільки коштує 1 кг шоколадних цукерок, 1 кг ірисок, а також у скільки разів шоколадні цукерки дорожче ірисок.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 22**

1. Що таке масиви.
2. Умовний оператор.
3. Дано 2 числа знайти більше з них.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

(підпис)

О.В. Бабич

(прізвище та ініціали)

Викладач \_\_\_\_\_

підпис

Олійник В.В.

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 23**

1. Множинний вибір.
2. Цикл с умовою.
3. Написати програму що виводить на екран повідомлення «Вам більше 18 років », якщо вести значення змінної більше 18.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_ від „\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)  
Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 24**

1. Цикл do-while.
2. Масив рядків.
3. Відомо, що X кг шоколадних цукерок коштує А рублів, а Y кг ірисок варто В рублів. Визначити, скільки коштує 1 кг шоколадних цукерок, 1 кг ірисок, а також у скільки разів шоколадні цукерки дорожче ірисок.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_ від „\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)  
Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)



МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 25**

1. Матриці.
2. Робота з файлами
3. Дано ціле число. Якщо воно є позитивним, то додати до нього 1; якщо негативним, то відняти від нього 2; якщо нульовим, то замінити його на 10. Вивести отримане число.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

О.В. Бабич

(підпис)

(прізвище та ініціали)

Викладач \_\_\_\_\_

Олійник В.В.

підпис

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 26**

1. Робота з двійковими файлами.
2. Вказівники.
3. Дано цілі числа K і N ( $N > 0$ ). Вивести N раз K.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення

Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_

О.В. Бабич

(підпис)

(прізвище та ініціали)

Викладач \_\_\_\_\_

Олійник В.В.

підпис

(прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 27**

1. Робота з текстовими файлами.
2. Функції рядків.
3. Дано два числа. Вивести порядковий номер меншого з них.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)

Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 28**

1. Процедури.
2. Рекурсії.
3. Дано ціле число  $N (> 0)$ . Використовуючи один цикл, знайти суму  
 $1! + 2! + 3! + \dots + N!$

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)

Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 29**

1. Операції над рядками.
2. Множинний вибір
3. Дано тризначне число. Використовуючи одну операцію ділення остачі, вивести першу цифру даного числа (сотні).

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)  
Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

---

МІНІСТЕРСТВО ОСВІТИ І НАУКИ, УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Освітньо-кваліфікаційний рівень молодший спеціаліст  
Напрямок підготовки 12 Інформаційні технології

Спеціальність 121 Інженерія програмного забезпечення  
Навчальна дисципліна „Програмування”

**Завдання № 30**

1. Типи даних. Змінні.
2. Робота з двійковими файлами.
3. Дано двозначне число знайти суму його цифр.

---

Затверджено на засіданні ЦК Дисциплін інженерії програмного забезпечення  
Протокол №\_\_ від „\_\_” \_\_\_\_\_ року

Голова циклової комісії \_\_\_\_\_ О.В. Бабич  
(підпис) (прізвище та ініціали)  
Викладач \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

### **Додаткові зауваження:**

- якщо студент не має поточних не перезданих «двійок» – іспит складається у вигляді фінального тесту (згенерований програмний продукт, який містить 60 випадкових запитань)
- якщо є «двійки», «н/а» тощо – студент складає «повний іспит», пише відповіді на теоретичні запитання з екзаменаційного білету, а також виконує тестове завдання та практичне завдання.
- якщо є сертифікат курсу від Cisco Networking Academy і успішно виконав всі практичні роботи – студент звільняється від екзамену й отримує «відмінно»
- додаткові преференції на екзамені отримують учасники фахових конкурсів, змагань та олімпіад (за умови успішного виконання всіх практичних робіт)

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ «ПОЛТАВСЬКИЙ  
ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Циклова комісія дисциплін програмної інженерії

**МЕТОДИЧНІ ВКАЗІВКИ ДО ВИКОНАННЯ КУРСОВИХ  
РОБІТ З ДИСЦИПЛІНИ  
«Програмування»**

**галузь знань:** 12 Інформаційні технології

**спеціальність:** 121 Інженерія програмного забезпечення

**спеціалізація:** Розробка програмного забезпечення

МЕТОДИЧНІ ВКАЗІВКИ ДО ВИКОНАННЯ КУРСОВИХ РОБІТ ДЛЯ  
ЗДОБУВАЧІВ ОСВІТИ СПЕЦІАЛЬНОСТІ 121 «ІНЖЕНЕРІЯ  
ПРОГРАМНОГО ЗАБЕЗПЕЧЕННЯ»: МЕТОДИЧНІ ВКАЗІВКИ /  
Відокремлений структурний підрозділ «Полтавський політехнічний фаховий  
коледж Національного технічного університету «Харківський політехнічний  
інститут»; [уклад.: В.В.Олійник]. – Полтава: ВСП ППФК НТУ “ХПІ”, 2025. –  
27 с.

Укладачі:

*В.В. Олійник*, викладач, Відокремлений структурний підрозділ «Полтавський  
політехнічний фаховий коледж Національного технічного університету  
«Харківський політехнічний інститут»

Видання містить методичні вказівки щодо виконання курсової роботи,  
змісту та оформлення пояснювальної записки, використання нормативів та  
стандартів, організації та проходження захисту роботи, критерії оцінювання  
результатів роботи здобувачів освіти, тощо.

Призначене для здобувачів освіти 2 курсу напряму підготовки 121  
«Інженерія програмного забезпечення» денної форми навчання.

Методичні вказівки розглянуто і затверджено на засіданні циклової  
комісії дисциплін програмної інженерії.

Протокол від «30» серпня 2025 року № 1

Голова циклової комісії \_\_\_\_\_ / Олександр БАБИЧ/

## **ЗМІСТ**

|                                                     |     |
|-----------------------------------------------------|-----|
| 1. ЗАГАЛЬНІ ПОЛОЖЕННЯ.....                          | 340 |
| 2. ПОРЯДОК ВИКОНАННЯ КУРСОВОЇ РОБОТИ.....           | 342 |
| 2.1. Постановка задачі.....                         | 342 |
| 2.2. Проектування програми.....                     | 342 |
| 2.3. Написання програми.....                        | 342 |
| 2.4. Тестування програми.....                       | 342 |
| 3. ЗМІСТ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ.....                 | 344 |
| 4. ПРАВИЛА ОФОРМЛЕННЯ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ.....    | 349 |
| 4.1. Загальні вимоги.....                           | 349 |
| 4.2. Нумерація.....                                 | 349 |
| 4.3. Оформлення цитат і списку літератури.....      | 351 |
| 4.4. Оформлення додатків.....                       | 354 |
| 5. ПОРЯДОК ВИКОНАННЯ ТА ЗАХИСТУ КУРСОВОЇ РОБОТИ.... | 355 |
| 5.1. Хід виконання та захисту КР.....               | 355 |
| 5.2. Критерії оцінювання КР.....                    | 356 |
| 6. ТЕМИ ІНДИВІДУАЛЬНИХ ЗАВДАНЬ НА КУРСОВУ РОБОТУ..  | 357 |
| СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ.....                 | 358 |
| ДОДАТОК А.....                                      | 359 |
| ДОДАТОК Б.....                                      | 359 |

## 1. ЗАГАЛЬНІ ПОЛОЖЕННЯ

Виконання курсової роботи (КР) з дисципліни «Основи програмування та алгоритмічні мови» є обов'язковою складовою навчального плану підготовки молодших фахових бакалаврів за спеціальністю 121 «Інженерія програмного забезпечення».

Головна мета КР полягає у закріпленні, поглибленні та узагальненні базових теоретичних знань, якими здобувач освіти оволодів під час вивчення дисципліни «Основи програмування та алгоритмічні мови», їх застосуванні до комплексного вирішення конкретного завдання. Такий підхід повністю відповідає концепції формування висококваліфікованих фахівців у галузі інформаційних технологій, котрі набувають не тільки знань, але й навичок та вмінь, якими повинні володіти фахові молодші бакалаври освітнього закладу з напрямку «Програмна інженерія».

Основними завданнями написання курсової роботи є:

- закріплення, поглиблення та узагальнення знань, якими здобувач освіти оволодів під час вивчення курсу;
- надбання досвіду роботи з літературними та фондовими матеріалами, вміння узагальнювати та аналізувати наукову інформацію, виробляти власне ставлення до наукової чи практичної проблеми;
- набуття навичок використання основ алгоритмізації та програмування на алгоритмічних мовах високого рівня;
- набуття здобувачами освіти теоретичних знань та практичних навичок в області використання сучасних систем створення програмного забезпечення та освоєння принципів та методів сучасних технологій програмування;
- вироблення вміння застосовувати методи обчислювальної математики та прикладного програмування для розв'язання прикладних задач;
- проведення ґрунтовного аналізу отриманих результатів і формування змістовних висновків стосовно їх якості.



Під час виконання курсової роботи здобувач освіти повинен продемонструвати:

- вміння збирати і аналізувати відповідні матеріали про об'єкт дослідження, використовуючи сучасні джерела інформації, включаючи Інтернет-ресурси;
- спроможність проводити необхідні обґрунтування для розробки програмного забезпечення різного призначення, тощо;
- здатність доводити розв'язання поставленої задачі до логічного завершення;
- вміння аналізувати отримані результати і робити відповідні висновки.

Курсова робота є самостійною роботою здобувача освіти. Відповідальність за правильність аналітичних висновків, результатів розрахунків і моделювання, а також оформлення несе здобувач освіти – автор КР.

## **2. ПОРЯДОК ВИКОНАННЯ КУРСОВОЇ РОБОТИ**

Основні етапи виконання курсової роботи:

- постановка задачі;
- проектування програми;
- написання програми;
- тестування програми;
- оформлення пояснювальної записки;
- захист курсової роботи.

### **2.1. Постановка задачі**

Постановка задачі є самостійним етапом роботи по виконанню КР. На цьому етапі визначається перелік функцій (дій), які виконує програма, і пропонується інтерфейс користувача, з яким пов'язуються функції програми.

### **2.2. Проектування програми**

На цьому етапі розробляється сценарій роботи програми, її функціональна схема, алгоритмічне забезпечення, формати вхідних, вихідних та проміжних даних, проектується інтерфейс користувача.

### **2.3. Написання програми**

На етапі кодування створюються текст програми, лістинги повинні бути детальним чином прокоментовані і повністю відповідати розробленим алгоритмам.

### **2.4. Тестування програми**

На цьому етапі складається план тестування, який враховує всі особливості програми. Тестовий набір необхідно узгодити з керівником

курсного проекту. Після успішного тестування програми можна переходити до наступних етапів. В разі невдалого тестування треба повернутись до попередніх етапів курсного проектування. Результати тестування та виявлені обмеження програми необхідно задокументувати

### 3. ЗМІСТ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ

Основними документами, що представляють КР, є пояснювальна записка та комплекс програм на хмарному сховищі. Текст пояснювальної записки до курсової роботи повинен бути викладений лаконічно, у обґрунтованому стилі. Не дозволяється переписування літературних джерел та використання не опрацьованих студентом Інтернет-оглядів.

Пояснювальна записка виконується на аркушах формату А4 згідно ДСТУ 3008-95. У випадку необхідності окремі ілюстрації можуть виконуватись на аркушах більших форматів.

Обов'язковими структурними частинами пояснювальної записки є:

- титульний лист,
- зміст,
- вступ,
- основна частина,
- висновки,
- перелік посилань,
- додатки до пояснювальної записки.

Титульний лист повинен бути встановленого зразку. На ньому вказується назва міністерства, університету, інституту, факультету, кафедри і тема курсової роботи (у точній відповідності із індивідуальним завданням). Його зразок наведений у Додатку А. Титульний лист не нумерується як розділ, не вноситься до змісту і не нумерується як сторінка.

Зміст характеризує структуру КР. Він повинен вміщувати в собі назви усіх розділів, підрозділів, пунктів та підпунктів курсової роботи, а також перелік додатків. Усі назви повинні бути записані так само як вони сформульовані в КР. Зміст розміщується на окремій сторінці, як розділ зміст не нумерується.

У вступі коротко розкривається призначення КР, ціль роботи, сутність вирішуваної задачі, дається загальна постановка завдання і методи його розв'язання. Вступ як розділ не нумерується.

Основна частина пояснювальної записки може складатися з розділів, підрозділів. Кожний розділ починають з нової сторінки. У розділах основної частини подають:

- постановку задачі,
- викладення використовуваних методів,
- опис алгоритму,
- опис програмного забезпечення,
- результати його тестування,
- інструкцію користувача,
- аналіз і узагальнення результатів дослідження.

В *постановці задачі* висвітлюється інформаційна сутність задачі - вся інформація, необхідна для програмної реалізації поставленого завдання, усі вхідні і вихідні дані.

У *другому розділі* розкриваються теоретичні основи використаних методів розрахунків, оцінки похибок тощо. В цьому розділі студент має продемонструвати свою обізнаність в питаннях теорії і методології тих методів обчислювальної математики, які використовуються для розв'язання поставленої задачі.

При *описі алгоритму* з вичерпною повнотою викладають

- логіку алгоритму,
- спосіб формування результатів вирішення задачі,
- послідовність етапів,
- розрахункові формули,
- стандартні (запозичені) алгоритми з обов'язковим посиланням на джерело (джерело повинне бути вказане у списку літератури),
- зв'язки між частинами і операціями алгоритму.

Не слід наводити детальний алгоритм, до окремого оператора. Однак, не слід і значно спрощувати його. Орієнтуватися необхідно на те, щоб можна було повністю зрозуміти нюанси алгоритму, авторські нововведення та здобутки. Загальновідомі і широко розповсюджені алгоритми, за узгодженням з викладачем, можна вказати скорочено (наприклад, в одному блоці).

Алгоритми представляються в словесному вигляді або у вигляді блок-схем. Після кожної блок-схеми необхідно надати детальне пояснення роботи алгоритму з посиланням на номери блоків.

Блок-схеми виконуються відповідно до вимог, зазначених у Додатку Б. В підписах блоків треба вказувати дію, яку вони виконують, а не приводити відповідні оператори мови програмування.

Опис програмного забезпечення включає:

- опис функціональної структури програмного забезпечення,
- опис функцій частин програмного забезпечення.

У даному розділі проектуються основні структурні компоненти програми (функції, бібліотеки тощо). Спроектowana функціональна структура представляється у вигляді функціональної схеми програми.

Після розробки даної схеми необхідно описати кожен її блок, його призначення і застосування.

Опис специфікації функцій здійснюється у вигляді наступної таблиці:

| № п/п | Назва функції | Призначення функції | Опис вхідних параметрів | Опис вихідних параметрів | Заголовний файл |
|-------|---------------|---------------------|-------------------------|--------------------------|-----------------|
|       |               |                     |                         |                          |                 |

В специфікації описуються усі функції користувача, які використовуються в програмі.

Також необхідно привести таблицю стандартних функцій, які були використанні у програмі. Таблиця повинна містити в собі назву функції, її призначення, назву стандартної бібліотеки, якій вона належить.

У розділі *тестування* розробляються та наводяться тести, виконання яких дозволяє пересвідчитись у правильності роботи програми, надається

план тестування розробленого програмного забезпечення та демонстраційні приклади.

При розробці тестів визначаються усі можливі напрямки обчислювального процесу, аналізуються особливості їх реалізації.

План тестування включає формування наборів відповідних тестових даних та очікувані результати.

Демонстраційні приклади показують хід розв'язання поставленої задачі за різних умов (у вигляді певних проміжних результатів, таблиць, графіків, рисунків, скріншотів із детальними коментарями та поясненнями).

У *інструкції користувача* описується призначення програми; вимоги до системи (апаратні та програмні) - вимоги до процесора, розміру оперативної пам'яті, версії операційної систем, встановлені драйвера (keytus.com) та інше); наводиться докладна інструкція по роботі з програмою, в якій описується склад програмного забезпечення (імена всіх файлів, з яких складається програма із зазначенням їх розміру і призначення), варіанти використання програми – опис інтерфейсу (зовнішній вигляд, засоби керування та їх призначення) та послідовність дій для виконання тієї чи іншої функції програми.

Описуючи інтерфейс користувача, обов'язково використовувати рисунки.

При *аналізі і узагальненні результатів* КР вказується точність обчислень, здійснюється оцінка достовірності результатів, опис можливих критичних ситуацій і рекомендації по їх ліквідації, аналізується складність алгоритмів та ефективність використаних підходів.

У висновках в реферативній формі повинні бути описані результати, отримані студентом на кожному із етапів виконання роботи (аналітичному, етапі проектування програмного забезпечення, експериментальному дослідженню, аналізу отриманих результатів), а також висновки щодо досягнення мети курсової роботи. Тут необхідно наголосити на якісних і

кількісних показниках здобутих результатів, обґрунтувати їх достовірність. Висновки як розділ не нумеруються.

Перелік посилань повинен включати усі літературні джерела, на які є посилання у тексті пояснювальної записки. Список повинен формуватися в порядку посилань за текстом і вміщувати бібліографічні відомості офіційно виданих книжок, статей, патентів, депонованих рукописів тощо. Як розділ перелік літератури не нумерується.

В додатки включають допоміжний матеріал: проміжні математичні доведення, формули та розрахунки; таблиці, графіки, скріншоти тощо, які не увійшли до пояснювальної записки, але потрібні для пояснень.

У додатках також повинні міститися лістинги програми.

Текст програми (функцій) повинен мати коментарі. Наводиться призначення усіх ідентифікаторів (імена констант, змінних, типів даних), які використовуються у програмі, а також функцій користувача. Кожна така функція повинна бути документована із зазначенням не тільки її призначення, а й опису аргументів (параметрів). При підключенні власних заголовних файлів необхідно вказати в коментарях засоби відповідної бібліотеки, які будуть використовуватися у програмі. На початку кожного заголовного файлу користувача необхідно вказати в коментарях його призначення.

Текст програми (функцій) повинен відповідати розробленим алгоритмам.



## **4. ПРАВИЛА ОФОРМЛЕННЯ ПОЯСНЮВАЛЬНОЇ ЗАПИСКИ**

### **4.1. Загальні вимоги**

Пояснювальна записка має бути представлена в електронному та друкованому вигляді.

Електронна версія зберігається в банку даних відділення. Файл із копією курсової роботи здається керівнику разом із друкованим примірником за 3 дні до захисту. Формат файлу – pdf.

Текстову частину роботи необхідно друкувати на одному боці аркуша білого паперу формату А4 (210×297 мм). В окремих випадках, для більш наочного подання таблиць та ілюстрацій, можна використовувати папір формату А3 (297×420 мм).

Оптимальний обсяг текстової частини роботи (без додатків) має складати 20 аркушів. Обсяг додатків жорстко не лімітується, але пропонується мінімальний об'єм у кількості 5 аркушів.

Форматування пояснювальної записки:

- поля: ліве – 30 мм, верхнє і нижнє – 20 мм, праве – 15 мм;
- шрифт: Times New Roman, 14 pt;
- міжрядковий інтервал – 1.5 pt;
- вирівнювання: назв розділів і підрозділів – з лівого боку без відступу, основного тексту – по ширині.

Заголовки структурних частин курсової роботи «ЗМІСТ», «ВСТУП», «РОЗДІЛ», «ВИСНОВКИ», «ДОДАТКИ», «СПИСОК ЛІТЕРАТУРИ» друкуються великими літерами. Кожну структурну частину роботи потрібно починати з нової сторінки.

### **4.2. Нумерація**

Нумерацію сторінок, розділів, підрозділів, пунктів, підпунктів, рисунків, таблиць, формул подають арабськими цифрами без знака «№».

Першою сторінкою курсової роботи є титульний аркуш, який включають до загальної нумерації, але номер сторінки на ньому не ставлять. На всіх наступних сторінках номер проставляють у правому верхньому куті сторінки без крапки в кінці.

Такі структурні частини роботи, як «ЗМІСТ», «ВСТУП», «ВИСНОВКИ», «ДОДАТКИ», «СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ» не мають порядкового номеру. Номер розділу ставлять перед його назвою, після номеру ставлять крапку, потім з прописними буквами друкують назву розділу (шрифт напівжирний Times New Roman, кегль 16, всі букви верхнього регістру, інтервал перед 12 пт, після 3.)

Підрозділи нумерують у межах кожного розділу. Номер підрозділу складається з номера розділу і порядкового номера підрозділу, між якими ставлять крапку. В кінці номера підрозділу повинна стояти крапка. Потім у тому ж рядку наводять назву підрозділу (шрифт напівжирний Times New Roman, кегль 14, інтервал перед 12 пт, після 3.)

Наприклад, «2.3.» – перший пункт третього підрозділу другого розділу.

Ілюстрації (фотографії, скріншоти, креслення, схеми, графіки, рисунки, карти) і таблиці необхідно подавати безпосередньо після тексту, де вони згадані вперше, або на наступній сторінці. Ілюстрації позначають словом «Рисунок» і нумерують послідовно в межах розділу, за винятком ілюстрацій, поданих у додатках. Номер ілюстрації повинен складатися з номера розділу і порядкового номера ілюстрації, між якими ставиться крапка. Номер, назва і пояснювальний підпис (у разі необхідності) повинні міститися безпосередньо під ілюстрацією.

Таблиці нумерують послідовно в межах розділу (за винятком таблиць, поданих у додатках). В правому верхньому куті над відповідним заголовком таблиці розміщують напис «Таблиця» із зазначенням її номера. Номер таблиці повинен складатися з номера розділу і порядкового номера таблиці, між якими ставиться крапка.

Формули в курсовій роботі нумерують в межах розділу. Номер формули повинен складатися з номера розділу і порядкового номера формули, між якими ставиться крапка. Номер формули пишуть в круглих дужках і розміщують біля правого поля аркуша на рівні відповідної формули.

Посилання в тексті роботи на ілюстрації, таблиці, формули вказують порядковим номером в круглих дужках, наприклад, «... у формулі (2.1)».

#### **4.3. Приклади оформлення бібліографічного опису у списку джерел**

##### **КНИГИ**

##### **ОДИН АВТОР**

1. Горбунова А. В. Управління економічною захищеністю підприємства: теорія і методологія : монографія. Запоріжжя : ЗНУ, 2017. 240 с.
2. Дробот О. В. Професійна свідомість керівника : навч. посіб. Київ : Талком, 2016. 340 с.

##### **ДВА АВТОРИ**

1. Аванесова Н. Е., Марченко О. В. Стратегічне управління підприємством та сучасним містом: теоретико-методичні засади : монографія. Харків : Щедра садиба плюс, 2015. 196 с.

##### **ТРИ АВТОРИ**

1. Аніловська Г. Я., Марушко Н. С., Стоколоса Т. М. Інформаційні системи і технології у фінансах : навч. посіб. Львів : Магнолія 2006, 2015. 312 с.

##### **БЕЗ АВТОРА**

1. Країни пострадянського простору: виклики модернізації : зб. наук. пр. / редкол.: П. М. Рудяков (відп. ред.) та ін. Київ : Ін-т всесвітньої історії НАН України, 2016. 306 с.

##### **БАГАТОТОМНИЙ ДОКУМЕНТ**

1. Енциклопедія Сучасної України / редкол.: І. М. Дзюба та ін. Київ : САМ, 2016. Т. 17. 712 с.

##### **МАТЕРІАЛИ КОНФЕРЕНЦІЙ, З'ЇЗДІВ**

1. Микитів Г. В., Кондратенко Ю. Позатекстові елементи як засіб формування медіакультури читачів науково-популярних журналів. Актуальні проблеми медіаосвіти в Україні та світі : зб. тез доп. міжнар. наук.-практ. конф., м. Запоріжжя, 3-4 берез. 2016 р. Запоріжжя, 2016. С. 50–53.

2. Соколова Ю. Особливості впровадження проблемного навчання хімії в старшій профільній школі. Актуальні проблеми та перспективи розвитку медичних, фармацевтичних та природничих наук : матеріали ІІІ регіон. наук.-практ. конф., м. Запоріжжя, 29 листоп. 2014 р. Запоріжжя, 2014. С. 211–212.

#### ПРЕПРИНТИ

1. Панасюк М. І., Скорбун А. Д., Сплошной Б. М. Про точність визначення активності твердих радіоактивних відходів гамма-методами. Чорнобиль : Ін-т з проблем безпеки АЕС НАН України, 2006. 7, [1] с. (Препринт. НАН України, Ін-т проблем безпеки АЕС; 06-1).

#### СЛОВНИКИ

1. Кучеренко І. М. Право державної власності. Великий енциклопедичний юридичний словник / ред. Ю. С. Шемшученко. Київ, 2007. С. 673.
2. Сірий М. І. Судова влада. Юридична енциклопедія. Київ, 2003. Т. 5. С. 699.

#### ЗАКОНОДАВЧІ ТА НОРМАТИВНІ ДОКУМЕНТИ

1. Про освіту : Закон України від 05.09.2017 р. № 2145-VIII. Голос України. 2017. 27 верес. (№ 178-179). С. 10–22.
2. Інструкція щодо заповнення особової картки державного службовця : затв. наказом Нац. агентства України з питань Держ. служби від 05.08.2016 р. № 156. Баланс-бюджет. 2016. 19 верес. (№ 38). С. 15–16.

#### СТАНДАРТИ

1. ДСТУ 7152:2010. Видання. Оформлення публікацій у журналах і збірниках. [Чинний від 2010-02-18]. Вид. офіц. Київ, 2010. 16 с. (Інформація та документація).
2. ДСТУ 3582:2013. Бібліографічний опис. Скорочення слів і словосполучень українською мовою. Загальні вимоги та правила(ISO 4:1984, NEQ; ISO 832:1994, NEQ). [На заміну ДСТУ3582-97; чинний від 2013-08-22]. Вид. офіц. Київ : Мінекономрозвитку України, 2014. 15 с. (Інформація та документація).

#### КАТАЛОГИ

1. Історико-правова спадщина України : кат. вист. / Харків. держ. наук. б-ка ім. В. Г. Короленка; уклад.: Л. І. Романова, О. В. Земляніщина. Харків, 1996. 64 с.
2. Пам'ятки історії та мистецтва Львівської області : кат.-довід. / авт.-упоряд.: М. Зобків та ін. ; Упр. культури Львів. облдержадмін., Львів. іст. музей. Львів : Новий час, 2003. 160 с.

#### БІБЛІОГРАФІЧНІ ПОКАЗЧИКИ

1. Лисодєд О. В. Бібліографічний довідник з кримінології (1992-2002) / ред. О. Г. Кальман. Харків : Одісей, 2003. 128 с.

#### ДИСЕРТАЦІЇ

1. Левчук С. А. Матриці Гріна рівнянь і систем еліптичного типу для дослідження статичного деформування складених тіл : дис. ... канд. фіз.-мат. наук : 01.02.04. Запоріжжя, 2002. 150 с.
2. Вініченко О. М. Система динамічного контролю соціально-економічного розвитку промислового підприємства : дис. ... д-ра екон. наук : 08.00.04. Дніпро, 2017. 424 с.

#### АВТОРЕФЕРАТИ ДИСЕРТАЦІЙ

1. Гнатенко Н. Г. Групи інтересів у Верховній Раді України: сутність і роль у формуванні державної політики : автореф. дис. ... канд. політ. наук : 23.00.02. Київ, 2017. 20 с.
2. Кулініч О. О. Право людини і громадянина на освіту в Україні та конституційно-правовий механізм його реалізації : автореф. дис. ... канд. юрид. наук : 12.00.02. Маріуполь, 2015. 20 с.

#### ЕЛЕКТРОННІ РЕСУРСИ

1. Влада очима історії : фотовиставка. URL: <http://www.kmu.gov.ua/control/uk/photogallery/gallery?galleryId=15725757&> (дата звернення: 15.11.2017).
2. Шарая А. А. Принципи державної служби за законодавством України. Юридичний науковий електронний журнал. 2017. № 5. С. 115–118. URL: [http://lsej.org.ua/5\\_2017/32.pdf](http://lsej.org.ua/5_2017/32.pdf).
3. Ганзенко О. О. Основні напрями подолання правового нігілізму в Україні. Вісник Запорізького національного університету. Юридичні науки. Запоріжжя, 2015. № 3. – С. 20–27. – URL: <http://ebooks.znu.edu.ua/files/Fakhovivydannya/vznu/juridichni/VestUr2015v3/5.pdf>. (дата звернення: 15.11.2017).
4. Яцків Я. С., Маліцький Б. А., Бублик С. Г. Трансформація наукової системи України протягом 90-х років XX століття: період переходу до ринку. Наука та інновації. 2016. Т. 12, № 6. С. 6–14. DOI: <https://doi.org/10.15407/scin12.06.006>.

Список формується в порядку черговості згадування документа в тексті. Список використаних першоджерел має містити не менше п'яти посилань, з яких не менше трьох – друковані видання.

Звіт варто ретельно відредагувати, домагаючись стислості і точності викладу матеріалу, технічної грамотності, науковості.

#### **4.4. Оформлення додатків**

Додаток повинен мати заголовок, надрукований на початку сторінки із вирівнюванням по лівому краю сторінки без відступів. Друкується слово «Додаток» і велика літера, що позначає додаток, після крапки друкується назва додатку. Додатки слід позначати послідовно великими літерами української абетки, за винятком літер Г, Ґ, Є, І, Ї, Й, О, Ч, Ї, П

Нумерація рисунків у додатку відбувається в межах додатку. Перед кожним номером ставлять позначення додатка (літеру) і крапку, наприклад А.1 – перший рисунок додатка А. Таблиці і формули нумерують аналогічним чином.

## **5. ПОРЯДОК ВИКОНАННЯ ТА ЗАХИСТУ КУРСОВОЇ РОБОТИ**

### **5.1. Хід виконання та захисту КР**

Керівник здійснює контроль за ходом виконання здобувачем освіти курсової роботи, надає йому необхідну консультативну допомогу.

Протягом семестру здобувач освіти демонструє викладачу поточні результати роботи над проектом.

Після перевірки роботи викладач призначає день, час і місце захисту.

Напередодні захисту здобувачу освіти необхідно повторити теоретичний матеріал, що стосується роботи, та переглянути безпосередньо її зміст.

Захист КР проводиться у відкритій формі перед комісією та глядацькою аудиторією.

За результатами захисту, у відповідності до критеріїв оцінювання, що наведені у розділі 7 даних вказівок, комісія виставляє здобувачу освіти оцінку.

На оцінку за КР впливають:

- якість розробленого програмного забезпечення;
- якість розробленої програмної документації;
- компетентність та загальна ерудиція здобувача освіти при відповідях на запитання під час захисту;
- ступінь виконання графіку підготовки курсової роботи.

Якщо здобувач освіти подав на захист не самостійно виконану роботу, про що свідчить його некомпетентність у рішеннях та матеріалах роботи, КР до захисту не допускається, що супроводжується записом "не допущений" у екзаменаційній відомості. Такий самий запис робиться у випадку, якщо КР не готовий на час захисту. В цих випадках запис "не допущений" еквівалентний отриманню оцінки "незадовільно".

## 5.2. Критерії оцінювання КР

Виконання курсової роботи може бути оцінено таким чином:

Оцінка **«відмінно»** – зміст і оформлення звіту та обов’язкових документів відповідає вимогам. В ході курсової роботи створено програмний продукт. Здобувач освіти дає повні та точні відповіді на всі запитання членів комісії.

Оцінка **«добре»** – є несуттєві зауваження щодо змісту та оформлення звіту й обов’язкових документів. Програмний продукт відповідає вимогам замовника. У відповідях на запитання членів комісії здобувач освіти припускається окремих неточностей, хоча загалом має міцні знання.

Оцінка **«задовільно»** – є зауваження щодо оформлення роботи та обов’язкових документів. Необхідна документація підготована, однак є окремі помилки. Створений програмний продукт не надає повної функціональності або має значні недоліки. При відповідях на запитання членів комісії здобувач освіти відчувається невпевнено, збивається, припускається помилок, не виявляє міцних знань.

Оцінка **«незадовільно»** – неякісне оформлення роботи та обов’язкових документів. Документація підготована, однак є окремі помилки. Програмний продукт не створено або він є нефункціональним. На запитання членів комісії здобувач освіти не дає задовільних відповідей. Можливо, здобувач освіти з певних причин взагалі не виконав курсову роботу.



## **6. ТЕМИ ІНДИВІДУАЛЬНИХ ЗАВДАНЬ НА КУРСОВУ РОБОТУ**

Тематика курсових робіт визначається змістом робочої навчальної програми з дисципліни «Основи програмування та алгоритмічні мови».

Здобувач освіти повинен написати порівняно складну програму. Програма повинна відповідати всім критеріям якості програмного продукту. Для виконання курсової роботи використовується мова програмування C/C++ або інша, за погодженням з викладачем.

Порядок виконання курсової роботи включає розробку алгоритмічного і програмного забезпечення, проектування зручного інтерфейсу користувача, та аналіз одержаних результатів, оформлення пояснювальної записки.

Студент може запропонувати власну тему, обґрунтувавши її актуальність та доцільність виконання відповідної розробки. Тематика індивідуальних завдань оновлюється щороку.

## СПИСОК ВИКОРИСТАНОЇ ЛІТЕРАТУРИ

1. Методичні вказівки до виконання дипломних робіт для здобувачів освіти спеціальності 121 Інженерія програмного забезпечення спеціальності «Розробка програмного забезпечення» / Укл.: Бабич Олдр. В., Бабич Олена В. – Полтава: ВСП «ППФК НТУ “ХПІ”», 2020. – 51 с.
2. ГОСТ2.105-95 ЄСКД “Загальні вимоги до текстових документів”.
3. ГОСТ19.105-78 ЄСПД “Загальні вимоги до програмних документів”.
4. ГОСТ19.404-79 ЄСПД “Пояснювальна записка. Вимоги до змісту та оформленню”.
5. ГОСТ19.781-90 ЄСПД “Забезпечення систем обробки інформації: програми, терміни та визначення.”
6. ГОСТ19.701-90 (ISO 5807-85) ЄСПД “Схеми алгоритмів, даних і систем. Позначення умовні графічні та правила виконання”.
7. ГОСТ19.401-78 ЄСПД “Текст програми. Вимоги до змісту та оформленню”.
8. ГОСТ19.402-78 ЄСПД “Опис програми”.
9. ГОСТ19.104-78 ЄСПД “Основні написи”.
10. ГОСТ19.202-78 ЄСПД “Специфікація. Вимоги до змісту та оформленню”.
11. ДСТУ 8302:2015 «Інформація та документація. Бібліографічне посилання. Загальні вимоги та правила складання» – введений з 01 липня 2016 р.

## ДОДАТОК А

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
ВІДОКРЕМЛЕНИЙ СТРУКТУРНИЙ ПІДРОЗДІЛ  
«ПОЛТАВСЬКИЙ ПОЛІТЕХНІЧНИЙ ФАХОВИЙ КОЛЕДЖ  
НАЦІОНАЛЬНОГО ТЕХНІЧНОГО УНІВЕРСИТЕТУ  
«ХАРКІВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ»

Циклова комісія дисциплін програмної інженерії

### КУРСОВА РОБОТА

молодшого спеціаліста

на тему \_\_\_\_\_ Шляхи метро  
\_\_\_\_\_  
\_\_\_\_\_

Виконав: здобувач освіти 2 курсу,  
групи \_\_\_\_\_  
спеціалізації «Розробка програмного  
забезпечення»

Донець Д.О.

\_\_\_\_\_  
(прізвище та ініціали)

Керівник \_\_\_\_\_ Олійник В.В.  
(підпис) (прізвище та ініціали)

Полтава – 2021

## ДОДАТОК Б

Блок-схеми алгоритмів виконуються за наступними ДЕСТами:

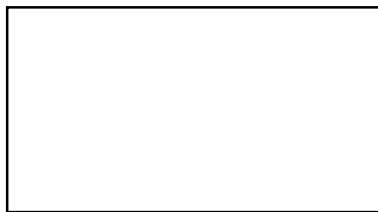
- ДЕСТ 19.002-80 Схеми алгоритмів і програм. Правила виконання
- ДЕСТ 19.003-80 Схеми алгоритмів і програм. Позначення умовні графічні.

Блок-схеми алгоритмів повинні виконуватися в електронному вигляді (Microsoft Visio тощо).

Зображення схем

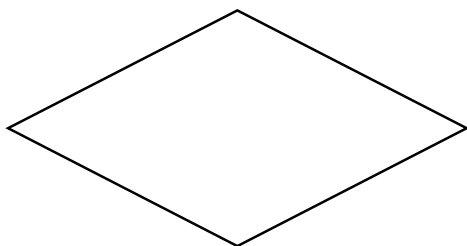
Окремі функції алгоритмів і програм, з урахуванням ступеня їх деталізації, відображаються у вигляді умовних графічних позначень - символів.

Блок-схема є формою подання алгоритму за допомогою графічних символів. Графічні символи, їхні розміри, а також правила побудови блок-схем визначені державними стандартами. Розглянемо часто вживані графічні символи (повний список містить 42 символи).



**Процес.** Виконання операції або групи операцій, у результаті чого змінюється значення, форма подання або розташування даних.

Усередині символу або ж у вигляді коментарю природною мовою або у вигляді формули записуються дії, які проводяться при виконанні операції або групи операцій.

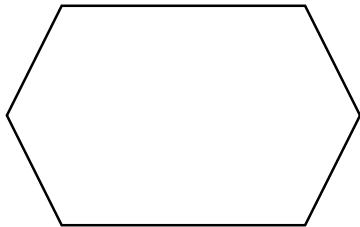


**Рішення.** Вибір напрямку виконання алгоритму або програми залежно від деяких змінних умов.

Символ використовується для зображення уніфікованих структур:

— РОЗГАЛУЖЕННЯ ПОВНЕ

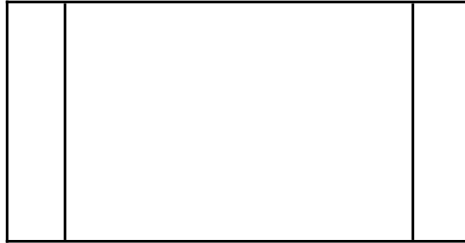
- РОЗГАЛУЖЕННЯ НЕПОВНЕ
- ВИБІР
- ЦИКЛ-ДО
- ЦИКЛ-ПОКИ



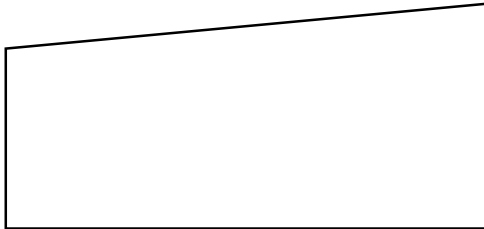
**Модифікація.** Виконання операцій, що міняють команди або групу команд, що змінюють програму.

Символ використовується для зображення уніфікованої структури ЦИКЛ З ПАРАМЕТРОМ. Усередині символу записується параметр циклу із вказаними початковим і кінцевим значеннями, а також крок зміни циклу, якщо він не дорівнює одиниці.

**Визначений процес.** Використання раніше створених і окремо описаних алгоритмів або програм (процедур, функцій, програмних модулів). Символ служить для вказівки звертання до процедур, функцій, програмних модулів.



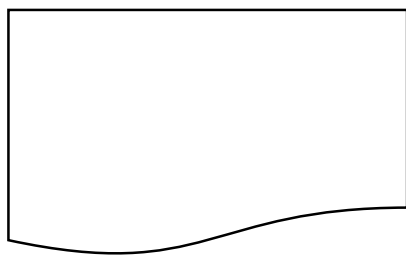
**Ручне введення.** Введення даних оператором у процес обробки за допомогою пристрою, безпосередньо сполученого з комп'ютером (наприклад, клавіатури).



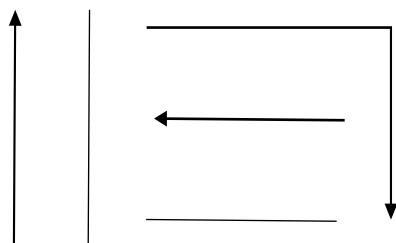
**Дисплей.** Введення-виведення даних у випадку, якщо безпосередньо підключений до комп'ютера пристрій відтворює дані й дозволяє операторові вносити зміни в процесі їхньої обробки.



**Документ.** Введення-виведення даних, носієм яких служить папір.



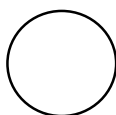
Лінія потоку . Вказівка послідовності зв'язків між символами.



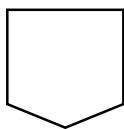
Перелічимо деякі правила зображення ліній потоку:

- лінії потоку повинні бути паралельні лініям зовнішньої рамки блок-схеми (межам аркуша, на якому зображена блок-схема);
- напрямок лінії потоку — зверху-донизу і зліва-направо береться за основний й стрілками не позначається, в інших випадках напрямок лінії потоку позначається стрілками;
- зміна напрямку лінії потоку провадиться під кутом 90 градусів.

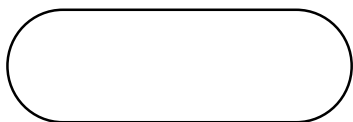
**З'єднувач.** Вказівка зв'язку між перерваними лініями потоку, що зв'язують символи. Якщо блок-схема складається з декількох частин, розташованих на одній сторінці, то лінія потоку однієї частини закінчується символом З'ЄДНУВАЧ, а лінія потоку на продовженні блок-схеми починається із цього ж символу. Усередині символів З'ЄДНУВАЧ ставляться однакові порядкові номери, які відповідають розірваній лінії потоку.



**Міжсторінковий з'єднувач.** Вказівка зв'язку між роз'єднаними частинами схем алгоритмів і програм, розташованих на різних аркушах. Даний символ служить для тих же цілей, що й з'єднувач, але при розташуванні частин блок-схеми на різних сторінках.



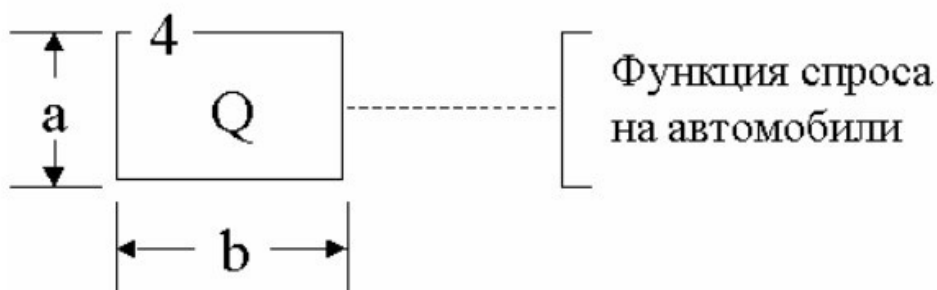
**Пуск- зупинка.** Початок, кінець, переривання процесу обробки даних або виконання програми.



**Коментар.** Зв'язок між елементами схеми й поясненнями. Дозволяє включати в блок-схему пояснення, формули й іншу інформацію.



Розміри символів повинні задовольняти співвідношенню  $b = 1.5a$  ( $a$  і  $b$  зазначені на рисунку). На цьому ж рисунку показаний приклад використання символу КОМЕНТАР.



Кожному символу на блок-схемі привласнюється порядковий номер. Для прикладу на рисунку символу ПРОЦЕС привласнений порядковий номер 4.



## ОПИС МЕТОДИЧНОГО ТА ТЕХНІЧНОГО ЗАБЕЗПЕЧЕННЯ

Для успішного опанування дисципліни «Програмування» кожному здобувачеві освіти потрібен доступ до персонального комп'ютера або ноутбука з наступними характеристиками:

- процесор: принаймні 2-ядерний Intel або AMD 1.7 ГГц або кращий
- дисковий простір: принаймні 120 Гб вільного місця для встановлення ПЗ
- RAM: принаймні 2 Гб (у випадку використання Linux), 4 Гб або більше (Windows)
- мережна карта з доступом в Інтернет
- відеокарта/монітор: принаймні 11,6” Wide XGA (WXGA) або кращий
- Microsoft Mouse або сумісний маніпулятор
- звукова карта з гучномовцями та мікрофоном
- операційна система – будь-яка (Windows 10, Linux, MacOS)
- бажаною є наявність веб-камери

**Вільне програмне забезпечення, використовуване під час виконання практичних робіт:**

- Dev-C++ - <https://www.dev-cpp.com/>
- Code::Blocks - <https://www.codeblocks.org/downloads/>
- Visual Studio 2026. - <https://visualstudio.microsoft.com/>
- Visual Studio Code - <https://code.visualstudio.com/download>
- CLion - <https://www.jetbrains.com/ru-ru/clion/>
- Qt Creator - <https://www.qt.io/development/tools/qt-creator-ide>
- Xcode - <https://apps.apple.com/us/app/xcode/id497799835?mt=12>
- GitHub (GitHub Enterprise Cloud, доступний в рамках партнерської угоди між коледжем та GitHub Inc.) - <https://github.com/>

## РЕКОМЕНДОВАНА ЛІТЕРАТУРА ТА ІНТЕРНЕТ-РЕСУРСИ

### Книги та навчальні посібники:

1. Трофименко О.Г., Прокоп Ю.В., Швайко І.Г. С++. Основи програмування: Навчальний посібник. – Одеса: Фенікс, 2010. – 314 с.
2. Грицюк Ю.І., Рак Т.Є. Програмування мовою С++: Навчальний посібник. – Львів: Вид-во ЛДУ БЖД, 2011. – 292 с.
3. Іванов Є.О., Ліндер Я.М., Жереб К.А. Основи мови програмування С++: Посібник першокурсника – Київ: Вид-во ЛОГОС, 2020. – 90 с.
4. Л. І. Кублій, Алгоритми та структури даних, основи алгоритмізації: Підручник. – Київ : КПІ ім. Ігоря Сікорського, 2022 – 528 с.
5. О. В. ГАЛКІН, М. М. ВЕРЕС., Мова програмування С++:Конспект лекцій. –Київ: ДП «Видавничий дім «Персонал»,2017 – 260с.
6. С++ Notes for Professionals [PDF] – [https://drive.google.com/file/d/18dChg\\_eFuObpuCvIY1AZjxjMS3CC0\\_z/view?usp=sharing](https://drive.google.com/file/d/18dChg_eFuObpuCvIY1AZjxjMS3CC0_z/view?usp=sharing)